Multi-Spacecraft Concept and Autonomy Tool (MuSCAT) Version 2

Dr. Saptarshi Bandyopadhyay,
Jet Propulsion Laboratory, California Institute of Technology
Email: saptarshi.bandyopadhyay@jpl.nasa.gov

October 13, 2026

Contents

1 Introduction **14**
- 1.1 Coding Conventions . . . 19
- 1.2 Coding Tips for Multi-Mission Usage . . . 19
- 1.3 Acknowledgment . . . 19

2 Implementing Custom Missions in MuSCAT **20**
- 2.1 Overview of Mission Implementation . . . 20
- 2.2 Understanding the Mission Structure . . . 20
- 2.3 MuSCAT Code Organization . . . 21
  - 2.3.1 Top-Level Directory Structure . . . 21
  - 2.3.2 Mission Files . . . 21
  - 2.3.3 True_SC: Core Spacecraft Systems . . . 22
  - 2.3.4 Class Interfaces and Update Functions . . . 22
    - 2.3.4.1 Main Update Functions . . . 22
    - 2.3.4.2 Extension Without Modification . . . 23
    - 2.3.4.3 Custom Function Implementation . . . 23
  - 2.3.5 True_Sensors_Actuators: Hardware Components . . . 24
  - 2.3.6 Software_SC: Flight Software . . . 24
  - 2.3.7 True_Environment: Environmental Models . . . 25
  - 2.3.8 Main Simulation Engine . . . 25
- 2.4 Step-by-Step Guide to Implementing a Mission . . . 26
  - 2.4.1 Customization Best Practices . . . 26
  - 2.4.2 Creating a New Mission File . . . 26
  - 2.4.3 Basic Mission Configuration . . . 27
  - 2.4.4 Time and Storage Configuration . . . 27
    - 2.4.4.1 Understanding the Dual-Loop Architecture . . . 27
    - 2.4.4.2 Storage Configuration . . . 28
  - 2.4.5 Environment Configuration . . . 28
  - 2.4.6 Ground Station Configuration . . . 29
    - 2.4.6.1 Ground Station Fundamentals . . . 29
    - 2.4.6.2 Ground Station Parameters . . . 30
    - 2.4.6.3 Simulation Impact . . . 30
  - 2.4.7 Spacecraft Configuration . . . 31
    - 2.4.7.1 Spacecraft Physical Properties . . . 31
    - 2.4.7.2 Spacecraft Hardware Complement . . . 31
      - 2.4.7.2.1 Simulation Impact . . . 32
      - 2.4.7.2.2 Selecting Appropriate Hardware . . . 33
    - 2.4.7.3 Initial Position and Attitude . . . 33

2.4.7.3.1 Position and Velocity Initialization . . . 33
2.4.7.3.2 Attitude Initialization . . . 34
2.4.7.3.3 Simulation Impact . . . 35
2.4.8 Configuring Spacecraft Subsystems . . . 35
2.4.8.1 Power Subsystem . . . 35
2.4.8.2 Data Handling Subsystem . . . 36
2.4.8.3 Communication Subsystem . . . 36
2.4.8.4 Sensors and Actuators . . . 37
2.4.9 Flight Software Configuration . . . 39
2.4.9.1 Executive Software . . . 39
2.4.9.2 Attitude & Orbit Estimation . . . 39
2.4.9.3 Orbit & Attitude Control . . . 39
2.4.9.4 Communication & Resource Management . . . 40
2.4.10 Final Initialization and Simulation Execution . . . 40
2.5 Understanding SPICE Integration . . . 41
2.5.1 What is SPICE? . . . 41
2.5.2 Why MuSCAT Uses SPICE . . . 41
2.5.3 SPICE Navigation Dynamics Mode . . . 42
2.5.4 SPICE Kernel Types Used in MuSCAT . . . 42
2.5.5 SPICE Setup in MuSCAT . . . 42
2.5.6 Creating SPICE Kernels for Your Mission . . . 43
2.5.7 SPICE Integration with Main Simulation Loop . . . 44
2.6 Creating Custom Flight Software . . . 44
2.6.1 Executive Software . . . 44
2.6.2 Attitude & Orbit Controllers . . . 44
2.7 Simulation Execution and Analysis . . . 45
2.7.1 Main Simulation Loop . . . 45
2.7.2 Visualizing Results . . . 45
2.8 Advanced Topics . . . 46
2.8.1 Creating Custom Hardware Components . . . 46
2.8.1.1 Custom Hardware Component Development Steps . . . 46
2.8.1.2 Integration with MuSCAT's Core Features . . . 47
2.8.2 Multi-Spacecraft Missions . . . 47
2.8.3 Monte Carlo Simulations . . . 48
2.9 Conclusion . . . 48
2.10 Troubleshooting and Best Practices . . . 49
2.10.1 Common Issues and Solutions . . . 49
2.10.2 Performance Optimization . . . 50
2.10.3 Units and Conventions . . . 50
2.10.4 Glossary of Terms . . . 51
2.10.5 Known Limitations . . . 51
2.11 Practical Examples and Design Patterns . . . 52
2.11.1 Common Mode Implementation Patterns . . . 52
2.11.1.1 Sun-Safe Mode . . . 52
2.11.1.2 Science Observation Mode . . . 53
2.11.2 Implementing Common Mission Types . . . 53
2.11.2.1 Earth-Orbiting Remote Sensing Satellite . . . 53
2.11.2.2 Interplanetary Science Mission . . . 54

2.11.3 Common Attitude Control Implementations . . . 55
2.11.3.1 PD Controller . . . 55
2.11.4 Custom Hardware Example: Laser Rangefinder . . . 55

3 Environment Classes **58**
3.1 `Storage` . . . 58
3.1.1 Inputs / Outputs . . . 58
3.1.1.1 Inputs / Dependencies . . . 58
3.1.1.2 Outputs . . . 59
3.1.2 Mathematical Equations . . . 59
3.1.2.1 Number of Storage Steps . . . 59
3.1.3 CML 2–5 Suitability . . . 60
3.1.3.1 Capabilities adequate for CML 2–5 formulation . . . 60
3.1.3.2 Gaps relevant to CML 2–5 use . . . 61
3.2 `True_Ground_Station` . . . 62
3.2.1 Inputs / Outputs . . . 62
3.2.1.1 Inputs / Dependencies . . . 62
3.2.1.2 Outputs . . . 62
3.2.2 Mathematical Equations . . . 63
3.2.2.1 Instantaneous Data Volume from Data Rate and Time Step . . . 63
3.2.2.2 Cumulative Total Data Transmitted and Received Over Time . . . 63
3.2.3 CML 2–5 Suitability . . . 64
3.2.3.1 What is included and correctly implemented . . . 64
3.2.3.2 What is missing or a gap for CML 2–5 use . . . 64
3.3 `True_GS_Radio_Antenna` . . . 66
3.3.1 Inputs / Outputs . . . 66
3.3.1.1 Inputs / Dependencies . . . 66
3.3.1.2 Outputs . . . 66
3.3.2 Mathematical Equations . . . 67
3.3.2.1 Ground Antenna Geodetic-to-Cartesian Position Transform . . . 67
3.3.2.2 Link Margin / Required E_b/N_0 Equation (Intended, Not Yet Implemented) . . . 67
3.3.3 CML 2–5 Suitability . . . 68
3.3.3.1 What is included and correctly implemented . . . 68
3.3.3.2 What is not included / gaps . . . 68
3.4 `Software_GS_Enterprise` . . . 70
3.4.1 Inputs / Outputs . . . 70
3.4.1.1 Inputs / Dependencies . . . 70
3.4.1.2 Outputs . . . 70
3.4.2 CML 2–5 Suitability . . . 70
3.4.2.1 What is included and correctly implemented . . . 70
3.4.2.2 What is missing / gap . . . 71
3.5 `True_Solar_System` . . . 72
3.5.1 Inputs / Outputs . . . 72
3.5.1.1 Inputs / Dependencies . . . 72
3.5.1.2 Outputs . . . 72
3.5.2 Mathematical Equations . . . 73

3.5.2.1 Standard Gravitational Parameter from Mass and the Gravitational Constant . . . 73
3.5.2.2 Heliocentric State Vector Decomposition from SPICE Ephemerides . 73
3.5.3 CML 2–5 Suitability . . . 74
3.5.3.1 What is included and correctly implemented . . . 74
3.5.3.2 What is not included or is a gap . . . 74
3.6 True_Stars . . . 76
3.6.1 Inputs / Outputs . . . 76
3.6.1.1 Inputs / Dependencies . . . 76
3.6.1.2 Outputs . . . 76
3.6.2 Mathematical Equations . . . 76
3.6.2.1 Proper-Motion-Corrected Epoch Coordinates . . . 76
3.6.2.2 Star Unit Vector from Right Ascension and Declination . . . 77
3.6.3 CML 2–5 Suitability . . . 77
3.6.3.1 What is included and correctly implemented . . . 77
3.6.3.2 What is not included / gaps . . . 78
3.7 True_Target_SPICE . . . 79
3.7.1 Inputs / Outputs . . . 79
3.7.1.1 Inputs / Dependencies . . . 79
3.7.1.2 Outputs . . . 79
3.7.2 Mathematical Equations . . . 80
3.7.2.1 Pole Right Ascension/Declination Rotation Matrix . . . 80
3.7.2.2 Time-Varying Prime Meridian Angle and Full Body-to-J2000 Rotation 80
3.7.2.3 Earth Prime Meridian from Julian Date UT1 (Sidereal Motion) . . . 81
3.7.3 CML 2–5 Suitability . . . 81
3.7.3.1 What is included and correctly implemented . . . 81
3.7.3.2 What is not included / gaps . . . 82
3.8 True_Target_Computed . . . 84
3.8.1 Inputs / Outputs . . . 84
3.8.1.1 Inputs / Dependencies . . . 84
3.8.1.2 Outputs . . . 84
3.8.2 Mathematical Equations . . . 85
3.8.2.1 Sun-Centered Equations of Motion and Third-Body Gravitational Perturbations . . . 85
3.8.2.2 Solar Radiation Pressure (SRP) Disturbance Force . . . 86
3.8.2.3 Yarkovsky Thermal Parameters . . . 86
3.8.2.4 Yarkovsky Along-Track Acceleration . . . 87
3.8.2.5 Area-Weighted Mean Radius from the Shape Model . . . 87
3.8.2.6 Fourth-Order Runge–Kutta (RK4) Propagation . . . 88
3.8.3 CML 2–5 Suitability . . . 88
3.8.3.1 What is included and correctly implemented . . . 88
3.8.3.2 What is not included / gaps . . . 89
3.9 True_Time . . . 90
3.9.1 Inputs / Outputs . . . 90
3.9.1.1 Inputs / Dependencies . . . 90
3.9.1.2 Outputs . . . 90
3.9.2 Mathematical Equations . . . 91
3.9.2.1 Dual-Rate Time-Step Synchronization (Attitude Sub-Stepping) . . . 91

3.9.2.2 Total Number of Simulation Time Steps . . . 91
3.9.2.3 Conversion from Simulation-Relative Time to Absolute Ephemeris Date . . . 92
3.9.3 CML 2–5 Suitability . . . 92
3.9.3.1 What is included and correctly implemented . . . 92
3.9.3.2 What is not included / gaps . . . 93

4 SC Physics Based Simulation Layer Classes 94
4.1 `True_SC_ADC` . . . 94
4.1.1 Inputs / Outputs . . . 94
4.1.1.1 Inputs / Dependencies . . . 94
4.1.1.2 Outputs . . . 95
4.1.2 Mathematical Equations . . . 96
4.1.2.1 Quaternion Kinematics . . . 96
4.1.2.2 Rigid-Body Rotational Dynamics (Euler's Equation) . . . 96
4.1.2.3 Axis–Angle to Rotation Matrix (Rodrigues Formula) . . . 96
4.1.2.4 Total Spacecraft Angular Momentum . . . 97
4.1.3 CML 2–5 Suitability . . . 97
4.1.3.1 What is included and correctly implemented . . . 97
4.1.3.2 What is not included / gaps . . . 98
4.2 `True_SC_Body` . . . 99
4.2.1 Inputs / Outputs . . . 99
4.2.1.1 Inputs / Dependencies . . . 99
4.2.1.2 Outputs . . . 99
4.2.2 Mathematical Equations . . . 100
4.2.2.1 Cuboid Moment of Inertia per Unit Mass . . . 100
4.2.2.2 Composite Center of Mass . . . 101
4.2.2.3 Total Moment of Inertia via the Parallel Axis Theorem . . . 101
4.2.2.4 Outward Face Normal and Area (Shape Face Geometry) . . . 101
4.2.3 CML 2–5 Suitability . . . 102
4.2.3.1 What is included and correctly implemented . . . 102
4.2.3.2 What is not included / gaps . . . 102
4.3 `True_SC_Data_Handling` . . . 104
4.3.1 Inputs / Outputs . . . 104
4.3.1.1 Inputs / Dependencies . . . 104
4.3.1.2 Outputs . . . 105
4.3.2 Mathematical Equations . . . 105
4.3.2.1 Instantaneous Data Generated by a Source . . . 105
4.3.2.2 Instantaneous Data Removed by a Source . . . 106
4.3.2.3 Net Instantaneous Data Change . . . 106
4.3.2.4 Onboard Memory Fill/Drain with Capacity Clipping . . . 106
4.3.3 CML 2–5 Suitability . . . 107
4.3.3.1 What is included and correctly implemented . . . 107
4.3.3.2 What is not included / gaps . . . 108
4.4 `True_SC_Gravity_Gradient` . . . 109
4.4.1 Inputs / Outputs . . . 109
4.4.1.1 Inputs / Dependencies . . . 109
4.4.1.2 Outputs . . . 110

4.4.2 Mathematical Equations . . . 110
4.4.2.1 Line-of-sight Vector in the Body Frame . . . 110
4.4.2.2 Gravity-Gradient Disturbance Torque . . . 110
4.4.3 CML 2–5 Suitability . . . 111
4.4.3.1 What is included and correctly implemented . . . 111
4.4.3.2 What is not included / gaps . . . 111
4.5 `True_SC_Orbit_Propagation` . . . 113
4.5.1 Inputs / Outputs . . . 113
4.5.1.1 Inputs / Dependencies . . . 113
4.5.1.2 Outputs . . . 114
4.5.2 Mathematical Equations . . . 115
4.5.2.1 Absolute-Frame Equations of Motion (n-Body Perturbed Two-Body Dynamics) . . . 115
4.5.2.2 Target-Relative Equations of Motion with Target Gravity Field . . . 115
4.5.2.3 Fourth-Order Runge-Kutta Time Integration . . . 116
4.5.2.4 Line-of-Sight Occultation Test (Point-to-Line-Segment Distance) . . . 116
4.5.3 CML 2–5 Suitability . . . 117
4.5.3.1 What is included and correctly implemented . . . 117
4.5.3.2 What is not included / gaps . . . 117
4.6 `True_SC_Power` . . . 119
4.6.1 Inputs / Outputs . . . 119
4.6.1.1 Inputs / Dependencies . . . 119
4.6.1.2 Outputs . . . 119
4.6.2 Mathematical Equations . . . 120
4.6.2.1 Net Energy Balance per Time Step . . . 120
4.6.2.2 Power Consumption with Loss Margin . . . 121
4.6.2.3 Battery Charging (Excess Energy) . . . 121
4.6.2.4 Battery Discharging (Energy Deficit) and Power Emergency . . . 121
4.6.2.5 Per-Hardware Energy Accumulation . . . 122
4.6.3 CML 2–5 Suitability . . . 122
4.6.3.1 What is included and correctly implemented . . . 122
4.6.3.2 What is not included / gaps . . . 122
4.7 `True_SC_SRP` . . . 124
4.7.1 Inputs / Outputs . . . 124
4.7.1.1 Inputs / Dependencies . . . 124
4.7.1.2 Outputs . . . 125
4.7.2 Mathematical Equations . . . 125
4.7.2.1 Solar Incidence Angle per Face . . . 125
4.7.2.2 SRP Force per Face . . . 125
4.7.2.3 SRP Torque per Face . . . 126
4.7.3 CML 2–5 Suitability . . . 126
4.7.3.1 What is included and correctly implemented . . . 126
4.7.3.2 What is not included / gaps . . . 127

5 SC Sensors and Actuators Classes **128**
5.1 `True_SC_Battery` . . . 128
5.1.1 Inputs / Outputs . . . 128
5.1.1.1 Inputs / Dependencies . . . 128

5.1.1.2 Outputs . . . 129
5.1.2 Mathematical Equations . . . 129
5.1.2.1 State of Charge . . . 129
5.1.2.2 Non-negativity (Depletion) Constraint . . . 129
5.1.3 CML 2–5 Suitability . . . 130
5.1.3.1 What is included and correctly implemented . . . 130
5.1.3.2 What is not included / gaps . . . 130
5.2 `True_SC_Camera` . . . 132
5.2.1 Inputs / Outputs . . . 132
5.2.1.1 Inputs / Dependencies . . . 132
5.2.1.2 Outputs . . . 133
5.2.2 Mathematical Equations . . . 133
5.2.2.1 Boresight-to-Target Angle and Field-of-View Visibility Test . . . 133
5.2.2.2 Instantaneous Image Data Volume . . . 134
5.2.2.3 Target-Body Surface Point to Longitude/Latitude Mapping . . . 134
5.2.3 CML 2–5 Suitability . . . 134
5.2.3.1 What is included and correctly implemented . . . 134
5.2.3.2 What is not included / gaps . . . 135
5.3 `True_SC_Communication_Link` . . . 136
5.3.1 Inputs / Outputs . . . 136
5.3.1.1 Inputs / Dependencies . . . 136
5.3.1.2 Outputs . . . 137
5.3.2 Mathematical Equations . . . 137
5.3.2.1 Data Rate Selection: Given Rate vs. Eclipse/No-Visibility . . . 137
5.3.2.2 Onboard Data Storage Rate Limiting . . . 138
5.3.2.3 Link Margin Computation (Intended but Not Implemented) . . . 138
5.3.3 CML 2–5 Suitability . . . 138
5.3.3.1 What is included and correctly implemented . . . 138
5.3.3.2 What is not included / gaps . . . 139
5.4 `True_SC_Chemical_Thruster` . . . 140
5.4.1 Inputs / Outputs . . . 140
5.4.1.1 Inputs / Dependencies . . . 140
5.4.1.2 Outputs . . . 140
5.4.2 Mathematical Equations . . . 141
5.4.2.1 Thrust Force Vector (Body to Inertial Frame) . . . 141
5.4.2.2 Disturbance Torque About Center of Mass . . . 142
5.4.2.3 Propellant Mass Flow Rate (Rocket Equation Relation) . . . 142
5.4.2.4 Impulsive Delta-V Applied . . . 142
5.4.3 CML 2–5 Suitability . . . 143
5.4.3.1 What is included and correctly implemented . . . 143
5.4.3.2 What is not included / gaps . . . 143
5.5 `True_SC_Fuel_Tank` . . . 145
5.5.1 Inputs / Outputs . . . 145
5.5.1.1 Inputs / Dependencies . . . 145
5.5.1.2 Outputs . . . 145
5.5.2 Mathematical Equations . . . 146
5.5.2.1 Propellant Mass Depletion . . . 146
5.5.3 CML 2–5 Suitability . . . 146

5.5.3.1 What is included and correctly implemented . . . 146
5.5.3.2 What is not included / gaps . . . 147
5.6 True_SC_Generic_Sensor . . . 148
5.6.1 Inputs / Outputs . . . 148
5.6.1.1 Inputs / Dependencies . . . 148
5.6.1.2 Outputs . . . 148
5.6.2 Mathematical Equations . . . 149
5.6.3 CML 2–5 Suitability . . . 149
5.6.3.1 What is included and correctly implemented . . . 149
5.6.3.2 What is not included / gaps . . . 149
5.7 True_SC_IMU . . . 151
5.7.1 Inputs / Outputs . . . 151
5.7.1.1 Inputs / Dependencies . . . 151
5.7.1.2 Outputs . . . 151
5.7.2 Mathematical Equations . . . 152
5.7.2.1 Simple Mode: Rate Measurement with White Noise . . . 152
5.7.2.2 Rate Bias Mode: Bias Random-Walk Propagation . . . 152
5.7.2.3 Rate Bias Mode: Measured Rate with Noise and Bias . . . 152
5.7.2.4 Discretization of ARW and RRW Standard Deviations from PSD . . . 153
5.7.3 CML 2–5 Suitability . . . 153
5.7.3.1 What is included and correctly implemented . . . 153
5.7.3.2 What is not included / gaps . . . 154
5.8 True_SC_Magnetorquer . . . 155
5.8.1 Inputs / Outputs . . . 155
5.8.1.1 Inputs / Dependencies . . . 155
5.8.1.2 Outputs . . . 156
5.8.2 Mathematical Equations . . . 156
5.8.2.1 Tilted-Dipole Ambient Magnetic Field . . . 156
5.8.2.2 Commanded Dipole Moment with Noise and Saturation . . . 156
5.8.2.3 Dipole Moment Vector and Control Torque . . . 157
5.8.2.4 Power Consumption from Lookup Table Interpolation . . . 157
5.8.3 CML 2–5 Suitability . . . 157
5.8.3.1 What is included and correctly implemented . . . 157
5.8.3.2 What is not included / gaps . . . 158
5.9 True_SC_Onboard_Computer . . . 159
5.9.1 Inputs / Outputs . . . 159
5.9.1.1 Inputs / Dependencies . . . 159
5.9.1.2 Outputs . . . 159
5.9.2 Mathematical Equations . . . 160
5.9.3 CML 2–5 Suitability . . . 160
5.9.3.1 What is included and correctly implemented . . . 160
5.9.3.2 What is not included / gaps . . . 160
5.10 True_SC_Onboard_Clock . . . 162
5.10.1 Inputs / Outputs . . . 162
5.10.1.1 Inputs / Dependencies . . . 162
5.10.1.2 Outputs . . . 162
5.10.2 Mathematical Equations . . . 163
5.10.2.1 Measurement Noise Draw . . . 163

5.10.2.2 Clock Measurement Vector . . . 163
5.10.2.3 Measurement Update Trigger . . . 163
5.10.3 CML 2–5 Suitability . . . 164
5.10.3.1 What is included and correctly implemented . . . 164
5.10.3.2 What is not included / gaps . . . 164
5.11 `True_SC_Onboard_Memory` . . . 165
5.11.1 Inputs / Outputs . . . 165
5.11.1.1 Inputs / Dependencies . . . 165
5.11.1.2 Outputs . . . 165
5.11.2 Mathematical Equations . . . 166
5.11.2.1 State of Data Storage (Percent Full) . . . 166
5.11.2.2 Mode-Dependent Instantaneous Power Consumption . . . 166
5.11.3 CML 2–5 Suitability . . . 167
5.11.3.1 What is included and correctly implemented . . . 167
5.11.3.2 What is not included / gaps . . . 167
5.12 `True_SC_Radio_Antenna` . . . 169
5.12.1 Inputs / Outputs . . . 169
5.12.1.1 Inputs / Dependencies . . . 169
5.12.1.2 Outputs . . . 169
5.12.2 Mathematical Equations . . . 170
5.12.2.1 Instantaneous Power Consumption by TX/RX Mode . . . 170
5.12.2.2 Data-Rate Bookkeeping (Generated/Removed) . . . 170
5.12.2.3 Link Margin / E_b/N_0 Parameters (Declared but Not Computed) . . . 171
5.12.3 CML 2–5 Suitability . . . 171
5.12.3.1 What is included and correctly implemented . . . 171
5.12.3.2 What is not included / gaps . . . 172
5.13 `True_SC_RCS_Thruster` . . . 173
5.13.1 Inputs / Outputs . . . 173
5.13.1.1 Inputs / Dependencies . . . 173
5.13.1.2 Outputs . . . 174
5.13.2 Mathematical Equations . . . 174
5.13.2.1 True Commanded Thrust with Random Noise . . . 174
5.13.2.2 Thrust Force Vector: Body Frame and Rotation to Inertial Frame . . . 174
5.13.2.3 Control Torque About the Center of Mass . . . 175
5.13.2.4 Propellant Mass Flow Rate (Rocket Equation) . . . 175
5.13.2.5 Reaction Force Contribution to Orbital Disturbance . . . 175
5.13.3 CML 2–5 Suitability . . . 175
5.13.3.1 What is included and correctly implemented . . . 175
5.13.3.2 What is not included / gaps . . . 176
5.14 `True_SC_Reaction_Wheel` . . . 177
5.14.1 Inputs / Outputs . . . 177
5.14.1.1 Inputs / Dependencies . . . 177
5.14.1.2 Outputs . . . 177
5.14.2 Mathematical Equations . . . 178
5.14.2.1 Moment of Inertia of the Wheel . . . 178
5.14.2.2 Maximum Wheel Acceleration . . . 178
5.14.2.3 Commanded Torque from Angular Acceleration . . . 178
5.14.2.4 Angular Velocity Update with Noise and Rate Limiting . . . 179

5.14.2.5 Wheel Angular Momentum . . . 179
5.14.2.6 Reaction Torque Applied to the Spacecraft Bus . . . 179
5.14.2.7 Saturation Flag . . . 179
5.14.2.8 Power Consumption from Angular Velocity . . . 180
5.14.3 CML 2–5 Suitability . . . 180
5.14.3.1 What is included and correctly implemented . . . 180
5.14.3.2 What is not included / gaps . . . 181
5.15 True_SC_Remote_Sensing . . . 182
5.15.1 Inputs / Outputs . . . 182
5.15.1.1 Inputs / Dependencies . . . 182
5.15.1.2 Outputs . . . 183
5.15.2 Mathematical Equations . . . 183
5.15.2.1 Measurement Cadence (Temporal Sampling Gate) . . . 183
5.15.2.2 Limb-Angle Visibility Test . . . 183
5.15.2.3 Boresight Field-of-View Test . . . 184
5.15.2.4 Surface Point Geodetic Mapping . . . 184
5.15.2.5 Monostatic Coverage Fraction . . . 184
5.15.3 CML 2–5 Suitability . . . 185
5.15.3.1 What is included and correctly implemented . . . 185
5.15.3.2 What is not included / gaps . . . 185
5.16 True_SC_RPS . . . 187
5.16.1 Inputs / Outputs . . . 187
5.16.1.1 Inputs / Dependencies . . . 187
5.16.1.2 Outputs . . . 187
5.16.2 Mathematical Equations . . . 188
5.16.2.1 Radioisotope Decay-Based Power Output . . . 188
5.16.2.2 Instantaneous Power Generated (Health Gating) . . . 188
5.16.3 CML 2–5 Suitability . . . 189
5.16.3.1 What is included and correctly implemented . . . 189
5.16.3.2 What is not included / gaps . . . 189
5.17 True_SC_Science_Processor . . . 191
5.17.1 Inputs / Outputs . . . 191
5.17.1.1 Inputs / Dependencies . . . 191
5.17.1.2 Outputs . . . 192
5.17.2 Mathematical Equations . . . 192
5.17.2.1 Beam Footprint Radius and Gaussian Divergence Width . . . 193
5.17.2.2 Momentum Transfer Efficiency . . . 193
5.17.2.3 EP Thruster Force on Target . . . 193
5.17.2.4 EP Thruster Torque on Target About the Target Centroid . . . 194
5.17.3 CML 2–5 Suitability . . . 194
5.17.3.1 What is included and correctly implemented . . . 194
5.17.3.2 What is not included / gaps . . . 195
5.18 True_SC_Science_Radar . . . 196
5.18.1 Inputs / Outputs . . . 196
5.18.1.1 Inputs / Dependencies . . . 196
5.18.1.2 Outputs . . . 197
5.18.2 Mathematical Equations . . . 197
5.18.2.1 Boresight-to-Surface-Point Angular Separation . . . 197

5.18.2.2 Field-of-View / Nadir-Point Selection Criterion . . . 197
5.18.2.3 Monostatic Coverage Accumulation . . . 198
5.18.2.4 Instantaneous Data Volume Generated per Measurement . . . 198
5.18.3 CML 2–5 Suitability . . . 198
5.18.3.1 What is included and correctly implemented . . . 198
5.18.3.2 What is not included / gaps . . . 199
5.19 `True_SC_Solar_Panel` . . . 200
5.19.1 Inputs / Outputs . . . 200
5.19.1.1 Inputs / Dependencies . . . 200
5.19.1.2 Outputs . . . 200
5.19.2 Mathematical Equations . . . 201
5.19.2.1 Sun Incidence Angle . . . 201
5.19.2.2 Maximum Power (Inverse-Square Solar Flux Attenuation) . . . 201
5.19.2.3 Instantaneous Power Generated (Cosine Law with Eclipse/Health Gating) . . . 202
5.19.2.4 Panel Face Area (Heron's Formula) and Total Active Area . . . 202
5.19.2.5 Cuboid Mass Properties (Inertia Tensor and Volume) . . . 202
5.19.3 CML 2–5 Suitability . . . 203
5.19.3.1 What is included and correctly implemented . . . 203
5.19.3.2 What is not included / gaps . . . 203
5.20 `True_SC_Star_Tracker` . . . 205
5.20.1 Inputs / Outputs . . . 205
5.20.1.1 Inputs / Dependencies . . . 205
5.20.1.2 Outputs . . . 205
5.20.2 Mathematical Equations . . . 206
5.20.2.1 Sun-Exclusion Field-of-View Test . . . 206
5.20.2.2 Simple Small-Angle Attitude Error Model . . . 206
5.20.2.3 Bias + First-Order Gauss-Markov (FOGM) Error Budget . . . 207
5.20.3 CML 2–5 Suitability . . . 207
5.20.3.1 What is included and correctly implemented . . . 207
5.20.3.2 What is not included / gaps . . . 208
5.21 `True_SC_Sun_Sensor` . . . 209
5.21.1 Inputs / Outputs . . . 209
5.21.1.1 Inputs / Dependencies . . . 209
5.21.1.2 Outputs . . . 209
5.21.2 Mathematical Equations . . . 210
5.21.2.1 Sensor Boresight Direction in the Reference Frame . . . 210
5.21.2.2 True Sun-Direction Vector . . . 210
5.21.2.3 Field-of-View Visibility Test . . . 210
5.21.2.4 Measurement Noise Model (Small-Angle Quaternion Perturbation) . . . 210
5.21.2.5 Truth-Mode Output . . . 211
5.21.3 CML 2–5 Suitability . . . 211
5.21.3.1 What is included and correctly implemented . . . 211
5.21.3.2 What is not included / gaps . . . 211

6 SC System-Level and Functional-Level Autonomy Software Layer Classes 213
6.1 `Software_SC_Communication` . . . 213
6.1.1 Inputs / Outputs . . . 213
6.1.1.1 Inputs / Dependencies . . . 213
6.1.1.2 Outputs . . . 214
6.1.2 Mathematical Equations . . . 215
6.1.3 CML 2–5 Suitability . . . 215
6.1.3.1 What is included and correctly implemented . . . 215
6.1.3.2 What is not included / gaps . . . 215
6.2 `Software_SC_Control_Attitude` . . . 217
6.2.1 Inputs / Outputs . . . 217
6.2.1.1 Inputs / Dependencies . . . 217
6.2.1.2 Outputs . . . 218
6.2.2 Mathematical Equations . . . 218
6.2.2.1 Quaternion Attitude Error . . . 218
6.2.2.2 PD Attitude Control Law with Feed-Forward and Gyroscopic Compensation . . . 218
6.2.2.3 Asymptotically-Stable Quaternion-Feedback Control Law . . . 219
6.2.2.4 Reaction-Wheel Command Allocation (Pseudo-Inverse) . . . 219
6.2.2.5 Thruster Torque Allocation via Constrained Least Squares (KKT) . . . 219
6.2.2.6 SLERP-Based Slew Trajectory Generation . . . 220
6.2.3 CML 2–5 Suitability . . . 220
6.2.3.1 What is included and correctly implemented . . . 220
6.2.3.2 What is not included / gaps . . . 221
6.3 `Software_SC_Control_Orbit` . . . 222
6.3.1 Inputs / Outputs . . . 222
6.3.1.1 Inputs / Dependencies . . . 222
6.3.1.2 Outputs . . . 223
6.3.2 Mathematical Equations . . . 223
6.3.2.1 Delta-V Magnitude Tracking and Execution Threshold . . . 223
6.3.3 CML 2–5 Suitability . . . 224
6.3.3.1 What is included and correctly implemented . . . 224
6.3.3.2 What is not included / gaps . . . 224
6.4 `Software_SC_Data_Handling` . . . 226
6.4.1 Inputs / Outputs . . . 226
6.4.1.1 Inputs / Dependencies . . . 226
6.4.1.2 Outputs . . . 227
6.4.2 Mathematical Equations . . . 227
6.4.2.1 Total Onboard Data Storage . . . 227
6.4.2.2 Mean State of Data Storage . . . 227
6.4.3 CML 2–5 Suitability . . . 228
6.4.3.1 What is included and correctly implemented . . . 228
6.4.3.2 What is not included / gaps . . . 228
6.5 `Software_SC_Estimate_Attitude` . . . 230
6.5.1 Inputs / Outputs . . . 230
6.5.1.1 Inputs / Dependencies . . . 230
6.5.1.2 Outputs . . . 231
6.5.2 Mathematical Equations . . . 231

6.5.2.1 Truth-Mode Attitude Estimate (Trivial Pass-Through) . . . 231
6.5.2.2 KF-Mode Quaternion Propagation (Gyro Kinematics) . . . 231
6.5.2.3 KF-Mode Error-State Covariance Propagation . . . 232
6.5.2.4 KF-Mode Star-Tracker Measurement Update . . . 232
6.5.2.5 Estimated Body-Frame Angular Momentum . . . 232
6.5.3 CML 2–5 Suitability . . . 233
6.5.3.1 What is included and correctly implemented . . . 233
6.5.3.2 What is not included / gaps . . . 233
6.6 Software_SC_Estimate_Orbit . . . 235
6.6.1 Inputs / Outputs . . . 235
6.6.1.1 Inputs / Dependencies . . . 235
6.6.1.2 Outputs . . . 235
6.6.2 Mathematical Equations . . . 236
6.6.2.1 Truth-Mode State Pass-Through (No Estimation Error) . . . 236
6.6.2.2 Visibility-Gated Linear Error-Growth Model . . . 236
6.6.3 CML 2–5 Suitability . . . 237
6.6.3.1 What is included and correctly implemented . . . 237
6.6.3.2 What is not included / gaps . . . 237
6.7 Software_SC_Executive . . . 239
6.7.1 Inputs / Outputs . . . 239
6.7.1.1 Inputs / Dependencies . . . 239
6.7.1.2 Outputs . . . 239
6.7.2 Mathematical Equations . . . 240
6.7.2.1 Mode-Value Lookup (String-to-Index Encoding) . . . 240
6.7.3 CML 2–5 Suitability . . . 241
6.7.3.1 What is included and correctly implemented . . . 241
6.7.3.2 What is not included / gaps . . . 241
6.8 Software_SC_Power . . . 243
6.8.1 Inputs / Outputs . . . 243
6.8.1.1 Inputs / Dependencies . . . 243
6.8.1.2 Outputs . . . 243
6.8.2 Mathematical Equations . . . 244
6.8.2.1 Mean State of Charge . . . 244
6.8.3 CML 2–5 Suitability . . . 244
6.8.3.1 What is included and correctly implemented . . . 244
6.8.3.2 What is not included / gaps . . . 245

7 Main File **246**
7.1 main_v3 . . . 246

8 Mission Classes **247**
8.1 Mission_DART . . . 247
8.2 Mission_NISAR . . . 248
8.3 Mission_NISAR_2SC . . . 249

Bibliography **250**

Chapter 1

Introduction

Multi-Spacecraft Concept and Autonomy Tool (MuSCAT) open-source simulation software offers an integrated platform for conducting low-fidelity simulations of single/multiple cruising/orbiting spacecraft mission concepts and test of autonomy algorithms. MuSCAT encompasses various spacecraft subsystems (Fig. 1.2) such as (i) Orbit Propagation, (ii) Attitude Determination and Control, (iii) Power Management, (iv) Data Handling, (v) Communication (including direct-to-Earth and inter-spacecraft), and (vi) Scientific Instruments. It provides mission designers with a means to quantitatively verify if the mission concept meets level-1 science functional requirements. This document provides a detailed description of the software architecture and use cases of MuSCAT.

MuSCAT's architecture (Figs. 1.2, 1.3) separates across multiple layers:

- a *physics-based simulation* layer representing ground-truth spacecraft subsystems (denoted by `True_SC_*` classes) and all environment behaviors (denoted by `True_*`),
- a *hardware* layer representing spacecraft's sensor measurements and actuator commands on this physics-based simulation layer. These classes are denoted by `True_SC_*`.
- two *autonomous flight software* layers denoted by `Software_SC_*` classes:
  - the *functional-level onboard autonomy* layer represents the software for each spacecraft's subsystems, which intake measurements from the sensors and send commands to the actuators,
  - the *system-level onboard autonomy* layer represents the top-level decision making happening on the spacecraft, that controls the ConOps of the spacecraft and manages all the actions of the functional-level onboard autonomy layers.

A dual-rate time loop advances orbital/subsystem states at a coarse time step ($\Delta t \sim 5$ s) and attitude states at a fine time step ($\Delta t_{\text{att}} \sim 0.1$ s), which is the key computational choice that keeps MuSCAT tractable for long-duration simulations.

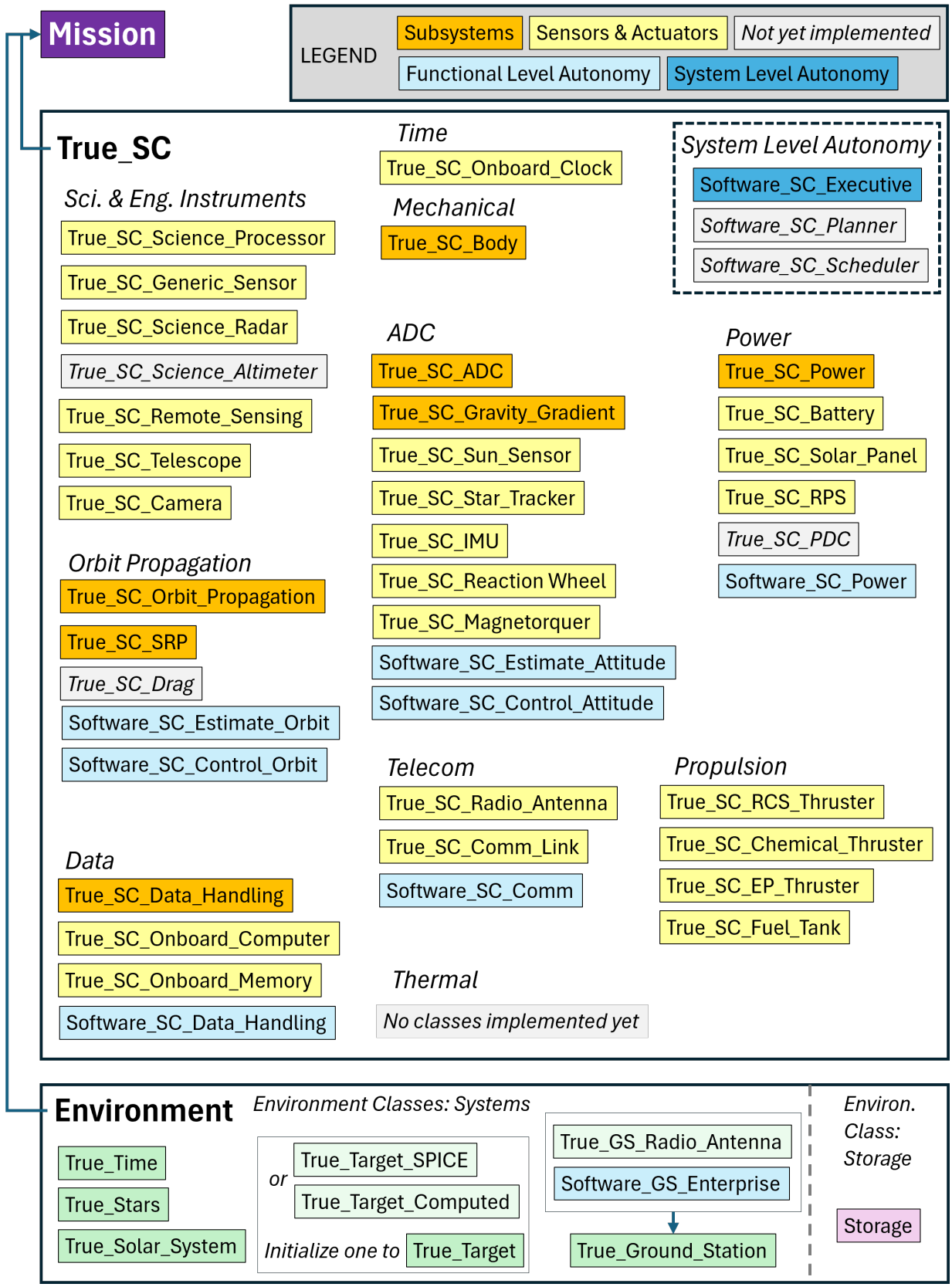


Figure 1.1: Diagram showing all the classes that are inside MuSCAT, and their conceptual relationship to, e.g., spacecraft subsystems. This is distinct from their MuSCAT codebase architectural layer, which is shown in the next figure. Classes in gray are currently under development.

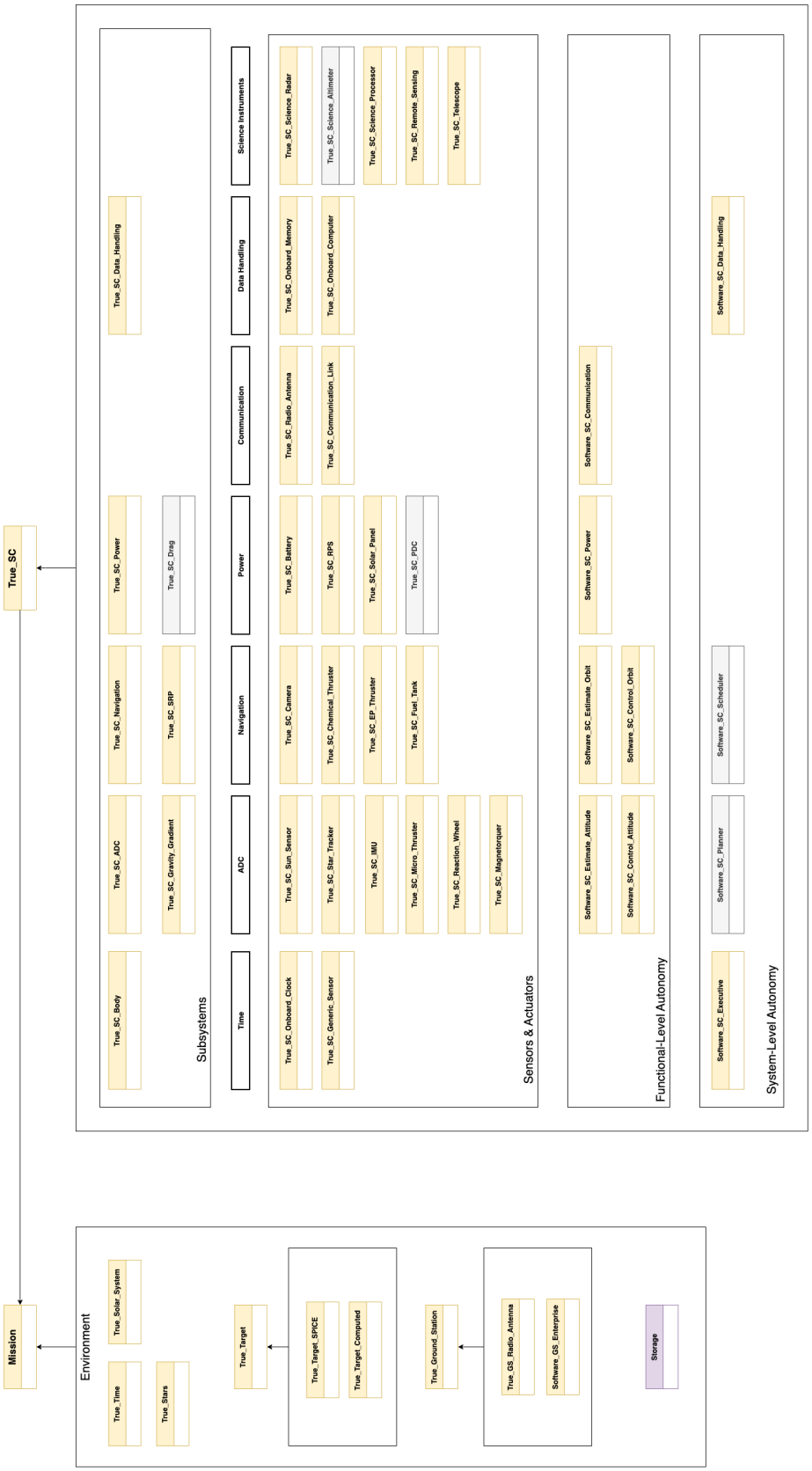


Figure 1.2: Diagram showing all the classes that are inside MuSCAT, and their architectural layer. Classes in gray are currently under development.

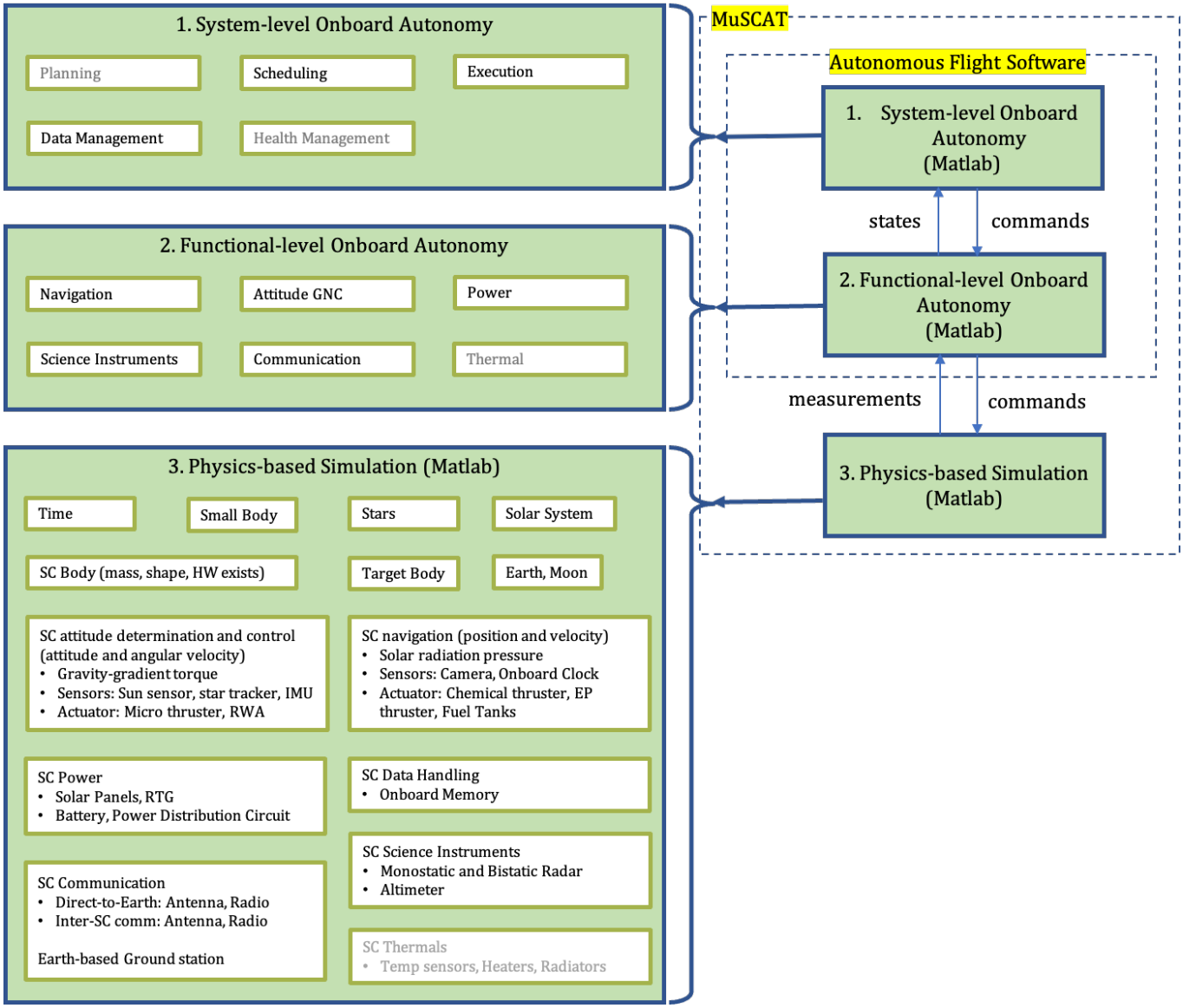


Figure 1.3: Block diagram that shows the different architectural layers inside the MuSCAT simulator. The blocks in gray are currently under development.

1.1 Coding Conventions

- Items marked with # are not initialized, and computed internally
- In naming functions,
  `update` is used when the function is called often (every time step),
  `compute` is used when it is called rarely (or only once).
- All functions are prefixed with `func_`, and colored green.
- All classes within the physics-based simulation represent real objects like Solar System or small body, and are prefixed with `True_`, and those that belong to the spacecraft are prefixed with `True_SC_`.
- All classes within the autonomous flight software that only exist within the spacecraft's onboard computer are prefixed with `Software_SC_`.

1.2 Coding Tips for Multi-Mission Usage

- Within each class, use `data` to track all mission-specific variables.
- If a component uses different power in different spacecraft modes, use `data.instantaneous_power_consumed_per_SC_mode` to track that.

1.3 Acknowledgment

This research was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration. ©2026 California Institute of Technology. Government sponsorship acknowledged.

The authors also thank the following people for their invaluable time and technical expertise in constructing this code base:

- Thibault Wartel from Institut supérieur de l'aéronautique et de l'espace (ISAE-SUPAERO), France
- Cesc Casanovas Gasso from International Space University, France
- Ian Aenishanslin from Institut polytechnique des sciences avancées (IPSA), France
- Carmine Buonagura from Politecnico di Milano, Italy
- Chris Agia from Stanford, USA
- Kazu Echigo from University of Washington, USA
- Minduli C Wijayatunga from University of Auckland, New Zealand

Chapter 2

Implementing Custom Missions in MuSCAT

2.1 Overview of Mission Implementation

The Multi-Spacecraft Concept and Autonomy Tool (MuSCAT) provides a flexible framework for simulating spacecraft missions. This chapter explains in detail how to implement your own custom mission using MuSCAT's modular architecture.

At its core, MuSCAT follows an object-oriented approach where various spacecraft subsystems, environmental elements, and software components are instantiated as objects and connected together to form a complete mission simulation. The central data structure that ties everything together is the `mission` structure, which serves as the global state for the entire simulation.

2.2 Understanding the Mission Structure

The `mission` structure contains all the objects and parameters that define your mission:

- **mission.name**: Name identifier for the mission
- **mission.num_SC**: Number of spacecraft in the mission
- **mission.num_target**: Number of target bodies (e.g., asteroids, planets)
- **mission.frame**: Reference frame type ('Absolute', 'Relative', or 'Combined')
- **mission.true_time**: Time-related parameters and functions
- **mission.storage**: Data storage configuration
- **mission.true_stars**: Star catalog for attitude determination
- **mission.true_solar_system**: Solar system bodies
- **mission.true_target**: Target bodies (e.g., asteroids)
- **mission.true_ground_station**: Ground station parameters
- **mission.true_SC**: Array of spacecraft objects

Each spacecraft contains numerous subsystems, all properly organized within the structure:

- **mission.true_SC{i}.true_SC_body**: Physical properties and geometry
- **mission.true_SC{i}.true_SC_navigation**: Position, velocity, and orbital parameters
- **mission.true_SC{i}.true_SC_adc**: Attitude determination and control
- **mission.true_SC{i}.true_SC_power**: Power subsystem
- **mission.true_SC{i}.true_SC_data_handling**: Data management
- **mission.true_SC{i}.true_SC_[hardware]**: Various hardware components
- **mission.true_SC{i}.software_SC_[software]**: Flight software components

2.3 MuSCAT Code Organization

Understanding the organization of MuSCAT's codebase is essential for effectively implementing and customizing missions. The codebase follows a modular structure that separates different aspects of spacecraft simulation into distinct directories, each with a specific purpose.

2.3.1 Top-Level Directory Structure

The MuSCAT codebase is organized into several key directories:

- **/Mission/**: Contains mission definition files that serve as the entry points for simulations
- **/Main/**: Contains the main simulation engine that executes the time steps
- **/True_SC/**: Houses the core spacecraft system classes (body, navigation, power, etc.)
- **/True_Sensors_Actuators/**: Contains sensor and actuator hardware implementations
- **/True_Environment/**: Defines environmental models (solar system, stars, etc.)
- **/Software_SC/**: Implements flight software components
- **/Supporting_Functions/**: Contains utility functions used throughout the codebase
- **/Documentation/**: Contains detailed documentation files
- **/Output/**: Default location for simulation results and visualizations

2.3.2 Mission Files

The `/Mission/` directory is where users should focus most of their attention when creating new simulations. Each mission file (e.g., `Mission_DART.m`) follows the same general structure:

1. Configure mission parameters (name, number of spacecraft, etc.)
2. Set up time and storage parameters
3. Configure environmental elements (stars, solar system, targets)

4. Set up ground stations
5. For each spacecraft:
    - Define physical properties
    - Set initial position and attitude
    - Configure hardware components
    - Set up flight software
6. Perform final initialization
7. Execute the simulation
8. Generate and save results

These mission files serve as the "glue" that brings together all the individual components into a cohesive simulation.

2.3.3 True_SC: Core Spacecraft Systems

The `/True_SC/` directory contains the fundamental spacecraft subsystem classes:

- **True_SC_Body.m**: Defines the physical properties of the spacecraft, including mass, inertia, and shape
- **True_SC_Navigation.m**: Handles position and velocity dynamics
- **True_SC_ADC.m**: Manages attitude dynamics and control
- **True_SC_Power.m**: Tracks power generation, consumption, and distribution
- **True_SC_Data_Handling.m**: Manages onboard data flows

These core classes form the backbone of each spacecraft model. They work together to provide a complete representation of the physical spacecraft and its core functionalities.

2.3.4 Class Interfaces and Update Functions

All MuSCAT classes follow a consistent interface pattern that enables the simulation loop to update each component appropriately. Understanding this pattern is key to extending MuSCAT with custom components.

2.3.4.1 Main Update Functions

Each class in MuSCAT has a corresponding `func_main_[className]` function that the simulation loop calls to update that component's state. For example:

- `func_main_true_SC_navigation`: Updates spacecraft position and velocity
- `func_main_true_SC_adc`: Updates spacecraft attitude
- `func_main_true_SC_star_tracker`: Updates star tracker measurements

- `func_main_software_SC_executive`: Executes decision-making logic

These functions follow a consistent pattern:

```
function func_main_[className](obj, mission, i_SC, [optional parameters])
    % Update the object's state based on current mission state
    % obj: The object instance being updated
    % mission: The global mission structure
    % i_SC: The spacecraft index (for spacecraft components)
    % [optional parameters]: Component-specific parameters

    % Update code specific to this component
    % ...

    % Update stored data if needed
    func_store_data(obj, mission);
end
```

2.3.4.2 Extension Without Modification

It's important to note that users should **never modify** these core update functions directly. Instead, MuSCAT provides several approaches for customization:

1. **Mode Selectors**: Most classes have a `mode_[className]_selector` parameter that can be set to choose different behavioral modes. For example, setting `mode_true_SC_navigation_dynamics_selector = 'Absolute Dynamics'` selects the absolute dynamics mode.

2. **Mission-Specific Extensions**: Flight software classes are designed to be extended with mission-specific implementations. For instance, the `Software_SC_Executive` class allows you to add a new function named `func_software_SC_executive_YourMission` that implements your custom decision logic.

3. **Custom Component Creation**: You can create entirely new component classes in your mission directory, inheriting from existing parent classes where appropriate.

2.3.4.3 Custom Function Implementation

When implementing custom functions for flight software, follow this pattern:

```
function obj = func_software_SC_[component]_YourMission(obj, mission, i_SC)
    % Your custom implementation
    % ...

    % Return the updated object
end
```

By strictly adhering to these patterns, you ensure that your custom components will integrate seamlessly with the main simulation loop.

2.3.5 True_Sensors_Actuators: Hardware Components

The `/True_Sensors_Actuators/` directory contains all the individual hardware components that can be added to a spacecraft:

- **Sensors**: Star trackers, sun sensors, IMUs, cameras, etc.
- **Actuators**: Reaction wheels, thrusters, etc.
- **Communication**: Radio antennas, communication links, etc.
- **Power**: Solar panels, batteries, etc.
- **Data**: Onboard computers, memory, etc.

Each hardware component is represented by a MATLAB class that inherits from a common parent class. This object-oriented approach allows for easy extension with new components and consistent interfaces across all hardware.

The number of each hardware component is specified in the mission file and directly influences how the simulation behaves. For example, specifying 6 sun sensors will result in 6 separate hardware objects being created and included in the simulation loop. Each component will be updated during the appropriate part of the simulation cycle.

2.3.6 Software_SC: Flight Software

The `/Software_SC/` directory contains the flight software classes that control the spacecraft's autonomous behavior:

- **Software_SC_Executive.m**: The main decision-making component that determines spacecraft operating modes
- **Software_SC_Estimate_Attitude.m**: Algorithms for estimating spacecraft attitude
- **Software_SC_Estimate_Orbit.m**: Algorithms for estimating spacecraft orbit
- **Software_SC_Control_Attitude.m**: Algorithms for controlling spacecraft attitude
- **Software_SC_Control_Orbit.m**: Algorithms for controlling spacecraft orbit
- **Software_SC_Communication.m**: Logic for managing communications
- **Software_SC_Power.m**: Logic for managing power
- **Software_SC_Data_Handling.m**: Logic for managing data

These software components interact with the hardware models and implement the autonomous behavior of the spacecraft. They are executed at each appropriate time step in the simulation loop.

2.3.7 True_Environment: Environmental Models

The `/True_Environment/` directory contains models of the space environment:

- **True_Solar_System.m**: Models of solar system bodies
- **True_Stars.m**: Star catalog for attitude determination
- **True_Target.m**: Models of mission targets (e.g., asteroids)
- **True_Ground_Station.m**: Earth-based ground stations
- **True_SRP.m**: Solar radiation pressure model
- **True_Gravity_Gradient.m**: Gravity gradient torque model

These environmental models provide the context in which the spacecraft operates and influence various aspects of the simulation, from orbital dynamics to sensor measurements.

2.3.8 Main Simulation Engine

The `/Main/` directory contains the main simulation engine in `main_v3.m`. This script implements the time-stepping algorithm that advances the simulation state:

1. Update simulation time and date
2. Update solar system and target bodies
3. For each spacecraft:
    - Update environmental effects (solar radiation pressure, gravity gradient)
    - Update spacecraft body, position, and velocity
    - Update onboard clock and computer systems
    - Execute the Executive software to determine operating mode
    - Update all sensors, actuators, and subsystems
    - Run the attitude dynamics loop at a higher frequency
    - Update communication systems
    - Update power and data handling systems
4. Update ground station systems
5. Save data and update visualizations

The main simulation engine is responsible for calling all the appropriate update functions for each component in the correct order, ensuring that dependencies between components are properly handled.

2.4 Step-by-Step Guide to Implementing a Mission

2.4.1 Customization Best Practices

When implementing your own mission in MuSCAT, it's critical to follow these best practices to ensure maintainability and compatibility with future updates:

1. **Never Modify Core Files**: The core MuSCAT files (`/True_SC/`, `/True_Sensors_Actuators/`, `/Main/`, etc.) should never be directly edited. These files contain the fundamental simulation engine and component models that are designed to be used as-is.
2. **Create Mission-Specific Directories**: For each new mission, create a dedicated directory in `/Mission/` (e.g., `/Mission/YourMission/`) to contain all mission-specific files.
3. **Use Extension Mechanisms**: MuSCAT provides specific extension points (via mode selectors and mission-specific functions) that allow you to customize behavior without modifying core files.
4. **Document Custom Components**: When creating mission-specific extensions, thoroughly document their purpose, interfaces, and requirements to facilitate future maintenance.
5. **Follow Naming Conventions**: Use consistent naming patterns (e.g., `func_software_SC_executive_YourMission`) to ensure that your custom functions are correctly discovered and invoked by the simulation engine.

By adhering to these practices, you ensure that:

- Your mission can be easily upgraded when new MuSCAT versions are released
- You can transfer your custom components between different MuSCAT installations
- Collaboration with other users is simplified through clear separation of core and custom code
- Debugging is easier since modifications are isolated to specific mission files

2.4.2 Creating a New Mission File

1. Navigate to the `/Mission` directory in the MuSCAT codebase
2. Copy `Mission_DART.m` to a new file with your mission name (e.g., `Mission_YourMission.m`)
3. Open the new file and begin customizing according to your mission requirements

Each mission file follows a structured format, with initialization of different subsystems organized into sections. Let's examine each section.

2.4.3 Basic Mission Configuration

The first step is to define basic mission parameters:

```
%% Mission Definition
mission = [];
mission.name = 'YourMission';      % Name of the Mission
mission.num_SC = 1;                % Number of Spacecraft
mission.num_target = 1;            % Number of Target bodies
mission.frame = 'Absolute';        % Frame type: 'Absolute', 'Relative', or 'Combined'
mission.flag_stop_sim = 0;         % Boolean flag to stop simulation if needed
```

2.4.4 Time and Storage Configuration

The time configuration is a critical aspect of MuSCAT as it defines the temporal resolution and duration of your simulation. MuSCAT employs a dual-loop architecture to efficiently handle different timescales for spacecraft dynamics:

```
%% Time Configuration
init_data = [];
init_data.t_initial = 0;                                    % [sec] Initial time
init_data.t_final = 10000;                                  % [sec] Final time
init_data.time_step = 5;                                    % [sec] Simulation time step
init_data.t_initial_date_string = '01-JAN-2025 00:00:00';  % Starting date
init_data.time_step_attitude = 0.1;                 % [sec] Attitude dynamics time step
mission.true_time = True_Time(init_data);
```

2.4.4.1 Understanding the Dual-Loop Architecture

MuSCAT's main simulation engine in `main_v3.m` implements two nested time loops:

1. **Outer Loop (Orbital Dynamics)**: Runs at the slower rate specified by `time_step` (typically 5 seconds). This loop handles orbital dynamics, subsystem operations, communications, power systems, and other slower-changing phenomena.

2. **Inner Loop (Attitude Dynamics)**: Runs at the faster rate specified by `time_step_attitude` (typically 0.1 seconds). This nested loop simulates attitude dynamics and control, which require higher temporal resolution due to the faster rotational motion of spacecraft.

This dual-loop architecture is essential because attitude dynamics typically evolve much more rapidly than orbital dynamics. For example, a spacecraft might complete a full rotation in minutes while its orbit might take hours or days to complete. Using a single timestep would either:

- Be too slow for accurate attitude simulation (if using the orbital timestep)
- Be computationally prohibitive for long-duration missions (if using the attitude timestep for everything)

In the `main_v3.m` file, you'll see this implemented as:

```
%% Time Loop (Outer Loop)
for k = 1:1:mission.true_time.num_time_steps
    % Update Time, Date, Storage, Solar System, etc.

    %% Attitude Dynamics Loop (Inner Loop)
    for k_attitude = 1:1:mission.true_time.num_time_steps_attitude
        % Update attitude, sensors, actuators, etc.
    end

    % Continue with other subsystems...
end
```

2.4.4.2 Storage Configuration

Similarly, storage configuration determines how frequently simulation data is saved:

```
%% Storage Configuration
init_data = [];
init_data.time_step_storage = 1;                          % Data storage interval
init_data.time_step_storage_attitude = 0.5;               % Attitude data storage
↪  interval
init_data.flag_visualize_SC_attitude_orbit_during_sim = 0; % Don't show attitude during
↪  sim
init_data.flag_realtime_plotting = 0;                     % Show mission data during sim
init_data.flag_save_plots = 1;                            % Save plots to disk
init_data.flag_save_video = 0;                            % Save animation to video
mission.storage = Storage(init_data, mission);
```

The storage configuration follows the same dual-timescale approach as the main simulation:

- `time_step_storage`: How often to save orbital and subsystem data (in seconds)
- `time_step_storage_attitude`: How often to save attitude data (in seconds)

Setting appropriate storage intervals is crucial for managing memory usage. Setting them too small can lead to memory issues with large simulations, while setting them too large might miss important transient behaviors.

The visualization flags control real-time display and post-simulation outputs:

- `flag_visualize_SC_attitude_orbit_during_sim`: Toggles 3D visualization during simulation
- `flag_realtime_plotting`: Enables data plots during simulation (can slow performance)
- `flag_save_plots`: Saves figures to disk after simulation completes
- `flag_save_video`: Records video of 3D visualization (memory-intensive)

2.4.5 Environment Configuration

Configure the stars catalog, solar system bodies, and target bodies:

```
%% Star Catalog Configuration
mission.true_stars = True_Stars(mission);
mission.true_stars.maximum_magnitude = 10;  % Maximum star magnitude to include

%% Solar System Configuration
init_data = [];
init_data.SS_body_names = ["Sun", "Earth", "Mars"];  % Relevant solar system bodies
mission.true_solar_system = True_Solar_System(init_data, mission);

%% Target Body Configuration
for i_target = 1:1:mission.num_target
    init_data = [];
    init_data.target_name = 'Bennu';        % Target asteroid name
    mission.true_target{i_target} = True_Target_SPICE(init_data, mission);
end
```

2.4.6 Ground Station Configuration

Ground stations are essential elements of spacecraft missions, providing the Earth-based communications infrastructure for telemetry, tracking, and command. In MuSCAT, ground stations are modeled with realistic visibility constraints, antenna properties, and communication capabilities.

2.4.6.1 Ground Station Fundamentals

The ground station model in MuSCAT accounts for:

- **Earth Rotation**: Ground stations rotate with Earth, affecting visibility periods
- **Antenna Characteristics**: Gain patterns, pointing capabilities, and frequency-dependent properties
- **Link Budget Calculations**: Realistic modeling of signal strength, noise, and data rate capabilities
- **Visibility Constraints**: Line-of-sight calculations based on Earth's position and rotation

To configure a ground station, first specify the number of antennas:

```
%% Ground Station Configuration
init_data = [];
init_data.num_GS_radio_antenna = 1;  % Number of ground station antennas
mission.true_ground_station = True_Ground_Station(init_data, mission);
```

Then, configure each antenna with appropriate communication parameters:

```
%% Ground Station Radio Antenna Configuration
for i_HW = 1:1:mission.true_ground_station.num_GS_radio_antenna
    init_data = [];

    % Basic configuration
    init_data.antenna_type = 'High Gain';
    init_data.mode_true_GS_radio_antenna_selector = 'RX';  % Receive mode
```

```
    % Link budget parameters
    init_data.antenna_gain = 90;            % [dB] Very high gain (DSN-like)
    init_data.noise_temperature = 100;      % [K] System noise temperature
    init_data.beamwidth = 0.1;              % [MHz] Receiver bandwidth
    init_data.energy_bit_required = 4.2;    % [dB] Required Eb/N0
    init_data.line_loss = 0;                % [dB] Transmission line losses
    init_data.coding_gain = 7.3;            % [dB] Error correction coding gain

    % Create the antenna object
    mission.true_GS_radio_antenna{i_HW} = True_GS_Radio_Antenna(init_data, mission,
    ↪  i_HW);
end
```

2.4.6.2 Ground Station Parameters

The key parameters affecting ground station performance include:

- **Antenna Gain**: Higher gain provides better signal strength but narrower beamwidth requiring more precise pointing. The value of 90 dB represents a large Deep Space Network (DSN) class antenna.
- **Noise Temperature**: Lower values (in Kelvin) improve signal-to-noise ratio. Typical values range from 20K (very cold, space-pointing receivers) to 290K (room temperature).
- **Beamwidth**: The receiver's frequency bandwidth in MHz. Narrower bandwidths reduce noise but limit data rate.
- **Eb/N0 Requirement**: Energy per bit to noise density ratio required for reliable communication. Higher values increase reliability but reduce achievable data rate.
- **Coding Gain**: The improvement in effective signal-to-noise ratio achieved through error correction coding.

2.4.6.3 Simulation Impact

During simulation, the ground station model:

1. Calculates its position based on Earth's rotation
2. Determines visibility to each spacecraft based on line-of-sight
3. Computes achievable data rates based on distance, antenna parameters, and pointing
4. Transfers data between spacecraft and ground when links are established

The ground station configuration directly affects communication windows and data throughput, which in turn influence mission planning, data collection strategies, and onboard storage requirements.

2.4.7 Spacecraft Configuration

2.4.7.1 Spacecraft Physical Properties

Configure the physical properties of each spacecraft, including shape, mass, and moment of inertia:

```
%% Spacecraft Body Configuration
i_SC = 1;  % First spacecraft

init_data = [];
init_data.i_SC = i_SC;

% Body shape model
init_data.shape_model{1} = [];
init_data.shape_model{1}.Vertices = [0 0 0;
    0.3 0 0;
    0.3 0 0.1;
    0 0 0.1;
    0 0.2 0;
    0.3 0.2 0;
    0.3 0.2 0.1;
    0 0.2 0.1]; % [m]
init_data.shape_model{1}.Faces = [1 2 3;
    1 4 3;
    2 3 7;
    2 6 7;
    3 4 8;
    3 7 8;
    1 4 8;
    1 5 8;
    1 2 6;
    1 5 6;
    5 6 7;
    5 8 7];

init_data.shape_model{1}.Face_reflectance_factor =
↪  0.6*ones(size(init_data.shape_model{1}.Faces,1),1);
init_data.shape_model{1}.type = 'cuboid';
init_data.shape_model{1}.mass = 11; % [kg] Dry mass

% Additional mass components
init_data.mass.supplement{1}.mass = 0.5; % [kg]
init_data.mass.supplement{1}.location = [0.1 0 0]; % [m]
init_data.mass.supplement{1}.MI_over_m = zeros(3,3); % [m^2]

init_data.mode_COM_selector = 'update';  % Compute Center of Mass dynamically
init_data.mode_MI_selector = 'update';   % Compute Moment of Inertia dynamically
```

2.4.7.2 Spacecraft Hardware Complement

When designing a spacecraft in MuSCAT, the hardware complement defines what components are present on the spacecraft and in what quantities. This configuration directly impacts how the

simulation behaves by determining:

- What hardware-specific loops will be executed during simulation
- How many instances of each component will be created and updated
- What capabilities the spacecraft will have during the mission
- The physical and performance characteristics of the spacecraft

The hardware complement is specified through the `num_hardware_exists` structure:

```
% Define hardware complement
init_data.num_hardware_exists.num_onboard_clock = 1;      % Timing system
init_data.num_hardware_exists.num_camera = 1;             % Imaging system
init_data.num_hardware_exists.num_solar_panel = 3;        % Power generation
init_data.num_hardware_exists.num_battery = 2;            % Energy storage
init_data.num_hardware_exists.num_onboard_memory = 2;     % Data storage
init_data.num_hardware_exists.num_sun_sensor = 6;         % Sun direction sensing
init_data.num_hardware_exists.num_star_tracker = 3;       % Star-based attitude
↪  determination
init_data.num_hardware_exists.num_imu = 1;                % Inertial measurement
init_data.num_hardware_exists.num_micro_thruster = 12;    % Small attitude control
↪  thrusters
init_data.num_hardware_exists.num_chemical_thruster = 1;  % Main propulsion
init_data.num_hardware_exists.num_reaction_wheel = 3;     % Momentum exchange devices
init_data.num_hardware_exists.num_communication_link = 2; % Data links (up/downlink)
init_data.num_hardware_exists.num_radio_antenna = 1;      % Communication hardware
init_data.num_hardware_exists.num_fuel_tank = 1;          % Propellant storage
init_data.num_hardware_exists.num_onboard_computer = 2;   % Computing resources
```

2.4.7.2.1 Simulation Impact Each number specified here has direct consequences for the simulation:

1. **Loop Execution**: In `main_v3.m`, the simulation iterates through each hardware type according to the specified count. For example, with 6 sun sensors:

```
for i_HW =
↪  1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_sun_sensor
    func_main_true_SC_sun_sensor(mission.true_SC{i_SC}.true_SC_sun_sensor{i_HW
    ↪  }, mission,
    ↪  i_SC);
end
```

2. **Resource Consumption**: Each hardware component consumes power and generates data, affecting the overall spacecraft power balance and data handling requirements.
3. **Redundancy and Failure Tolerance**: Multiple instances of critical components (e.g., computers, batteries) provide redundancy for fault tolerance.
4. **Capability Coverage**: The placement and orientation of sensors (like sun sensors or star trackers) determine the spacecraft's ability to determine attitude in different orientations.
5. **Control Authority**: The number and arrangement of reaction wheels and thrusters determine the spacecraft's control capabilities.

2.4.7.2.2 Selecting Appropriate Hardware When designing your mission, consider these questions when selecting hardware:

- What sensing capabilities are required for the mission objectives?
- What control authority is needed for attitude and orbital maneuvers?
- How much power generation and storage is needed?
- What data rates and storage capacities are required?
- What level of redundancy is appropriate for mission criticality?

After defining the hardware complement, you must create a `True_SC_Body` object that will encapsulate this information:

```
mission.true_SC{i_SC}.true_SC_body = True_SC_Body(init_data, mission);
```

This object becomes the foundation for all subsequent hardware initialization and will be used to calculate mass properties and other physical characteristics of the spacecraft.

2.4.7.3 Initial Position and Attitude

The initial state of the spacecraft—its position, velocity, and attitude—defines the starting point of the simulation and has profound impacts on mission trajectory, operations, and results. In MuSCAT, there are multiple methods to specify these initial conditions, each serving different purposes.

2.4.7.3.1 Position and Velocity Initialization There are three primary methods to specify the initial position and velocity of a spacecraft:

1. **SPICE-Based Initialization**: Using NASA's SPICE toolkit for high-fidelity ephemeris data
2. **Direct Specification**: Explicitly defining position and velocity vectors
3. **Orbital Elements**: Specifying Keplerian orbital elements

The SPICE-based method is shown below and is preferred for missions with predefined trajectories:

```
%% Initialize Spacecraft's Position and Velocity
init_data = [];
init_data.spice_filename =
↪  '../../MuSCAT_Supporting_Files/SC_data/YourMission/trajectory.bsp';
cspice_furnsh(init_data.spice_filename)

init_data.spice_name = '-110'; % SPICE ID for the spacecraft
init_data.SC_pos_vel = cspice_spkezr(init_data.spice_name, mission.true_time.date,
                                     'J2000', 'NONE', 'SUN');
init_data.position = init_data.SC_pos_vel(1:3)'; % [km]
init_data.velocity = init_data.SC_pos_vel(4:6)'; % [km/sec]
```

Alternatively, you can directly specify position and velocity:

```
init_data.position = [149598023, 0, 0]; % [km] Position in Sun-centered frame
init_data.velocity = [0, 29.78, 0];     % [km/sec] Velocity in Sun-centered frame
```

The dynamics mode selection determines how the position and velocity will evolve during simulation:

```
init_data.mode_true_SC_navigation_dynamics_selector = 'Absolute Dynamics';
```

MuSCAT currently supports two navigation dynamics modes:

- **Absolute Dynamics**: Full orbital dynamics with gravitational forces
- **SPICE**: Position/velocity updated directly from SPICE kernels

After configuration, create the navigation object:

```
mission.true_SC{i_SC}.true_SC_navigation = True_SC_Navigation(init_data, mission);
```

2.4.7.3.2 Attitude Initialization Spacecraft attitude defines the orientation in three-dimensional space. In MuSCAT, attitudes can be specified using various representations:

1. **Quaternions**: A four-parameter representation avoiding singularities
2. **Modified Rodrigues Parameters (MRP)**: A three-parameter representation
3. **Euler Angles**: Classical roll, pitch, yaw angles (not singularity-free)
4. **Direction Cosine Matrix (DCM)**: 3×3 rotation matrix

The example below uses Modified Rodrigues Parameters, then converts to quaternions:

```
%% Initialize Spacecraft's Attitude
init_data = [];
% Method 1: Starting with MRP
init_data.SC_MRP_init = [0.1 0.2 0.3]; % Modified Rodrigues Parameters
init_data.SC_omega_init = [0 0 0.001]; % [rad/sec] Angular velocity

% Convert MRP to quaternion
init_data.SC_e_init = init_data.SC_MRP_init/norm(init_data.SC_MRP_init);
init_data.SC_Phi_init = 4*atand(init_data.SC_MRP_init(1)/init_data.SC_e_init(1)); %
↪  [deg]
init_data.SC_beta_v_init = init_data.SC_e_init * sind(init_data.SC_Phi_init/2);
init_data.SC_beta_4_init = cosd(init_data.SC_Phi_init/2);

init_data.attitude = [init_data.SC_beta_v_init, init_data.SC_beta_4_init]; %
↪  [quaternion]
init_data.attitude = func_quaternion_properize(init_data.attitude); % [quaternion]
↪  properized
init_data.angular_velocity = init_data.SC_omega_init;
```

Alternatively, you can specify quaternions directly:

```
% Method 2: Direct quaternion specification
init_data.attitude = [0, 0, 0, 1]; % Identity quaternion (no rotation)
init_data.angular_velocity = [0, 0, 0]; % [rad/sec] No initial rotation
```

The attitude dynamics mode determines how the orientation will evolve:

```
init_data.mode_true_SC_attitude_dynamics_selector = 'Rigid';
```

Currently, MuSCAT only supports the 'Rigid' mode, which implements standard rigid body dynamics. After configuration, create the attitude dynamics and control object:

```
mission.true_SC{i_SC}.true_SC_adc = True_SC_ADC(init_data, mission);
```

2.4.7.3.3 Simulation Impact The initial state significantly influences the simulation:

- **Orbital Period and Evolution**: Initial position and velocity determine the spacecraft's orbit, affecting visibility of ground stations, targets, and available sunlight
- **Pointing Capabilities**: Initial attitude affects which sensors can view their targets and which solar panels receive sunlight
- **Thermal Conditions**: Orientation relative to the Sun impacts thermal conditions
- **Communication Opportunities**: Position influences when communication with Earth is possible
- **Scientific Observations**: Initial state determines when observation targets are visible

When implementing your mission, carefully consider the initial state based on mission objectives, launch conditions, and operational requirements.

2.4.8 Configuring Spacecraft Subsystems

Configure various spacecraft subsystems such as power, data handling, communications, and sensors:

2.4.8.1 Power Subsystem

```
%% Initialize Spacecraft's Power
init_data = [];
init_data.power_loss_rate = 0.05; % [float] 5% power loss in distribution and conversion
mission.true_SC{i_SC}.true_SC_power = True_SC_Power(init_data, mission);

%% Initialize Spacecraft's Solar Panels
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_solar_panel
    init_data = [];
    init_data.instantaneous_power_consumed = 0.01; % [W]
    init_data.instantaneous_data_rate_generated = (1e-3)*8; % [kbps]

    % Define solar panel geometry
    init_data.shape_model = [];
    init_data.shape_model.Vertices = [0 0 0; 0.2 0 0; 0.2 0 -0.6; 0 0 -0.6];
    init_data.shape_model.Faces = [1 2 3; 1 4 3];
```

```
    init_data.shape_model.Face_reflectance_factor_solar_cell_side = [0.01; 0.01];
    init_data.shape_model.Face_reflectance_factor_opposite_side = [0.5; 0.5];
    init_data.shape_model.Face_orientation_solar_cell_side = [0 -1 0];
    init_data.shape_model.type = 'cuboid';

    init_data.mass = 0.24; % [kg]
    init_data.type = 'passive_deployed';
    init_data.packing_fraction = 0.74; % Packing fraction of solar cells
    init_data.solar_cell_efficiency = 0.28; % Efficiency of each solar cell

    mission.true_SC{i_SC}.true_SC_solar_panel{i_HW} = True_SC_Solar_Panel(init_data,
    ↪  mission, i_SC, i_HW);
end

%% Initialize Spacecraft's Battery
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_battery
    init_data = [];
    init_data.maximum_capacity = 40; % [W hr]
    init_data.charging_efficiency = 0.96; % [float <= 1]
    init_data.discharging_efficiency = 0.96; % [float <= 1]
    init_data.instantaneous_power_consumed = 1e-4; % [W]
    init_data.instantaneous_data_rate_generated = (1e-3)*8; % [kbps]

    mission.true_SC{i_SC}.true_SC_battery{i_HW} = True_SC_Battery(init_data, mission,
    ↪  i_SC, i_HW);
end
```

2.4.8.2 Data Handling Subsystem

```
%% Initialize Spacecraft's Data Handling
init_data = [];
init_data.mode_true_SC_data_handling_selector = 'Generic';
mission.true_SC{i_SC}.true_SC_data_handling = True_SC_Data_Handling(init_data, mission);

%% Initialize Spacecraft's Onboard Memory
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_onboard_memory
    init_data = [];
    init_data.maximum_capacity = 1e6; % [kb]
    init_data.instantaneous_power_consumed = 1; % [W]
    init_data.instantaneous_data_rate_generated = (1e-3)*8; % [kbps]

    mission.true_SC{i_SC}.true_SC_onboard_memory{i_HW} =
    ↪  True_SC_Onboard_Memory(init_data, mission, i_SC, i_HW);
end
```

2.4.8.3 Communication Subsystem

```
%% Initialize Spacecraft's Radio Antenna
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_radio_antenna
    init_data = [];
    init_data.location = [0 1 0];   % [unit vector] antenna physical axis in Body frame
```

```
    init_data.orientation = [0 0 1]; % [unit vector] antenna pointing direction

    % Antenna parameters
    init_data.antenna_type = "dipole";
    init_data.antenna_gain = 28.1;          % [dB]
    init_data.antenna_frequency = 8450;     % [MHz]
    init_data.tx_line_loss = 1;             % [dB]
    init_data.noise_temperature = 100;      % [K]
    init_data.maximum_data_rate = 1000;     % [kbps]
    init_data.TX_power_consumed = 50;       % [W]
    init_data.RX_power_consumed = 25;       % [W]

    mission.true_SC{i_SC}.true_SC_radio_antenna{i_HW} = True_SC_Radio_Antenna(init_data,
    ↪  mission, i_SC, i_HW);
end

%% Spacecraft Communication Links Configuration
for i_HW =
↪  1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_communication_link
    init_data = [];

    if i_HW == 1
        % Downlink: Spacecraft to Earth
        init_data.TX_spacecraft = i_SC;
        init_data.TX_spacecraft_Radio_HW = 1;
        init_data.RX_spacecraft = 0;        % Ground Station
        init_data.RX_spacecraft_Radio_HW = 1;
        init_data.given_data_rate = 360;    % [kbps]
    else
        % Uplink: Earth to Spacecraft
        init_data.TX_spacecraft = 0;        % Ground Station
        init_data.TX_spacecraft_Radio_HW = 1;
        init_data.RX_spacecraft = i_SC;
        init_data.RX_spacecraft_Radio_HW = 1;
        init_data.given_data_rate = 0;      % [kbps]
    end

    mission.true_SC{i_SC}.true_SC_communication_link{i_HW} =
    ↪  True_SC_Communication_Link(init_data, mission, i_SC, i_HW);
end
```

2.4.8.4 Sensors and Actuators

Configure sensors (cameras, star trackers, IMUs) and actuators (thrusters, reaction wheels):

```
%% Initialize Spacecraft's Cameras
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_camera
    init_data = [];
    init_data.instantaneous_power_consumed = 10; % [W]
    init_data.mode_true_SC_camera_selector = 'Simple';
    init_data.measurement_wait_time = 60; % [sec]
```

```
    init_data.location = [0.3 0.1 0.05]; % [m]
    init_data.orientation = [1 0 0]; % [unit vector]
    init_data.orientation_up = [0 0 1]; % [unit vector]

    init_data.resolution = [512 512];  % [x y] pixel
    init_data.field_of_view = 10; % [deg]
    init_data.flag_show_camera_plot = 0;
    init_data.flag_show_stars = 1;

    mission.true_SC{i_SC}.true_SC_camera{i_HW} = True_SC_Camera(init_data, mission,
    ↪  i_SC, i_HW);
end

%% Initialize Spacecraft's Star Trackers
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_star_tracker
    init_data = [];
    init_data.instantaneous_power_consumed = 1.5; % [W]
    init_data.mode_true_SC_star_tracker_selector = 'Simple with Sun outside FOV';
    init_data.measurement_wait_time = 0.1; % [sec]
    init_data.measurement_noise = 2e-4; % [rad]
    init_data.field_of_view = 90; % [deg]

    init_data.location = [0.3 0.15 0.05]; % [m]
    init_data.orientation = [1 0 0]; % [unit vector]

    mission.true_SC{i_SC}.true_SC_star_tracker{i_HW} = True_SC_Star_Tracker(init_data,
    ↪  mission, i_SC, i_HW);
end

%% Initialize Spacecraft's Reaction Wheels
for i_HW = 1:1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_reaction_wheel
    init_data = [];
    init_data.location = [0,0,0];
    init_data.radius = (43e-3)/2; % [m]
    init_data.mass = 0.137; % [kg]
    init_data.max_angular_velocity = 6500*2*pi/60; % [rad/s]
    init_data.max_torque = 3.2*1e-3; % [Nm]

    % Set orientation based on configuration (3 or 4 wheels)
    if mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_reaction_wheel == 3
        % 3-wheel configuration along principal axes
        switch i_HW
            case 1
                init_data.orientation = [1, 0, 0];  % X-axis
            case 2
                init_data.orientation = [0, 1, 0];  % Y-axis
            case 3
                init_data.orientation = [0, 0, 1];  % Z-axis
        end
    end
```

```
    mission.true_SC{i_SC}.true_SC_reaction_wheel{i_HW} =
    ↪  True_SC_Reaction_Wheel(init_data, mission, i_SC, i_HW);
end
```

2.4.9 Flight Software Configuration

The final step is to configure the flight software components that control the spacecraft. Each software component serves a specific purpose in the autonomous operation of the spacecraft.

2.4.9.1 Executive Software

The Executive software was described in detail in Section 4.6.1. To briefly recap, it is responsible for determining the spacecraft's operating mode and coordinating subsystem activities. When creating a custom mission, you should:

1. Define the operating modes relevant to your mission in your mission configuration file
2. Create a custom implementation function in `Software_SC_Executive.m`
3. Set the `mode_software_SC_executive_selector` parameter to your custom function name

See Section 4.6.1 for implementation examples and code samples.

2.4.9.2 Attitude & Orbit Estimation

These components determine the spacecraft's current attitude and position:

```
%% Spacecraft Software: Attitude Estimation Configuration
init_data = [];
init_data.mode_software_SC_estimate_attitude_selector = 'Truth';
mission.true_SC{i_SC}.software_SC_estimate_attitude =
    Software_SC_Estimate_Attitude(init_data, mission, i_SC);

%% Spacecraft Software: Orbit Estimation Configuration
init_data = [];
init_data.mode_software_SC_estimate_orbit_selector = 'TruthWithErrorGrowth';
mission.true_SC{i_SC}.software_SC_estimate_orbit =
    Software_SC_Estimate_Orbit(init_data, mission, i_SC);
```

For initial mission development, you can use the 'Truth' estimators that simply read the actual spacecraft state. For more realistic simulations, you can implement custom estimators that process sensor data.

2.4.9.3 Orbit & Attitude Control

The control components determine how to move the spacecraft to desired states:

```
%% Spacecraft Software: Orbit Control Configuration
init_data = [];
init_data.max_time_before_control = 0.5*60*60 + 900;  % 45 minutes
init_data.mode_software_SC_control_orbit_selector = 'YourMission';
mission.true_SC{i_SC}.software_SC_control_orbit =
```

```
    Software_SC_Control_Orbit(init_data, mission, i_SC);

%% Spacecraft Software: Attitude Control Configuration
init_data = [];
init_data.mode_software_SC_control_attitude_selector = 'YourMission Control';
init_data.control_gain = [1 0.2];    % Controller gain parameters
mission.true_SC{i_SC}.software_SC_control_attitude =
    Software_SC_Control_Attitude(init_data, mission, i_SC);
```

You'll need to implement custom control algorithms that match the mode selector names provided here.

2.4.9.4 Communication & Resource Management

These components manage communication opportunities and onboard resources:

```
%% Spacecraft Software: Communication Configuration
init_data = [];
init_data.mode_software_SC_communication_selector = 'YourMission';
init_data.attitude_error_threshold_deg = 1;  % Max attitude error for comms
mission.true_SC{i_SC}.software_SC_communication =
    Software_SC_Communication(init_data, mission, i_SC);
```

The communication software determines when to establish links with ground stations and manages the flow of data to and from the spacecraft.

2.4.10 Final Initialization and Simulation Execution

After configuring all subsystems, several final initialization steps must be performed before running the simulation. These steps ensure that all spacecraft properties are properly calculated and that the simulation environment is ready:

```
%% Final Things to Do Before Running the Simulation

% Initialize mass properties
func_update_SC_body_total_mass_COM_MI(mission.true_SC{i_SC}.true_SC_body);

% Initialize power and data handling storage
func_initialize_store_HW_power_consumed_generated(mission.true_SC{i_SC}.true_SC_power,
↪  mission);
func_initialize_store_HW_data_generated_removed(mission.true_SC{i_SC}.true_SC_data_hand
↪  ling,
↪  mission);

% Clean up temporary variables
clear init_data i_SC i_HW i_target

% Visualize the spacecraft in 3D
func_visualize_SC(mission.storage, mission, true);

% Save initial state
```

```
save([mission.storage.output_folder, 'all_data.mat'], '-v7.3')

%% Execute Main File
run main_v3.m

%% Save All Data and Generate Plots
save([mission.storage.output_folder, 'all_data.mat'], '-v7.3')
func_visualize_simulation_data(mission.storage, mission);
```

The initialization process includes calculating the total mass, center of mass, and moment of inertia for the spacecraft body, setting up storage for power and data handling metrics, and cleaning up temporary variables used during configuration. The spacecraft is then visualized in 3D to provide a visual confirmation of the configuration.

After initialization, the main simulation file (`main_v3.m`) is executed, which runs the simulation according to the configured parameters. Once the simulation is complete, all data is saved to the specified output folder, and visualization functions are called to generate plots of the simulation results.

2.5 Understanding SPICE Integration

MuSCAT uses NASA's SPICE toolkit for accurate ephemeris calculations, providing precise positions and velocities of solar system bodies and spacecraft. This section explains how SPICE is integrated into MuSCAT and how to use it effectively for your mission.

2.5.1 What is SPICE?

SPICE (Spacecraft, Planet, Instrument, C-matrix, Events) is a NASA toolkit developed by the Navigation and Ancillary Information Facility (NAIF) at JPL. It provides essential information for mission design, planning, and science data analysis for space science missions. SPICE consists of data files called "kernels" and a suite of software tools that read and interpret these kernels to provide spacecraft and planetary ephemerides, instrument pointing information, time conversions, and other critical mission data.

2.5.2 Why MuSCAT Uses SPICE

MuSCAT integrates SPICE for several critical reasons:

- **Accurate Ephemerides**: SPICE provides high-precision positions and velocities of solar system bodies based on the latest planetary ephemerides (like DE430 or DE440), ensuring that simulations accurately represent the gravitational influences of planets and other bodies.
- **Realistic Trajectory Modeling**: Enables detailed simulation of spacecraft trajectories, including complex orbital maneuvers and gravity assists, which is essential for mission planning and validation.
- **Time System Conversions**: Provides robust time system conversions between various time standards (UTC, TDB, TDT, etc.), which is crucial for correlating events across different reference frames and mission phases.

- **Reference Frame Transformations**: Handles complex coordinate transformations between different reference frames (J2000, ICRF, body-fixed, etc.), allowing for accurate representation of spacecraft orientation and position relative to various celestial bodies.
- **Industry Standard**: SPICE is the de facto standard for deep space mission design at NASA and other space agencies, making MuSCAT compatible with industry practices and facilitating data exchange with other mission analysis tools.
- **Validation and Verification**: Using SPICE allows MuSCAT simulations to be validated against real mission data and other industry-standard tools, increasing confidence in simulation results.

2.5.3 SPICE Navigation Dynamics Mode

When using the 'SPICE' navigation dynamics mode in MuSCAT, the spacecraft's position and velocity are directly obtained from SPICE kernels rather than being propagated using orbital dynamics equations. This approach:

- Ensures consistency with mission design trajectories
- Allows for simulation of complex trajectories that may be difficult to model with standard orbital dynamics
- Provides a ground truth reference for validating other dynamics models
- Simplifies the simulation of multi-body trajectories (e.g., lunar or interplanetary missions)

2.5.4 SPICE Kernel Types Used in MuSCAT

MuSCAT uses several types of SPICE kernels:

- **SPK (Spacecraft and Planet Kernels)**: Contain ephemeris data for spacecraft and celestial bodies. These files define the positions and velocities of objects as functions of time.
- **LSK (Leapseconds Kernels)**: Provide information about leap seconds, allowing accurate conversion between different time systems.
- **PCK (Planetary Constants Kernels)**: Contain physical and cartographic constants for planets and satellites, such as size, shape, and orientation.
- **FK (Frame Kernels)**: Define reference frames needed for coordinate transformations.
- **IK (Instrument Kernels)**: Contain geometric and operational parameters of instruments (used for more advanced simulations).

2.5.5 SPICE Setup in MuSCAT

To use SPICE in your MuSCAT mission:

1. Ensure the SPICE toolkit (MICE for MATLAB) is correctly installed in `MuSCAT_Supporting_Files/SPICE/`

2. Collect necessary SPICE kernels for your mission:
   - Leapseconds kernel (LSK) for time conversions (e.g., `naif0012.tls`)
   - Planetary ephemeris kernel (SPK) for solar system bodies (e.g., `de430.bsp`)
   - Spacecraft trajectory kernel (SPK) for your spacecraft (e.g., `YourMission.bsp`)
   - Frame kernel (FK) if using custom reference frames
3. Load the required kernels in your mission file:

```
% Load necessary SPICE kernels
cspice_furnsh('../../MuSCAT_Supporting_Files/SPICE/naif0012.tls');  %
↪  Leapseconds kernel
cspice_furnsh('../../MuSCAT_Supporting_Files/SPICE/de430.bsp');     % Planetary
↪  ephemeris
init_data.spice_filename =
↪  '../../MuSCAT_Supporting_Files/SC_data/YourMission/trajectory.bsp';
cspice_furnsh(init_data.spice_filename);  % Your spacecraft trajectory
```

4. Retrieve position and velocity data:

```
init_data.spice_name = '-123';  % SPICE ID for your spacecraft (negative
↪  integer)
init_data.SC_pos_vel = cspice_spkezr(init_data.spice_name,
↪  mission.true_time.date,
                                     'J2000', 'NONE', 'SUN');
init_data.position = init_data.SC_pos_vel(1:3)'; % [km]
init_data.velocity = init_data.SC_pos_vel(4:6)'; % [km/sec]
```

5. Specify SPICE mode in navigation dynamics (if using SPICE for position/velocity updates):

```
init_data.mode_true_SC_navigation_dynamics_selector = 'SPICE';
```

2.5.6 Creating SPICE Kernels for Your Mission

You can create your own SPICE SPK kernel for a custom spacecraft trajectory:

1. Generate a trajectory using your preferred method (analytical, numerical integration, etc.)
2. Convert the trajectory to SPICE format using SPICE utilities:
   - For MATLAB, use MICE functions like `cspice_spkw08` or `cspice_spkw09`
   - For standalone processing, use the SPICE utility `mkspk`
3. Format the trajectory data as a time series of position and velocity vectors
4. Assign a NAIF ID for your spacecraft (usually a negative integer)
5. Create the SPK file with appropriate metadata (reference frame, time coverage, etc.)
6. Place your SPK file in the appropriate directory: `MuSCAT_Supporting_Files/SC_data/YourMission/`

2.5.7 SPICE Integration with Main Simulation Loop

During simulation, SPICE integration occurs at several key points:

1. **Initialization**: SPICE kernels are loaded and initial states are obtained
2. **Time Updates**: Each simulation step updates the current epoch used for SPICE queries
3. **Navigation Updates**: When using SPICE mode, the `func_main_true_SC_navigation` function retrieves updated position/velocity directly from SPICE
4. **Environmental Calculations**: SPICE may be used to determine positions of other bodies for gravity calculations
5. **Ground Station Visibility**: SPICE data helps determine line-of-sight between spacecraft and ground stations

This integration provides a seamless interface between MuSCAT's simulation environment and the high-fidelity ephemeris data provided by SPICE, ensuring accurate representation of spacecraft trajectories and celestial body positions throughout the simulation.

2.6 Creating Custom Flight Software

Flight software defines the autonomous behavior of your spacecraft. In MuSCAT, flight software is implemented as MATLAB classes in the `Software_SC` directory. This section explains how to create custom flight software for your mission.

2.6.1 Executive Software

The Executive software was described in detail in Section 4.6.1. To briefly recap, it is responsible for determining the spacecraft's operating mode and coordinating subsystem activities. When creating a custom mission, you should:

1. Define the operating modes relevant to your mission in your mission configuration file
2. Create a custom implementation function in `Software_SC_Executive.m`
3. Set the `mode_software_SC_executive_selector` parameter to your custom function name

See Section 4.6.1 for implementation examples and code samples.

2.6.2 Attitude & Orbit Controllers

Similarly, you can create custom orbit and attitude controllers for your mission:

1. Add your custom controller implementation to the appropriate file (`Software_SC_Control_Orbit.m` or `Software_SC_Control_Attitude.m`)
2. Implement the control logic specific to your mission requirements
3. Reference your controller in the mission file:

```
init_data.mode_software_SC_control_orbit_selector = 'YourMission';
mission.true_SC{i_SC}.software_SC_control_orbit =
↪  Software_SC_Control_Orbit(init_data, mission, i_SC);
```

2.7 Simulation Execution and Analysis

Once your mission configuration is complete, the simulation is executed by running the `main_v3.m` script. This script contains the main time loop that updates all components of the mission at each time step.

2.7.1 Main Simulation Loop

The main simulation loop in `main_v3.m` proceeds as follows:

1. Update simulation time and date
2. Update solar system and target bodies
3. For each spacecraft:
    - Update environmental effects (solar radiation pressure, gravity gradient)
    - Update spacecraft body, position, and velocity
    - Update onboard clock and computer systems
    - Execute the Executive software to determine operating mode
    - Update all sensors, actuators, and subsystems
    - Run the attitude dynamics loop at a higher frequency
    - Update communication systems
    - Update power and data handling systems
4. Update ground station systems
5. Save data and update visualizations

2.7.2 Visualizing Results

MuSCAT provides comprehensive visualization tools to analyze mission performance:

- `func_visualize_SC`: Creates a 3D visualization of the spacecraft
- `func_visualize_simulation_data`: Generates plots for various subsystems
- Real-time visualization can be enabled with `flag_realtime_plotting = 1`

Key visualizations include:

- Spacecraft configuration and ConOps
- Orbit and attitude dynamics
- Power generation and consumption
- Data handling and communication performance
- Sensor and actuator performance

2.8 Advanced Topics

2.8.1 Creating Custom Hardware Components

If the built-in hardware models don't meet your needs, you can create custom components. This should be done without modifying the core MuSCAT files, following the extension patterns described earlier.

2.8.1.1 Custom Hardware Component Development Steps

1. Create a new MATLAB class file in your mission-specific directory that inherits from the appropriate parent class:

```
classdef Your_Custom_Sensor < True_SC_Sensor
    % Your custom sensor implementation
    properties
        % Custom properties specific to your sensor
        custom_parameter_1
        custom_parameter_2
    end

    methods
        % Constructor
        function obj = Your_Custom_Sensor(init_data, mission, i_SC, i_HW)
            % Call parent constructor
            obj@True_SC_Sensor(init_data, mission, i_SC, i_HW);

            % Initialize custom properties
            obj.custom_parameter_1 = init_data.custom_parameter_1;
            obj.custom_parameter_2 = init_data.custom_parameter_2;
        end
    end
end
```

2. Create a corresponding `func_main_your_custom_sensor` function that follows the standard MuSCAT update pattern:

```
function func_main_your_custom_sensor(obj, mission, i_SC, i_HW)
    % Update the custom sensor state based on current mission state

    % Your custom update logic here
    % ...

    % Store data if needed
    func_store_data(obj, mission);
end
```

3. Add your component to the simulation loop by extending the appropriate hardware update section in a copy of `main_v3.m` that you create for your mission:

```
%% Custom Hardware Update
for i_SC = 1:1:mission.num_SC
```

```
        for i_HW = 1:1:mission.true_SC{i_SC}.num_your_custom_sensor
            func_main_your_custom_sensor(mission.true_SC{i_SC}.your_custom_sensor{
            ↪ i_HW},
                                        mission, i_SC, i_HW);
        end
    end
```

4. In your mission file, instantiate your custom hardware:

```
% Add custom hardware to the hardware complement
init_data.num_hardware_exists.num_your_custom_sensor = 2;

% Create the custom sensor objects
for i_HW = 1:1:init_data.num_hardware_exists.num_your_custom_sensor
    init_data = [];
    init_data.custom_parameter_1 = value1;
    init_data.custom_parameter_2 = value2;

    mission.true_SC{i_SC}.your_custom_sensor{i_HW} =
        Your_Custom_Sensor(init_data, mission, i_SC, i_HW);
end
```

2.8.1.2 Integration with MuSCAT's Core Features

To ensure your custom component works seamlessly with MuSCAT:

- **Power Subsystem Integration**: Implement proper power consumption tracking by setting `instantaneous_power_consumed` and updating it during operation.
- **Data Handling Integration**: Set `instantaneous_data_rate_generated` to track data generation and ensure it's properly routed through the data handling subsystem.
- **Visualization Support**: Implement `func_visualize` methods if your component has visual representation.
- **Error Handling**: Include appropriate error checks and failure modes to enhance simulation realism.
- **Documentation**: Thoroughly document your custom component's interfaces, parameters, and behaviors.

By creating custom components in this way, you maintain compatibility with future MuSCAT updates while extending its capabilities to meet your specific mission requirements.

2.8.2 Multi-Spacecraft Missions

To simulate missions with multiple spacecraft:

1. Set `mission.num_SC` to the desired number of spacecraft
2. Configure each spacecraft using a loop over `i_SC`
3. Implement inter-spacecraft communication using communication links

2.8.3 Monte Carlo Simulations

For uncertainty analysis, you can run Monte Carlo simulations:

1. Create a wrapper script that calls your mission file multiple times
2. Vary key parameters according to your uncertainty model
3. Collect and analyze the results across all simulations

2.9 Conclusion

MuSCAT provides a flexible, modular framework for simulating complex spacecraft missions. By following the steps outlined in this chapter, you can implement your own custom missions, from initial concept to detailed simulation and analysis.

The implementation process involves several key steps:

1. **Mission Definition**: Set up the basic parameters like mission name, number of spacecraft, and target bodies.
2. **Time and Storage Configuration**: Configure the dual-loop time steps to properly handle both orbital and attitude dynamics at appropriate temporal resolutions.
3. **Environment Setup**: Configure the star catalog, solar system bodies, target bodies, and ground stations that form the operational context for your mission.
4. **Spacecraft Configuration**: Define the physical properties, hardware complement, initial position, and attitude for each spacecraft in your mission.
5. **Subsystem Configuration**: Set up power, data handling, communications, sensors, and actuators with the specific properties required for your mission.
6. **Flight Software Implementation**: Configure the executive, estimation, and control algorithms that will govern the autonomous behavior of your spacecraft.
7. **Simulation Execution**: Run the main simulation loop to simulate your mission over the specified time period.
8. **Results Analysis**: Analyze the simulation results using MuSCAT's visualization tools to evaluate mission performance.

MuSCAT's modular architecture allows for extensive customization at every level. You can:

- Create new hardware components by implementing additional classes in the appropriate directories
- Develop custom flight software algorithms to implement novel autonomous behaviors
- Simulate complex multi-spacecraft missions with varied objectives and capabilities
- Perform trade studies by varying parameters and analyzing the impacts on mission performance
- Validate mission concepts and requirements before detailed design

The object-oriented nature of MuSCAT means that each component is self-contained with well-defined interfaces, making it straightforward to extend the functionality without disrupting existing capabilities. This modularity also facilitates incremental development, allowing you to start with a simple mission model and progressively add complexity as your understanding evolves.

By leveraging NASA's SPICE toolkit for ephemeris calculations, MuSCAT ensures high-fidelity position and velocity data for spacecraft and celestial bodies, enabling realistic simulation of mission scenarios in the complex dynamical environment of space.

Whether you're designing a simple Earth-orbiting satellite or a complex interplanetary mission with multiple spacecraft, MuSCAT provides the tools and framework to model, simulate, and evaluate your mission concept efficiently and effectively.

2.10 Troubleshooting and Best Practices

2.10.1 Common Issues and Solutions

When implementing missions in MuSCAT, you may encounter several common issues. This section provides guidance on identifying and resolving these problems:

- **Memory Errors**: For large simulations with many time steps or spacecraft:
  - Increase the storage intervals (`time_step_storage` and `time_step_storage_attitude`)
  - Use the `clear` command strategically to free memory during long simulations
  - Consider breaking the simulation into multiple segments and combining results afterward
- **SPICE-Related Errors**: When working with SPICE:
  - Ensure MICE is properly installed and on the MATLAB path
  - Verify kernel coverage spans your entire simulation time period
  - Check for NAIF ID conflicts between different objects
  - Use `cspice_furnsh` with absolute paths to avoid path-related issues
- **Simulation Instability**: If your simulation becomes unstable:
  - Reduce the time step, especially for attitude dynamics
  - Check for physical inconsistencies (e.g., mismatched units, unrealistic parameters)
  - Verify that actuator limitations (e.g., maximum torque) are realistic
  - Implement controllers with appropriate stability margins
- **Missing Data in Results**: If expected data is missing:
  - Verify that storage flags are enabled for relevant components
  - Check that sensor/actuator update times align with storage times
  - Ensure `func_store_data` is called in all custom components

2.10.2 Performance Optimization

For complex missions or Monte Carlo simulations, performance optimization becomes critical:

1. **Time Step Optimization**:
   - Use the largest time step that maintains accuracy for your specific mission
   - Consider variable time stepping for different mission phases
   - Balance the attitude time step with the dynamics of your spacecraft
2. **Hardware Selection**:
   - Only include hardware components necessary for your mission
   - Use a reasonable number of sensors/actuators (e.g., 4-6 sun sensors rather than dozens)
   - Group hardware with similar functions when possible
3. **Vectorization**:
   - Rewrite custom component code to use MATLAB's vectorized operations
   - Avoid loops when matrix/vector operations can accomplish the same task
   - Pre-allocate arrays for storing time-series data
4. **Visualization**:
   - Disable real-time visualization during long simulations
   - Generate plots after the simulation completes rather than during execution
   - Consider lower-resolution shape models for faster 3D rendering

2.10.3 Units and Conventions

MuSCAT uses a consistent set of units and conventions throughout the codebase:

- **Time**: Seconds (s) for simulation time; UTC for absolute dates
- **Length**: Meters (m) for spacecraft dimensions; Kilometers (km) for orbital distances
- **Mass**: Kilograms (kg)
- **Angles**: Radians (rad) internally
- **Angular Velocity**: Radians per second (rad/s)
- **Force**: Newtons (N)
- **Torque**: Newton-meters (N·m)
- **Power**: Watts (W)
- **Energy**: Watt-hours (W·hr) for batteries
- **Data**: Kilobits (kb) for storage; Kilobits per second (kbps) for rates

- **Reference Frames**:
  - J2000 for inertial references
  - Body-fixed frames for spacecraft-relative quantities

Consistent use of these units is essential for correct simulation behavior.

2.10.4 Glossary of Terms

- **ADC**: Attitude Determination and Control
- **ConOps**: Concept of Operations
- **DCM**: Direction Cosine Matrix
- **DSN**: Deep Space Network
- **Eb/N0**: Energy per bit to noise density ratio
- **FK**: Frame Kernel (SPICE)
- **IMU**: Inertial Measurement Unit
- **LSK**: Leapseconds Kernel (SPICE)
- **MICE**: MATLAB Interface to SPICE
- **MRP**: Modified Rodrigues Parameters
- **NAIF**: Navigation and Ancillary Information Facility
- **PCK**: Planetary Constants Kernel (SPICE)
- **SPK**: Spacecraft and Planet Kernel (SPICE)
- **SRP**: Solar Radiation Pressure

2.10.5 Known Limitations

As with any simulation framework, MuSCAT has some known limitations:

- **Physical Fidelity**: Some physical effects are simplified or not modeled:
  - Thermal effects on spacecraft materials and instruments
  - Detailed propellant slosh dynamics
  - Advanced radiation effects on electronics
  - Detailed structural flexibility
- **Scalability**: Performance may degrade with very complex missions:
  - Formations with many (¿10) spacecraft
  - Very long duration missions (¿1 year) at high temporal resolution

- **Navigation Models**: Some specialized navigation techniques lack detailed models:
  - Optical navigation using small bodies
  - Inter-spacecraft relative navigation
  - GPS-like navigation in cislunar space
- **Environment Models**: Some environmental effects have simplified models:
  - Higher-order gravitational terms
  - Atmospheric drag variation with solar activity
  - Detailed magnetic field modeling

Users should be aware of these limitations when interpreting simulation results and consider supplementing MuSCAT with specialized tools for detailed analysis of these effects when necessary.

2.11 Practical Examples and Design Patterns

This section provides concrete examples of common tasks and design patterns to help you implement your missions effectively.

2.11.1 Common Mode Implementation Patterns

The following examples illustrate typical implementations for common spacecraft operating modes:

2.11.1.1 Sun-Safe Mode

This mode orients the spacecraft to maximize solar panel exposure while keeping sensitive instruments safe:

```
% In your Executive implementation function
if strcmp(obj.this_sc_mode, 'Sun-Safe')
    % Get Sun direction in spacecraft body frame
    sun_body = mission.true_SC{i_SC}.true_SC_navigation.sun_direction_body;

    % Set attitude target to align solar panels with Sun
    % Assuming Z-axis is normal to solar panels
    target_quaternion = func_quaternion_from_two_vectors([0 0 1], sun_body);

    % Point solar panels at Sun with moderate rotation rate
    mission.true_SC{i_SC}.software_SC_control_attitude.target_attitude =
    ↪  target_quaternion;
    mission.true_SC{i_SC}.software_SC_control_attitude.target_rate = [0 0 0];

    % Disable components to save power
    mission.true_SC{i_SC}.true_SC_camera{1}.flag_executive = 0;
    mission.true_SC{i_SC}.true_SC_micro_thruster{1}.flag_executive = 0;

    % Enable power-critical components
    mission.true_SC{i_SC}.software_SC_estimate_attitude.flag_executive = 1;
    mission.true_SC{i_SC}.software_SC_control_attitude.flag_executive = 1;
end
```

2.11.1.2 Science Observation Mode

This mode points instruments at a target of interest:

```
% In your Executive implementation function
if strcmp(obj.this_sc_mode, 'Science Observation')
    % Get target direction in inertial frame
    target_inertial =
    ↪  mission.true_SC{i_SC}.true_SC_navigation.target_direction_inertial;

    % Convert to required quaternion (assuming camera points along X-axis)
    target_quaternion = func_quaternion_from_two_vectors([1 0 0], target_inertial);

    % Point instrument and stabilize
    mission.true_SC{i_SC}.software_SC_control_attitude.target_attitude =
    ↪  target_quaternion;
    mission.true_SC{i_SC}.software_SC_control_attitude.target_rate = [0 0 0];

    % Enable science instruments
    mission.true_SC{i_SC}.true_SC_camera{1}.flag_executive = 1;

    % Log start of observation if mode just activated
    if ~strcmp(obj.data.previous_mode, 'Science Observation')
        disp(['Science observation started at t = ', ...
              num2str(mission.true_time.time), ' seconds']);
        obj.data.observation_start_time = mission.true_time.time;
    end
end
```

2.11.2 Implementing Common Mission Types

The following patterns demonstrate implementations for common mission types:

2.11.2.1 Earth-Orbiting Remote Sensing Satellite

For an Earth-observing satellite, consider this implementation pattern:

```
%% Mission Definition
mission = [];
mission.name = 'EarthObsSat';
mission.num_SC = 1;
mission.num_target = 1;  % Earth as target
mission.frame = 'Absolute';

%% Initialize Spacecraft's Position and Velocity
init_data = [];
% Sun-synchronous orbit at 700 km altitude
orbit_altitude = 700;  % [km]
earth_radius = 6378.1;  % [km]
orbit_radius = earth_radius + orbit_altitude;
orbit_velocity = sqrt(398600.4415 / orbit_radius);  % [km/s]
```

```
% Position at ascending node
init_data.position = [0, -orbit_radius, 0];  % [km]
init_data.velocity = [orbit_velocity, 0, 0];  % [km/s]
init_data.mode_true_SC_navigation_dynamics_selector = 'Absolute Dynamics';

% Key operational modes
init_data.sc_modes = {'Nadir Pointing', 'Target Imaging',
                      'Data Downlink', 'Safe Mode'};

% Payload-specific hardware
init_data.num_hardware_exists.num_camera = 2;  % Multispectral + Panchromatic
init_data.num_hardware_exists.num_solar_panel = 2;  % Two deployable panels
init_data.num_hardware_exists.num_star_tracker = 2;  % Redundant attitude determination
init_data.num_hardware_exists.num_reaction_wheel = 4;  % 4-wheel configuration
init_data.num_hardware_exists.num_magnetorquer = 3;  % For wheel desaturation
```

2.11.2.2 Interplanetary Science Mission

For an interplanetary probe, use this implementation pattern:

```
%% Mission Definition
mission = [];
mission.name = 'MarsMission';
mission.num_SC = 1;
mission.num_target = 1;  % Mars as target
mission.frame = 'Absolute';

%% SPICE-Based Trajectory
init_data = [];
init_data.spice_filename =
↪  '../../MuSCAT_Supporting_Files/SC_data/MarsMission/trajectory.bsp';
cspice_furnsh(init_data.spice_filename);
cspice_furnsh('../../MuSCAT_Supporting_Files/SPICE/de430.bsp');  % Planetary ephemeris

init_data.spice_name = '-123';  % SPICE ID for the spacecraft
init_data.SC_pos_vel = cspice_spkezr(init_data.spice_name, mission.true_time.date,
                                     'J2000', 'NONE', 'SUN');
init_data.position = init_data.SC_pos_vel(1:3)';  % [km]
init_data.velocity = init_data.SC_pos_vel(4:6)';  % [km/sec]
init_data.mode_true_SC_navigation_dynamics_selector = 'SPICE';

% Mission-specific modes
init_data.sc_modes = {'Cruise', 'TCM', 'Science',
                      'Mars Orbit Insertion', 'Communication', 'Safe Mode'};

% Deep space hardware
init_data.num_hardware_exists.num_high_gain_antenna = 1;  % For long-distance comms
init_data.num_hardware_exists.num_camera = 3;  % Navigation + Science instruments
init_data.num_hardware_exists.num_chemical_thruster = 1;  % Main propulsion
init_data.num_hardware_exists.num_reaction_wheel = 4;  % Attitude control
```

2.11.3 Common Attitude Control Implementations

The following examples show implementations for common attitude control laws:

2.11.3.1 PD Controller

A simple Proportional-Derivative controller for attitude:

```
function obj = func_software_SC_control_attitude_PD(obj, mission, i_SC)
    % Get current attitude error quaternion
    att_error_q = func_quaternion_error(mission.true_SC{i_SC}.software_SC_estimate_atti⌋
    ↪  tude.attitude,
    ↪  ...
                                    obj.target_attitude);

    % Convert quaternion error to angle-axis representation
    [error_axis, error_angle] = func_quaternion_to_axis_angle(att_error_q);

    % Calculate error vectors
    att_error_vec = error_axis * error_angle;
    rate_error = mission.true_SC{i_SC}.software_SC_estimate_attitude.angular_velocity -
    ↪  obj.target_rate;

    % PD controller with gains
    Kp = obj.control_gain(1);  % Proportional gain
    Kd = obj.control_gain(2);  % Derivative gain

    % Calculate desired control torque
    desired_torque = -Kp * att_error_vec - Kd * rate_error;

    % Apply control to reaction wheels
    for i_wheel =
    ↪  1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_reaction_wheel
        % Get wheel axis
        wheel_axis = mission.true_SC{i_SC}.true_SC_reaction_wheel{i_wheel}.orientation;

        % Project desired torque onto wheel axis
        wheel_torque = dot(desired_torque, wheel_axis) * wheel_axis;

        % Set wheel torque (with limits)
        max_torque = mission.true_SC{i_SC}.true_SC_reaction_wheel{i_wheel}.max_torque;
        commanded_torque = min(max_torque, max(wheel_torque, -max_torque));

        mission.true_SC{i_SC}.true_SC_reaction_wheel{i_wheel}.input_torque =
        ↪  commanded_torque;
    end
end
```

2.11.4 Custom Hardware Example: Laser Rangefinder

Below is an example of implementing a custom laser rangefinder for proximity operations:

```
classdef Custom_Laser_Rangefinder < True_SC_Sensor
    properties
        max_range          % Maximum detection range [m]
        min_range          % Minimum detection range [m]
        range_accuracy     % Range measurement accuracy [m]
        field_of_view      % Field of view [deg]
        target_reflectivity % Target reflectivity factor [0-1]
        last_measurement   % Last valid range measurement [m]
        is_target_detected % Flag indicating target detection
    end

    methods
        % Constructor
        function obj = Custom_Laser_Rangefinder(init_data, mission, i_SC, i_HW)
            % Call parent constructor
            obj@True_SC_Sensor(init_data, mission, i_SC, i_HW);

            % Initialize properties
            obj.max_range = init_data.max_range;
            obj.min_range = init_data.min_range;
            obj.range_accuracy = init_data.range_accuracy;
            obj.field_of_view = init_data.field_of_view;
            obj.target_reflectivity = init_data.target_reflectivity;
            obj.last_measurement = -1;  % Invalid measurement
            obj.is_target_detected = 0;

            % Set standard properties
            obj.instantaneous_power_consumed = 2.5;  % [W]
            obj.instantaneous_data_rate_generated = 0.1 * 8;  % [kbps]
        end
    end
end

% Corresponding update function
function func_main_custom_laser_rangefinder(obj, mission, i_SC, i_HW)
    % Skip if not active
    if ~obj.flag_executive
        obj.is_target_detected = 0;
        obj.last_measurement = -1;
        return;
    end

    % Get target position in spacecraft body frame
    target_pos_inertial = mission.true_target{1}.position;
    sc_pos_inertial = mission.true_SC{i_SC}.true_SC_navigation.position;
    sc_att_q = mission.true_SC{i_SC}.true_SC_adc.attitude;

    % Calculate relative position vector in body frame
    rel_pos_inertial = target_pos_inertial - sc_pos_inertial;
    rel_pos_body = func_quaternion_rotate_vector(sc_att_q, rel_pos_inertial);
```

```
    % Calculate range and direction
    true_range = norm(rel_pos_body);
    direction = rel_pos_body / true_range;

    % Check if target is within detector range and FOV
    sensor_axis = obj.orientation;
    angle_to_target = acosd(dot(sensor_axis, direction));

    % Determine if target is detected
    if true_range >= obj.min_range && true_range <= obj.max_range && ...
       angle_to_target <= obj.field_of_view/2

        % Target is within FOV and range limits
        obj.is_target_detected = 1;

        % Add noise to measurement based on accuracy
        noise = randn(1) * obj.range_accuracy;
        obj.last_measurement = true_range + noise;
    else
        % No valid detection
        obj.is_target_detected = 0;
        obj.last_measurement = -1;
    end

    % Store data
    func_store_data(obj, mission);
end
```

These examples demonstrate typical implementation patterns for various MuSCAT components. You can use them as starting points for your own mission implementations, adapting them to your specific requirements.

Chapter 3

Environment Classes

3.1 Storage

The Storage class manages data logging and visualization for the entire simulation. It is instantiated once per mission and is responsible for deciding when simulation data should be saved (based on independent time steps for the main simulation loop and the faster attitude dynamics loop), for driving real-time plotting of spacecraft attitude and orbit during the simulation, and for generating post-simulation dashboards, plots, and PDF/Excel reports summarizing the mission (orbit, attitude, power, data handling, telecom, and payload performance). Its constructor also creates the mission's output folder and initializes plotting and report parameters from the mission's initialization data.

3.1.1 Inputs / Outputs

3.1.1.1 Inputs / Dependencies

- init_data **(constructor-time):** time_step_storage, time_step_storage_attitude (optional, 0 = inherit from True_Time), flag_realtime_plotting, wait_time_realtime_plotting, flag_save_plots, flag_save_video, quiver_auto_scale_factor, flag_write_report, report_authors, mode_storage_viz_selector, mode_selector_name, num_viz_SC, data.
- True_Time **(mission-level, read at construction):** mission.true_time.time, t_initial, t_final, time_step, time_step_attitude are read to size the storage buffers (num_storage_steps, num_storage_steps_attitude) and seed time_prev_storage/time_prev_storage_attitude. func_update_storage_flag and func_update_storage_flag_attitude re-read mission.true_time.time, time_attitude, k, k_attitude, num_time_steps, num_time_steps_attitude every call to decide whether to store this step.
- mission **(struct fields, not a True_Environment class):** mission.name, mission.num_SC, mission.num_target, mission.frame are read for output-folder naming and report generation.
- True_SC / Software_SC_Executive **(per spacecraft, in** func_update_realtime_plot**):** reads mission.true_SC{i_SC}.software_SC_executive.this_sc_mode to detect mode changes that trigger an immediate real-time plot refresh.

- **Many hardware classes (in `func_visualize_simulation_data`):** reads `mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.*` and iterates cameras/radars/remote-sensing/telescopes to close their video file handles (`.data.myVideo`).

3.1.1.2 Outputs

- **Internal / stored outputs:** `flag_store_this_time_step`, `flag_store_this_time_step_attitude`, `k_storage`, `k_storage_attitude`, `num_storage_steps`, `num_storage_steps_attitude`, `output_folder`, `plot_parameters`, `report_parameters`, `flag_stop_sim`.
- **Reported to essentially every other True/Software class in the codebase:** `mission.storage.flag_store_this_time_step` / `flag_store_this_time_step_attitude` and `mission.storage.k_storage` / `k_storage_attitude` are read inside the `func_update_*_store` method of nearly all other classes (confirmed via grep across `True_Environment`, `True_SC`, `True_Sensors_Actuators`, `Software_SC`) to gate and index their own `obj.store.*` writes — this is the central storage-synchronization mechanism of the whole simulation.
- **Reported to `Main/main_v3.m` (main loop):** `mission.storage.flag_stop_sim` is read each loop iteration to decide whether to break out of the simulation.
- **Reported to plotting/report functions (`func_plot_*`, `func_visualize_*`):** `mission.storage.plot_parameters` (colors, markers, fonts) and `mission.storage.output_folder` are read throughout the plotting/report-generation code to format and save figures/Excel output.
- External write-back: `True_SC_Orbit_Propagation.m` sets `mission.storage.flag_stop_sim = 1` when it detects a spacecraft has crashed into a target, i.e. this is one case where an external class writes back into `Storage`'s own property rather than merely reading it.

3.1.2 Mathematical Equations

3.1.2.1 Number of Storage Steps

$$N_{\text{storage}} = \left\lceil \frac{t_{\text{final}} - t_{\text{initial}}}{\Delta t_{\text{storage}}} \right\rceil + 1 \tag{3.1}$$

Here, t_{initial} and t_{final} are the simulation start and end times [sec], and $\Delta t_{\text{storage}}$ is the storage time step [sec], i.e. the interval at which simulation data is logged (as opposed to the underlying integration time step, which may be finer). N_{storage} is the resulting number of discrete storage indices (rows/columns) that must be pre-allocated to hold the time history of any stored variable over the full simulation. The ceiling operator $\lceil \cdot \rceil$ ensures that a partial final interval still gets its own storage slot, and the $+1$ term accounts for the storage slot needed at t_{initial} itself (i.e., the state must be stored once before any time has elapsed).

In the MATLAB code, this formula is implemented twice in the `Storage` constructor, once for the main simulation loop and once for the (optional) attitude dynamics loop, which may run on a different, typically finer, time step. For the main loop, it appears as `obj.num_storage_steps = ceil( (mission.true_time.t_final - mission.true_time.t_initial) / obj.time_step_storage ) + 1;` where `obj.time_step_storage` is $\Delta t_{\text{storage}}$ (set just above, either equal to the main simulation time step `mission.true_time.time_step` or to a user-specified `init_data.time_step_storage`), and `mission.true_time.t_initial` /

`mission.true_time.t_final` are t_{initial} and t_{final}. The resulting `obj.num_storage_steps` is used elsewhere in the codebase to pre-size storage arrays for every logged quantity.

The identical formula is repeated for the attitude-loop storage as `obj.num_storage_steps_attitude = ceil( (mission.true_time.t_final - mission.true_time.t_initial) / obj.time_step_storage_attitude ) + 1;` using `obj.time_step_storage_attitude` in place of $\Delta t_{\text{storage}}$ (defaulting to `mission.true_time.time_step_attitude` or to `obj.time_step_storage` if no attitude-specific storage step is specified). This yields `obj.num_storage_steps_attitude`, the array length used for variables logged at the attitude-dynamics-loop rate.

3.1.3 CML 2–5 Suitability

3.1.3.1 Capabilities adequate for CML 2–5 formulation

Reading through the implementation confirms that `Storage` correctly delivers several features that are directly useful for rapid, iterative trade-space work at CML 2–5:

- **Storage cadence decoupled from integration step.** The constructor lets `time_step_storage` either default to the dynamics time step (`mission.true_time.time_step`, when the user passes 0) or be set independently, and the same pattern is repeated for a separate attitude-loop storage cadence (`time_step_storage_attitude`). This lets an engineer run fine-grained dynamics while logging coarsely, which is exactly the kind of performance/fidelity trade useful in early formulation sweeps.

- **Robust step-boundary logic.** `func_update_storage_flag` and `func_update_storage_flag_attitude` correctly guard against floating-point drift in the elapsed-time comparison via a `numerical_accuracy_factor` (0.99) multiplier, and both explicitly force a final store when the simulation reaches its last time step. This is a genuinely correct, verified implementation detail that prevents silently dropping the last data point.

- **Non-blocking real-time visualization for fast iteration.** `func_update_realtime_plot` throttles updates using a configurable `viz_update_interval`, but also triggers an out-of-cadence redraw whenever any spacecraft's software-executive mode changes, and calls `drawnow limitrate` rather than a blocking `drawnow`. This gives a formulation engineer live visual feedback on mode transitions without stalling the simulation.

- **Automated stakeholder-ready reporting.** `func_initialize_report` and `func_visualize_simulation_data` use MATLAB's `mlreportgen` API to auto-generate a titled, table-of-contents PDF report documenting mission name, spacecraft/target counts, frame convention, and simulation/storage time steps, and separately write an Excel file combining variable-name headers with logged data. This directly supports briefing mission stakeholders on trade-study configurations without hand-built slides.

- **Self-organizing, non-destructive output management.** Each run creates a uniquely named, timestamped output folder via `mkdir`, so successive trade iterations do not overwrite each other's plots/reports/videos.

- **Memory-conscious post-processing.** `func_visualize_simulation_data` explicitly closes figures between spacecraft, closes all open video writers, and processes one spacecraft's plots at a time "to limit memory usage" – a real, verifiable safeguard against runaway memory accumulation during long batch post-processing.

3.1.3.2 Gaps relevant to CML 2–5 use

The class also has limitations a formulation engineer should know before relying on it for a new or larger mission:

- **Toolbox and environment dependencies.** Report generation depends on the MATLAB Report Generator toolbox and numerous figure/`drawnow` calls that assume an interactive, licensed, display-capable MATLAB session – a practical barrier for headless/batch execution of large trade sweeps.
- **Minimal input validation.** Constructor options are set via `isfield` checks with silent defaults, and there is no validation that Excel variable names and data stay consistent before being written out.

3.2 True_Ground_Station

The True_Ground_Station class tracks all data transmitted to and received by the mission's ground station(s). It aggregates the instantaneous and cumulative data volumes contributed by each piece of hardware (e.g., spacecraft radio antennas) that transmits to or receives from the ground, keeping a running list of contributing hardware and their individual data totals for use in telecom analysis and reporting.

3.2.1 Inputs / Outputs

3.2.1.1 Inputs / Dependencies

- init_data **(constructor-time):** num_GS_radio_antenna, num_GS_enterprise (optional, default 0), data (optional).
- Storage **(mission-level):** mission.storage.num_storage_steps sizes the obj.store.* arrays at construction; mission.storage.flag_store_this_time_step and mission.storage.k_storage gate/index every store update in func_update_true_ground_station_store.
- True_Time **(mission-level):** mission.true_time.time_step is read inside func_initialize_list_HW_data_transmitted/_received and func_update_instantaneous_data_transmitted/_received to convert a hardware equipment's instantaneous data rate [kbps] into data transmitted/received per step [kb].
- True_GS_Radio_Antenna **and** True_SC_Communication_Link **("equipment" argument):** these HW/link objects are passed in as equipment to func_initialize_list_HW_data_transmitted/received and func_update_instantaneous_data_transmitted/received; the class reads equipment.name, equipment.instantaneous_data_rate_transmitted/received, and equipment.instantaneous_data_transmitted/received_per_sample (whichever property exists) to register/accumulate per-equipment transmit/receive totals.

3.2.1.2 Outputs

- **Internal / stored outputs:** instantaneous_data_transmitted/received, total_data_transmitted/received, list_HW_data_transmitted/received, array_HW_data_transmitted/received, and their time-history equivalents under obj.store.
- **Reported to** True_GS_Radio_Antenna **(own methods called by that class):** True_GS_Radio_Antenna's constructor calls func_initialize_list_HW_data_transmitted/received(mission.true_ground_station, obj, mission), and its main method calls func_update_instantaneous_data_transmitted/received(mission.true_ground_station, obj, mission) — i.e. the antenna drives updates of True_Ground_Station's instantaneous/array data counters.
- **Reported to** True_SC_Communication_Link **(same pattern):** the SC-side communication link similarly calls func_update_instantaneous_data_transmitted/received to add its data to the ground station's totals.
- **Reported to** Main/main_v3.m**:** func_main_true_ground_station(mission.true_ground_station, mission, i_SC) is called once per main-loop iteration to roll instantaneous data

into totals, store, and reset instantaneous counters. `mission.true_ground_station.num_GS_radio_antenna` / `num_GS_enterprise` are read in the main loop to iterate per-antenna and per-enterprise updates.

- **Reported to Mission setup scripts and plotting (`func_plot_GS_data_received.m`, `func_plot_GS_ground_track.m`, various `func_visualize_SC_*`/`func_software_SC_executive_*`):** `num_GS_radio_antenna` is read to size loops over ground stations; `obj.store.total_data_received` is read directly in `func_plot_GS_data_received.m` for reporting/plotting.
- No external class writes back into `True_Ground_Station`'s own properties — all external interaction occurs through its own `func_initialize_list_HW_data_*` / `func_update_instantaneous_data_*` methods, which it owns.

3.2.2 Mathematical Equations

3.2.2.1 Instantaneous Data Volume from Data Rate and Time Step

$$\Delta D_k^{\text{tx}} = \sum_{j=1}^{N_{\text{HW}}} r_j^{\text{tx}} \cdot \Delta t, \qquad \Delta D_k^{\text{rx}} = \sum_{j=1}^{N_{\text{HW}}} r_j^{\text{rx}} \cdot \Delta t \tag{3.2}$$

Here, ΔD_k^{tx} and ΔD_k^{rx} denote the instantaneous data volumes (in kilobits, kb) transmitted and received by the ground station during simulation time step k, respectively. r_j^{tx} and r_j^{rx} are the instantaneous data rates (kb/s) of the j-th piece of ground station hardware/equipment currently transmitting or receiving data, Δt is the simulation time step duration (s, given by `mission.true_time.time_step`), and N_{HW} is the number of hardware units contributing to the transmission or reception event at that time step. In cases where equipment reports a fixed per-sample data volume instead of a rate, the contribution is taken directly as that per-sample value rather than being multiplied by Δt.

In the MATLAB code, this equation is implemented in `func_update_instantaneous_data_transmitted` and `func_update_instantaneous_data_received`. For each piece of equipment passed into these methods, the code checks whether the object has the property `instantaneous_data_rate_transmitted` (or `instantaneous_data_rate_received`); if so, it computes `this_instantaneous_data_transmitted = equipment.instantaneous_data_rate_transmitted * mission.true_time.time_step` (analogously for received data). If instead the equipment exposes `instantaneous_data_transmitted_per_sample` (or `instantaneous_data_received_per_sample`), that per-sample value is used directly. This per-equipment contribution is then accumulated into `obj.instantaneous_data_transmitted` (or `obj.instantaneous_data_received`), and simultaneously added into the corresponding slot of `obj.array_HW_data_transmitted` (or `obj.array_HW_data_received`) for that specific hardware unit, identified by matching `equipment.name` against `obj.list_HW_data_transmitted` (or `obj.list_HW_data_received`). The same logic seeds the initial per-hardware values in `func_initialize_list_HW_data_transmitted` and `func_initialize_list_HW_data_received`.

3.2.2.2 Cumulative Total Data Transmitted and Received Over Time

$$D_k^{\text{tx}} = D_{k-1}^{\text{tx}} + \Delta D_k^{\text{tx}}, \qquad D_k^{\text{rx}} = D_{k-1}^{\text{rx}} + \Delta D_k^{\text{rx}} \tag{3.3}$$

D_k^{tx} and D_k^{rx} represent the cumulative total data (kb) transmitted and received by the ground station up through simulation step k, while D_{k-1}^{tx} and D_{k-1}^{rx} are the corresponding cumulative totals from the previous step. ΔD_k^{tx} and ΔD_k^{rx} are the instantaneous (per-time-step) data volumes defined

above, accumulated across all contributing hardware during step k. This is a discrete-time running summation (equivalent to a Riemann sum of data rate over the mission timeline) that accumulates every instantaneous contribution into a running mission total.

This equation is implemented in `func_main_true_ground_station`, the main per-time-step update method of the class. The lines `obj.total_data_transmitted = obj.total_data_transmitted + obj.instantaneous_data_transmitted` and `obj.total_data_received = obj.total_data_received + obj.instantaneous_data_received` perform the running accumulation, using whatever instantaneous values were most recently populated by `func_update_instantaneous_data_transmitted`/`func_update_instantaneous_data_received`. Immediately after, `func_update_true_ground_station_store` logs the updated totals into `obj.store.total_data_transmitted` and `obj.store.total_data_received` at the current storage index `mission.storage.k_storage` (only when `mission.storage.flag_store_this_time_step == 1`). Finally, `obj.instantaneous_data_transmitted` and `obj.instantaneous_data_received` are reset to zero at the end of `func_main_true_ground_station`, so the instantaneous accumulators restart cleanly for the next time step while the cumulative totals persist.

3.2.3 CML 2–5 Suitability

3.2.3.1 What is included and correctly implemented

`True_Ground_Station` provides a solid, generic bookkeeping layer for aggregate ground-segment data flow that is well matched to CML 2–5 trade studies, where the goal is comparative sizing rather than certified performance prediction. Concretely, the class implements:

- **Multi-hardware data aggregation.** `func_initialize_list_HW_data_transmitted` and `func_initialize_list_HW_data_received` build a name-keyed registry so that an arbitrary number of ground-station radio antennas or software "enterprise" interfaces can each contribute to a shared ledger without the class needing to know their types in advance. This is directly useful for trading ground-network architectures (e.g., single dish vs. multi-antenna arrays, DSN vs. commercial ground networks) at CML 2–3 fidelity.

- **Dual-mode data-rate ingestion.** The update methods accept either a continuous rate (`instantaneous_data_rate_transmitted` $\times$ time step) or a discrete per-sample quantity, covering both streaming-downlink and packetized/burst communication models.

- **Running totals for downlink/uplink volume estimation.** Cumulative totals are accumulated every call to `func_main_true_ground_station`, giving mission designers a straightforward way to estimate cumulative downlinked/uplinked volume – a common CML 2–5 telecom-budget deliverable.

- **Per-time-step and cumulative time-history storage.** The `store` structure records instantaneous and total data-volume histories at every storage step, enabling post-run plotting of data-volume time histories and pass-by-pass comparisons across design alternatives.

- **Consistent, defensive interface.** Every update path validates hardware names against the registry and raises an explicit error rather than silently mis-aggregating, and the constructor safely defaults optional fields when absent.

3.2.3.2 What is missing or a gap for CML 2–5 use

The class is purely a data-flow accountant; it contains no telecommunications physics whatsoever:

- **No link-budget or link-margin computation.** There is no calculation of received signal power, path loss, E_b/N_0, or link margin anywhere in this class. The sibling class `True_GS_Radio_Antenna` declares the relevant link-budget fields, but these are only ever assigned from `init_data` and never used in any link-margin formula elsewhere in MuSCAT. Consequently, MuSCAT can report *how much data was moved* but not *whether the link could actually close* at a given range, antenna size, or data rate.
- **No data buffering, queuing, or storage-capacity model.** Totals are unbounded running sums with no onboard/ground buffer size, overflow logic, or downlink-queue draining a solid-state recorder.
- **No contact-window, scheduling, or line-of-sight gating logic.** The class only sums whatever instantaneous values it is handed each step; visibility gating and contact scheduling must be implemented and enforced correctly upstream.
- **No atmospheric, weather, or environmental attenuation.** There is no rain fade, tropospheric/ionospheric attenuation, or elevation-angle-dependent loss modeling.
- **No error/retransmission or protocol-level modeling.** Data is aggregated as an idealized raw kilobit quantity with no bit-error-rate, packet-loss, or coding-overhead representation.

3.3 True_GS_Radio_Antenna

The True_GS_Radio_Antenna class models an individual ground station radio antenna (dipole or high-gain), which can operate in transmit (TX) or receive (RX) mode. It stores the antenna's link-budget parameters (gain, noise temperature, beamwidth, required energy per bit, line loss, coding gain) and geographic location, computes the resulting instantaneous data rates, and reports these contributions up to the True_Ground_Station class.

3.3.1 Inputs / Outputs

3.3.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name (optional), antenna_type, mode_true_GS_radio_antenna_selector, optional link-margin parameters (antenna_gain, noise_temperature, beamwidth, energy_bit_required, line_loss, coding_gain), optional latitude/longitude, data.
- True_Target_SPICE **(mission-level, Earth-only):** if the first target is named 'Earth', mission.true_target{1}.radius is read to convert the antenna's lat/long into a Cartesian position via cspice_latrec.
- Storage **(mission-level):** mission.storage.num_storage_steps sizes obj.store.* arrays; mission.storage.flag_store_this_time_step and k_storage gate/index store updates.
- True_Ground_Station **(mission-level, called from within this class):** the constructor calls func_initialize_list_HW_data_transmitted/received(mission.true_ground_station, obj, mission); the main method calls func_update_instantaneous_data_transmitted/received(mission.true_ground_station, obj, mission) whenever flag_executive == 1 and health == 1.
- True_SC_Communication_Link **(external, writes into this object — see below):** the link class sets this antenna's flag_executive, mode_true_GS_radio_antenna_selector, and instantaneous_data_rate_transmitted/received to command TX/RX behavior for the current step.

3.3.1.2 Outputs

- **Internal / stored outputs:** health, flag_executive, instantaneous_data_rate_transmitted/received, latitude/longitude/position, and obj.store.flag_executive, store.mode_TX_RX, store.instantaneous_data_rate_transmitted/received.
- **Reported to** True_Ground_Station**:** via func_update_instantaneous_data_transmitted/received, this antenna's name and instantaneous_data_rate_transmitted/received (or per-sample equivalents) feed into True_Ground_Station's instantaneous/array/total data-transmitted/received accounting.
- **Reported to** True_SC_Communication_Link**:** the link reads mission.true_GS_radio_antenna{i}.name, .flag_executive, and writes mode/rate fields (see write-back note below) to coordinate TX/RX pairing between SC and GS.
- **Reported to** Main/main_v3.m**:** func_main_true_GS_radio_antenna is invoked once per antenna per main-loop step.

- **Reported to plotting/mission-specific executive code (`func_plot_GS_data_received.m`, `func_plot_GS_ground_track.m`, mission-specific `func_visualize_SC_*` and `func_software_SC_executive_*` functions, e.g. the NISAR variants):** `.name`, `.latitude`, `.longitude`, `.position`, and `.store.instantaneous_data_rate_received` are read for ground-track plotting, pass-detection, and reporting.

- External write-back: yes — `True_SC_Communication_Link.m` (and its mission-specific variants, e.g. `func_communication_link_MultiSC_per_GS.m`) directly assigns `mission.true_GS_radio_antenna{i}.flag_executive`, `.mode_true_GS_radio_antenna_selector`, `.instantaneous_data_rate_transmitted`, and `.instantaneous_data_rate_received` to command this antenna's behavior each step; mission-specific communication scripts (e.g. for DART and NISAR) also set `.mode_true_GS_radio_antenna_selector` directly to switch TX/RX mode.

3.3.2 Mathematical Equations

3.3.2.1 Ground Antenna Geodetic-to-Cartesian Position Transform

$$\mathbf{r}_{\text{antenna}} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = R_{\oplus} \begin{bmatrix} \cos(\lambda)\cos(\phi) \\ \cos(\lambda)\sin(\phi) \\ \sin(\lambda) \end{bmatrix} \tag{3.4}$$

Here $R_{\oplus}$ is the (spherical) radius of the Earth target body [km], ϕ is the antenna's geodetic longitude [rad], measured East-positive/West-negative, and λ is the antenna's geodetic latitude [rad], defined over $[-\pi/2, \pi/2]$. The resulting $\mathbf{r}_{\text{antenna}} = [x,\, y,\, z]^T$ is the antenna's position [km] expressed in the body-fixed Cartesian frame of the Earth target, obtained by treating the antenna as lying exactly on a sphere of radius $R_{\oplus}$ (i.e., a spherical, not oblate-ellipsoid, Earth model).

In the code, this is implemented in the constructor of `True_GS_Radio_Antenna.m`. The object properties `obj.latitude` and `obj.longitude` are read from `init_data` in degrees and converted to radians via `deg2rad(obj.longitude)` and `deg2rad(obj.latitude)`. The transform itself is delegated to the SPICE toolkit routine `cspice_latrec(radius, longitude, latitude)`, called as `cspice_latrec(mission.true_target{1}.radius, deg2rad(obj.longitude), deg2rad(obj.latitude))'`, using `mission.true_target{1}.radius` as $R_{\oplus}$. This call only executes when the mission's first true target is Earth (`strcmp(mission.true_target{1}.name, 'Earth')`), and the transposed 3-vector result is stored in `obj.position` with units of km. This position is used elsewhere (e.g., visibility/line-of-sight checks against spacecraft) to determine when ground-spacecraft communication links are geometrically possible.

3.3.2.2 Link Margin / Required E_b/N_0 Equation (Intended, Not Yet Implemented)

$$\left(\frac{E_b}{N_0}\right)_{\text{actual}} = P_r + G_{\text{ant}} + G_{\text{code}} - L_{\text{line}} - 10\log_{10}(k) - 10\log_{10}(T_{\text{noise}}) - 10\log_{10}(R_b) \tag{3.5}$$

$$\text{Link Margin} = \left(\frac{E_b}{N_0}\right)_{\text{actual}} - \left(\frac{E_b}{N_0}\right)_{\text{required}} \tag{3.6}$$

P_r is the received carrier power [dBW/dBm], G_{ant} is the receiving antenna gain [dB] (field `antenna_gain`), G_{code} is the coding gain [dB] from forward error correction (field `coding_gain`), L_{line} is the RF line/pointing loss [dB] (field `line_loss`), k is Boltzmann's constant [J/K], T_{noise} is the system noise temperature [K] (field `noise_temperature`), R_b is the transmitted data rate [bps], and $(E_b/N_0)_{\text{required}}$ is the minimum energy-per-bit-to-noise-density ratio [dB] needed for the chosen modulation/coding scheme to achieve the target bit error rate (field `energy_bit_required`). The `beamwidth` field

[MHz] corresponds to the receiver's noise/RF bandwidth B, which would enter a companion noise-power term $N = kT_{\text{noise}}B$ in a full link-budget/carrier-to-noise formulation. A positive Link Margin indicates the link can close (support) the specified data rate at the required error performance; a negative margin means it cannot.

This equation is *not currently computed anywhere in the MuSCAT codebase* — it represents the standard telecommunications link-budget relationship that the declared parameters are clearly intended to support, based on their names and units. The properties `antenna_gain`, `noise_temperature`, `beamwidth`, `energy_bit_required`, `line_loss`, and `coding_gain` are declared and initialized identically in both `True_GS_Radio_Antenna.m` (ground antenna) and `True_SC_Radio_Antenna.m` (spacecraft antenna), under a comment block explicitly labeled "Optional (only for Link Margin Calculations)". Sample numeric values are set in mission configuration files such as `Mission_DART.m`, confirming these are meant to feed a link-margin computation. However, the actual arithmetic is stubbed out: in `True_SC_Communication_Link.m`, method `func_communication_link_generic`, the branch guarded by `obj.flag_compute_data_rate == 1` immediately raises `error('Write code to compute Link Margin!')` instead of performing the calculation. In all current missions, `flag_compute_data_rate` is instead set to 0, so the data rate is simply taken directly from a user-specified `given_data_rate` [kbps], bypassing any link-budget computation entirely.

3.3.3 CML 2–5 Suitability

3.3.3.1 What is included and correctly implemented

`True_GS_Radio_Antenna` provides the core bookkeeping needed for early-formulation ground-link trade studies.

- **Explicit TX/RX mode selector.** Each antenna instance carries an explicit TX/RX mode selector that is checked at every storage step, so a mission can model separate transmit and receive antennas or a single antenna switching roles over a pass.
- **Geodetic-to-Cartesian position via SPICE.** The class computes the ground antenna's fixed geodetic position (latitude/longitude) and converts it to a Cartesian position on the Earth's surface via SPICE (`cspice_latrec`), which is adequate for representing a ground station's location in an Earth-centered analysis at CML 2–5 fidelity.
- **Per-antenna data-rate rollup.** Instantaneous transmitted and received data rates are computed each active step and rolled up into the parent `True_Ground_Station` object, giving a clean per-hardware-element contribution to station-level throughput.
- **Optional link-budget parameterization.** The link-budget properties are explicitly marked optional and only populated `if isfield(init_data, 'antenna_gain')` – meaning a user can skip link-budget parameterization entirely and instead drive the antenna with a fixed, pre-specified data rate (as all current missions, e.g. `Mission_DART.m`, in fact do). For CML 2–3 trade studies, where the analyst typically already has a rough data-rate allocation from a separate telecom trade and simply wants to propagate its downstream effects, this is a reasonable and appropriately lightweight simplification rather than a deficiency.

3.3.3.2 What is not included / gaps

The most significant gap is that the link-budget fields, although declared and in some missions initialized, are never actually used in any Eb/N0 or link-margin arithmetic anywhere in the codebase.

- **Link-margin path never implemented.** As noted in the previous subsection, `func_communication_link_generic`'s link-margin branch simply raises an error, and every current mission instead hard-codes a `given_data_rate`. For CML 4–5 work, where telecom closure is often a discriminating factor between concept variants, MuSCAT cannot answer that question internally – the user must compute link margin and an achievable data rate in an external tool and feed the result in as `given_data_rate`.
- **No environmental or pointing-loss effects.** Beyond this central gap, this class itself contains no model of atmospheric or rain/weather attenuation, no antenna pointing-loss computation tied to actual off-boresight pointing angle (`line_loss` is a static declared value, not a function of geometry), and no multipath or ground-clutter effects.
- **Spherical Earth position model.** The ground-station position model is also limited to a spherical body representation (a single scalar radius passed to `cspice_latrec`), so oblate/ellipsoidal Earth geodesy is not represented, and gain roll-off vs. elevation angle is not modeled in this file.
- **Consistent with CML 2–5 scope.** These omissions are consistent with a CML 2–5 scope but should be understood by users who need higher-fidelity RF performance predictions.

3.4 Software_GS_Enterprise

The Software_GS_Enterprise class represents the ground-based mission operations/command-and-control software (e.g., an "Enterprise" system) that plans and issues commands to the ground station and, through it, to the spacecraft fleet. Its behavior is mode-dependent (e.g., Generic, Blue, Red), with the constructor, main update, and storage logic all dispatching to mode-specific implementations.

This class contains no mathematical equations of its own: it only performs a switch-based dispatch to mode-specific helper functions (func_software_GS_enterprise_blue, func_software_GS_enterprise_red, etc.) and storage-flag branching, with no arithmetic computed directly in the class file.

3.4.1 Inputs / Outputs

3.4.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name (optional), mode_software_SC_enterprise_selector (one of 'Generic', 'Blue', 'Red'), data (optional).
- Storage **(mission-level):** mission.storage.flag_store_this_time_step gates func_update_software_GS_enterprise_store.
- **Mode-specific constructor/update/store functions (**func_software_GS_enterprise_constructor_blue/red**,** func_software_GS_enterprise_blue/red**,** func_software_GS_enterprise_store_blue/red**):** dispatched via obj.mode_software_SC_enterprise_selector; no implementations of these mode-specific functions were found anywhere in the current codebase (only the 'Generic' no-op path is exercised), so their concrete cross-class dependencies could not be verified.

3.4.1.2 Outputs

- **Internal / stored outputs:** obj.data, obj.store (populated only by the unverified Blue/Red mode functions; empty under 'Generic').
- **Reported to** Main/main_v3.m**:** func_main_software_GS_enterprise(mission.software_GS_enterprise{i_HW}, mission, i_HW) is called once per enterprise instance per main-loop step; mission.true_ground_station.num_GS_enterprise (not this class's own property) is read by the main loop to determine how many enterprise instances to iterate.
- No other class in the codebase was found reading mission.software_GS_enterprise properties directly (grep across the whole codebase found only the single call site in main_v3.m); this class currently has no verified downstream consumers beyond the main loop invocation itself.
- No external class writes back into this class's properties.

3.4.2 CML 2–5 Suitability

3.4.2.1 What is included and correctly implemented

Software_GS_Enterprise provides a clean, well-defined extension point for modeling ground-segment ("enterprise") behavior: a mode selector (mode_software_SC_enterprise_selector) dis-

patches consistently across all three lifecycle hooks (constructor, main update, and store) via matching `switch–case` blocks. This gives a formulation team a ready-made interface into which ground-based command-and-control or scheduling logic could be inserted later, without needing to restructure the class. For the common case where a CML 2–5 study does not need to model ground-segment autonomy or decision-making, the `'Generic'` mode is a genuine, correctly implemented no-op: it is safe to select, performs no operations in any of the three dispatch points, and introduces no risk of runtime error. For spacecraft-focused trade studies, which is the large majority of MuSCAT use cases, selecting `'Generic'` is sufficient and appropriate.

3.4.2.2 What is missing / gap

The `'Blue'` and `'Red'` modes are stubs only: each case branch calls a helper function (`func_software_GS_enterprise_constructor_blue`, `func_software_GS_enterprise_blue`, `func_software_GS_enterprise_store_blue`, and the corresponding `_red` variants) that does not exist anywhere in the MuSCAT codebase. Selecting either mode will therefore throw a MATLAB undefined-function runtime error the first time the constructor, main-update, or store method executes. In its current state, this class offers no actual ground-enterprise decision-making, scheduling, or command-planning capability beyond the placeholder dispatch interface:

- No adversarial ("Red") or friendly ("Blue") ground command-and-control logic exists to exercise.
- Any mission configuration that sets the selector to `'Blue'` or `'Red'` will fail at runtime rather than degrade gracefully.
- There is no fallback or warning; the failure mode is a hard error, not a silent no-op.

For most CML 2–5 spacecraft-centric trade studies – where ground-segment autonomy and enterprise-level decision logic are rarely the object of study – this gap is minor and can simply be avoided by using `'Generic'` mode. However, it should be flagged as a real limitation for any study that specifically intends to model ground-based scheduling, command planning, or ground-in-the-loop decision-making, since no such capability currently exists to build on.

3.5 True_Solar_System

The True_Solar_System class maintains the positions and velocities of the Sun, planets, and other solar system bodies of interest, computed via SPICE ephemeris kernels. It stores each body's physical properties (radius, mass, gravitational parameter) alongside physical constants (solar constant, speed of light, gravitational constant, Stefan–Boltzmann constant) used throughout the simulation for gravity, solar radiation pressure, and thermal calculations.

3.5.1 Inputs / Outputs

3.5.1.1 Inputs / Dependencies

- init_data **(constructor-time):** SS_body_names (string array of solar-system body names to track, e.g. ["Sun","Earth"]); internally matched against a hardcoded catalog (func_load_all_SS_body_data) of name/SPICE-ID/radius/mu/mass/color for Sun through Neptune and major moons.
- True_Time **(mission-level):** mission.true_time.date, prev_date, time_position_array, num_time_steps_position_array are read in func_main_true_solar_system to query SPICE (cspice_spkezr) for each body's position/velocity at the current time and across the position-array time grid.
- Storage **(mission-level):** mission.storage.num_storage_steps sizes obj.store.SS_body{i}.position/velocity; mission.storage.flag_store_this_time_step and k_storage gate/index store updates.
- **SPICE kernel files (not a MuSCAT class):** multiple .bsp/.tls kernels are loaded via cspice_furnsh at construction to supply ephemerides.

3.5.1.2 Outputs

- **Internal / stored outputs:** SS_body{i}.position/velocity/position_array, index_Sun/Earth/Moon, physical constants (solar_constant_AU, AU_distance, light_speed, gravitational_constant, sigma), and obj.store.SS_body{i}.position/velocity.
- **Reported to** True_Target_Computed**:** heavily read — mission.true_solar_system.SS_body{i}.position/mu, index_Sun, num_SS_body, solar_constant_AU, AU_distance, light_speed, sigma, gravitational_constant are all used in func_update_SRP_disturbance_force, func_update_SS_Body_disturbance_force, and func_update_Yarkovsky_disturbance_force to compute SRP, N-body gravitational perturbation, and Yarkovsky forces on the target body.
- **Reported to** True_SC_Orbit_Propagation**,** True_SC_SRP**,** True_SC_Camera**,** True_SC_Star_Tracker**,** True_SC_Sun_Sensor**,** True_SC_Telescope**,** True_SC_Solar_Panel**:** these all read SS_body{index_Sun}.position (and sometimes index_Earth) for Sun-pointing, illumination, SRP, and visibility geometry calculations.
- **Reported to** Software_SC_Executive **/ mission-specific pointing and plotting code (e.g. for DART and NISAR):** read SS_body, index_Sun, AU_distance etc. for Lambert targeting, pointing-vector computation, and dashboard/trajectory plotting; obj.store.SS_body{i}.position/velocity is read directly by func_plot_orbit_visualization.m and func_post_process_NISAR_output_to_Excel.m for reporting.

- No external class writes back into this class's properties.

3.5.2 Mathematical Equations

3.5.2.1 Standard Gravitational Parameter from Mass and the Gravitational Constant

$$\mu_i = G\, m_i \tag{3.7}$$

Here μ_i is the standard gravitational parameter of solar system body i, expressed in units of $\mathrm{km}^3\,\mathrm{s}^{-2}$, and m_i is the mass of that body in kilograms. G is Newton's universal gravitational constant, expressed in the mixed unit system $\mathrm{km}^3\,\mathrm{kg}^{-1}\,\mathrm{s}^{-2}$ so that the resulting μ_i comes out directly in $\mathrm{km}^3\,\mathrm{s}^{-2}$, the unit convention used throughout the propagator for two-body dynamics.

This relationship is explicitly evaluated in the MATLAB code, rather than simply hard-coded, for several of the minor Saturnian moons. In the constructor of `True_Solar_System`, the gravitational constant is initialized as `obj.gravitational_constant = 6.67430e-20;` (in $\mathrm{km}^3\,\mathrm{kg}^{-1}\,\mathrm{s}^{-2}$). Inside the method `func_load_all_SS_body_data(obj)`, each solar system body's data is assembled into a local structure `this_data`. For most major bodies (Sun, Mercury, Venus, Earth, Moon, Mars, Jupiter, Saturn, Uranus, Neptune, and their barycenters, and Pluto Barycenter) `this_data.mu` is a directly specified numerical value taken from an external reference. However, for the smaller Saturnian satellites — Mimas, Enceladus, Tethys, Dione, Rhea, Titan, Hyperion, Iapetus, Phoebe, and Helene — no independently measured μ value is supplied; instead, `this_data.mass` is set from a literature value, and `this_data.mu` is computed on the fly via `this_data.mu = obj.gravitational_constant * this_data.mass;`, directly implementing Equation 3.7. The resulting per-body structures are collected into the cell array `all_SS_body_data`, matched against the requested body list `init_data.SS_body_names` to populate `obj.SS_body{i}`, from which `mu` is later available to any orbit-propagation or dynamics code that needs the gravitational parameter of these moons.

3.5.2.2 Heliocentric State Vector Decomposition from SPICE Ephemerides

$$\mathbf{X}_i(t) = \begin{bmatrix} \mathbf{r}_i(t) \\ \mathbf{v}_i(t) \end{bmatrix}, \qquad \mathbf{r}_i(t) \in \mathbb{R}^3\ [\mathrm{km}], \qquad \mathbf{v}_i(t) = \frac{d\mathbf{r}_i}{dt} \in \mathbb{R}^3\ [\mathrm{km/s}] \tag{3.8}$$

Here $\mathbf{X}_i(t)$ is the six-element Cartesian state vector of solar system body i at epoch t, expressed in the inertial J2000 reference frame centered on the Sun. It is composed of the position vector $\mathbf{r}_i(t)$ (three components, in km) and its time derivative, the velocity vector $\mathbf{v}_i(t)$ (three components, in km/s). This is the fundamental two-body/N-body state representation used by JPL's SPICE ephemeris system: for a chosen target body, observer, reference frame, and aberration correction, SPICE returns the instantaneous position and velocity relative to the observer by interpolating precomputed trajectory (SPK) kernels.

In the method `func_main_true_solar_system(obj, mission)`, this decomposition is carried out explicitly for every tracked body. The call `body_pos_vel_this_time = cspice_spkezr(obj.SS_body{i}.spice_name, mission.true_time.date, 'J2000', 'NONE', 'SUN')` retrieves the 6x1 state vector $\mathbf{X}_i(t)$ of body i relative to the Sun, in the J2000 frame, with no aberration correction, at the current mission epoch `mission.true_time.date`. The code then splits this vector according to Equation 3.8: `obj.SS_body{i}.position = body_pos_vel_this_time(1:3,1)';` extracts $\mathbf{r}_i(t)$ and `obj.SS_body{i}.velocity = body_pos_vel_this_time(4:6,1)';` extracts $\mathbf{v}_i(t)$. The same decomposition is applied in vectorized form over a full array of epochs to populate `obj.SS_body{i}.position_array`, which stores the trajectory of each body across the mission timeline. These values are subsequently cached each storage step by `func_update_solar_system_store(obj,`

`mission)` into `obj.store.SS_body{i}.position` and `obj.store.SS_body{i}.velocity`; the underlying orbital dynamics are entirely encapsulated inside the SPICE kernel interpolation itself, not exposed as MATLAB source code in this file.

3.5.3 CML 2–5 Suitability

3.5.3.1 What is included and correctly implemented

`True_Solar_System.m` provides solar-system body ephemerides and physical constants that meet or exceed the fidelity typically required for CML 2–5 trade studies. Rather than relying on simplified two-body or Keplerian analytic ephemerides (the norm at CML 2–3), the class loads real JPL SPICE kernels (`de442.bsp`, `mar099.bsp`, `jup347.bsp`, `sat455.bsp`, `ura182.bsp`, `nep105.bsp`) and calls `cspice_spkezr` to obtain true, time-tagged position and velocity vectors, meaning the "truth" ephemeris used in MuSCAT is actually *higher fidelity* than what a formulation-level trade study normally demands.

- **Broad, configurable body coverage.** The constructor loops over `init_data.SS_body_names` and matches against an internal library covering the Sun, all major planets, the Earth-Moon system, Pluto, and numerous named barycenters, plus a substantial list of Saturnian moons. Users can select only the bodies relevant to their mission scenario.
- **Standard, correct physical constants.** The solar constant, AU distance, speed of light, gravitational constant, and Stefan–Boltzmann constant are all standard, correctly valued, and bundled directly on the object for downstream SRP and thermal calculations.
- **Per-body physical data for immediate reuse.** Each body carries radius, mass, gravitational parameter μ, and a color tag for plotting.
- **Time-history support.** In addition to instantaneous position/velocity, the class propagates a full `position_array` over the mission's stored time grid and maintains a `store` structure.

3.5.3.2 What is not included or is a gap

Several implementation choices are worth flagging, though most are unlikely to matter for CML 2–5 work specifically:

- **No runtime validation of kernel time coverage.** Each kernel's valid interval is documented only in code comments; there is no runtime check that the mission epoch falls within a loaded kernel's coverage.
- **No error handling around kernel loading.** A missing or unreadable kernel file would produce a raw SPICE/MATLAB error rather than a descriptive MuSCAT-level message.
- **Geometric (uncorrected) states only.** States are retrieved with aberration correction `'NONE'`, i.e., no light-time or stellar aberration correction. This is standard and acceptable at CML 2–5.
- **Point-mass gravity only; no oblateness/harmonics.** Every body is characterized by a scalar radius and point-mass μ; nothing in this file models oblateness, consistent with CML 2–5 fidelity.

- **Inconsistent provenance of μ.** For major bodies, μ is taken from cited or unattributed sources; for several Saturnian moons, μ is instead computed on the fly as $G\times$ mass – a minor internal inconsistency rather than a fidelity problem.
- **No ephemeris uncertainty/covariance.** Only deterministic state vectors are returned, which is not generally required at CML 2–5 but would be a gap for higher-CML navigation-sensitive analyses.

3.6 True_Stars

The True_Stars class loads the Smithsonian Astrophysical Observatory (SAO) star catalog 'saoNAN.txt', which includes catalog IDs, visual magnitudes, Right Ascension (RA), and Declination (Dec). It computes each star's unit vector direction (from its RA and Dec, adjusted for proper motion to the simulation's start year). This star field is used by star tracker and camera classes to simulate realistic background star observations.

3.6.1 Inputs / Outputs

3.6.1.1 Inputs / Dependencies

- True_Time **(mission-level, constructor argument** mission **only, no** init_data**):** mission.true_time.t_initial_date_string is parsed to extract the simulation start year YYYY, used to apply proper-motion correction to each star's catalog RA/Dec.
- **SAO Star Catalog file** saoNAN.txt **(not a MuSCAT class):** read directly via fopen/textscan at construction to obtain star name, magnitude, RA, Dec, and proper-motion rates for all catalog stars.
- No other True_Environment or Software class properties are read by this class — construction depends only on mission.true_time and the static catalog file.

3.6.1.2 Outputs

- **Internal / stored outputs:** num_stars, sao_name, magnitude_visible, all_stars_unit_vector (unit vectors in J2000 from RA/Dec), maximum_magnitude. This class has no obj.store / time-history storage — its properties are static for the whole simulation (computed once at construction).
- **Reported to** True_SC_Camera **and** True_SC_Telescope**:** read mission.true_stars.all_stars_unit_vector and magnitude_visible (filtered by maximum_magnitude) to render/detect visible stars within a camera or telescope's field of view.
- **Reported to Mission setup scripts:** every Mission_*.m script reads/overrides mission.true_stars.maximum_magnitude right after construction (e.g. mission.true_stars.maximum_magnitude = 10) to tune the star catalog's visible-magnitude cutoff per mission.
- **Reported to plotting code (**func_make_video_*, func_plot_formation_earth.m, func_plot_image.m**, and other mission-specific plotting):** read star unit vectors/magnitudes for rendering star fields in simulated camera/telescope imagery and dashboard videos.
- No external class writes back into this class's properties, aside from Mission setup scripts directly setting maximum_magnitude immediately after construction (a one-time configuration override, not a runtime write-back).

3.6.2 Mathematical Equations

3.6.2.1 Proper-Motion-Corrected Epoch Coordinates

$$\mathrm{RA}(t) = \frac{\alpha_0 + \mu_\alpha\,(Y - 2000)}{15}, \qquad \mathrm{DEC}(t) = \delta_0 + \mu_\delta\,(Y - 2000) \tag{3.9}$$

Here α_0 and δ_0 are the catalog Right Ascension and Declination of a star at the J2000.0 reference epoch, as tabulated in the SAO Star Catalog (Right Ascension is stored in units of time-arcseconds, hence the division by 15 to convert to hours, since 15° of RA corresponds to 1 hour of Earth rotation). The quantities μ_α and μ_δ are the star's proper motion rates in Right Ascension and Declination (change per year), and Y is the epoch year of the simulation. The term $(Y - 2000)$ is the number of years elapsed since the J2000.0 catalog epoch, so the correction linearly propagates each star's cataloged position forward (or backward) in time to account for its apparent drift due to proper motion. The results $\mathrm{RA}(t)$ and $\mathrm{DEC}(t)$ are the epoch-adjusted equatorial coordinates used for all subsequent computations.

In the code, this correction is computed once, outside the per-star loop, immediately after the SAO catalog is read into the cell array `M` via `textscan`. The epoch year `YYYY` is derived from `mission.true_time.t_initial_date_string`. The catalog fields are `M{5}` (base RA, α_0), `M{7}` (RA proper motion, μ_α), `M{6}` (base Dec, δ_0), and `M{8}` (Dec proper motion, μ_δ). The MATLAB statements `RA=(M{5}+(M{7}*(YYYY-2000)))/15;` and `DEC=M{6}+(M{8}*(YYYY-2000));` implement the equation above elementwise across all cataloged stars, producing the vectors `RA` and `DEC` indexed by star number `i` inside the subsequent `for i=1:1:obj.num_stars` loop.

3.6.2.2 Star Unit Vector from Right Ascension and Declination

$$\hat{\mathbf{s}}_i = R_y(\delta_i)\, R_z(\alpha_i)\, \hat{\mathbf{x}}, \qquad R_z(\alpha_i) = \begin{bmatrix} \cos\alpha_i & -\sin\alpha_i & 0 \\ \sin\alpha_i & \cos\alpha_i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad R_y(\delta_i) = \begin{bmatrix} \cos\delta_i & 0 & \sin\delta_i \\ 0 & 1 & 0 \\ -\sin\delta_i & 0 & \cos\delta_i \end{bmatrix} \tag{3.10}$$

Here $\hat{\mathbf{x}} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^T$ is the unit vector along the inertial frame's x-axis, which by convention points toward the vernal equinox at the reference epoch. $\alpha_i = 15\,\mathrm{RA}_i$ is the Right Ascension of the i-th star converted from hours to degrees and then to radians, and δ_i is the Declination of the i-th star in radians; both are the epoch/proper-motion-corrected values from the previous equation. $R_z(\alpha_i)$ is the rotation matrix about the inertial z-axis (the celestial pole direction) by angle α_i, which sweeps the x-axis eastward along the celestial equator to the star's RA. $R_y(\delta_i)$ is the rotation matrix about the y-axis by angle δ_i, which tilts the resulting vector out of the equatorial plane to the star's Declination. The combined rotation $R_y(\delta_i)R_z(\alpha_i)$, applied to $\hat{\mathbf{x}}$, yields $\hat{\mathbf{s}}_i$, the unit vector in the inertial frame pointing from the observer toward the i-th star, with all quantities dimensionless except the angles, which are in radians.

In the code, this is computed inside the `for i=1:1:obj.num_stars` loop in the `True_Stars` constructor. For each star index `i`, `x_hat = [1 0 0]'` defines $\hat{\mathbf{x}}$, and the epoch-corrected RA and Dec are converted to radians via `RA_angle = deg2rad(RA(i)*15)` and `Dec_angle = deg2rad(DEC(i))` (the multiplication by `15` converts RA from hours to degrees before conversion to radians). The matrix `Rot_Z_star_RA` implements $R_z(\alpha_i)$ and `Rot_Y_star_Dec` implements $R_y(\delta_i)$, exactly matching the sine/cosine layout above. These are combined as `Rot_RA_dec = Rot_Y_star_Dec * Rot_Z_star_RA`, and the star's unit vector is obtained via `obj.all_stars_unit_vector(i,:) = (Rot_RA_dec*x_hat)'`, which stores $\hat{\mathbf{s}}_i^T$ as row `i` of the property `all_stars_unit_vector`, a `num_stars`-by-3 matrix holding the inertial-frame direction of every star in the SAO catalog for later use (e.g., by the star-tracker or camera model).

3.6.3 CML 2–5 Suitability

3.6.3.1 What is included and correctly implemented

`True_Stars` is built on a genuine astronomical catalog rather than a synthetic or uniformly-

distributed toy star field: it reads right ascension, declination, and visual magnitude directly from the Smithsonian Astrophysical Observatory (SAO) catalog file. For CML 2–5 attitude-determination trade studies (e.g., sizing a star tracker field-of-view or sensitivity threshold), this preserves the real angular density of stars on the sky and the real magnitude distribution, so trades on FOV size versus expected star count, or on detector sensitivity versus magnitude-limited catalog completeness, are grounded in physically representative inputs rather than an arbitrary random field. The class also applies a proper-motion correction to the mission's initial epoch, using the mission start year parsed from `mission.true_time.t_initial_date_string` – a nice touch of fidelity beyond a bare static catalog lookup. Each star's inertial unit vector is computed via a standard RA/Dec rotation, and the full set of vectors, catalog IDs, and visual magnitudes are exposed as public properties, giving downstream star-tracker/camera models everything needed to run magnitude-limit and sensor-sensitivity trades at a level of fidelity appropriate for early formulation.

3.6.3.2 What is not included / gaps

Several simplifications and portability concerns are apparent directly from the code:

- **No double-star/binary handling.** Each catalog row is treated as a single point source with a single unit vector and magnitude; there is no logic to detect or merge close/binary pairs, though this is a second-order effect at CML 2–5.
- **No stellar aberration or parallax correction.** Only proper motion is applied. This omission is generally acceptable at CML 2–5, but it means the star positions are not fully time- and observer-consistent inertial directions.
- **No sensor/detection model in this class.** `True_Stars` only produces geometric "truth" (positions and catalog magnitudes); any star-tracker measurement fidelity (centroiding noise, magnitude-dependent detection probability) must live in a separate hardware class.
- **Catalog depth/vintage not documented in code.** SAO is an older, shallower catalog than modern astrometric catalogs (e.g., Hipparcos/Gaia); trade studies requiring very faint magnitude limits or high-precision modern astrometry should be aware of this.
- **Single fixed-epoch proper-motion correction.** The proper-motion shift is computed once, at construction, using the mission's initial year as a static offset; star positions are fixed for the entire simulation rather than time-varying through a multi-year mission.

3.7 True_Target_SPICE

The True_Target_SPICE class represents the mission's primary target body (e.g., an asteroid, planet, or moon) whose position and velocity are propagated directly from SPICE kernels rather than integrated from dynamics. It loads the target's gravity field, shape model, rotation parameters (pole orientation, prime meridian, rotation rate), and SPICE identifiers, and updates the target's state and body-fixed-to-inertial rotation matrix at each time step.

3.7.1 Inputs / Outputs

3.7.1.1 Inputs / Dependencies

- init_data **(constructor-time):** target_name (selects a hardcoded case: 'Bennu', 'Apophis', 'Toutatis', 'Itokawa', '1996HW1', 'Earth', 'IBD_Asteroid', 'Enceladus', 'Uranus', 'Moon'); for 'Earth' only, optional gravity_degree_harmonics. Each case hardcodes gravity-field file, shape-model file, SPICE kernel file/ID, and pole RA/Dec/prime-meridian.
- True_Time **(mission-level):** mission.true_time.date, prev_date, time_position_array, num_time_steps_position_array, t_initial, t_initial_date, time are read in the constructor and in func_main_true_target/func_update_target_rotation_matrix to query SPICE for position/velocity and to compute the body's current rotation angle (prime-meridian + rotation-rate × elapsed time).
- Storage **(mission-level):** mission.storage.num_storage_steps sizes obj.store.position/velocity; mission.storage.flag_store_this_time_step and k_storage gate/index store updates.
- **External SPICE kernels and gravity/shape-model files (not MuSCAT classes):** GravityField, readObj, and multiple cspice_furnsh kernel loads supply the physical/geometric model.

3.7.1.2 Outputs

- **Internal / stored outputs:** position, velocity, position_array, rotation_matrix, radius, mu, mass, shape_model, gravity_field, and obj.store.position/velocity.
- **Reported to** True_SC_Orbit_Propagation**:** reads mission.true_target{i}.position, velocity, position_array, name, radius for relative-to-target orbit propagation and for SC-crash detection against the target's radius.
- **Reported to** True_GS_Radio_Antenna**:** reads mission.true_target{1}.radius (Earth-only) at construction to convert ground-station lat/long into a body-fixed Cartesian position.
- **Reported to** True_SC_Camera**,** True_SC_Telescope**,** True_SC_Science_Radar**,** True_SC_Remote_Sensing**,** True_SC_Gravity_Gradient**,** True_SC_Magnetorquer**:** read rotation_matrix, position, mu, name, gravity_degree_harmonics, shape_model for body-fixed-frame transforms, ground-track computation, gravity-gradient torque, and surface imaging/radar geometry.

- **Reported to `Software_SC_Estimate_Orbit` and mission-specific `Software_SC_Executive` code (e.g. for DART and NISAR):** read `position`, `name`, `rotation_matrix` for truth-based orbit estimation and target-relative pointing/ground-track logic.
- **Reported to plotting/report code (`func_plot_orbit_estimator.m`, `func_plot_orbit_visualization.m`, `func_plot_target_shape.m`, `func_display_mission_dashboard.m`, `func_post_process_NISAR_output_to_Excel.m`):** read `obj.store.position/velocity`, `shape_model`, `name` for visualization and Excel export.
- No external class writes back into this class's properties.

3.7.2 Mathematical Equations

3.7.2.1 Pole Right Ascension/Declination Rotation Matrix

$$\mathbf{R}_Z(\alpha) = \begin{bmatrix} \cos(90^\circ+\alpha) & -\sin(90^\circ+\alpha) & 0 \\ \sin(90^\circ+\alpha) & \cos(90^\circ+\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{R}_X(\delta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(90^\circ-\delta) & -\sin(90^\circ-\delta) \\ 0 & \sin(90^\circ-\delta) & \cos(90^\circ-\delta) \end{bmatrix} \tag{3.11}$$

$$\mathbf{R}_{\text{pole}} = \mathbf{R}_Z(\alpha)\,\mathbf{R}_X(\delta) \tag{3.12}$$

Here α is the target's pole Right Ascension (`pole_RA`, in degrees) and δ is the target's pole Declination (`pole_Dec`, in degrees), both specified in the J2000 inertial frame and constant for a given target (for Earth they are themselves linear functions of Julian centuries computed once at t_{initial}). $\mathbf{R}_Z(\alpha)$ is a rotation about the Z-axis by $90^\circ + \alpha$ and $\mathbf{R}_X(\delta)$ is a rotation about the X-axis by $90^\circ - \delta$; their product $\mathbf{R}_{\text{pole}}$ orients the body's mean spin axis (and hence its equator and prime-meridian reference) relative to the J2000 frame, independent of time.

In the code, this is implemented in the constructor of `True_Target_SPICE`. The matrix $\mathbf{R}_Z(\alpha)$ is built as `Rot_Z_pole_RA` using `cosd(90 + obj.pole_RA)` and `sind(90 + obj.pole_RA)` entries, and $\mathbf{R}_X(\delta)$ is built as `Rot_X_pole_Dec` using `cosd(90 - obj.pole_Dec)` and `sind(90 - obj.pole_Dec)` entries. These are multiplied together and stored as `obj.rotation_matrix_pole_RA_Dec = Rot_Z_pole_RA * Rot_X_pole_Dec`, computed once at construction time (per the NAIF PCK orientation-model convention referenced in the code comments) and reused on every subsequent call to `func_update_target_rotation_matrix` rather than being recomputed at each time step.

3.7.2.2 Time-Varying Prime Meridian Angle and Full Body-to-J2000 Rotation

$$\theta_{\text{PM}}(t) = \theta_{\text{PM},0} + \dot{\theta}\,(t - t_0) \tag{3.13}$$

$$\mathbf{R}_{Z,\text{PM}}(\theta_{\text{PM}}) = \begin{bmatrix} \cos\theta_{\text{PM}} & \sin\theta_{\text{PM}} & 0 \\ -\sin\theta_{\text{PM}} & \cos\theta_{\text{PM}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3.14}$$

$$\mathbf{R}_{\text{J2000}\leftarrow\text{body}}(t) = \left[\mathbf{R}_{Z,\text{PM}}(\theta_{\text{PM}}(t))\,\mathbf{R}_{\text{pole}}\right]^T \tag{3.15}$$

Here $\theta_{\text{PM},0}$ is the target's prime-meridian angle at the initial epoch (`prime_meridian`, in degrees), $\dot{\theta}$ is the target's rotation rate (`rotation_rate`, in deg/sec), t is the current mission time (`mission.true_time.time`) and t_0 is the initial mission time (`mission.true_time.t_initial`), both in seconds, so that $\theta_{\text{PM}}(t)$ is the instantaneous rotation angle of the body's prime meridian about its spin axis at time t. $\mathbf{R}_{\text{pole}}$ is the constant pole-orientation matrix from the previous equation,

and $\mathbf{R}_{\mathrm{J2000}\leftarrow\mathrm{body}}(t)$ is the final time-varying rotation matrix that maps vectors from the target's body-fixed frame into the J2000 inertial frame.

This is implemented in `func_update_target_rotation_matrix` and handled by `cspice_pxform` for targets possessing a dedicated SPICE body-fixed frame (i.e., Solar System bodies, moons, minor planets, and comets - See Appendix A for a list of those objects pre-encoded within MuSCAT). For targets without a dedicated SPICE body-fixed frame, this is still implemented in `func_update_target_rotation_matrix` but in the otherwise branch of the switch on `obj.name`. The code computes `theta_PM = obj.prime_meridian + obj.rotation_rate * (mission.true_time.time - mission.true_time.t_initial)`, builds `Rot_Z_PM` from `cosd(theta_PM)` and `sind(theta_PM)` terms, and forms `rot = Rot_Z_PM * obj.rotation_matrix_pole_RA_Dec`, the J2000-to-body-frame rotation. This is then transposed (`rot = rot';`) to obtain the body-to-J2000 rotation, which is returned and assigned to `obj.rotation_matrix` inside `func_main_true_target` at every time step. Note that `obj.rotation_rate` itself is set once in the constructor as `obj.rotation_rate = 360/obj.rotation_period`, converting the body's rotation period (in seconds) into a mean rotation rate in degrees per second.

3.7.2.3 Earth Prime Meridian from Julian Date UT1 (Sidereal Motion)

$$\theta_{\mathrm{PM,Earth,0}} = \mathrm{mod}\Big(2\pi\big[\,0.7790572732640 + 1.00273781191135448\,(\mathrm{JD}_{\mathrm{UT1}} - 2451545.0)\,\big],\ 2\pi\Big) \times \frac{180^\circ}{\pi} \tag{3.16}$$

Here $\mathrm{JD}_{\mathrm{UT1}}$ is the Julian Date in the UT1 time system corresponding to the mission's initial epoch, 2451545.0 is the Julian Date of the J2000.0 epoch, the constants 0.7790572732640 (revolutions) and 1.00273781191135448 (revolutions per UT1 day) are the Earth Rotation Angle sidereal-motion coefficients taken from NASA TP 20220014814, Section 4.3.2, and $\theta_{\mathrm{PM,Earth,0}}$ is the resulting Earth prime-meridian angle at the initial epoch, wrapped into $[0^\circ, 360^\circ)$ and expressed in degrees.

This is implemented in the `'Earth'` case of the constructor switch statement. The code first loads a leapseconds kernel via `cspice_furnsh` on `naif0012.tls`, then obtains the UTC Julian Date string via `cspice_et2utc(mission.true_time.t_initial_date, 'J', 6)`, parses it into `JD_UTC`, and sets `JD_UT1 = JD_UTC` as an approximation (the code notes UTC tracks UT1 to within ± 0.9 s). It then evaluates the sidereal-motion polynomial directly as `obj.prime_meridian = rad2deg(wrapTo2Pi(2*pi*(0.7790572732640 + 1.00273781191135448 * (JD_UT1 - 2451545.0))))`, storing the result in `obj.prime_meridian` (degrees) as the Earth-specific initial value later consumed as $\theta_{\mathrm{PM,0}}$ in the time-varying prime-meridian equation above. This special-cased formula overrides the generic pole/prime-meridian handling used for other bodies, though the code also subsequently loads the SPICE `pck00011.tpc` kernel, and Earth's actual rotation matrix is computed at runtime via `cspice_pxform('IAU_EARTH','J2000',...)` in `func_update_target_rotation_matrix` rather than via the `theta_PM` formula.

3.7.3 CML 2–5 Suitability

3.7.3.1 What is included and correctly implemented

For the nine currently hard-coded target bodies (Bennu, Apophis, Toutatis, Itokawa, 1996HW1, Earth, IBD_Asteroid, Enceladus, Uranus), `True_Target_SPICE` already provides fidelity that is at or above what is typically required for CML 2–3 target-body dynamics, and adequate for CML 4–5 refinement.

- **High-fidelity shape and gravity models.** The six small bodies load a true polyhedral shape model via `readObj` from vendor/PDS-derived `.obj` mesh files. The three large bodies load a simplified spherical primitive. They all load a real spherical-harmonic gravity field through the `GravityField` class from body-specific harmonics text files. This is materially higher-fidelity than is normally expected at CML 2–3, where a point-mass or low-degree ellipsoid approximation would suffice for a small body.
- **SPICE-kernel state propagation.** State propagation for every body is read directly from SPICE kernels via `cspice_spkezr`, so position/velocity accuracy is limited only by kernel quality, not by any approximate propagator.
- **Two-tier rotation handling.** Rotation is handled two ways depending on frame availability: Earth, Enceladus, Uranus, Moon use full SPICE frame transforms via `cspice_pxform`, which is high-fidelity; other bodies lacking a dedicated SPICE PCK frame fall back to a pole-RA/Dec plus linear prime-meridian rotation model, a standard and reasonable medium-fidelity approach.
- **Breadth without new modeling code.** Taken together, the class already spans a broad diversity of target types out of the box, enabling many plausible CML 2–5 mission-concept trade studies to be run without any new modeling code, provided the target of interest is one of the nine already implemented.

3.7.3.2 What is not included / gaps

The target list is a closed, hard-coded `switch/case` enumeration.

- **Closed target enumeration.** Adding any target body not already named requires a developer to write a new case block supplying a shape-model mesh file, a gravity-harmonics file, and pole RA/Dec/prime-meridian parameters – source-code-level development work, not a user-facing configuration option, and a real friction point at CML 2, where the target body itself is often still an open trade variable.

  To add a new target body, the user needs to provide the following information:

  - Rotation period
  - Gravity file with spherical-harmonics gravity parameters, that can be read by `GravityField` function
  - Shape model or an image of the target body (that can be wrapped around a sphere)
  - SPICE filename and ID
  - Pole information either provided by SPICE or computed using RA, Dec, Pre Meredian

  Then the developer writes a new case block with this information.
- **Non-tumbling bodies.** Toutatis is explicitly flagged in a run-time `warning('Toutatis is tumbling!')` yet its rotation still falls through to the same simple single-axis pole-RA/Dec plus linear-prime-meridian rotation model used for non-tumbling bodies, so the rotational fidelity does not actually reflect its known tumbling state.
- **SPICE-ephemeris-only scope.** This class is intentionally SPICE-ephemeris-only: it performs no dynamics integration, no perturbation modeling, and provides no mechanism to apply

control or disturbance forces to the target body itself – that capability instead lives in the sibling `True_Target_Computed` class. Users should treat `True_Target_SPICE` as the low-effort, high-fidelity-ephemeris option to use only when a suitable SPICE trajectory kernel for the target already exists.

3.8 True_Target_Computed

The True_Target_Computed class also represents the mission's primary target body, but propagates its position and velocity by numerically integrating the equations of motion (via RK4) rather than reading them directly from SPICE. In addition to gravity from the Sun and other solar system bodies, it accounts for disturbance forces from solar radiation pressure and the Yarkovsky effect, as well as control forces/torques imparted on the target (e.g., by spacecraft electric propulsion thrusters), and computes shape-model-derived quantities such as the area-weighted mean radius.

3.8.1 Inputs / Outputs

3.8.1.1 Inputs / Dependencies

- init_data **(constructor-time):** target_name (currently only 'Apophis' and 'Apophis 25-meter-radius' are implemented; other names raise an error), each case hardcoding gravity-field file, shape-model file, SPICE kernel file/ID, pole RA/Dec/prime-meridian, and Yarkovsky/SRP material parameters (Bond_Albedo, SRP_reflectance_factor, Gamma, eps).
- True_Time **(mission-level):** mission.true_time.date, prev_date, time_position_array, time_step, time_step_position_array, t_initial, time are read for SPICE queries, numerical integration (RK4) time stepping, and rotation-angle computation.
- True_Solar_System **(mission-level, heavily used):** gravitational_constant (mass from mu), SS_body{i}.position/mu for every tracked body (N-body perturbation acceleration), index_Sun, solar_constant_AU, AU_distance, light_speed, sigma (SRP and Yarkovsky force calculations).
- Storage **(mission-level):** mission.storage.num_storage_steps sizes the extensive obj.store.* arrays (position, velocity, acceleration, SPICE variants, control/disturbance/SRP/Yarkovsky forces); flag_store_this_time_step and k_storage gate/index updates.
- Integrators **and** func_orbit_inertial_static_SB_v6_Sun_centered **(supporting functions, not classes):** used for RK4 numerical integration of the target's dynamics-based trajectory.
- True_SC_Science_Processor **(same mission, all spacecraft/HW instances, external mission repository):** not instantiated by any mission script within MuSCAT_Matlab_v2 itself (every Mission_*.m in that repository uses True_Target_SPICE instead), but this class is designed to also be instantiated and actively driven by mission-specific code living in a separate, firewalled mission repository that adds MuSCAT_Matlab_v2 to its MATLAB path and reuses this class unmodified. In that configuration, obj.control_force/obj.control_torque are read-and-accumulated-into externally every step by a mission-specific science-processor plugin (reached through True_SC_Science_Processor's dispatch) — see the write-back note below.

3.8.1.2 Outputs

- **Internal / stored outputs:** dynamics-integrated position/velocity/acceleration/position_array plus parallel SPICE-only position_SPICE/velocity_SPICE/acceleration_SPICE/position_array_SPICE (kept for comparison), control_force/control_torque,

`disturbance_force`, `SRP_disturbance_force`, `Yarkovsky_disturbance_force`, `rotation_matrix`, `area_weighted_mean_radius/min_radius/max_radius`, and the full `obj.store.*` time histories of all of the above.

- **Not instantiated within `MuSCAT_Matlab_v2` alone:** none of the `Mission_*.m` scripts inside `MuSCAT_Matlab_v2` instantiate this class (they all use `True_Target_SPICE`); within that repository in isolation it is a parallel/alternative implementation with no live consumers.
- **Instantiated and consumed by mission-specific code in a separate, firewalled mission repository:** `mission.true_target{i_target} = True_Target_Computed(init_data, mission)` is called directly in that repository's mission script, making this the live target-truth class there; its `position`, `velocity`, `rotation_matrix`, `shape_model`, `mu`, `radius` are consumed by the same classes documented for `True_Target_SPICE` (orbit propagation, gravity-gradient, estimate-orbit, plotting), since both target classes expose the same property names and are interchangeable from a consumer's point of view.
- **External write-back confirmed (in that mission repository):** the mission's science-processor plugin sets `mission.true_target{1}.control_force = mission.true_target{1}.control_force + EP_thruster_force` and the analogous accumulation line for `control_torque`, once per EP-thruster-equipped science-processor instance per step. This is exactly the historical `True_Target_Computed.func_update_SB_control_force`-style coupling described for `True_SC_Science_Processor` below, just implemented as a direct accumulation in the mission's plugin rather than a dedicated method on this class. Of the two, only `control_force` is actually consumed by the dynamics: `func_orbit_inertial_static_SB_v6_Sun_centered.m` adds it into the integrated translational acceleration as `(1e-3 * obj.control_force'/obj.mass)`; `control_torque` is stored and plotted but never integrated anywhere, since the target's attitude is currently prescribed kinematically (prime meridian plus a constant rotation rate) rather than dynamically. Both `control_force` and `control_torque` are reset to zero at the end of every `func_main_true_target` call.

3.8.2 Mathematical Equations

3.8.2.1 Sun-Centered Equations of Motion and Third-Body Gravitational Perturbations

$$\ddot{\vec{r}}_{\mathrm{Ast}} = -\mu_{\odot}\frac{\vec{r}_{\mathrm{Ast}}}{|\vec{r}_{\mathrm{Ast}}|^3} + \sum_{\substack{i=1 \\ i\neq\odot}}^{N_{\mathrm{SS}}} \mu_i \left(\frac{\vec{r}_{i\to\mathrm{Ast}}}{|\vec{r}_{i\to\mathrm{Ast}}|^3} - \frac{\vec{r}_{i\to\odot}}{|\vec{r}_{i\to\odot}|^3} \right) + \frac{\vec{F}_{\mathrm{control}} + \vec{F}_{\mathrm{dist}}}{m_{\mathrm{Ast}}} \tag{3.17}$$

Here $\vec{r}_{\mathrm{Ast}}$ is the position of the target body (asteroid) relative to the Sun in the J2000 inertial frame [km]; $\mu_{\odot}$ and μ_i are the standard gravitational parameters GM [km^3/s^2] of the Sun and of the i-th solar-system perturbing body (N_{SS} bodies total), respectively; $\vec{r}_{i\to\mathrm{Ast}} = \vec{r}_{\mathrm{Ast}} - \vec{r}_i$ is the vector from body i to the asteroid, and $\vec{r}_{i\to\odot} = -\vec{r}_i$ is the vector from body i to the Sun, both in km; $\vec{F}_{\mathrm{control}}$ and $\vec{F}_{\mathrm{dist}}$ are the net control (electric-propulsion thruster reaction) and disturbance (SRP + Yarkovsky) forces [N] acting on the asteroid, and m_{Ast} is its mass [kg]. The first sum term is the direct attraction of each other solar-system body on the asteroid; the subtracted term is the indirect (Sun-centered-frame) acceleration of the Sun itself due to that same body, which must be removed because the whole state vector is expressed relative to the Sun.

This equation is the right-hand side evaluated inside `func_orbit_inertial_static_SB_v6_Sun_centered.m`, which is passed as `odefun` to the fixed-step integrator inside `True_Target_`

`Computed.func_main_true_target`. For each solar-system body index `i_SS_body`, the code computes `R_SS_Body_to_Ast` (i.e. $\vec{r}_{i\to\mathrm{Ast}}$) via SPICE calls to `cspice_spkezr`, accumulates `acc = acc - (mu_SS_body * R_SS_Body_to_Ast / dist_SS_Body_to_Ast^3)` for the direct term, and for every non-Sun body adds back `mu_SS_body * R_SS_Body_to_Sun / dist_SS_Body_to_Sun^3` to correct for the Sun-centered frame. The disturbance and control accelerations are added as `(1e-3 * obj.control_force' / obj.mass) + (1e-3 * obj.disturbance_force' / obj.mass)` (the factor 10^{-3} converts N/kg = m/s^2 to km/s^2). Separately, `True_Target_Computed.func_update_SS_Body_disturbance_force` independently computes and stores just the direct per-body Newtonian accelerations $\mu_i\vec{r}_{i\to\mathrm{Ast}}/|\vec{r}_{i\to\mathrm{Ast}}|^3$ in `obj.acceleration_SS_body` for diagnostic/telemetry purposes.

3.8.2.2 Solar Radiation Pressure (SRP) Disturbance Force

$$\vec{F}_{\mathrm{SRP}} = C_R\,\frac{S_0}{c}\left(\frac{r_{\mathrm{AU}}}{r_{\odot\to\mathrm{Ast}}}\right)^2 A_{\mathrm{proj}}\,\hat{r}_{\odot\to\mathrm{Ast}} \tag{3.18}$$

$C_R = 1 + \frac{4}{9}A_B$ is the (dimensionless) SRP reflectance factor, where A_B is the Bond albedo; S_0 is the solar constant at 1 AU [W/m^2]; c is the speed of light [m/s]; r_{AU} is the length of one astronomical unit [km]; $r_{\odot\to\mathrm{Ast}}$ is the instantaneous Sun-to-target distance [km]; $A_{\mathrm{proj}} = \pi R^2$ is the target's projected (cross-sectional) area [m^2], approximated as a sphere of radius R; and $\hat{r}_{\odot\to\mathrm{Ast}}$ is the unit vector pointing from the Sun to the target, so that the SRP force always pushes the body radially outward from the Sun. $\vec{F}_{\mathrm{SRP}}$ is in Newtons.

This is implemented in `True_Target_Computed.func_update_SRP_disturbance_force`. The code forms `R_Sun_to_SB` as `obj.position` minus the Sun's position, takes `distance_SB_Sun = norm(R_Sun_to_SB)`, and sets `SB_projected_area = pi*((obj.radius * 1e3)^2)` (converting `obj.radius` from km to m). The scalar magnitude `F_SRP_magnitude` is computed as `obj.SRP_reflectance_factor * (mission.true_solar_system.solar_constant_AU / mission.true_solar_system.light_speed) * (mission.true_solar_system.AU_distance / distance_SB_Sun)^2 * SB_projected_area`, where `obj.SRP_reflectance_factor` was set in the constructor as `1 + (4/9)*obj.Bond_Albedo`. The result is directed along `func_normalize_vec(R_Sun_to_SB)` and stored in `obj.SRP_disturbance_force`, which is then added into `obj.disturbance_force` inside `func_main_true_target`.

3.8.2.3 Yarkovsky Thermal Parameters

$$\Phi = S_0\left(\frac{r_{\mathrm{AU}}}{r}\right)^2, \qquad T_* = \left[\frac{(1-A_B)\,\Phi}{\varepsilon\,\sigma}\right]^{1/4}, \qquad \Theta_d = \frac{\Gamma\sqrt{\omega}}{\varepsilon\,\sigma\,T_*^3}, \qquad \Theta_s = \frac{\Gamma\sqrt{n}}{\varepsilon\,\sigma\,T_*^3} \tag{3.19}$$

Φ is the solar flux [W/m^2] at the target's current heliocentric distance r [km], scaled from the 1-AU solar constant S_0 [W/m^2] using the AU length r_{AU} [km]; T_* is the characteristic (sub-solar equilibrium) surface temperature scale [K], with A_B the Bond albedo, ε the surface emissivity, and σ the Stefan–Boltzmann constant [W m^{-2} K^{-4}]; Θ_d and Θ_s are the dimensionless diurnal and seasonal thermal parameters, built from the thermal inertia Γ [J m^{-2} K^{-1} s$^{-1/2}$], the body's rotation (diurnal) angular rate ω [rad/s], and the orbital mean motion n [rad/s].

These quantities are computed in `True_Target_Computed.func_update_Yarkovsky_disturbance_force`. The flux `Phi = S0 * (AU/r)^2` uses `S0 = mission.true_solar_system.solar_constant_AU` and `AU = mission.true_solar_system.AU_distance`, with `r = norm(r_vec)` the Sun-to-target distance. `Tstar` is computed directly from `obj.Bond_Albedo`, `Phi`, `obj.eps`, and `sigma = mission.true_solar_system.sigma`. The rotation rate is `omega = 2*pi/obj.rotation_period`, while the mean motion `n = sqrt(muSun / a^3)` is derived from the osculating semi-major axis `a` obtained from the vis-viva relation `a = 1 / (2/r - (v^2)/muSun)`

using the heliocentric speed `v`. Finally `Theta_d = (obj.Gamma * sqrt(omega)) / (obj.eps * sigma * Tstar^3)` and `Theta_s = (obj.Gamma * sqrt(n)) / (obj.eps * sigma * Tstar^3)` implement the two thermal parameters using the body's thermal inertia `obj.Gamma` and emissivity `obj.eps`.

3.8.2.4 Yarkovsky Along-Track Acceleration

$$W(\Theta) = \frac{0.5\,\Theta}{1 + \Theta + 0.5\,\Theta^2}, \qquad a_t = \frac{4}{9}\,\frac{(1 - A_B)\,\Phi}{c\,\rho\,R}\Big[W(\Theta_d)\cos\gamma \;-\; W(\Theta_s)\sin^2\gamma\Big] \tag{3.20}$$

$W(\Theta)$ is the Lowry/Vokrouhlický-type thermal-lag efficiency function evaluated at either the diurnal (Θ_d) or seasonal (Θ_s) thermal parameter; a_t is the resulting along-track (tangential, i.e. along the orbital velocity direction) acceleration magnitude [m/s^2] produced by anisotropic thermal re-emission (the Yarkovsky effect); c is the speed of light [m/s]; ρ is the target's bulk density [kg/m^3]; R is its mean radius [m]; and γ is the obliquity, the angle between the target's spin pole and its orbit-normal vector [rad]. The first term ($\propto \cos\gamma$) is the diurnal Yarkovsky component, and the second (subtracted, $\propto \sin^2\gamma$) is the seasonal component.

The obliquity γ (`obliq_rad`) is computed by first getting the orbital inclination and RAAN from `Rv2Coe(rv_SB, muSun)`, forming the orbit-normal unit vector `n_hat`, forming the spin-pole unit vector `s_hat` from `obj.pole_RA` and `obj.pole_Dec`, and taking `obliq_rad = acos(dot(s_hat, n_hat))`. The efficiency function is implemented as the anonymous function `W = @(Th) (0.5*Th) ./ (1 + Th + 0.5*Th.^2)` and evaluated at `Theta_d` and `Theta_s`. The bulk density is `rho = obj.mass/((4/3)*pi*(R_m^3))` with `R_m = obj.radius*1000` [m]. The leading coefficient is `coef_mps2 = (4/9) * ((1 - obj.Bond_Albedo) * Phi) / (c * rho * R_m)`, and the two components are formed as `a_t_diurnal_mps2 = coef_mps2 * W(Theta_d) * cos(obliq_rad)` and `a_t_seasonal_mps2 = -coef_mps2 * W(Theta_s) * sin(obliq_rad)^2`, summed into `a_t_mps2`. This scalar is multiplied by the heliocentric along-track unit vector `that = v_vec / v` and converted to km/s^2 (`aY_kmps2 = (a_t_mps2/1000)*that`) before being converted to a force via `obj.Yarkovsky_disturbance_force = (1e3) * aY_kmps2 * obj.mass`, stored as an [N] vector and added into `obj.disturbance_force` in `func_main_true_target`.

3.8.2.5 Area-Weighted Mean Radius from the Shape Model

$$V_{\text{tot}} = \sum_{k=1}^{N_f} \frac{1}{6}\,\vec{v}_{1,k}\cdot(\vec{v}_{2,k}\times\vec{v}_{3,k}), \qquad \vec{r}_{\text{COM}} = \frac{1}{V_{\text{tot}}}\sum_{k=1}^{N_f}\frac{\vec{v}_{1,k}+\vec{v}_{2,k}+\vec{v}_{3,k}}{4}\,V_k, \qquad \bar{R} = \frac{\displaystyle\sum_{k=1}^{N_f} R_k\,A_k}{\displaystyle\sum_{k=1}^{N_f} A_k} \tag{3.21}$$

N_f is the number of triangular faces in the shape model; $\vec{v}_{1,k}, \vec{v}_{2,k}, \vec{v}_{3,k}$ are the position vectors (relative to the shape-model origin) of the three vertices of face k; $V_k = \frac{1}{6}\vec{v}_{1,k}\cdot(\vec{v}_{2,k}\times\vec{v}_{3,k})$ is the signed volume of the tetrahedron formed by the origin and face k, and V_{tot} is their sum, i.e. the total enclosed (signed) volume; $\vec{r}_{\text{COM}}$ is the volumetric center-of-mass of the body, obtained as the volume-weighted average of each tetrahedron's centroid $(\vec{v}_{1,k}+\vec{v}_{2,k}+\vec{v}_{3,k})/4$; A_k is the surface area of face k (after re-centering vertices on $\vec{r}_{\text{COM}}$); R_k is the distance from the center of mass to the centroid of face k; and $\bar{R}$ is the resulting area-weighted mean radius.

This computation appears directly in the `True_Target_Computed` constructor. The shape model vertices/faces `V = obj.shape_model.Vertices`, `F = obj.shape_model.Faces` are used to extract per-face vertex triplets `v1, v2, v3`; `tetra_volumes = dot(v1, cross(v2,v3,2), 2)/6` and `total_volume_signed = sum(tetra_volumes)` implement V_k and V_{tot}; `tetra_centroids`

`= (v1+v2+v3)/4` and `COM_SB = sum(tetra_centroids .* tetra_volumes, 1) / total_volume_signed` implement $\vec{r}_{\mathrm{COM}}$. Vertices are then re-centered (`V = V - COM_SB`), face areas are recomputed as `face_areas = 0.5 * vecnorm(cross(v2-v1, v3-v1, 2), 2, 2)` (A_k), face centroids as `face_centroids = (v1+v2+v3)/3`, and face radii as `face_radii = vecnorm(face_centroids, 2, 2)` (R_k). The final area-weighted mean radius is stored (converted from m to km) as `obj.area_weighted_mean_radius = sum(face_radii .* face_areas) / total_area * 1e-3`, with the extremes stored in `obj.min_radius` and `obj.max_radius`.

3.8.2.6 Fourth-Order Runge–Kutta (RK4) Propagation

$$\begin{aligned} k_1 &= f(t_n, \vec{X}_n), \qquad k_2 = f\left(t_n + \tfrac{\Delta t}{2}, \vec{X}_n + \tfrac{\Delta t}{2} k_1\right), \\ k_3 &= f\left(t_n + \tfrac{\Delta t}{2}, \vec{X}_n + \tfrac{\Delta t}{2} k_2\right), \qquad k_4 = f(t_n + \Delta t, \vec{X}_n + \Delta t\, k_3) \end{aligned} \tag{3.22}$$

$$\vec{X}_{n+1} = \vec{X}_n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right) \tag{3.23}$$

$\vec{X}_n = [\vec{r}_{\mathrm{Ast}}, \vec{v}_{\mathrm{Ast}}]$ is the 6-element target state (position and velocity) at time t_n; $f(t, \vec{X}) = [\vec{v}, \vec{a}(t, \vec{X})]$ is the equation-of-motion function returning the state derivative (velocity and acceleration, per the Sun-centered equation of motion above); Δt is the integration time step [s]; and k_1–k_4 are the standard RK4 stage derivatives used to advance the state from t_n to $t_{n+1} = t_n + \Delta t$.

This is implemented generically in `Integrators.rk4` (in `Integrators.m`), where `odefun` plays the role of f, `state` is $\vec{X}_n$, and `dt` is Δt; the four stages `k1..k4` are combined as `state_next = state + (k1 + 2*k2 + 2*k3 + k4) * (dt/6)`. In `True_Target_Computed.func_main_true_target`, this integrator is driven with `odefun = @(t,X) func_orbit_inertial_static_SB_v6_Sun_centered(t, X, obj, mission)` (the Sun-centered equation of motion described earlier) inside a loop over `this_time_array`, calling `Integrators.rk4(...,this_time_array(ii-1), X(ii-1,:).', mission.true_time.time_step_position_array)` to propagate `[obj.position, obj.velocity]` forward; the final row of `X` updates `obj.position` and `obj.velocity` for the current mission time step.

3.8.3 CML 2–5 Suitability

3.8.3.1 What is included and correctly implemented

`True_Target_Computed.m` propagates the primary target body's Sun-centered state via fixed-step RK4 integration driven by a dynamics function that accumulates several physically distinct perturbation sources into a single disturbance force.

- **N-body point-mass gravity.** Newtonian point-mass accelerations are computed from every body in `mission.true_solar_system`.
- **Solar radiation pressure.** A solar radiation pressure model is included, using cross-sectional area, solar constant at 1 AU, speed of light, and a Bond-albedo-derived reflectance factor.
- **Full Yarkovsky thermophysical model.** The model computes orbital obliquity from orbital elements and pole RA/Dec, solar flux at the current heliocentric distance, a radiative-equilibrium temperature scale, and both diurnal and seasonal dimensionless thermal parameters built from thermal inertia, emissivity, spin rate, and mean motion, combined through the standard efficiency function to produce along-track accelerations.

- **Control-force feedback.** Closed-loop control-force/torque feedback from spacecraft electric-propulsion thrusters acting on the target is included, including sanity checks on momentum-transfer efficiency.
- **Shape-derived geometric utility.** The constructor computes a genuinely useful geometric utility: an area-weighted mean radius, minimum radius, and maximum radius derived directly from the shape model via a tetrahedron signed-volume/centroid method.
- **Overall fidelity assessment.** Taken together, this level of physics fidelity – N-body perturbations, SRP, a full diurnal/seasonal Yarkovsky thermophysical model, and control-force feedback from actuation – exceeds what is typically demanded at CML 2–3 and meets or approaches CML 4–5 rigor for small-body proximity-operations trade studies, such as asteroid deflection/redirect concepts where Yarkovsky and SRP perturbations matter for long-duration station-keeping or deflection-verification trades.

3.8.3.2 What is not included / gaps

The target-body switch-case in the constructor currently implements only two entries.

- **Closed target-body enumeration.** Only `'Apophis'` and `'Apophis 25-meter-radius'` are implemented (differing only in gravity-field file and harmonic degree); any other value throws `error('Invalid Target type')`. A user wishing to run this same orbital/Yarkovsky/SRP physics engine on a different target – e.g., Bennu or Itokawa, both of which already have SPICE-only support in the sibling `True_Target_SPICE.m` class – would need to hand-add a new case here supplying a gravity field file, shape model file, pole RA/Dec, rotation period, Bond albedo, thermal inertia, and emissivity for that body. This is real code-level rework specific to each new body, not a configuration change, and is a barrier to the rapid target-flexibility expected at CML 2 concept trades, even though the underlying integration and force-modeling engine itself is reusable once the parameters are supplied.
- **Fixed-step integration with no adaptive control.** The numerical integration is fixed-step RK4 with no adaptive step-size control or local error estimation (unlike the commented-out `ode15s`/`ode45` calls left in the code, which are not used); there is no runtime check, warning, or diagnostic if the fixed step size is too coarse for the perturbation magnitudes present, so integration error could silently accumulate over long mission durations.
- **Single-target structure.** This class is structured to track exactly one target body: the constructor stores a single `obj.name`, a single set of physical/thermal parameters, and a single position/velocity state (in contrast to `True_Solar_System`, which holds an indexed cell array with a corresponding body count). Consequently, any mission formulation trade study that needs to compare multiple candidate target bodies simultaneously, or model a binary/multi-asteroid system, would need to instantiate and manage multiple `True_Target_Computed` objects externally, since the class itself provides no internal multi-target bookkeeping.

3.9 True_Time

The True_Time class tracks all time-related variables for the simulation, including the dual-rate time loop: a coarse simulation time step for orbital/subsystem dynamics and a finer time step for the attitude dynamics loop. It converts between simulation time and calendar date (via SPICE), maintains time-loop counters, and prints simulation progress (elapsed/remaining time and memory usage) at configurable intervals.

3.9.1 Inputs / Outputs

3.9.1.1 Inputs / Dependencies

- init_data **(constructor-time):** t_initial, t_final, time_step, t_initial_date_string, optional time_step_attitude (defaults to time_step), optional print_progress_steps, optional time_step_position_array (defaults to min(10, time_step, time_step_attitude)), optional data.
- Storage **(mission-level, read only in store methods):** mission.storage.num_storage_steps, num_storage_steps_attitude size the obj.store.time/date/time_attitude arrays in func_initialize_time_store; mission.storage.flag_store_this_time_step and flag_store_this_time_step_attitude, k_storage, k_storage_attitude gate/index the two store-update methods.
- **External helper** cal2sec **(not a MuSCAT class):** converts t_initial_date_string into t_initial_date (seconds from J2000).
- No other True_Environment or Software class is read by True_Time — it is foundational and self-contained aside from the Storage coupling needed to size/gate its own store buffers.

3.9.1.2 Outputs

- **Internal / stored outputs:** time, date, time_attitude, k, k_attitude, t_initial, t_final, time_step, time_step_attitude, num_time_steps, num_time_steps_attitude, time_position_array, num_time_steps_position_array, prev_time, prev_date, and obj.store.time/date/time_attitude.
- **Reported to virtually every True_Environment, True_SC, True_Sensors_Actuators and Software_SC class:** mission.true_time.time, date, time_step, time_step_attitude, k, k_attitude, num_time_steps, time_position_array, prev_date are read pervasively (84 files reference mission.true_time.*) for: SPICE ephemeris queries (True_Solar_System, True_Target_SPICE/Computed), storage-step gating logic (via Storage), data-rate-to-data conversion (True_Ground_Station, True_GS_Radio_Antenna), dynamics integration time grids, and progress printing in func_update_true_time_date.
- **Reported to** Storage**:** Storage's constructor and func_update_storage_flag/_attitude read mission.true_time.time/time_attitude/t_initial/t_final/k/num_time_steps/k_attitude/num_time_steps_attitude to size storage buffers and decide when to flip the store flags each step — the inverse coupling to the dependency noted above.
- **Reported to** Main/main_v3.m**:** the main loop calls func_update_true_time_date/func_update_true_time_attitude once per step/sub-step to advance simulation time, and reads mission.true_time.k/num_time_steps to control loop bounds.

- **Reported to plotting/report code:** `obj.store.time` is read as the x-axis for nearly all time-series plots (`func_plot_orbit_estimator.m`, `func_plot_telecom.m`, `func_plot_GS_data_received.m`, etc.) and `t_initial_date_string/t_final/time_step` are read for report text in `Storage.func_initialize_report`.
- No external class writes back into this class's properties — a codebase-wide search for `mission.true_time.<field> =` assignments outside `True_Time.m` found none.

3.9.2 Mathematical Equations

3.9.2.1 Dual-Rate Time-Step Synchronization (Attitude Sub-Stepping)

$$N_{\text{att}} = \frac{\Delta t}{\Delta t_{\text{att}}}, \qquad \Delta t_{\text{att}} \leq \Delta t, \qquad N_{\text{att}} \in \mathbb{Z} \tag{3.24}$$

Here Δt (`time_step`, in seconds) is the coarse simulation time step used to advance the main orbital/subsystem loop, and Δt_{att} (`time_step_attitude`, in seconds) is the finer time step used to integrate the attitude dynamics, which typically evolve much faster than orbital/translational states and therefore require a shorter numerical-integration step. N_{att} (`num_time_steps_attitude`, dimensionless integer) is the number of attitude-dynamics sub-steps that must be executed within each single coarse time step so that the two loops re-synchronize exactly at the end of every coarse step.

In the constructor, if `init_data.time_step_attitude` is not supplied, `obj.time_step_attitude` defaults to `obj.time_step` (i.e., no sub-stepping, $N_{\text{att}} = 1$). The code first enforces $\Delta t_{\text{att}} \leq \Delta t$, raising an error (`'Attitude time step bigger than Position time step!'`) if `obj.time_step_attitude > obj.time_step`. It then computes `obj.num_time_steps_attitude = (obj.time_step/obj.time_step_attitude)` and checks that this quotient is an integer via `isinteger(int32(obj.num_time_steps_attitude))`, raising an error if it is not. During the simulation, `func_update_true_time_date` advances `obj.time` by one full `obj.time_step` per call (the coarse loop index `k`), while the inner attitude loop calls `func_update_true_time_attitude` N_{att} times per coarse step, each call advancing `obj.time_attitude` by `obj.time_step_attitude` (indexed by `k_attitude`), guaranteeing that `obj.time_attitude` exactly equals `obj.time` after the N_{att}-th sub-step.

3.9.2.2 Total Number of Simulation Time Steps

$$N_{\text{steps}} = \left\lceil \frac{t_f - t_0}{\Delta t} \right\rceil \tag{3.25}$$

Here t_0 (`t_initial`, in seconds) and t_f (`t_final`, in seconds) are the simulation-relative start and end times of the mission (both measured on the same internal clock, with t_0 typically 0), Δt (`time_step`, in seconds) is the coarse simulation time step, and N_{steps} (`num_time_steps`, dimensionless integer) is the resulting total number of coarse-loop iterations required to advance the simulation clock from t_0 to t_f. The ceiling operation ensures that the loop runs at least far enough to reach or pass t_f even when $(t_f - t_0)$ is not an exact integer multiple of Δt.

This is computed once in the constructor as `obj.num_time_steps = ceil((obj.t_final - obj.t_initial)/obj.time_step)`, immediately after `obj.t_initial`, `obj.t_final`, and `obj.time_step` are assigned from `init_data`. `obj.num_time_steps` subsequently governs the size of the main simulation loop (the range of the loop index `k` passed into `func_update_true_time_date`), determines the default value of `obj.print_progress_steps` (`ceil((obj.num_time_steps)/1000)` when

not explicitly specified), and is used inside `func_update_true_time_date` to estimate the remaining number of iterations (`obj.num_time_steps - obj.k`) for the progress/ETA printout.

3.9.2.3 Conversion from Simulation-Relative Time to Absolute Ephemeris Date

$$d(t) = d_0 + t, \qquad d_0 = \text{cal2sec}\,(s_0) \tag{3.26}$$

Here t (`time`, in seconds) is the current simulation-relative time measured from the start of the run, d_0 (`t_initial_date`, in seconds past J2000) is the absolute SPICE ephemeris time corresponding to the mission's start epoch, s_0 (`t_initial_date_string`) is the human-readable calendar-date string specifying that start epoch (format `DD-MMM-YYYY HH:MM:SS`), and $d(t)$ (`date`, in seconds past J2000) is the absolute ephemeris time corresponding to simulation-relative time t. This relationship is the bridge between MuSCAT's internal relative clock, which the dynamics/propagation loops operate on, and the absolute time system used by SPICE for all ephemeris, attitude kernel, and celestial-body lookups.

In the constructor, `obj.t_initial_date` is computed once via `obj.t_initial_date = cal2sec(obj.t_initial_date_string)`, converting the user-supplied calendar string into seconds past J2000, and the initial absolute date is set with `obj.date = obj.t_initial_date + obj.time` (with `obj.time` initialized to `obj.t_initial`). Thereafter, every time `obj.time` is advanced, `obj.date` is recomputed using the same affine relationship: inside `func_update_true_time_date`, after `obj.time = obj.time + obj.time_step`, the line `obj.date = obj.t_initial_date + obj.time` updates the absolute date; the same pattern appears in `func_set_time`, where an externally supplied `time` value is assigned to `obj.time` and `obj.date` is recomputed identically. Because `obj.t_initial_date` is fixed for the whole run, `obj.date` always tracks `obj.time` by this constant offset, and it is `obj.date` (not `obj.time`) that is passed to SPICE-based routines elsewhere in MuSCAT for ephemeris and attitude-kernel evaluation.

3.9.3 CML 2–5 Suitability

3.9.3.1 What is included and correctly implemented

`True_Time` provides a dual-rate clock architecture that is appropriate for CML 2–5 point-design and trade-space work.

- **Dual-rate clock with safety checks.** The class maintains a coarse `time_step` for orbital/subsystem dynamics and an optional finer `time_step_attitude` for attitude dynamics, and it enforces a correctly implemented safety check on the relationship between the two: an explicit error is raised if the attitude time step is larger than the main time step, and a second check (`num_time_steps_attitude = time_step/time_step_attitude`, tested with `isinteger(int32(...))`) raises an error if the main step is not evenly divisible by the attitude step. The same integer-divisibility pattern is applied to the auxiliary `time_step_position_array` used for storing/interpolating position data. These guards prevent a common class of simulation-timing bug in which sub-loops silently drift out of sync with the parent loop.

- **SPICE-anchored absolute epoch.** The class also ties simulation-relative time to an absolute epoch via SPICE: the user-supplied calendar string is converted with `cal2sec` into seconds past J2000, and every subsequent date update is expressed in that same SPICE ephemeris-time frame, which is exactly the representation needed to interface correctly with SPICE ephemeris and attitude-kernel lookups elsewhere in MuSCAT.

- **Configurable fidelity/runtime tradeoff.** All of the key time steps are user-configurable with sensible defaults, which lets a formulation engineer easily trade fidelity against runtime – a central need during rapid CML 2–3 iteration on many concept variants.
- **Progress/ETA estimator.** Finally, `func_update_true_time_date` includes a built-in progress/ETA estimator with periodic MATLAB memory-usage reporting, which is genuinely useful for monitoring and budgeting long batch trade-study runs.

3.9.3.2 What is not included / gaps

The divisibility checks, while useful, are not bulletproof, and the class supports only fixed time-stepping.

- **Non-bulletproof divisibility check.** The divisibility checks, while a reasonable and useful safety net, are implemented via a simple floating-point division followed by an integer check; this is not a bulletproof integer-divisibility test and could in principle mis-flag certain time-step ratios due to floating-point rounding, though this is a minor code-robustness note rather than a fidelity concern.
- **Fixed time-stepping only.** More significant for trade studies is that `True_Time` implements only *fixed* time-stepping: both the main loop step and the attitude-loop step are set once in the constructor and held constant for the entire run, with no adaptive or variable-step-size logic anywhere in the class. A trade study that needs finer temporal resolution only during a specific fast-dynamics event (e.g., a slew, a close flyby, or a burn) has no built-in mechanism to refine the step locally; the user must either run the entire simulation at the finer step or manually segment/re-run the timeline with different configurations.
- **No explicit leap-second handling.** The class also contains no explicit handling of leap-second transitions – there is no logic in this file that special-cases a leap second crossing the simulation window. This is likely a non-issue in practice, since SPICE itself is expected to absorb leap-second bookkeeping internally, but users should be aware that `True_Time` itself performs no independent verification of this.
- **ETA estimator assumes constant per-step cost.** Lastly, the progress/ETA estimator computes the average elapsed wall-clock time per completed step and linearly extrapolates it to estimate time remaining; this assumes a roughly constant per-step computational cost and could produce misleading ETAs if downstream event-driven code causes per-step cost to vary significantly over the run – a minor usability caveat rather than a simulation-fidelity gap.

Chapter 4

SC Physics Based Simulation Layer Classes

4.1 True_SC_ADC

The True_SC_ADC class represents the "truth" attitude determination and control state of a single spacecraft: its quaternion attitude, angular velocity, and the torques and momenta that drive their evolution. It owns the propagation of rigid-body rotational dynamics (currently only a 'Rigid' mode is implemented; any other mode_true_SC_attitude_dynamics_selector throws an error), converting the current quaternion and body rates forward in time using a 4th-order Runge-Kutta integrator, and it maintains the derived rotation matrix used elsewhere in the simulation to transform vectors between the body frame and the inertial frame.

Interface-wise, True_SC_ADC does not generate torques itself: control_torque is accumulated externally by actuator classes (e.g. thrusters, reaction wheels) and disturbance_torque is accumulated externally by environment classes such as solar radiation pressure and gravity-gradient models, both of which are reset by True_SC_ADC after each attitude time step. It queries mission.true_SC{i_SC}.true_SC_body.total_MI (the spacecraft's total mass-moment-of-inertia matrix) and loops over mission.true_SC{i_SC}.true_SC_reaction_wheel{i} objects to sum reaction-wheel angular momentum before integrating. The class also logs a full time history (attitude, angular velocity, torques, and momenta) into its store structure at each attitude storage step, which downstream analysis/plotting code consumes.

4.1.1 Inputs / Outputs

4.1.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** attitude (initial quaternion, properized via func_quaternion_properize), angular_velocity, and mode_true_SC_attitude_dynamics_selector (currently only 'Rigid' is implemented).
- True_SC_Body **(same spacecraft):** total_MI, read in func_true_SC_attitude_dynamics_rigid (to compute total_sc_momentum) and, via func_ode_attitude_dynamics, inside the RK4 integration of Euler's rigid-body equation.
- True_SC_Reaction_Wheel **(same spacecraft, all instances):** total_momentum, summed in func_update_total_wheel_momentum into obj.total_wheel_momentum, iterated over mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_reaction_wheel.

- `mission.true_time`: `time_step_attitude` (integration step for the RK4 call) and `k_attitude`/`num_time_steps_attitude` (to decide when to reset `disturbance_torque` at the end of a main time step).
- `mission.storage`: attitude-storage bookkeeping fields (`num_storage_steps_attitude`, `flag_store_this_time_step_attitude`, `k_storage_attitude`) used to size and index `obj.store`.
- **Indirectly via `func_ode_attitude_dynamics` (called from the RK4 integrator):** re-reads `mission.true_SC{i_SC}.true_SC_body.total_MI` and `mission.true_SC{i_SC}.true_SC_adc.{control_torque, disturbance_torque, total_wheel_momentum}` (i.e. its own just-updated properties) at each integration sub-step.

4.1.1.2 Outputs

- **Internal / stored outputs:** `attitude`, `angular_velocity`, `rotation_matrix`, `dot_angular_velocity`, `total_torque`, `total_wheel_momentum`, `total_sc_momentum` are computed each attitude-loop step and the first five plus momentum terms are stored in `obj.store`.
- **Reported to / read by `True_SC_Gravity_Gradient` and `True_SC_SRP`:** both read `rotation_matrix` to rotate body-frame vectors (lever arms, face normals) into the inertial frame, and both *write* into `disturbance_torque` (accumulating `disturbance_torque = disturbance_torque + obj.disturbance_torque_G2`, respectively `+ obj.disturbance_torque_SRP_body`) – see the explicit write-back note below.
- **Reported to / read by actuator classes (`True_SC_Reaction_Wheel`, `True_SC_RCS_Thruster`, `True_SC_Magnetorquer`):** each writes its generated torque into `control_torque` (accumulating `control_torque = control_torque + ...`).
- **Reported to / read by disturbance-only actuator classes (`True_SC_Chemical_Thruster`, `True_SC_EP_Thruster`):** each accumulates its own disturbance torque contribution into `disturbance_torque`.
- **Reported to / read by many sensor classes (`True_SC_Sun_Sensor`, `True_SC_Star_Tracker`, `True_SC_IMU`, `True_SC_Camera`, `True_SC_Telescope`, `True_SC_Remote_Sensing`, `True_SC_Science_Radar`, `True_SC_Solar_Panel`):** these read `rotation_matrix` (and in some cases `attitude`/`angular_velocity`) to compute boresight/pointing geometry, Sun incidence, or line-of-sight visibility; they do not write to `True_SC_ADC`.
- **Reported to / read by Software_SC estimator and control classes** (e.g. `Software_SC_Estimate_Attitude`, `Software_SC_Control_Attitude` and mission-specific variants such as `func_update_software_SC_control_attitude_DART/Oracle/Slew`): these read `attitude`, `angular_velocity`, `total_wheel_momentum` as truth/estimation inputs, and in several DART-specific control functions *write directly* into `attitude`, `angular_velocity`, and `control_torque` (see below).
- **Explicit external write-back (uncommon but present):** unlike most leaf hardware classes, `True_SC_ADC`'s own properties ARE written to by other classes: `True_SC_Reaction_Wheel`, `True_SC_RCS_Thruster`, and `True_SC_Magnetorquer` set/accumulate `control_torque`; `True_SC_Chemical_Thruster`, `True_SC_EP_Thruster`, `True_SC_Gravity_Gradient`, and `True_SC_SRP` accumulate `disturbance_torque`; and mission-specific attitude-control/estimation functions (e.g. `func_update_software_SC_control_attitude_Oracle.m`,

`func_update_software_SC_control_attitude_Slew.m`, `func_update_software_SC_control_attitude_DART.m`) directly overwrite `attitude`, `angular_velocity`, and `control_torque` as part of "perfect control"/oracle attitude maneuvering logic, and plotting/validation scripts (`func_make_video_camera.m`, `imu_model_validation.m`, `st_model_validation.m`) overwrite `attitude`/`angular_velocity` for visualization or test purposes.

4.1.2 Mathematical Equations

4.1.2.1 Quaternion Kinematics

$$\dot{\boldsymbol{\beta}} = \frac{1}{2}\,\Xi(\boldsymbol{\beta})\,\boldsymbol{\omega}, \qquad \Xi(\boldsymbol{\beta}) = \begin{bmatrix} \beta_4 I_{3\times 3} + [\boldsymbol{\beta}_{123}\times] \\ -\boldsymbol{\beta}_{123}^T \end{bmatrix} \tag{4.1}$$

Here $\boldsymbol{\beta} = [\beta_1, \beta_2, \beta_3, \beta_4]^T$ is the unit quaternion (dimensionless) describing the orientation of the inertial frame I with respect to the body frame B, with $\boldsymbol{\beta}_{123} = [\beta_1, \beta_2, \beta_3]^T$ the vector part and β_4 the scalar part (JPL convention, scalar last). $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ is the body angular velocity in rad/sec, and $[\boldsymbol{\beta}_{123}\times]$ is the 3×3 skew-symmetric cross-product matrix of the vector part.

In the code this is evaluated in the helper function `func_ode_attitude_dynamics.m` (called from `func_true_SC_attitude_dynamics_rigid`), where the state `beta` corresponds to `obj.attitude` and `omega` to `obj.angular_velocity`. The matrix $\Xi(\boldsymbol{\beta})$ is built as `Zbeta`, with `Zbeta(1:3,1:3) = beta(4)*eye(3) + skew(beta(1:3))` and `Zbeta(4,1:3) = -beta(1:3)'`, and `beta_dot = 0.5*Zbeta*omega` is returned as the first four components of `Xdot`.

4.1.2.2 Rigid-Body Rotational Dynamics (Euler's Equation)

$$J\dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_{ctrl} - \boldsymbol{\omega} \times (J\boldsymbol{\omega} + \mathbf{h}_{rw}) + \boldsymbol{\tau}_{dist} \tag{4.2}$$

J is the spacecraft's total mass-moment-of-inertia matrix (3×3, kg·m^2) about its center of mass, $\boldsymbol{\omega}$ is the body angular velocity (rad/sec), $\dot{\boldsymbol{\omega}}$ its time derivative (rad/sec^2), $\boldsymbol{\tau}_{ctrl}$ is the applied control torque (N·m, typically from thrusters and/or reaction wheels), $\boldsymbol{\tau}_{dist}$ is the applied disturbance torque (N·m, e.g. from SRP or gravity-gradient), and $\mathbf{h}_{rw}$ is the total angular momentum stored in the reaction wheels (kg·m^2/s).

In `func_ode_attitude_dynamics.m`, J is `mission.true_SC{i_SC}.true_SC_body.total_MI`, $\boldsymbol{\tau}_{ctrl}$ is `obj.control_torque`, $\boldsymbol{\tau}_{dist}$ is `obj.disturbance_torque`, $\mathbf{h}_{rw}$ is `obj.total_wheel_momentum` (populated by `func_update_total_wheel_momentum`), and $\dot{\boldsymbol{\omega}}$ is computed via the left-division `omega_dot = total_MI \ (control_torque' - cross(omega, total_MI*omega + total_wheel_momentum') + disturbance_torque')`, stored back in `obj.dot_angular_velocity`. Both this equation and the kinematics equation above are packaged into a single state derivative and advanced one attitude time step (`mission.true_time.time_step_attitude`) using a classical 4th-order Runge-Kutta step, `Integrators.rk4`, inside `func_true_SC_attitude_dynamics_rigid`. The resulting quaternion is re-"properized" (sign convention enforced and renormalized) via `func_quaternion_properize` before being stored back in `obj.attitude`.

4.1.2.3 Axis–Angle to Rotation Matrix (Rodrigues Formula)

$$C = \cos(\phi)\, I_{3\times 3} + (1 - \cos(\phi))\, \mathbf{a}\mathbf{a}^T + \sin(\phi)\, [\mathbf{a}\times] \tag{4.3}$$

ϕ is the principal rotation angle (rad), $\mathbf{a}$ is the unit Euler rotation axis (dimensionless, expressed in body-frame components derived from the quaternion vector part), $[\mathbf{a}\times]$ is the skew-symmetric cross-product matrix of $\mathbf{a}$, and C is the resulting 3×3 direction cosine (rotation) matrix that maps a vector from the body frame B to the inertial frame I.

Within `func_update_true_SC_ADC_rotation_matrix`, the axis is extracted as `SC_True_e_current = obj.attitude(1:3)/norm(obj.attitude(1:3))` and the angle as `SC_True_Phi_current = 2*acos(obj.attitude(4))`; these are passed to `func_create_rotation_matrix`, which returns C and is stored as `obj.rotation_matrix`. If the quaternion vector part is exactly zero (no rotation), `obj.rotation_matrix` defaults to the identity matrix.

4.1.2.4 Total Spacecraft Angular Momentum

$$\mathbf{L}_{sc} = \mathbf{h}_{rw} + J\boldsymbol{\omega} \tag{4.4}$$

$\mathbf{L}_{sc}$ is the total angular momentum of the spacecraft body-plus-wheels system, expressed in the body frame (kg·m^2/s); $\mathbf{h}_{rw}$, J, and $\boldsymbol{\omega}$ are as defined above. This bookkeeping quantity is useful for verifying angular momentum conservation/exchange between the wheels and the bus.

This is computed directly in `func_true_SC_attitude_dynamics_rigid` as `obj.total_sc_momentum = (obj.total_wheel_momentum' + mission.true_SC{i_SC}.true_SC_body.total_MI*obj.angular_velocity')'` immediately after the new attitude/rate state is obtained from the integrator, and is logged each attitude storage step as `store.total_sc_momentum_BF`.

4.1.3 CML 2–5 Suitability

4.1.3.1 What is included and correctly implemented

The class implements a physically consistent, quaternion-based rigid-body attitude propagator appropriate for CML 2–5 trade studies.

- **Quaternion kinematics and Euler's equation.** Proper quaternion kinematics ($\dot{\boldsymbol{\beta}} = \frac{1}{2}\Xi(\boldsymbol{\beta})\boldsymbol{\omega}$), Euler's rotational equation of motion including the gyroscopic cross-coupling term and reaction-wheel momentum exchange, and a 4th-order Runge-Kutta integration of the coupled state are all implemented – first-order-correct physics of the type appropriate for this maturity range.

- **Quaternion drift control.** Quaternion drift is actively controlled: every propagation step re-normalizes and applies a consistent sign ("properization") convention via `func_quaternion_properize`, which keeps long-duration formulation-level runs numerically well-behaved.

- **Self-consistent rotation matrix.** The rotation matrix used throughout the rest of the simulation is derived self-consistently from the same quaternion state each step, so attitude-dependent effects (pointing, frame transforms for SRP/GG/instrument classes) all see a coherent orientation.

- **Decoupled torque accumulation architecture.** Control and disturbance torque accumulation is cleanly decoupled from the ADC class itself (other subsystem classes contribute torques, and `True_SC_ADC` sums and resets them), which is a reasonable, extensible architecture for swapping in different actuator or disturbance models during concept trades.

- **Full time-history storage.** Full time-history storage of attitude, rate, torques, and momenta supports the kind of post-run plotting/analysis (e.g. momentum budget, torque budget) typically needed to compare point designs at CML 3–5.

4.1.3.2 What is not included / gaps

Several capabilities expected only at higher fidelity than CML 2–5 are absent, which is appropriate for this maturity range but should be flagged as known limitations.

- **Only rigid-body dynamics implemented.** Only one dynamics mode, `'Rigid'`, is implemented; any other selector string immediately throws `error('Havent written yet!')`, so flexible-body, sloshing, or multi-body articulated dynamics are not available even as placeholders.
- **No attitude control law.** There is no attitude control law in this class – no controller (e.g. PID, LQR) computing `control_torque` from an attitude/rate error against a guidance target; `control_torque` is purely a pass-through summed from whatever external actuator classes populate it, so any closed-loop pointing performance analysis must live entirely elsewhere.
- **No wheel saturation or dynamics.** The reaction-wheel momentum summation (`func_update_total_wheel_momentum`) simply adds up each wheel's `total_momentum` vector with no torque-limiting, saturation, friction, or wheel-speed dynamics modeled inside this class (those would need to exist in the reaction-wheel class itself).
- **No sensor models or knowledge-error propagation.** Sensor models (star trackers, gyros, sun sensors) and any associated noise, bias, or knowledge-error propagation are entirely absent – this class tracks only the true/truth attitude state, not an estimated one, so it cannot by itself support pointing-knowledge or control-error budgeting without a companion estimation class.
- **Identity-matrix fallback edge case.** The rotation-matrix update has a hard-coded fallback to the identity matrix when the quaternion vector part is exactly zero, which is a reasonable but unremarked edge case.
- **No actuator failure, desaturation, or flexible coupling.** There is no attitude actuator failure modeling, no momentum-management/desaturation logic, and no flexible appendage or fuel-slosh coupling, all of which would be expected only at higher-fidelity (post-CML 5) verification stages, so their absence is appropriate for this maturity range but should be flagged as a known limitation if used to argue pointing stability margins.

4.2 True_SC_Body

The True_SC_Body class represents the physical, rigid-body mass and shape properties of a single spacecraft in the simulation. It owns the spacecraft's shape model (vertices, faces, and per-face geometric properties used later for solar radiation pressure calculations), and it aggregates the mass, center of mass (COM), and moment of inertia (MI) contributions from the dry structural mass, any added/removed supplemental mass, propellant, and solar panels. It exposes two operating modes, selectable at initialization (mode_COM_selector and mode_MI_selector), that let a user either supply the total mass/COM/MI *a priori* ('given') or have the class compute them internally from the sub-component mass list ('update').

During the simulation, True_SC_Body interfaces with other spacecraft hardware classes such as True_SC_Fuel_Tank and True_SC_Solar_Panel (accessed through the mission.true_SC{i_SC} structure) to pull in time-varying propellant and solar-panel mass so that total mass, COM, and MI stay current as propellant is consumed. It uses a dirty flag (flag_update_SC_body_total_mass_COM_MI) so that the relatively expensive mass/COM/MI recomputation is only triggered when an upstream mass actually changes, rather than every timestep. Its outputs (total_mass, location_COM, total_MI) are logged to the store structure at each storage timestep and are consumed downstream by translational and rotational dynamics propagators and by force/torque models (e.g., SRP, thruster torques) that need the body's mass properties and face geometry.

4.2.1 Inputs / Outputs

4.2.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** name, rgb_color, mass.supplement (optional), mode_COM_selector ('given' or 'update'), total_mass/location_COM/total_volume (if mode_COM_selector='given'), shape_model (cell of structs with Vertices, Faces, Face_reflectance_factor, type, mass), num_hardware_exists (counts of every hardware type on the spacecraft), and mode_MI_selector/total_MI (if given).

- True_SC_Solar_Panel **(same spacecraft, all instances, treated as INPUT since** True_SC_Body **aggregates mass from hardware):** mass, shape_model.r_CM, shape_model.I_through_r_CM, read in func_update_SC_body_mass and folded into obj.mass.solar_panel{i_SP}.

- True_SC_Fuel_Tank **(same spacecraft, all instances, treated as INPUT):** instantaneous_fuel_mass and location, read in func_update_SC_body_mass and folded into obj.mass.propellant{i_FT}; True_SC_Fuel_Tank also sets obj.flag_update_SC_body_total_mass_COM_MI = 1 to request a mass/COM/MI recompute (see write-back note below).

- mission.storage: num_storage_steps, flag_store_this_time_step, k_storage used for storage array sizing/indexing.

4.2.1.2 Outputs

- **Internal / stored outputs:** total_mass, location_COM, total_MI are recomputed by func_update_SC_body_total_mass_COM_MI whenever flag_update_SC_body_total_mass_COM_MI is set, and stored each step in obj.store.

- **Reported to / read by** `True_SC_ADC`**:** `total_MI` is read in `func_true_SC_attitude_dynamics_rigid` (to compute `total_sc_momentum`) and in `func_update_total_wheel_momentum`'s loop bound (`num_hardware_exists.num_reaction_wheel`); `total_MI` is also re-read inside `func_ode_attitude_dynamics` during RK4 integration.
- **Reported to / read by** `True_SC_Gravity_Gradient` **and** `True_SC_SRP`**:** both read `total_MI` (gravity-gradient torque) and/or `shape_model/location_COM` (SRP face geometry and lever arms about the center of mass).
- **Reported to / read by actuator classes (**`True_SC_Chemical_Thruster`**,** `True_SC_EP_Thruster`**,** `True_SC_RCS_Thruster`**,** `True_SC_Reaction_Wheel`**):** read `location_COM` (lever-arm torque calculations) and `num_hardware_exists` (loop bounds for iterating over their own HW instances).
- **Reported to / read by orbit propagation ODE helpers** (`func_ode_orbit_inertial_absolute_dynamics(_v2)`, `func_orbit_inertial_static_relative_Target(_v2)`): read mass-related body properties as part of force/torque assembly for integration.
- **Reported to / read by nearly all** `Software_SC` **classes** (`Software_SC_Communication`, `Control_Attitude`, `Control_Orbit`, `Data_Handling`, `Estimate_Attitude`, `Estimate_Orbit`, `Executive`, `Power`) and by numerous DART mission-specific control/estimation functions (wheel/thruster command optimization, desaturation, maneuver planning): these read `num_hardware_exists` to size loops over hardware, and some read `total_MI`/`location_COM` for dynamics-aware control (e.g. `Software_SC_Estimate_Attitude`, `func_calculate_max_reaction_wheel_torque`, `func_calculate_max_thruster_torque`).
- **Reported to / read by plotting/visualization code (numerous** `func_plot_*`/`func_visualize_SC*` **functions):** read `shape_model`, `total_MI`, `location_COM`, `num_hardware_exists`, `rgb_color`, and `name` for rendering 3D spacecraft geometry and dashboards; purely read-only.
- **Explicit external write-back:** `True_SC_Fuel_Tank` and several mission scripts (e.g. `Mission_DART.m` and the NISAR mission variants) set `mission.true_SC{i_SC}.true_SC_body.flag_update_SC_body_total_mass_COM_MI = 1` directly, which is the mechanism by which fuel consumption (and mission-specific mass changes) trigger `True_SC_Body` to recompute its own total mass/COM/MI on the next `func_main_true_SC_body` call. This is a deliberate flag write, not a direct overwrite of the mass/COM/MI values themselves.

4.2.2 Mathematical Equations

4.2.2.1 Cuboid Moment of Inertia per Unit Mass

$$\mathbf{I}_{\text{through } r_{CM}} = \begin{bmatrix} \frac{1}{12}(W^2+H^2) & 0 & 0 \\ 0 & \frac{1}{12}(L^2+H^2) & 0 \\ 0 & 0 & \frac{1}{12}(L^2+W^2) \end{bmatrix} \tag{4.5}$$

Here L, W, H are the length, width, and height [m] of a shape's bounding box, obtained from the spread of its vertex coordinates along the body-frame x, y, z axes, and $\mathbf{I}_{\text{through } r_{CM}}$ [m^2] is the moment-of-inertia tensor of a uniform-density cuboid about its own centroid, *normalized by mass* (i.e., inertia per unit mass, not multiplied by mass yet).

In the code this is computed inside the constructor's shape-model loop (`case 'cuboid'`) as `L`, `W`, `H` from `shape.Vertices`, producing `I_through_r_CM = diag([1/12*(W^2+H^2), 1/12*(L^2+H^2), 1/12*(L^2+W^2)])`, which is then stored as `obj.shape_model{i_shape}.I_through_r_CM` and later copied into `obj.mass.dry{i_shape}.MI_over_m`. Only the `'cuboid'` shape type is implemented; any other `type` string raises an error.

4.2.2.2 Composite Center of Mass

$$\mathbf{r}_{COM}^{(n)} = \frac{m_{tot}^{(n-1)}\,\mathbf{r}_{COM}^{(n-1)} + m_n \mathbf{r}_n}{m_{tot}^{(n-1)} + m_n}, \qquad m_{tot}^{(n)} = m_{tot}^{(n-1)} + m_n \tag{4.6}$$

Here m_n [kg] is the mass of the n-th mass component (a dry-mass element, a supplemental mass, a propellant mass, or a solar panel mass), $\mathbf{r}_n$ [m] is that component's location in the body frame, $m_{tot}^{(n-1)}$ [kg] is the running total mass accumulated from all previously processed components, and $\mathbf{r}_{COM}^{(n)}$ [m] is the running center-of-mass estimate after incorporating component n. Applied sequentially over all components this yields the standard mass-weighted-average center of mass of the whole spacecraft.

This recursion is implemented in `func_update_SC_body_total_mass_COM_MI` inside the nested loop over `i_mass_class` (`obj.mass.dry`, `obj.mass.supplement`, `obj.mass.propellant`, `obj.mass.solar_panel`) and `i`: `obj.location_COM = ((obj.total_mass * obj.location_COM) + (this_mass_class{i}.mass * this_mass_class{i}.location))`, followed by `obj.total_mass = obj.total_mass + this_mass_class{i}.mass` and `obj.location_COM = (obj.location_COM / obj.total_mass)`. It only executes when `obj.mode_COM_selector` is `'update'`.

4.2.2.3 Total Moment of Inertia via the Parallel Axis Theorem

$$\mathbf{I}_{tot} = \sum_i \left[m_i\, \mathbf{I}_{\text{over } m,i} + m_i \left((\mathbf{r}_i^\top \mathbf{r}_i)\, \mathbf{I}_3 - \mathbf{r}_i \mathbf{r}_i^\top \right) \right] \tag{4.7}$$

Here m_i [kg] is the mass of component i, $\mathbf{I}_{\text{over } m,i}$ [m^2] is that component's inertia tensor about its own centroid, normalized by mass, $\mathbf{r}_i = \mathbf{r}_{COM} - \mathbf{r}_{i,\text{location}}$ [m] is the displacement vector from the component's centroid to the overall spacecraft COM, $\mathbf{I}_3$ is the 3×3 identity matrix, and $\mathbf{I}_{tot}$ [kg m^2] is the total spacecraft moment-of-inertia tensor about the spacecraft COM, summed over all dry, supplemental, propellant, and solar-panel mass elements.

This is implemented in `func_update_SC_body_total_mass_COM_MI`: `r_CM_i = obj.location_COM - this_mass_class{i}.location;` followed by `I_i = (this_mass_class{i}.mass * this_mass_class{i}.MI_over_m) + (this_mass_class{i}.mass * ((r_CM_i * r_CM_i' * eye(3)) - (r_CM_i' * r_CM_i)))`, accumulated into `obj.total_MI`. It executes only when `obj.mode_MI_selector` is `'update'`, and is explicitly labeled in the code as the Parallel Axis Theorem.

4.2.2.4 Outward Face Normal and Area (Shape Face Geometry)

$$\hat{\mathbf{n}}_i = \pm \frac{(\mathbf{V}_{i,1} - \mathbf{V}_{i,3}) \times (\mathbf{V}_{i,2} - \mathbf{V}_{i,3})}{\|(\mathbf{V}_{i,1} - \mathbf{V}_{i,3}) \times (\mathbf{V}_{i,2} - \mathbf{V}_{i,3})\|}, \qquad A_i = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2} \tag{4.8}$$

Here $\mathbf{V}_{i,1}$, $\mathbf{V}_{i,2}$, $\mathbf{V}_{i,3}$ [m] are the three vertex positions of triangular face i in the body frame, $\hat{\mathbf{n}}_i$ is the unit outward-pointing normal vector of that face (sign chosen to point away from the shape centroid), a, b, c [m] are the face's three edge lengths, s [m] is the semi-perimeter, and A_i [m^2] is the face area from Heron's formula. These per-face quantities, together with `Face_center`, are stored for use by downstream models (e.g., solar radiation pressure force/torque) that need illuminated-area and normal-direction information.

In the code this is computed in the constructor's face-orientation loop: `normal_vector_unsigned = cross(...)`, normalized, then flipped based on the sign of `acos(dot(out_vector,normal_vector_unsigned))` relative to the vector from the shape centroid (`SC_centroid`) to the face center, producing `Face_normal`; the edge lengths `a`, `b`, `c` and semi-perimeter `s` feed `Face_area(i) = sqrt(s*(s-a)*(s-b)*(s-c))`. Results are stored as `obj.shape_model{i_shape}.Face_center`, `Face_normal`, and `Face_area`.

4.2.3 CML 2–5 Suitability

4.2.3.1 What is included and correctly implemented

The class implements a physically correct, standard treatment of rigid-body mass properties suitable for early-to-mid formulation trade studies.

- **Standard COM and MI formulations.** The composite center-of-mass recursion and the moment-of-inertia aggregation both use textbook formulations (mass-weighted average COM; Parallel Axis Theorem for MI), and the cuboid inertia tensor matches the closed-form expression for a uniform rectangular box.
- **Physically distinct mass categories.** The class correctly separates mass into physically distinct categories (dry structure, supplemental mass, propellant, solar panels), which is appropriate for capturing how spacecraft mass properties evolve as propellant depletes or as mass is added/removed (e.g., sample collection), and it re-derives COM/MI on demand via a dirty-flag mechanism (`flag_update_SC_body_total_mass_COM_MI`) that is efficient without sacrificing correctness when propellant or panel mass actually changes.
- **Dual given/update mode selection.** The dual `'given'`/`'update'` mode selection for both COM and MI is well suited to CML 2–5 trade-space work, since it lets an analyst either quickly fix mass properties from a point design or let the tool compute them from more detailed subcomponent data as the concept matures.
- **Correct face-geometry processing.** The face-geometry processing (outward normal determination and Heron's-formula area) is geometrically correct and provides the data needed by downstream first-order force/torque models such as SRP, which is consistent with the medium-fidelity physics expected at these CMLs.

4.2.3.2 What is not included / gaps

Several simplifications are present, most of which are reasonable for this maturity level but would need generalization for higher-fidelity work.

- **Only cuboid shape implemented.** Shape modeling is limited to a single primitive: only `type = 'cuboid'` is implemented, and any other shape type causes the code to throw `error('Havent written yet!')`; more complex or composite spacecraft geometries (cylinders, multi-body appendages, non-rectangular buses) cannot currently be represented.
- **Diagonal-only inertia tensor.** The cuboid inertia tensor is also restricted to an axis-aligned, diagonal form (no off-diagonal products of inertia), which is a reasonable first-order approximation for CML 2–3 but would need generalization for higher-fidelity CML 4–5 point designs with asymmetric mass distributions.

- **Propellant treated as a point mass for inertia.** The propellant mass's own inertia contribution about its centroid is hard-coded to `zeros(3,3)` in `func_update_SC_body_mass` (with a more detailed cuboid/shape-based approximation left commented out in the source), so propellant is effectively treated as a point mass for inertia purposes — a known, explicitly-flagged simplification rather than an oversight, but still a gap if propellant MI contribution matters for the study.
- **No time-varying internal mass orientation, uncertainty, or V&V instrumentation.** There is no modeling of time-varying orientation of internal masses (e.g., a moving/rotating solar panel or gimbaled appendage changing its contribution to body MI beyond a simple mass/location/MI-per-mass update), no treatment of mass-property uncertainty or tolerancing, and no verification/validation instrumentation (e.g., unit tests, sanity checks that `total_MI` is positive-definite or symmetric) — all of which would be expected only at higher fidelity than CML 2–5 and are reasonably absent here.

4.3 True_SC_Data_Handling

True_SC_Data_Handling is the onboard bookkeeping class for spacecraft data volume: it tracks how much data is generated by instruments/subsystems, how much is removed (e.g., downlinked or consumed by onboard processing), and how that net change is deposited into or drained from the spacecraft's onboard memory (mass storage) elements each simulation time step. It is the spacecraft-side counterpart to link/communications accounting classes such as True_Ground_Station — where a ground station class accounts for data received over a downlink, True_SC_Data_Handling accounts for the data budget on the transmitting (spacecraft) end, including generation by many independent hardware/software sources and the finite-capacity memory banks that must buffer that data until it is removed.

The class interfaces with other mission.true_SC{i_SC} objects, most notably true_SC_body (for the count of onboard memory units, num_hardware_exists.num_onboard_memory) and true_SC_onboard_memory{i_memory} (each with an instantaneous_capacity and a maximum_capacity), which it fills or drains directly. Any other equipment/class object that produces or consumes data (cameras, recorders, radios, etc.) registers itself via func_initialize_list_HW_data_generated/func_initialize_list_HW_data_removed and then reports its contribution each step through func_update_instantaneous_data_generated(_attitude) and func_update_instantaneous_data_removed, which the class aggregates per-source (in array_HW_data_generated/array_HW_data_removed) and in total (in instantaneous_data_generated/instantaneous_data_removed). A top-level dispatcher, func_main_true_SC_data_handling, selects between a Generic mode and mission-specific named operating modes, and, for the Generic mode, applies the net data change to the chain of onboard memory objects, raising rate-limited warnings when all memories are simultaneously full (data would be lost) or empty (an underflow / more data was requested to be removed than exists).

4.3.1 Inputs / Outputs

4.3.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** mode_true_SC_data_handling_selector (e.g. 'Generic' or a mission-specific mode name) and optional data struct.
- True_SC_Body **(same spacecraft):** num_hardware_exists.num_onboard_memory, used as the loop bound in func_true_SC_data_handling_Generic when distributing instantaneous_data_change across memories.
- True_SC_Onboard_Memory **(same spacecraft, all instances):** instantaneous_capacity and maximum_capacity, read (and written) directly inside func_true_SC_data_handling_Generic to fill/drain memory banks as data is produced/removed.
- **Virtually every sensor/actuator/software class that generates or removes data** (passed in as the generic equipment argument to func_initialize_list_HW_data_generated, func_update_instantaneous_data_generated(_attitude), func_initialize_list_HW_data_removed, func_update_instantaneous_data_removed): reads equipment.name, equipment.instantaneous_data_type, equipment.instantaneous_data_rate_generated or instantaneous_data_generated_per_sample, and (for removal) instantaneous_data_rate_removed/instantaneous_data_removed_per_sample.

- `mission.true_time`: `time_step` and `time_step_attitude`, used to convert per-second data rates into per-step kb amounts.
- `mission.storage`: `num_storage_steps`, `flag_store_this_time_step`, `k_storage` for storage array sizing/indexing.

4.3.1.2 Outputs

- **Internal / stored outputs:** `instantaneous_data_change`, `instantaneous_data_generated`, `instantaneous_data_removed`, `list_HW_data_generated/removed`, `array_HW_data_generated/removed` are accumulated each step and stored in `obj.store`.
- **Call pattern used by nearly all `True_Sensors_Actuators` hardware classes** (`True_SC_Battery`, `Camera`, `Chemical_Thruster`, `EP_Thruster`, `Fuel_Tank`, `Generic_Sensor`, `IMU`, `Magnetorquer`, `Onboard_Clock`, `Onboard_Computer`, `Onboard_Memory`, `Radio_Antenna`, `RCS_Thruster`, `Reaction_Wheel`, `Remote_Sensing`, `RPS`, `Science_Processor`, `Science_Radar`, `Solar_Panel`, `Star_Tracker`, `Sun_Sensor`, `Telescope`) and by nearly all `Software_SC` classes (`Communication`, `Control_Attitude`, `Control_Orbit`, `Data_Handling`, `Estimate_Attitude`, `Estimate_Orbit`, `Executive`, `Power`): each calls `func_initialize_list_HW_data_generated` once at construction time to register itself, then calls `func_update_instantaneous_data_generated/_attitude` each step to report how much data it produced; `True_SC_Radio_Antenna` and `True_SC_Science_Processor` additionally call `func_initialize_list_HW_data_removed/func_update_instantaneous_data_removed` to report data consumed off of onboard memory (e.g. downlink, processing).
- **Reported to / read by `True_SC_Onboard_Memory`:** indirectly, via direct writes from `True_SC_Data_Handling` into `mission.true_SC{i_SC}.true_SC_onboard_memory{i_memory}.instantaneous_capacity` (fill/drain logic in `func_true_SC_data_handling_Generic`) – this is `True_SC_Data_Handling` writing into another class's property, not the reverse.
- **Reported to / read by plotting/dashboard code** (`func_plot_data_handling_visualization.m`, `func_plot_individual_data_usage.m`, `func_display_mission_dashboard.m`, `func_realtime_performance_plot.m`) and mission post-processing (`func_post_process_NISAR_output_to_Excel.m`): read `store.array_HW_data_generated/removed`, `store.list_HW_data_generated/removed`, and `instantaneous_data_*` for reporting; purely read-only.
- **No external class writes back into `True_SC_Data_Handling`'s own properties.** All interaction is `True_SC_Data_Handling` reading equipment metadata (passed by value/handle as the `equipment` argument) and itself writing into `True_SC_Onboard_Memory`'s capacity; no other class is observed assigning directly into `true_SC_data_handling.instantaneous_data_*` or `array_HW_data_*`.

4.3.2 Mathematical Equations

4.3.2.1 Instantaneous Data Generated by a Source

$$\Delta D_{\text{gen}}^{(j)} = \begin{cases} \dot{d}_{\text{gen}}^{(j)} \cdot \Delta t, & \text{type} = 0 \text{ (rate-based)} \\ d_{\text{gen,sample}}^{(j)}, & \text{type} = 1 \text{ (per-sample)} \end{cases} \tag{4.9}$$

Here $\Delta D_{\text{gen}}^{(j)}$ is the data volume [kb] generated by hardware/software source j during the current time step; $\dot{d}_{\text{gen}}^{(j)}$ [kb/s] is that source's instantaneous data generation rate, used when the source is continuous-rate type; Δt [s] is the simulation time step; and $d_{\text{gen,sample}}^{(j)}$ [kb] is a fixed per-sample data amount, used when the source instead generates data in discrete chunks (e.g., one image per exposure). The total generated in the step is the sum over all registered sources, $\Delta D_{\text{gen}} = \sum_j \Delta D_{\text{gen}}^{(j)}$.

In the code, the branch is selected by `equipment.instantaneous_data_type` (0 = rate-based, 1 = per-sample). The rate-based term is `equipment.instantaneous_data_rate_generated * mission.true_time.time_step` (or `time_step_attitude` inside `func_update_instantaneous_data_generated_attitude`, the attitude-control-loop variant), and the per-sample term is `equipment.instantaneous_data_generated_per_sample`. This is computed identically in `func_initialize_list_HW_data_generated` (to seed `array_HW_data_generated`) and in `func_update_instantaneous_data_generated`/`func_update_instantaneous_data_generated_attitude`, where the per-source result is accumulated into `obj.array_HW_data_generated(1,this_idx)` and, unless a specific target memory is passed in `varargin`, into the running total `obj.instantaneous_data_generated`.

4.3.2.2 Instantaneous Data Removed by a Source

$$\Delta D_{\text{rem}}^{(j)} = \begin{cases} \dot{d}_{\text{rem}}^{(j)} \cdot \Delta t, & \text{rate-based} \\ d_{\text{rem,sample}}^{(j)}, & \text{per-sample} \end{cases} \tag{4.10}$$

$\Delta D_{\text{rem}}^{(j)}$ is the data volume [kb] removed (consumed/downlinked) by source j in this step; $\dot{d}_{\text{rem}}^{(j)}$ [kb/s] is its removal rate (e.g., a radio's downlink drain rate) and Δt [s] the time step, used when `isprop(equipment,'instantaneous_data_rate_removed')`; $d_{\text{rem,sample}}^{(j)}$ [kb] is a fixed per-sample removal amount, used when `isprop(equipment,'instantaneous_data_removed_per_sample')` instead. The total removed in the step is $\Delta D_{\text{rem}} = \sum_j \Delta D_{\text{rem}}^{(j)}$.

This is implemented in `func_initialize_list_HW_data_removed` and `func_update_instantaneous_data_removed`, using `equipment.instantaneous_data_rate_removed * mission.true_time.time_step` or `equipment.instantaneous_data_removed_per_sample`. The per-source result is accumulated into `obj.array_HW_data_removed(1,this_idx)` and, unless a target memory index is supplied via `varargin`, into `obj.instantaneous_data_removed`.

4.3.2.3 Net Instantaneous Data Change

$$\Delta D = \Delta D_{\text{gen}} - \Delta D_{\text{rem}} \tag{4.11}$$

ΔD [kb] is the net change in onboard data volume for the current time step, i.e., the quantity that must be added to (if positive) or drained from (if negative) the onboard memory pool; ΔD_{gen} and ΔD_{rem} are the total generated and removed data volumes from the two preceding equations.

This is computed at the top of `func_true_SC_data_handling_Generic` as `obj.instantaneous_data_change = (obj.instantaneous_data_generated - obj.instantaneous_data_removed)`, and cached into `obj.data.store_instantaneous_data_change` for later logging.

4.3.2.4 Onboard Memory Fill/Drain with Capacity Clipping

$$c_i \leftarrow \text{clip}(c_i + \delta,\ 0,\ c_i^{\max}), \qquad \delta \text{ apportioned sequentially across } i = 1, \ldots, N_{\text{mem}} \tag{4.12}$$

c_i [kb] is the instantaneous stored data (`instantaneous_capacity`) in onboard memory unit i; $c_i^{\max}$ [kb] is that unit's maximum capacity (`maximum_capacity`); N_{mem} is the number of onboard

memory units on the spacecraft; and δ is the (remaining) net data change ΔD being apportioned. When $\Delta D > 0$, the code walks memories in index order $i = 1, \ldots, N_{\text{mem}}$, filling each unit up to its $c_i^{\max}$ before spilling the remainder into the next; if the remainder is still positive after all units are full, a "All Memories are Full!" warning is issued (rate-limited to 10 occurrences via `warning_counter_full`) and the excess is force-added to the last memory anyway (i.e., over-capacity is allowed rather than dropped). When $\Delta D < 0$, memories are walked in reverse order $i = N_{\text{mem}}, \ldots, 1$, draining each down to 0 before pulling from the previous one; if memory is exhausted before the deficit is satisfied, an "All Memories are Empty!" warning is issued (rate-limited via `warning_counter_empty`) and the last-touched memory's capacity is forced negative.

This logic lives entirely in `func_true_SC_data_handling_Generic`, operating on `mission.true_SC{i_SC}.true_SC_onboard_memory{i_memory}.instantaneous_capacity` and `.maximum_capacity`, with the memory count taken from `mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_onboard_memory`. Separately, `func_update_instantaneous_data_generated/_attitude` and `func_update_instantaneous_data_removed` support a direct-to-memory path: if a `chosen_memory` index and `i_SC` are passed via `varargin`, the per-source $\Delta D_{\text{gen}}^{(j)}$ or $\Delta D_{\text{rem}}^{(j)}$ is added to or subtracted from that specific memory's `instantaneous_capacity` directly, bypassing the aggregate `instantaneous_data_generated/instantaneous_data_removed` totals and the ordered-fill/drain logic above.

4.3.3 CML 2–5 Suitability

4.3.3.1 What is included and correctly implemented

The class provides a coherent, general-purpose data-budget ledger appropriate for early concept trades.

- **Extensible source registry with conservative accounting.** It supports an arbitrary, extensible list of data-generating and data-removing hardware/software sources (registered by name, so new instruments or radios can be added without modifying this class), two natural data-accounting models (continuous rate × time step, or discrete per-sample chunks), and it correctly conserves data volume by routing all net change through a single $\Delta D_{\text{gen}} - \Delta D_{\text{rem}}$ balance.

- **Multi-memory fill/drain algorithm.** The multi-memory fill/drain algorithm is a reasonable first-order model of a spacecraft with several onboard storage banks: it fills sequentially and drains in reverse order, correctly clips at each bank's `maximum_capacity` and at zero, and spills/pulls across banks so the aggregate capacity of the whole storage pool is used before declaring full/empty.

- **Explicit saturation detection and logging.** Saturation conditions (all memory full or empty) are explicitly detected and flagged with rate-limited warnings rather than silently failing, and both per-source and total data-generated/removed histories are logged via the `store` structure for post-run analysis (e.g., per-instrument data budget breakdowns), which is valuable for trade-space comparisons between instrument/mode configurations.

- **Mission-specific mode selector.** A mode selector (`Generic` vs. mission-specific named modes) also shows the architecture already anticipates mission-specific data-handling behaviors while defaulting to a reusable generic model — well suited to CML 2–3 style trade studies where a common first-order model must run across several candidate architectures.

4.3.3.2 What is not included / gaps

Several behaviors needed for higher-fidelity (CML 4–5) point-design verification are absent, though these gaps are consistent with, and acceptable for, CML 2–5 formulation-phase use.

- **Overflow and underflow are nonphysically clamped/unclamped.** When all memories are full, the "excess" data is not actually discarded or flagged as lost — it is added onto the last memory's `instantaneous_capacity` anyway, driving that unit's capacity above its `maximum_capacity` rather than modeling data loss, memory overwrite (e.g., ring-buffer overwrite of oldest data), or a hard downlink-priority queue; symmetrically, on empty-memory underflow the code forces a memory's capacity negative rather than clamping at zero, which is nonphysical and could propagate a negative-capacity state into later steps.
- **No flight-software-representative data management.** There is no flight-software-representative behavior such as data prioritization/quality-of-service between instruments, compression, packetization/framing overhead, CCSDS-style file or virtual-channel accounting, data recorder health/fault modeling, or memory read/write bandwidth limits (fill/drain is treated as instantaneous within a step, with no cap on how fast a given memory bank can physically accept or release data).
- **Mission-specific mode not implemented in this file.** The mission-specific mode is only referenced through a `switch`/case dispatch — its implementation is not present in this file, so its data-handling behavior cannot be assessed here.
- **Warning suppression limits long-run auditability.** Warning suppression after 10 occurrences means sustained full/empty conditions late in a long simulation would go unreported, which is acceptable for a first-order trade tool but insufficient for rigorous verification, where every over/underflow event would typically need to be logged and quantified (e.g., total lost data volume, not just a boolean warning).

4.4 True_SC_Gravity_Gradient

True_SC_Gravity_Gradient models the disturbance torque exerted on the spacecraft's rigid body by the gravity-gradient effect of the central body (target) it orbits. Because the spacecraft has finite extent and a non-spherical mass distribution, the gravitational force does not act uniformly through the center of mass; the resulting differential pull across the body produces a restoring torque that depends on the spacecraft's inertia tensor and its orientation relative to the local vertical (the line connecting the spacecraft center of mass to the central body). This class is one of several "disturbance" generators in the True_SC tree (alongside classes for solar radiation pressure, aerodynamic drag, etc.) that feed additive torque contributions into the attitude dynamics chain.

Architecturally, the class is instantiated per spacecraft (indexed by i_SC) and is driven by the mission superstructure: it reads the spacecraft's position relative to its target from true_SC_orbit_propagation, the body-to-inertial attitude rotation matrix from true_SC_adc, the total mass moment of inertia tensor from true_SC_body, and the target's gravitational parameter from true_target. It writes its computed torque both into its own property disturbance_torque_G2 and accumulates it directly into true_SC_adc.disturbance_torque, which is the running sum of all disturbance torques used by the attitude dynamics/control propagation. A boolean flag enable_G2 allows the effect to be switched on or off, and a store structure logs the torque history at each simulation storage step for post-run analysis.

4.4.1 Inputs / Outputs

4.4.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** enable_G2 (boolean flag enabling/disabling the gravity-gradient torque calculation).
- True_SC_Orbit_Propagation **(same spacecraft):** position_relative_target (converted to meters) and index_relative_target, used to form the target-to-spacecraft vector Rc and to select the correct mission.true_target entry.
- True_SC_ADC **(same spacecraft):** rotation_matrix, used (transposed) to rotate Rc from the inertial frame into the body frame (Rc_sc).
- True_SC_Body **(same spacecraft):** total_MI, used directly in the gravity-gradient torque formula (cross(Rc_sc, total_MI*Rc_sc)).
- mission.true_target{index_relative_target}: mu (gravitational parameter of the body about which the gravity gradient is computed, converted from km^3/s^2 to m^3/s^2).
- mission.storage: num_storage_steps, flag_store_this_time_step, k_storage for storage sizing/indexing.
- The constructor calls its own func_main_true_SC_gravity_gradient once before the first attitude-loop iteration, so True_SC_Orbit_Propagation, True_SC_ADC, and True_SC_Body must already be constructed and populated at that point (confirmed by construction order in Mission_DART.m: body, orbit propagation, and ADC are built before gravity gradient).

4.4.1.2 Outputs

- **Internal / stored outputs:** `disturbance_torque_G2` is recomputed every call to `func_main_true_SC_gravity_gradient` and stored in `obj.store.disturbance_torque_G2`.
- **Reported to / written into `True_SC_ADC`:** `disturbance_torque_G2` is immediately added into `mission.true_SC{i_SC}.true_SC_adc.disturbance_torque` inside `func_main_true_SC_gravity_gradient` itself – i.e. `True_SC_Gravity_Gradient` actively writes into `True_SC_ADC`'s accumulator rather than `True_SC_ADC` pulling the value.
- **Reported to / read by plotting code** (`func_plot_attitude_actuator_performance.m`): reads `enable_G2` and `store.disturbance_torque_G2` to compare gravity-gradient torque magnitude against other disturbance/control torques; purely read-only.
- **No external class writes back into this class's own properties** (i.e. no other class assigns into `true_SC_gravity_gradient.enable_G2` or `disturbance_torque_G2`); the only cross-class write observed is `True_SC_Gravity_Gradient` itself writing outward into `True_SC_ADC.disturbance_torque`, which is the class's intended mechanism for reporting its computed torque, not an external mutation of this class.

4.4.2 Mathematical Equations

4.4.2.1 Line-of-sight Vector in the Body Frame

$$\vec{R}_{c,B} = \mathbf{A}_{B/I}^{\top}\, \vec{R}_c \tag{4.13}$$

Here $\vec{R}_c$ is the spacecraft's position vector relative to the target (central) body's center, expressed in the inertial frame, with units of meters; $\mathbf{A}_{B/I}$ is the direction cosine (rotation) matrix that converts a vector expressed in the spacecraft body frame B into the inertial frame I; and $\vec{R}_{c,B}$ is the same line-of-sight vector re-expressed in the spacecraft body frame, also in meters. Because gravity-gradient torque depends on how the inertia tensor (which is naturally expressed in body coordinates) interacts with the local vertical direction, the position vector must first be rotated into the body frame before the torque can be evaluated.

In the code, $\vec{R}_c$ is `Rc`, obtained from `mission.true_SC{i_SC}.true_SC_orbit_propagation.position_relative_target` (given in km and converted to meters via the `1e3` factor). The rotation matrix $\mathbf{A}_{B/I}$ is `mission.true_SC{i_SC}.true_SC_adc.rotation_matrix`, and the transpose operation in `(...rotation_matrix')` implements $\mathbf{A}_{B/I}^{\top}$, converting the inertial-frame vector into the body frame. The result is stored in the local variable `Rc_sc`.

4.4.2.2 Gravity-Gradient Disturbance Torque

$$\vec{\tau}_{GG} = \frac{3\mu}{\|\vec{R}_{c,B}\|^5}\, \vec{R}_{c,B} \times \left(\mathbf{I}\, \vec{R}_{c,B}\right) \tag{4.14}$$

This is algebraically equivalent to the classic gravity-gradient torque expression $\vec{\tau}_{GG} = \frac{3\mu}{R^3}\, \hat{R} \times \left(\mathbf{I}\, \hat{R}\right)$ written in terms of the non-normalized vector $\vec{R}_{c,B} = R\hat{R}$, since dividing by R^5 instead of R^3 absorbs the two extra powers of R that come from using $\vec{R}_{c,B}$ in place of the unit vector $\hat{R}$ in both cross-product factors. Here μ is the standard gravitational parameter (GM) of the central/target body, in units of $\mathrm{m}^3/\mathrm{s}^2$; $\vec{R}_{c,B}$ is the body-frame line-of-sight vector from the previous equation, in meters, with magnitude $R = \|\vec{R}_{c,B}\|$; $\mathbf{I}$ is the spacecraft's total mass moment-of-inertia tensor (about

its center of mass), in kg m^2, expressed in the body frame; and $\vec{\tau}_{GG}$ is the resulting gravity-gradient disturbance torque vector, in N m, expressed in the body frame.

In the code this is computed in a single line as `obj.disturbance_torque_G2`. The gravitational parameter μ comes from `mission.true_target{...index_relative_target}.mu`, which is stored in km^3/s^2 and is multiplied by `1e9` to convert to m^3/s^2, consistent with `Rc_sc` being in meters. The norm term is `norm(Rc_sc)^5`. The inertia tensor $\mathbf{I}$ is `mission.true_SC{i_SC}.true_SC_body.total_MI`. The cross product $\vec{R}_{c,B} \times (\mathbf{I}\vec{R}_{c,B})$ is implemented via MATLAB's `cross(Rc_sc, ... total_MI * Rc_sc)`, and the whole expression is transposed (`'`) to store the torque as a row vector consistent with the property's $[1 \times 3]$ convention. The computation is only executed when `obj.enable_G2 == 1`; otherwise the torque is left at zero.

4.4.3 CML 2–5 Suitability

4.4.3.1 What is included and correctly implemented

The class implements the standard first-order rigid-body gravity-gradient torque formula.

- **Correct full inertia-tensor treatment.** It correctly transforms the target-relative position vector into the body frame before combining it with the full 3×3 inertia tensor, rather than assuming a diagonal or simplified inertia model.
- **Consistent unit handling.** Unit handling is explicit and internally consistent (km to m for position, km^3/s^2 to m^3/s^2 for μ).
- **Correct disturbance-torque accumulation.** The torque is accumulated additively into the spacecraft's total disturbance torque used by the ADC/attitude-dynamics propagation, which is the correct architectural pattern for a first-order disturbance-torque contributor.
- **Toggleable enable flag and logging.** An enable/disable flag (`enable_G2`) lets the effect be toggled for trade studies (e.g., comparing controlled attitude performance with and without gravity-gradient disturbance), and results are logged at each storage step for later plotting/analysis.
- **Fidelity matched to CML 2–5.** This level of fidelity — closed-form analytic torque from a point-mass gravity field and a lumped inertia tensor — is well matched to CML 2–5 trade-space and point-design work, where first-order physics is sufficient and full flight-software-representative fidelity is not required.

4.4.3.2 What is not included / gaps

The model treats the central body strictly as a point mass, with several resulting simplifications.

- **No higher-order gravity field terms.** It does not include any higher-order gravity field terms (e.g., J_2 or a spherical-harmonics expansion) that could couple additional gravity-gradient-like or gravity-field-asymmetry effects for close/irregular bodies, since the central body is modeled via μ alone.
- **Single-target limitation.** It only ever considers the single target referenced by `index_relative_target`; if a mission trade needed gravity-gradient contributions from multiple attracting bodies (e.g., a moon plus its primary), this class would need to be extended or called multiple times.

- **No noise or uncertainty modeling.** There is no noise, uncertainty, or Monte Carlo variation applied to μ, the inertia tensor, or the position vector, so the torque is purely deterministic given the current state — acceptable for early point-design analysis but a gap if later CML levels require statistical/robustness assessment.
- **No singularity guard.** The code has no input validation or guard against $\|\vec{R}_{c,B}\| \to 0$ (which would produce a singular $1/R^5$ term), though this is a low-risk edge case for realistic orbit geometries and consistent with the level of rigor expected at this maturity stage.

4.5 True_SC_Orbit_Propagation

The True_SC_Orbit_Propagation class is the "truth" orbital state generator for a single spacecraft in the simulation: it owns and advances the spacecraft's inertial position and velocity (both in an absolute Sun-centered J2000 frame and, in parallel, relative to a designated target body), and it exposes visibility flags (Sun, Earth) and a crash-detection flag that other classes (e.g. attitude, sensor, and navigation-filter classes) query each timestep. It plays the spacecraft-side analogue of True_Target_Computed/True_Target_SPICE, but for a moving, controllable spacecraft rather than a passive small body: it accepts control and disturbance force inputs (set externally by thruster/SRP models) and folds them into the equations of motion each propagation step. Its interface with the rest of the mission object is broad — it reads solar-system body ephemerides (mission.true_solar_system), target body states/gravity fields (mission.true_target), spacecraft mass (mission.true_SC{i_SC}.true_SC_body), and the master time grid (mission.true_time) — and it writes back updated position/velocity/visibility/crash state that downstream sensor and GN&C models consume.

The class supports several interchangeable dynamics "modes" selected via mode_true_SC_orbit_propagation_dynamics_selector: a SPICE mode that simply looks up a pre-computed trajectory from a furnished SPICE kernel; an Absolute Dynamics mode that numerically integrates the spacecraft state in the Sun-centered inertial frame under n-body perturbations; a Relative Dynamics mode that integrates the spacecraft state relative to a target body, including that target's own (possibly spherical-harmonic) gravity field plus third-body perturbations; and additional mission-specific modes that are dispatched to but not defined in this file. Regardless of mode, the class maintains both the absolute and target-relative representations of the state simultaneously (converting between them via the target's own position/velocity), updates Sun/Earth line-of-sight visibility using a point-to-line-segment eclipse test, checks for surface impact against every target body, and logs all of these quantities into a store structure for post-run analysis.

4.5.1 Inputs / Outputs

4.5.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** either position/velocity (absolute-frame init) or position_relative_target/velocity_relative_target/name_relative_target (relative-frame init), plus mode_true_SC_orbit_propagation_dynamics_selector ('SPICE', 'Absolute Dynamics', 'Relative Dynamics', or 'mission specific'), and (if SPICE mode) spice_filename/spice_name; optional data and mode_selector_name.

- mission.true_target{...} **(one or more target bodies):** name, position, velocity, radius, position_array – used to resolve index_relative_target, compute relative position/velocity, and to test for Sun/Earth occlusion and spacecraft-crash conditions (func_update_visible_Sun_Earth, func_update_flag_SC_crashed).

- mission.true_solar_system: SS_body{index_Sun}.position, SS_body{index_Earth}.position, index_Earth, num_SS_body, and each SS body's radius, used in func_update_visible_Sun_Earth to compute flag_visible_Sun/flag_visible_Earth via line-of-sight occlusion tests.

- mission.true_time: date, prev_date, time_position_array, time_step_position_array, used for SPICE queries and RK4 integration time grids.

- **Actuator classes writing forces (treated here as inbound accumulation, see Outputs below):** `True_SC_Chemical_Thruster`, `True_SC_EP_Thruster` add into `control_force`; `True_SC_RCS_Thruster` and `True_SC_SRP` add into `disturbance_force`.
- `mission.storage`**:** `num_storage_steps`, `flag_store_this_time_step`, `k_storage`, and `flag_stop_sim` (set to 1 by this class on a detected crash).
- **SPICE kernel system (external to the mission struct):** `cspice_furnsh`, `cspice_spkezr` calls directly query the loaded SPICE kernel for trajectory data when in `'SPICE'` mode.

4.5.1.2 Outputs

- **Internal / stored outputs:** `position`, `velocity`, `position_relative_target`, `velocity_relative_target`, `flag_visible_Sun`, `flag_visible_Earth`, `control_force`, `disturbance_force`, `oracle_position_error` are updated/reset each step and stored in `obj.store`; `position_array` and `flag_SC_crashed` are computed but not stored.
- **Reported to / read by** `True_SC_Gravity_Gradient` **and** `True_SC_SRP`**:** both read `position_relative_target/position` and `index_relative_target` to compute gravity-gradient and SRP geometry, and both write their resulting disturbance force/torque back (`True_SC_SRP` accumulates into `disturbance_force`; see write-back note).
- **Reported to / read by nearly all** `True_Sensors_Actuators` **classes with pointing/geometry needs** (`True_SC_Camera`, `Chemical_Thruster`, `Communication_Link`, `EP_Thruster`, `Magnetorquer`, `RCS_Thruster`, `Remote_Sensing`, `Science_Radar`, `Solar_Panel`, `Star_Tracker`, `Sun_Sensor`, `Telescope`): read `position`, `flag_visible_Sun`, `flag_visible_Earth` for eclipse/visibility/pointing calculations; thruster classes write back their generated force (see below).
- **Reported to / read by** `Software_SC_Estimate_Orbit` **and mission-specific executive/communication/control functions** (e.g. DART and NISAR variants of `func_software_SC_executive_*`, `func_communication_link_*`, `func_update_sw_SC_est_orbit_TruthWithErrorGrowth`, `func_compute_TCM_Lambert_Battin`): read `position/velocity/flag_visible_*` as orbit truth for estimation, link budgets, and maneuver planning.
- **Reported to / read by plotting/visualization code** (numerous `func_plot_*/func_visualize_SC*/func_make_video_*` functions): read `position`, `position_array`, `store.*` for trajectory rendering; `func_make_video_camera.m` is a plotting script that directly overwrites `position` with a value from a precomputed position array for rendering purposes (see write-back note).
- **Explicit external write-back:** `True_SC_Chemical_Thruster` and `True_SC_EP_Thruster` add their thrust force into `control_force`; `True_SC_RCS_Thruster` and `True_SC_SRP` add into `disturbance_force`. These are the intended force-accumulation mechanism (analogous to the ADC's torque accumulators) rather than incidental mutation. Separately, the plotting helper `func_make_video_camera.m` directly overwrites `obj.position` with a cached trajectory sample for video rendering – an unusual, visualization-only write-back that does not feed back into the simulation physics.

4.5.2 Mathematical Equations

4.5.2.1 Absolute-Frame Equations of Motion (n-Body Perturbed Two-Body Dynamics)

$$\ddot{\mathbf{R}} = \frac{\mathbf{F}_c + \mathbf{F}_d}{m} \; - \; \sum_i \mu_i \frac{\mathbf{R} - \mathbf{R}_i}{\|\mathbf{R} - \mathbf{R}_i\|^3} \; + \; \sum_{i \neq \text{Sun}} \mu_i \frac{\mathbf{R}_i}{\|\mathbf{R}_i\|^3} \tag{4.15}$$

Here $\mathbf{R}$ is the spacecraft position vector [km] in the Sun-centered J2000 inertial frame and $\ddot{\mathbf{R}}$ its inertial acceleration [km/s^2]; m is the spacecraft total mass [kg]; $\mathbf{F}_c$ and $\mathbf{F}_d$ are the instantaneous control force and disturbance force [N] acting through the spacecraft center of mass; the index i runs over every solar-system body and every mission target body, each with gravitational parameter μ_i [km^3/s^2] and Sun-centered position $\mathbf{R}_i$ [km]. The first summation is the direct gravitational attraction of each body on the spacecraft; the second ("indirect") summation adds each non-Sun body's own gravitational pull toward the Sun, which is required because the propagation frame is centered on the (non-inertially accelerating) Sun rather than on a true inertial origin.

In the code this is implemented in `Supporting_Functions/other/func_ode_orbit_inertial_absolute_dynamics_v2.m`, called from `func_update_SC_orbit_prop_position_velocity_Absolute_Dynamics`. $\mathbf{R}$ is `X(1:3)`; $\mathbf{F}_c/m$ and $\mathbf{F}_d/m$ come from `obj.control_force/obj.disturbance_force` divided by `mission.true_SC{i_SC}.true_SC_body.total_mass` (with a 10^{-3} unit conversion from N/kg to km/s^2); the direct-term loop over solar-system bodies uses `mission.true_solar_system.SS_body{i_SS_body}.mu` and positions from `cspice_spkezr`, and the indirect term is added whenever `i_SS_body ~= mission.true_solar_system.index_Sun`; an analogous loop adds gravity from every `mission.true_target{i_target}` using `.mu` and `.position_array` (interpolated in time), with the indirect term added unless the target's name is `'Sun'`. The resulting state derivative `Xdot = [X(4:6); acc]` is integrated forward via RK4 (see below) inside `func_update_SC_orbit_prop_position_velocity_Absolute_Dynamics`, and the new `obj.position/obj.velocity` are then differenced against the target's state to refresh `obj.position_relative_target/obj.velocity_relative_target`.

4.5.2.2 Target-Relative Equations of Motion with Target Gravity Field

$$\ddot{\mathbf{r}} = \frac{\mathbf{F}_c + \mathbf{F}_d}{m} \; + \; \text{Rot}^T \, \mathbf{a}_R(\mathbf{r}_R,\, n_{\max}) \; - \; \sum_{i \neq \text{target}} \mu_i \left(\frac{\mathbf{r} - \mathbf{R}_i}{\|\mathbf{r} - \mathbf{R}_i\|^3} - \frac{\mathbf{R}_i}{\|\mathbf{R}_i\|^3} \right), \qquad \mathbf{r}_R = \text{Rot}\,\mathbf{r} \tag{4.16}$$

Here $\mathbf{r}$ is the spacecraft position [km] relative to the designated target body, expressed in the (non-rotating) target-relative inertial frame, and $\mathbf{r}_R$ is that same position rotated into the target's body-fixed frame via the rotation matrix Rot; $\mathbf{a}_R$ is the target's own gravitational acceleration [km/s^2] evaluated in the body-fixed frame from a spherical-harmonic-type gravity model of degree/order $n_{\max}$; $\mathbf{F}_c$, $\mathbf{F}_d$, and m are as before; the sum over i again covers every other solar-system body and every other target body, each contributing a direct gravity term plus an indirect term accounting for the reference target's own acceleration due to that body (needed because the frame is centered on the moving target).

This is implemented in `Supporting_Functions/other/func_orbit_inertial_static_relative_Target_v2.m`, invoked from `func_update_SC_orbit_prop_position_velocity_Relative_Dynamics`. $\mathbf{r}$ is `X(1:3)` (`r_I`), Rot is `mission.true_target{index_relative_target}.rotation_matrix`, $\mathbf{r}_R$ is `r_R = Rot*r_I`, and $\mathbf{a}_R$ is returned by `mission.true_target{index_relative_target}.gravity_field.Acceleration(r_R, gravity_degree_harmonics)` then rotated back with `a_I = Rot' * a_R`. The perturbing-body loops mirror the absolute-dynamics case but use `cspice_spkezr` with the target's `spice_name` as observer (for solar-system bodies) and `interp1` of `position_array` (for other

targets), always adding the indirect term since the origin is the target rather than the Sun. The code also carries an optional, currently unused, state-transition-matrix branch (`flag_stm`) that would propagate $\dot{\Phi} = D\Phi$ with D built from the gravity-gradient $\partial\mathbf{a}/\partial\mathbf{r}$, but this path is dead code in the class's normal call (the function is always invoked with `nargin=5`, so `flag_stm` defaults to false).

4.5.2.3 Fourth-Order Runge-Kutta Time Integration

$$\begin{aligned} k_1 &= f(t_n, X_n), \quad k_2 = f\left(t_n + \tfrac{\Delta t}{2},\, X_n + \tfrac{\Delta t}{2} k_1\right), \\ k_3 &= f\left(t_n + \tfrac{\Delta t}{2},\, X_n + \tfrac{\Delta t}{2} k_2\right), \quad k_4 = f(t_n + \Delta t,\, X_n + \Delta t\, k_3), \\ X_{n+1} &= X_n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right) \end{aligned} \tag{4.17}$$

Here $X_n = [\mathbf{R}_n; \dot{\mathbf{R}}_n]$ (or $[\mathbf{r}_n; \dot{\mathbf{r}}_n]$ in the relative case) is the 6-element position/velocity state at the n-th time-grid point, f is the equation-of-motion function returning $\dot{X} = [\dot{\mathbf{R}}; \ddot{\mathbf{R}}]$, t_n is the current time [s] since the propagation epoch, and Δt is the fixed sub-step size [s].

This is the classical fourth-order Runge-Kutta scheme implemented generically in `Supporting_Functions/other/Integrators.m` (`Integrators.rk4`), and it is what `True_SC_Orbit_Propagation` actually calls in both `func_update_SC_orbit_prop_position_velocity_Absolute_Dynamics` and `func_update_SC_orbit_prop_position_velocity_Relative_Dynamics`: for each pair of consecutive entries in `mission.true_time.time_position_array`, `Integrators.rk4` is called with step size `mission.true_time.time_step_position_array` and the corresponding `func_ode_orbit_inertial_absolute_dynamics_v2` or `func_orbit_inertial_static_relative_Target_v2` handle, filling the array `X`. Note that MATLAB's variable-step `ode15s`/`ode45` calls appear directly above each RK4 loop but are commented out, i.e. the class is currently configured to always use fixed-step RK4 rather than the adaptive-tolerance `obj.ode_options` (set to `RelTol`/`AbsTol` of 10^{-14}) that is still constructed but no longer consumed on this path.

4.5.2.4 Line-of-Sight Occultation Test (Point-to-Line-Segment Distance)

$$d = \frac{\|(\mathbf{x}_0 - \mathbf{x}_1) \times (\mathbf{x}_0 - \mathbf{x}_2)\|}{\|\mathbf{x}_2 - \mathbf{x}_1\|}, \qquad t = \frac{-(\mathbf{x}_1 - \mathbf{x}_0) \cdot (\mathbf{x}_2 - \mathbf{x}_1)}{\|\mathbf{x}_2 - \mathbf{x}_1\|^2} \tag{4.18}$$

Here $\mathbf{x}_1$ is the position [km] of the illuminating/reference body (the Sun, or Earth), $\mathbf{x}_2$ is the spacecraft position, and $\mathbf{x}_0$ is the position of a candidate occulting body (any target or solar-system body); d [km] is the perpendicular distance from that occulting body's center to the infinite line through $\mathbf{x}_1$ and $\mathbf{x}_2$, and t is the dimensionless parameter locating the closest point on that line relative to the segment endpoints ($0 \le t \le 1$ means the closest point lies between $\mathbf{x}_1$ and $\mathbf{x}_2$).

This is implemented in `func_update_visible_Sun_Earth`: for each of Sun/Earth (`i_vis`), $\mathbf{x}_1$ is taken from `mission.true_solar_system.SS_body{index_Sun}.position` or the Earth's position (from the solar-system body list or, if Earth is itself the target, from `mission.true_target`), $\mathbf{x}_2$ is `obj.position`, and every other target/solar-system body is tested as $\mathbf{x}_0$ using its `.radius`. If d is less than the body's radius and $0 \le t \le 1$, the body is deemed to occult the line of sight and the corresponding `obj.flag_visible_Sun`/`obj.flag_visible_Earth` is set to 0. A companion (non-line-based) method, `func_update_flag_SC_crashed`, simply compares $\|\mathbf{x}_0 - \mathbf{x}_1\|$ against each target's radius at every stored time sample to set `obj.flag_SC_crashed` and stop the simulation.

4.5.3 CML 2–5 Suitability

4.5.3.1 What is included and correctly implemented

The class provides a physically reasonable, multi-fidelity orbit-propagation capability appropriate to concept-level formulation.

- **Multi-fidelity trajectory modes.** It supports both a SPICE-driven "as-flown/as-designed" trajectory mode and a self-consistent numerically integrated n-body dynamics mode, with the latter correctly including direct and indirect (frame-acceleration) perturbation terms from every modeled solar-system body and target, which is the standard formulation used in early-phase perturbed two-body trajectory design.
- **Target-body gravity field support.** The relative-dynamics mode additionally supports a target-body gravity field (spherical-harmonics-capable via `gravity_field.Acceleration` and a configurable `gravity_degree_harmonics`), which is well beyond bare point-mass fidelity and is useful for proximity-operations trade studies around small bodies.
- **Control/disturbance force injection.** Control and disturbance forces are cleanly injected into the equations of motion (and reset each step), giving a clear hook for closed-loop GN&C or SRP disturbance studies.
- **Consistent state tracking and analysis support.** The class also tracks absolute and target-relative states consistently, computes Sun/Earth visibility via a defensible point-to-line eclipse test against all modeled bodies, checks for surface impact against every target, and stores full time-history data for post-processing — all of which is the kind of medium-fidelity, self-contained physics needed to compare concept alternatives at CML 2–5 without requiring flight-software-representative verification.

4.5.3.2 What is not included / gaps

Several pieces are stubs or dead paths rather than missing physics per se.

- **Disabled adaptive integration and unreachable STM branch.** The adaptive-step `ode15s`/`ode45` integration calls are present but commented out in favor of a fixed-step RK4, so there is no automatic error control on the integration step size, and the STM/state-transition-matrix branch in the relative-dynamics ODE (`flag_stm`) is unreachable from this class's own call sites.
- **No internal non-gravitational perturbation modeling.** There is no explicit modeling of non-gravitational perturbations beyond whatever is externally supplied as `control_force`/`disturbance_force` (e.g. no SRP, drag, or relativistic corrections are computed inside this class itself).
- **No measurement/estimation-error model.** No measurement/estimation-error model is applied to the propagated state (the `oracle_position_error` and `manual_position_noise` properties exist but are not populated or applied anywhere in this file, and `manual_position_noise` is never even initialized).
- **Rigid single-body assumption and brittle error handling.** Finally, the class assumes rigid, single-body spacecraft mass with no attitude coupling into translational dynamics, and it silently errors out (`'Navigation initialization incorrect!'`, `'Navigation mode not`

`defined!'`) rather than gracefully degrading if inputs are malformed — acceptable for CML 2–5 trade studies but worth noting as a fidelity ceiling if the concept later needs to mature toward CML 6+.

4.6 True_SC_Power

True_SC_Power is the spacecraft-level power bookkeeping class in the True state ("truth") portion of the simulation. Its role is to aggregate, on every simulation time step, the instantaneous power drawn by every consuming piece of hardware (sensors, actuators, ADCS components, etc.) and the instantaneous power produced by every generating source (solar panels, RPS, etc.), convert the net result into an energy quantity for the time step, and then push that energy into (or pull it out of) the spacecraft's battery bank(s) via True_SC_Battery. It also maintains named, per-hardware-item energy ledgers (list_HW_energy_consumed/generated and the corresponding array_HW_energy_consumed/generated) so that the contribution of each subsystem to the total power budget can be tracked and stored over the mission timeline.

In this sense True_SC_Power acts as the mediator between individual hardware objects (which each expose an instantaneous_power_consumed or instantaneous_power_generated property) and the battery objects held in mission.true_SC{i_SC}.true_SC_battery. Hardware modules call func_update_instantaneous_power_consumed (or the attitude-loop variant func_update_instantaneous_power_consumed_attitude) and func_update_instantaneous_power_generated to report their power draw/output each step; the class then calls func_main_true_SC_power once per step to net everything together, charge or discharge the battery array in priority order, flag a power_emergency if all batteries are depleted, and log the results into obj.store for post-processing.

4.6.1 Inputs / Outputs

4.6.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** power_loss_rate (fractional power loss applied to consumed power), optional name, optional data.
- True_SC_Body **(same spacecraft):** num_hardware_exists.num_battery, used as the loop bound in func_main_true_SC_power when charging/discharging batteries.
- True_SC_Battery **(same spacecraft, all instances):** health, instantaneous_capacity, maximum_capacity, charging_efficiency, discharging_efficiency, read and written directly inside func_main_true_SC_power to route excess/deficit energy into/out of each battery.
- **Virtually every sensor/actuator class that consumes or generates power** (passed in as the generic equipment argument to func_initialize_list_HW_energy_consumed/generated and func_update_instantaneous_power_consumed(_attitude)/generated): reads equipment.name and equipment.instantaneous_power_consumed or instantaneous_power_generated.
- mission.true_time: time_step and time_step_attitude, used to convert instantaneous power (W) into energy (W-hr) for the current step.
- mission.storage: num_storage_steps, flag_store_this_time_step, k_storage for storage array sizing/indexing.

4.6.1.2 Outputs

- **Internal / stored outputs:** instantaneous_total_power_consumed/generated, instantaneous_energy, instantaneous_energy_unused, power_emergency, power_deficit,

list_HW_energy_consumed/generated, array_HW_energy_consumed/generated are recomputed each step and (the first four plus the HW arrays) stored in obj.store.

- **Call pattern used by nearly all True_Sensors_Actuators hardware classes** (True_SC_Battery, Camera, Chemical_Thruster, Communication_Link, EP_Thruster, Fuel_Tank, Generic_Sensor, IMU (initialize only), Magnetorquer (initialize only), Onboard_Clock, Onboard_Computer, Onboard_Memory, Radio_Antenna, RCS_Thruster (initialize only), Reaction_Wheel (initialize only), Remote_Sensing, RPS, Science_Processor (initialize only), Science_Radar, Solar_Panel, Star_Tracker (initialize only), Sun_Sensor (initialize only), Telescope): each calls func_initialize_list_HW_energy_consumed once at construction to register itself; power-consuming classes additionally call func_update_instantaneous_power_consumed/_attitude each step; True_SC_Solar_Panel and True_SC_RPS additionally call func_initialize_list_HW_energy_generated/func_update_instantaneous_power_generated to report generated power.

- **Reported to / read by True_SC_Battery:** indirectly, via direct writes from True_SC_Power into mission.true_SC{i_SC}.true_SC_battery{i_batt}.instantaneous_capacity (charge/discharge logic in func_main_true_SC_power) – True_SC_Power writes into the battery, not the reverse.

- **Reported to / read by mission-specific executive/communication functions** (func_software_SC_executive_DART.m, func_communication_link_MultiSC_per_GS.m, which themselves call func_update_instantaneous_power_consumed) and by post-processing/plotting code (func_post_process_NISAR_output_to_Excel.m, func_plot_individual_power_consumption.m, func_plot_power_visualization.m, func_display_mission_dashboard.m, func_realtime_performance_plot.m): read store.*, power_emergency, power_deficit for reporting/dashboards; purely read-only except the executive/communication functions which call the standard consumption-reporting API.

- **No external class writes back into True_SC_Power's own properties.** All interaction is True_SC_Power reading equipment metadata (passed as the equipment argument) and itself writing into True_SC_Battery's capacity; no other class is observed assigning directly into true_SC_power.instantaneous_* or array_HW_energy_*.

4.6.2 Mathematical Equations

4.6.2.1 Net Energy Balance per Time Step

$$E_{\text{net}} = (P_{\text{gen}} - P_{\text{cons}}) \cdot \frac{\Delta t}{3600} \tag{4.19}$$

Here P_{gen} [W] is the total instantaneous power generated by all sources (solar panels, RPS) summed over the current step, P_{cons} [W] is the total instantaneous power consumed by all hardware over the current step (already inflated by the power loss margin, see below), Δt [s] is the simulation time step, and E_{net} [W hr] is the net energy surplus (if positive) or deficit (if negative) available for that step.

This is computed in func_main_true_SC_power as obj.instantaneous_energy = (obj.instantaneous_total_power_generated - obj.instantaneous_total_power_consumed) * (mission.true_time.time_step/3600). The sign of obj.instantaneous_energy then determines whether the code proceeds to charge the batteries (positive branch) or discharge them (negative branch).

4.6.2.2 Power Consumption with Loss Margin

$$P_{\text{cons}} = \sum_j p_j \cdot (1 + f_{\text{loss}}) \tag{4.20}$$

p_j [W] is the instantaneous power drawn by hardware item j (e.g., a sensor or actuator), and f_{loss} [dimensionless, fraction] is the power loss rate representing harness/converter/regulation losses not otherwise modeled explicitly. P_{cons} [W] is the resulting total instantaneous power consumption used in the energy balance above.

This is implemented in `func_update_instantaneous_power_consumed` as `obj.instantaneous_total_power_consumed = obj.instantaneous_total_power_consumed + (equipment.instantaneous_power_consumed * (1 + obj.power_loss_rate))`, where `equipment.instantaneous_power_consumed` is p_j and `obj.power_loss_rate` is f_{loss}. The attitude-loop counterpart, `func_update_instantaneous_power_consumed_attitude`, applies the same loss factor but additionally scales p_j by the ratio $\Delta t_{\text{attitude}}/\Delta t$ (i.e. `mission.true_time.time_step_attitude/mission.true_time.time_step`) to account for attitude-determination-and-control hardware that runs on a finer sub-step than the main simulation loop.

4.6.2.3 Battery Charging (Excess Energy)

$$C_i \leftarrow \min\left(C_i + \eta_{c,i}\, E_{\text{net}},\ C_i^{\max}\right), \qquad E_{\text{net}} \leftarrow \max\left(E_{\text{net}} - \frac{C_i^{\max} - C_i}{\eta_{c,i}},\ 0\right) \tag{4.21}$$

C_i [W hr] is the instantaneous stored capacity of battery i, $C_i^{\max}$ [W hr] is that battery's maximum capacity, and $\eta_{c,i}$ [dimensionless, $0 < \eta_{c,i} \leq 1$] is its charging efficiency. E_{net} [W hr] is the leftover (positive) energy from the balance equation being distributed across the battery bank.

When `obj.instantaneous_energy > 0`, `func_main_true_SC_power` loops over batteries `mission.true_SC{i_SC}.true_SC_battery{i_batt}` (from last index to first) and, for each healthy battery (`health == 1`) that is not already full, either tops it up fully — `instantaneous_capacity = instantaneous_capacity + charging_efficiency*obj.instantaneous_energy` and zeroes `obj.instantaneous_energy` — or fills it to `maximum_capacity` and carries the remainder forward by decrementing `obj.instantaneous_energy` by `(maximum_capacity - instantaneous_capacity)/charging_efficiency`. Any energy left after all batteries are full is recorded as `obj.instantaneous_energy_unused` (documented as excess energy that would otherwise go to heat).

4.6.2.4 Battery Discharging (Energy Deficit) and Power Emergency

$$C_i \leftarrow \max\left(C_i - \frac{|E_{\text{net}}|}{\eta_{d,i}},\ 0\right), \qquad E_{\text{net}} \leftarrow \min\left(E_{\text{net}} + \eta_{d,i}\, C_i,\ 0\right) \tag{4.22}$$

$\eta_{d,i}$ [dimensionless, $0 < \eta_{d,i} \leq 1$] is the discharging efficiency of battery i, and the remaining symbols are as defined above, with $E_{\text{net}} < 0$ representing a power deficit that the battery bank must supply. If, after discharging all healthy batteries in index order ($i_{\text{batt}} = 1$ upward), E_{net} is still negative, a system-level power emergency is declared.

This is the negative branch of `func_main_true_SC_power`: for each healthy, non-empty battery it either drains enough to cover the deficit (`instantaneous_capacity = instantaneous_capacity - abs(obj.instantaneous_energy)/discharging_efficiency`, zeroing the deficit) or empties the battery (`instantaneous_capacity = 0`) and reduces the remaining deficit by `instantaneous_capacity*discharging_efficiency`. If `obj.instantaneous_energy` is still negative after the loop, the code sets `obj.power_emergency = true`, records `obj.power_deficit = abs(obj.instantaneous_energy)`, issues a rate-limited MATLAB `warning` (capped at

10 occurrences via `obj.warning_counter_empty`), and clamps `obj.instantaneous_energy` to 0 before the state is reset for the next step. Otherwise the emergency flag and counters are cleared.

4.6.2.5 Per-Hardware Energy Accumulation

$$A_k \leftarrow A_k + p_k \cdot \frac{\Delta t}{3600} \tag{4.23}$$

A_k [W hr] is the cumulative energy consumed (or generated) by the k-th named hardware item since its ledger entry was created, p_k [W] is that item's instantaneous power consumption (or generation), and Δt [s] is the (main or attitude) time step.

This bookkeeping appears in `func_initialize_list_HW_energy_consumed/func_initialize_list_HW_energy_generated` (first entry) and is updated every step in `func_update_instantaneous_power_consumed`, `func_update_instantaneous_power_consumed_attitude`, and `func_update_instantaneous_power_generated`, each of which increments the appropriate slot of `obj.array_HW_energy_consumed` or `obj.array_HW_energy_generated` indexed by matching `equipment.name` against `obj.list_HW_energy_consumed/generated`.

4.6.3 CML 2–5 Suitability

4.6.3.1 What is included and correctly implemented

The class implements a physically sensible, first-order power and energy balance appropriate for early concept-design trade studies.

- **First-order power/energy balance.** It nets total generated power against total consumed power (with an adjustable loss-rate margin `power_loss_rate` to approximate harness/regulation losses), converts the result to energy over the simulation time step, and routes any surplus or deficit through a bank of one or more batteries using distinct charging and discharging efficiencies per battery, correctly respecting each battery's `maximum_capacity` and `health` state and partially filling/draining across multiple batteries in priority order rather than assuming a single lumped battery.
- **Multi-rate handling and emergency flagging.** It also correctly tracks the case where instantaneous power draw needs different time-step scaling for the finer attitude-control loop versus the main loop, distinguishes unused excess energy (that would need to be dissipated as heat) from stored energy, and provides a power-emergency flag with a quantified deficit (`power_deficit`) and a rate-limited warning, which is useful for identifying infeasible power designs during trade-space exploration.
- **Per-hardware accounting.** Per-hardware energy accumulation (named lists/arrays for both consumers and generators) gives visibility into which subsystems drive the power budget, supporting subsystem-level trades.
- **Overall fidelity match to CML 2–5.** All of this is consistent with CML 2–5 expectations of medium-fidelity, first-order physics without requiring flight-software-level implementation.

4.6.3.2 What is not included / gaps

The model has several scope limitations that are acceptable at CML 2–5 but worth noting.

- **No thermal/environmental coupling or transient effects.** The model has no thermal or environmental coupling of its own (e.g., no temperature dependence of battery efficiency/capacity, no bus-voltage or regulation modeling, no explicit modeling of where "unused" energy actually goes as heat beyond the placeholder variable) and it does not model transient/dynamic effects such as inrush currents, duty-cycling within a step, or bus voltage drop under high load.

- **No corrective response to power emergencies.** The power-emergency handling is a simple flag-and-warn mechanism with no corrective/fault-response logic (e.g., automatic load shedding), so mission-level consequences of a power deficit must be handled elsewhere or not at all in the current model.

- **Fixed battery priority order, no fault redundancy.** Battery charge/discharge priority is a fixed index-based order (last-to-first for charging, first-to-last for discharging) rather than any configurable or capacity-based allocation strategy, and there is no cell-level or bus-level redundancy/fault modeling beyond the single `health` boolean per battery.

- **No tests or input validation.** Finally, the code has no unit tests or input validation (e.g., no check that `power_loss_rate` or efficiencies are within physically valid ranges), which is acceptable at CML 2–5 but would need to be hardened for later phases.

4.7 True_SC_SRP

True_SC_SRP models the disturbance force and torque that solar radiation pressure (SRP) exerts directly on the spacecraft's own body and solar panel surfaces, as distinguished from SRP effects computed elsewhere on a separate target body (e.g., in True_Target_Computed). The class discretizes the spacecraft exterior into a set of flat faces — both from the main body shape model and from any solar panels (each panel contributing two oppositely-facing sides with independent reflectance properties) — and stores, for each face, its area, unit-normal orientation in the body frame, reflectance factor, and center-of-pressure location relative to the body frame origin.

At each simulation step, func_main_true_SC_SRP loops over every illuminated face (i.e., those facing the Sun within 90° incidence), computes the SRP force contribution from solar flux attenuated by the inverse-square of the Sun–spacecraft distance, and accumulates it into a total inertial-frame disturbance force. It simultaneously computes the torque produced by each face's force acting at its center of pressure relative to the spacecraft center of mass, accumulating a total body-frame disturbance torque. These totals are added into the spacecraft's orbit-propagation disturbance force and ADC (attitude determination and control) disturbance torque accumulators, and are logged to the store structure for post-processing.

4.7.1 Inputs / Outputs

4.7.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** enable_SRP (boolean flag enabling/disabling the SRP calculation).
- True_SC_Body **(same spacecraft):** shape_model (Faces, Face_reflectance_factor, Face_area, Face_normal, Face_center for every body shape), read once at construction to build obj.face_data; location_COM, read each step in func_main_true_SC_SRP to compute the lever arm from the center of mass to each face's center of pressure.
- True_SC_Solar_Panel **(same spacecraft, all instances, if present):** shape_model.Faces, Face_reflectance_factor_solar_cell_side/_opposite_side, Face_area, Face_orientation_solar_cell_side, Face_center, read once at construction to build obj.solar_panels_face_data for both sides of each panel face.
- True_SC_Orbit_Propagation **(same spacecraft):** position, read each step to compute the spacecraft-to-Sun vector for every illuminated face.
- True_SC_ADC **(same spacecraft):** rotation_matrix, read each step (and its transpose) to rotate face normals/lever arms between body and inertial frames.
- mission.true_solar_system: SS_body{index_Sun}.position, solar_constant_AU, light_speed, AU_distance, used in the SRP force-magnitude formula (inverse-square flux attenuation divided by the speed of light).
- mission.storage: num_storage_steps, flag_store_this_time_step for storage array sizing/indexing.

4.7.1.2 Outputs

- **Internal / stored outputs:** `disturbance_torque_SRP_body` and `disturbance_force_SRP_inertial` are recomputed every call to `func_main_true_SC_SRP` (summed over all body and solar-panel faces) and stored in `obj.store`.
- **Reported to / written into** `True_SC_Orbit_Propagation`: `disturbance_force_SRP_inertial` is added directly into `mission.true_SC{i_SC}.true_SC_orbit_propagation.disturbance_force` inside `func_main_true_SC_SRP` itself.
- **Reported to / written into** `True_SC_ADC`: `disturbance_torque_SRP_body` is added directly into `mission.true_SC{i_SC}.true_SC_adc.disturbance_torque` inside the same function call.
- **Reported to / read by plotting code** (`func_plot_attitude_actuator_performance.m`): reads `enable_SRP` and `store.disturbance_torque_SRP_body` to compare SRP torque magnitude against other disturbance/control torques; purely read-only.
- **No external class writes back into this class's own properties** (no other class assigns into `true_SC_SRP.enable_SRP`, `disturbance_torque_SRP_body`, `disturbance_force_SRP_inertial`, `face_data`, or `solar_panels_face_data`); the only cross-class writes observed are `True_SC_SRP` itself pushing its computed force/torque outward into `True_SC_Orbit_Propagation` and `True_SC_ADC`, which is the class's intended reporting mechanism, not an external mutation of this class.

4.7.2 Mathematical Equations

4.7.2.1 Solar Incidence Angle per Face

$$\theta_i = \cos^{-1}(\hat{n}_i \cdot \hat{s}_i) \tag{4.24}$$

Here $\hat{n}_i$ is the unit outward normal of face i, expressed in the inertial (J2000) frame by rotating the body-frame face normal through the spacecraft attitude rotation matrix; $\hat{s}_i$ is the unit vector pointing from the face's center of pressure toward the Sun, also expressed in the inertial frame; and θ_i is the resulting solar incidence angle in degrees. A face is only illuminated (i.e., contributes SRP) when $|\theta_i| < 90°$.

In the code, $\hat{n}_i$ corresponds to `SC_face_normal_normalized_inertial`, computed as `mission.true_SC{i_SC}.true_SC_adc.rotation_matrix * face_i_body` (normalized), where `face_i_body` is `obj.face_data(i).orientation` (or the solar panel equivalent). The Sun-pointing vector $\hat{s}_i$ corresponds to `faceCP_sun_normalized_inertial`, built from the negative of the vector sum of the spacecraft's inertial position, the rotated face center-of-pressure offset `faceCP_sc_body`, minus the Sun's position (`mission.true_solar_system.SS_body{...index_Sun}.position`). The angle itself is `incidence = real(acosd(dot(...)))`, and the illumination test is `if abs(incidence) < 90`.

4.7.2.2 SRP Force per Face

$$\mathbf{F}_{SRP,i} = \Phi_{AU}\,\frac{1}{c}\left(\frac{d_{AU}}{d_{Sun}}\right)^2 A_i\,(1+\rho_i)\,\cos\theta_i\,(-\hat{s}_i) \tag{4.25}$$

Here Φ_{AU} is the solar constant (solar flux at 1 AU, W/m^2); c is the speed of light (m/s); d_{AU} is the length of one astronomical unit (m), used together with the instantaneous Sun-to-spacecraft

distance d_{Sun} (m) to scale the flux by inverse-square law; A_i is the face area (m^2); $\rho_i \in [0, 1]$ is the face's reflectance factor; θ_i is the incidence angle from the previous equation; and $-\hat{s}_i$ is the unit vector pointing from the Sun toward the face (i.e., the force acts opposite the face-to-sun direction, pushing the face away from the Sun). $\mathbf{F}_{SRP,i}$ is the resulting force vector (N) in the inertial frame for face i. The total spacecraft SRP force is the sum of $\mathbf{F}_{SRP,i}$ over all illuminated faces (body and solar panel faces).

In the code, Φ_{AU} is `mission.true_solar_system.solar_constant_AU`, c is `mission.true_solar_system.light_speed`, d_{AU} is `mission.true_solar_system.AU_distance`, and d_{Sun} is `norm(mission.true_SC{i_SC}.true_SC_orbit_propagation.position - mission.true_solar_system.SS_body{...index_Sun}.position)`. A_i and ρ_i are `face_area` and `face_reflectance_factor` pulled from `obj.face_data` or `obj.solar_panels_face_data`. The magnitude is computed as `F_SRP_magnitude`, and the vector as `F_SRP_vector_J2000 = F_SRP_magnitude * -faceCP_sun_normalized_inertial`. Face contributions are summed into `obj.disturbance_force_SRP_inertial`, which is later added to `mission.true_SC{i_SC}.true_SC_orbit_propagation.disturbance_force`.

4.7.2.3 SRP Torque per Face

$$\boldsymbol{\tau}_{SRP,i} = (\mathbf{r}_{cp,i} - \mathbf{r}_{COM}) \times \left(R^T \mathbf{F}_{SRP,i}\right) \tag{4.26}$$

Here $\mathbf{r}_{cp,i}$ is the body-frame location of face i's center of pressure (m); $\mathbf{r}_{COM}$ is the body-frame location of the spacecraft center of mass (m), so their difference is the lever arm from the center of mass to the center of pressure; R is the spacecraft's body-to-inertial attitude rotation matrix, so $R^T\mathbf{F}_{SRP,i}$ rotates the inertial-frame SRP force back into the body frame; and $\boldsymbol{\tau}_{SRP,i}$ is the resulting SRP disturbance torque (N·m) about the center of mass, expressed in the body frame. The total SRP torque is the sum of $\boldsymbol{\tau}_{SRP,i}$ over all illuminated faces.

In the code, $\mathbf{r}_{cp,i}$ is `faceCP_sc_body`, $\mathbf{r}_{COM}$ is `mission.true_SC{i_SC}.true_SC_body.location_COM`, and the lever arm is `this_face_lever_arm_body = faceCP_sc_body - ...location_COM'`. The rotated force is `force_on_surface_at_cp_body = (mission.true_SC{i_SC}.true_SC_adc.rotation_matrix)' * F_SRP_vector_J2000`, and the per-face torque is `this_face_torque_body = cross(this_face_lever_arm_body, force_on_surface_at_cp_body)`. Accumulated torque is stored in `obj.disturbance_torque_SRP_body`, which is added to `mission.true_SC{i_SC}.true_SC_adc.disturbance_torque`.

4.7.3 CML 2–5 Suitability

4.7.3.1 What is included and correctly implemented

The class implements a physically reasonable first-order flat-plate SRP model appropriate for early trade studies.

- **Face-based geometry decomposition.** It decomposes the spacecraft exterior into discrete faces (body faces plus two-sided solar panel faces) with independent area, orientation, reflectance, and center-of-pressure metadata, which is sufficient granularity to capture gross force/torque effects of spacecraft geometry and solar panel articulation on both translational dynamics (via the inertial-frame disturbance force fed to orbit propagation) and rotational dynamics (via the body-frame disturbance torque fed to the ADC disturbance torque accumulator).

- **Standard first-order SRP formulations.** The inverse-square attenuation of solar flux with Sun distance, the per-face illumination (self-shadowing by incidence angle) test, the (1 +

$\rho)\cos\theta$ reflectance/incidence scaling of force magnitude, and the lever-arm cross-product torque calculation about the true center of mass are all standard, correct first-order SRP formulations well matched to CML 2–5 medium-fidelity needs.

- **Integration and usability.** The model is toggled by a simple `enable_SRP` flag, is cleanly integrated into the broader mission disturbance-accumulation architecture, and includes storage/logging hooks for downstream analysis, all of which support rapid trade-space iteration typical of concept formulation.

4.7.3.2 What is not included / gaps

The model uses only simple flat-plate optics, and several second-order effects are absent.

- **Simplified optics model.** The model uses only simple flat-plate optics via a single scalar reflectance factor per face and $(1+\rho)\cos\theta$ scaling; it does not separate specular, diffuse, and absorptive components (e.g., no distinct specular/diffuse reflectivity coefficients or associated tangential force terms), which limits fidelity beyond first-order estimates.

- **No inter-face shadowing or eclipse.** There is no inter-face shadowing/self-occlusion logic beyond the simple 90° incidence cutoff per face — faces are treated independently, so one part of the spacecraft (e.g., a solar panel) blocking sunlight from reaching another face is not modeled, and no eclipse/occultation by planetary bodies is referenced in this file.

- **Fixed, externally supplied shape data.** The reflectance factors, face areas, orientations, and center-of-pressure locations are taken as fixed, externally supplied shape-model data with no visible temperature-, wavelength-, or degradation-dependent variation, and solar panel articulation is only reflected insofar as the panel orientation data passed in already accounts for it (no dynamic gimbal/tracking logic appears in this class).

- **No uncertainty quantification or added coupling.** There is no explicit uncertainty quantification, no thruster/attitude coupling beyond accumulating into the disturbance torque, and the commented-out call to compute SRP before the first ADC iteration (to avoid double counting) suggests a known first-step accounting subtlety that a reviewer should be aware of.

- **Acceptable for CML 2–5.** None of these gaps are unusual or disqualifying for CML 2–5 formulation-level analysis, but they mark the boundary beyond which higher-fidelity (CML 5+/Phase A-B) SRP modeling would be needed.

Chapter 5

SC Sensors and Actuators Classes

5.1 True_SC_Battery

True_SC_Battery implements the electrical energy storage element of a spacecraft's power subsystem within the True-model (truth/simulation) layer of MuSCAT. It is a handle class instantiated once per battery unit (indexed by i_HW) attached to a given spacecraft, and its sole state-tracking responsibility is to maintain the battery's instantaneous stored energy (instantaneous_capacity) and the derived state of charge (state_of_charge) at each simulation time step. It does not itself model charging or discharging dynamics; rather, it exposes the battery's identity, capacity, and efficiency parameters to True_SC_Power, which is the class actually responsible for computing power generation, consumption, and the resulting energy flow into or out of the battery.

The interface with True_SC_Power is bidirectional and constructor/step-driven: at construction, True_SC_Battery calls func_initialize_list_HW_energy_consumed on mission.true_SC{i_SC}.true_SC_power (and an analogous call into True_SC_Data_Handling) to register itself as a power-consuming (and data-generating) hardware element. At each time step, func_main_true_SC_battery recomputes state_of_charge from the current instantaneous_capacity, logs both to the store structure, and then invokes func_update_instantaneous_power_consumed and func_update_instantaneous_data_generated on the power and data-handling classes, respectively, which are presumably where True_SC_Power feeds energy back into (or draws energy from) obj.instantaneous_capacity. Within this file, the only safeguard against unphysical behavior is a check that instantaneous_capacity has not gone negative, which throws a hard error if violated.

5.1.1 Inputs / Outputs

5.1.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** name (optional), instantaneous_power_consumed, instantaneous_data_rate_generated, maximum_capacity, charging_efficiency, discharging_efficiency.
- mission.storage: num_storage_steps, flag_store_this_time_step, k_storage — used only for sizing/populating obj.store, not physical truth.
- **No other True/Software class is read by** True_SC_Battery**.** Unlike most hardware classes, the battery's own state-of-charge update (instantaneous_capacity) is driven entirely from the outside (by True_SC_Power, see below) rather than computed internally from any other class's truth state.

5.1.1.2 Outputs

- **Internal / stored outputs:** `instantaneous_capacity` and `state_of_charge` are computed each call to `func_main_true_SC_battery` (as `state_of_charge = 100*instantaneous_capacity/maximum_capacity`) and stored in `obj.store`.
- **Reported to `True_SC_Power`:** the battery registers with `True_SC_Power` at construction via `func_initialize_list_HW_energy_consumed` and each step via `func_update_instantaneous_power_consumed`, exposing `instantaneous_power_consumed` and `name` so `True_SC_Power` can accumulate it into the SC's total consumed power/energy ledger.
- **Reported to `True_SC_Data_Handling`:** `instantaneous_data_rate_generated` is registered via `func_initialize_list_HW_data_generated` and consumed by `func_update_instantaneous_data_generated`, which folds it into `True_SC_Data_Handling`'s accumulated data volume.
- **External class writes back into this class's properties: yes.** `True_SC_Power.func_main_true_SC_power` directly reads AND writes `mission.true_SC{i_SC}.true_SC_battery{i_batt}.instantaneous_capacity` (charging it up when there is excess generated power, discharging it when consumption exceeds generation), gated by `true_SC_battery{i_batt}.health`, `charging_efficiency`, and `discharging_efficiency`. This is the primary mechanism by which the battery's state of charge actually evolves — `True_SC_Battery` itself only recomputes the derived percentage from whatever capacity `True_SC_Power` has left it with.

5.1.2 Mathematical Equations

5.1.2.1 State of Charge

$$\mathrm{SoC} = 100 \times \frac{C_{\mathrm{inst}}}{C_{\max}} \tag{5.1}$$

Here SoC is the battery state of charge in percent, C_{inst} is the instantaneous stored energy of the battery in units of Watt-hours, and $C_{\max}$ is the maximum energy storage capacity of the battery, also in Watt-hours.

This is computed twice in the file: once in the constructor and once per simulation step in `func_main_true_SC_battery`, in both cases as `obj.state_of_charge = 100*obj.instantaneous_capacity/obj.maximum_capacity`. The variable `obj.instantaneous_capacity` is initialized equal to `obj.maximum_capacity`, i.e., the battery starts fully charged, and is otherwise updated externally (presumably by `True_SC_Power`, not within this file). `obj.maximum_capacity` is a fixed parameter taken from `init_data.maximum_capacity` at construction and is not modified anywhere in this class, so no capacity-fade or aging model is present.

5.1.2.2 Non-negativity (Depletion) Constraint

$$C_{\mathrm{inst}} \geq 0 \quad \text{(enforced as a hard error if violated)} \tag{5.2}$$

C_{inst} is the same instantaneous battery capacity as above. No physical variables besides C_{inst} enter this check.

This constraint is implemented in `func_main_true_SC_battery` as `if obj.instantaneous_capacity < 0, error('Battery SoC is negative!')`. It is a validity/sanity check on the externally-updated capacity value rather than a computed physical model; there is no clamping, no discharge-limit model, and no charge/discharge rate-limit equation in this file. The properties

`charging_efficiency` and `discharging_efficiency` are declared and initialized from `init_data` but are never referenced in any equation or update logic within `True_SC_Battery.m`; their application (if any) must occur in `True_SC_Power`.

5.1.3 CML 2–5 Suitability

5.1.3.1 What is included and correctly implemented

The class correctly implements the basic bookkeeping needed for a first-order battery model at trade-space fidelity.

- **Capacity and state-of-charge bookkeeping.** A fixed maximum capacity parameter, an instantaneous capacity state variable initialized to full charge, and a straightforward percentage state-of-charge calculation are consistent at both initialization and each simulation step.
- **Correct storage/logging integration.** It correctly wires itself into the mission's storage/logging framework (`mission.storage.num_storage_steps`, `func_update_true_SC_battery_store`) so that `instantaneous_capacity` and `state_of_charge` time histories are recorded only on designated storage steps, which is appropriate and consistent with how other True-model hardware classes appear to log state.
- **Clean separation of concerns.** It also correctly registers itself with `True_SC_Power` and `True_SC_Data_Handling` at construction and calls their update functions each step, maintaining a clean separation of concerns where the battery tracks capacity/SoC while power generation and consumption physics live elsewhere.
- **Negative-capacity sanity check.** The negative-capacity error is a reasonable, cheap sanity check appropriate for catching gross modeling errors during point-design trade studies.
- **Appropriate fidelity for CML 2–5.** For CML 2–5 purposes, where medium/first-order fidelity is acceptable, this level of detail (capacity, SoC, efficiency parameters, health flag) is a reasonable representation of a battery without requiring flight-software-representative charge-controller logic.

5.1.3.2 What is not included / gaps

Several omissions are present, though they are consistent with, and acceptable for, CML 2–5 scope.

- **No charge/discharge equations in this file.** No charge or discharge equations are present in this file: `instantaneous_capacity` is only initialized here and is otherwise mutated externally (presumably in `True_SC_Power`), so this class alone cannot be validated for energy-balance correctness.
- **Unused efficiency parameters.** The declared `charging_efficiency` and `discharging_efficiency` parameters are stored but never used in any formula within `True_SC_Battery.m`, so it is unclear from this file alone whether charge/discharge losses are actually applied anywhere.
- **No degradation, thermal, or rate-limit modeling.** There is no model of capacity fade, cycle-life degradation, temperature-dependent capacity or efficiency (despite a `temperature` property being tracked and fixed at 10 °C with no update logic), voltage/current limits, depth-of-discharge limits, or maximum charge/discharge rate constraints.

- **Health flag never read or transitioned.** The `health` property is defined with only two documented states (off / nominal) and is never read or transitioned within this file, so no degraded-health or fault-injection behavior is implemented here.
- **No graceful depletion handling.** The only protective mechanism is a hard error on negative capacity, with no graceful handling (e.g., load-shedding, safe-mode transition) of an impending depletion condition. These omissions are consistent with, and acceptable for, CML 2–5 scope, since flight-software-representative battery management (BMS logic, thermal derating, aging) is not expected at this maturity level.

5.2 True_SC_Camera

True_SC_Camera models the truth-state behavior of a spacecraft imaging sensor (e.g., a navigation or science camera) within the MuSCAT simulation. It maintains the camera's fixed mounting geometry in the spacecraft body frame (location, orientation, orientation_up), its imaging parameters (resolution, field_of_view), and its operational state (health, temperature, measurement_wait_time). On each executive-commanded step it determines whether a designated target body falls within the camera's field of view by rotating the boresight vector into the inertial/target frame using the spacecraft's true attitude and comparing the resulting angular offset to the half-cone field of view. It also supports a discretized "science point" mesh over the target body (loaded via func_load_science_points_v2 and converted to longitude/latitude via cspice_reclat) to accumulate monostatic imaging coverage statistics over the simulation.

The class interfaces closely with other True_SC objects: it queries true_SC_adc for the current attitude rotation matrix, true_SC_orbit_propagation for spacecraft-to-target relative position, and true_target for target position, shape, and orientation (and true_solar_system for the Sun's position, used only to illuminate the rendered scene). It reports generated data volume to true_SC_data_handling (via func_update_instantaneous_data_generated) and reports power draw to true_SC_power (via func_update_instantaneous_power_consumed), and it is driven by a flag_executive set by the spacecraft's software executive. A second, more elaborate mission-specific mode is referenced but implemented outside this file.

5.2.1 Inputs / Outputs

5.2.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed, instantaneous_data_generated_per_pixel, instantaneous_data_compression (optional), mode_true_SC_camera_selector, measurement_wait_time, location, orientation, orientation_up, resolution, field_of_view, flag_show_camera_plot, flag_show_stars (optional), flag_show_coverage_plot (optional), wait_time_visualize_SC_camera_coverage_during_sim (optional), num_points, data (optional passthrough).
- True_SC_ADC **(same spacecraft):** rotation_matrix, used to rotate the camera's fixed body-frame orientation/orientation_up into the inertial frame for both the plotting view and the target-visibility test (func_true_SC_camera_target_visible).
- True_SC_orbit_propagation **(same spacecraft):** position, position_relative_target, index_relative_target, used to compute the SC-to-target line of sight and camera plotting geometry.
- mission.true_target{...}**:** position, rotation_matrix, radius, name — the current observation target's truth state, used for the target-visibility angle test and the coverage/plot visualization.
- mission.true_solar_system**:** SS_body{index_Sun}.position, used only for lighting the optional 3-D camera plot.
- mission.true_stars **(optional, if** flag_show_stars**):** all_stars_unit_vector, magnitude_visible, maximum_magnitude, used only for the plot's star field.

- `mission.true_SC{i_SC}.software_SC_executive: time`, `this_sc_mode`, used for plot titling/timing of coverage visualization.
- `mission.true_time: time`, used to gate measurement cadence against `measurement_wait_time`.

5.2.1.2 Outputs

- **Internal / stored outputs:** `flag_target_visible`, `data.flag_take_picture`, `flag_executive`, `instantaneous_data_generated_per_sample`, `instantaneous_power_consumed`, `monostatic_num_point_observed`, `monostatic_observed_point` are computed in `func_true_SC_camera_Simple/func_coverage_Generic` and stored in `obj.store`.
- **Reported to `True_SC_Power`:** `instantaneous_power_consumed` is registered/updated via `func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed`.
- **Reported to `True_SC_Data_Handling`:** `instantaneous_data_generated_per_sample` (only non-zero when a picture is actually taken) is registered/updated via `func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated`.
- **Read by `Supporting_Functions/mission_specific/DART/func_update_sw_SC_est_orbit_TruthWithErrorGrowth.m`:** this DART-specific estimator reads `flag_target_visible` to decide whether to use the camera's visibility truth in its orbit-estimate error-growth logic.
- **Set (commanded) by `Supporting_Functions/mission_specific/DART/func_software_SC_executive_DART.m` and `func_update_software_SC_communication_DART.m`:** these DART software-executive/communication routines write `mission.true_SC{i_SC}.true_SC_camera{i_HW}.flag_executive` (set to 1 to activate the primary camera, or reset to 0), which is the standard executive-flag command pattern used across all True sensors/actuators to request a measurement — this is the one case where an external (software) class writes into the camera's own property, but it is the generic "take a measurement now" command flag common to all sensors, not a measurement/physics override.

5.2.2 Mathematical Equations

5.2.2.1 Boresight-to-Target Angle and Field-of-View Visibility Test

$$\hat{x}_{\text{cam}} = R_{B\to I}\,\hat{x}_{\text{cam,body}}, \qquad \theta_{\text{cam}} = \cos^{-1}(\hat{r}_T \cdot \hat{x}_{\text{cam}})\,, \qquad \text{visible} = \begin{cases} 1, & \theta_{\text{cam}} \le \text{FOV} \\ 0, & \text{otherwise} \end{cases} \tag{5.3}$$

Here $\hat{x}_{\text{cam,body}}$ is the camera boresight unit vector expressed in the spacecraft body frame, $R_{B\to I}$ is the spacecraft attitude rotation matrix from body to inertial (or target-relative) frame, $\hat{r}_T$ is the unit vector from the spacecraft to the target, θ_{cam} is the resulting boresight-to-target angle, and FOV is the camera's (half-angle) field of view.

This is implemented in `func_true_SC_camera_target_visible`: `Target_relative_pos_hat` is computed via `func_normalize_vec` of the target-minus-spacecraft position, `x_true_hat` is `mission.true_SC{i_SC}.true_SC_adc.rotation_matrix * obj.orientation'`, `Camera_angle = acosd(dot(Target_relative_pos_hat, x_true_hat))`, and the result sets `obj.flag_target_visible` by comparing `Camera_angle` against `obj.field_of_view`. The same boresight vector and an analogous angular test (against `2*field_of_view`) are reused in `func_true_SC_camera_Simple` to cull background stars (`dot_product_angle_array`, `flag_in_FOV`) before rendering.

5.2.2.2 Instantaneous Image Data Volume

$$D_{\text{sample}} = c \cdot d_{\text{pixel}} \cdot N_x \cdot N_y \tag{5.4}$$

Here D_{sample} is the data volume generated by a single captured image [kb], $c \leq 1$ is a (optional) data compression factor, d_{pixel} is the data generated per pixel [kb], and $N_x \times N_y$ is the pixel resolution of the detector array.

This is implemented directly in `func_true_SC_camera_Simple` as `obj.instantaneous_data_generated_per_sample = obj.instantaneous_data_compression * obj.instantaneous_data_generated_per_pixel * obj.resolution(1) * obj.resolution(2)`, and is subsequently pushed to the spacecraft data-handling ledger via `func_update_instantaneous_data_generated`.

5.2.2.3 Target-Body Surface Point to Longitude/Latitude Mapping

$$(\rho,\ \lambda,\ \phi) = \text{reclat}\left(\mathbf{p}_i\right), \qquad \text{lon}_i = \text{rad2deg}(\lambda), \qquad \text{lat}_i = \text{rad2deg}(\phi) \tag{5.5}$$

Here $\mathbf{p}_i$ is the i-th point of the science mesh (a unit-sphere point on the target body), and $\text{reclat}(\cdot)$ is the standard rectangular-to-latitudinal coordinate transform returning radius ρ, longitude λ, and latitude ϕ (in radians).

This is implemented via the SPICE call `[radius, lon, lat] = cspice_reclat(obj.pos_points(i,:)')` inside the constructor, storing `obj.spherical_points(i,:) = [rad2deg(lon), rad2deg(lat)]`; the same transform is reused in `func_visualize_SC_camera_coverage_during_sim` to project the spacecraft's own sub-target position into longitude/latitude for coverage plotting.

5.2.3 CML 2–5 Suitability

5.2.3.1 What is included and correctly implemented

The class implements a physically reasonable first-order camera model appropriate for trade-space and point-design studies.

- **Body-frame pointing and FOV visibility test.** A fixed boresight/up-vector mounting in the body frame is rotated by the true spacecraft attitude to determine pointing, and a simple conical field-of-view visibility test against a target body is applied.
- **Pixel-driven data volume generation.** Pixel-resolution-driven data volume generation with an optional compression factor is fed into the spacecraft data-handling and power ledgers.
- **Monostatic coverage accumulation.** A monostatic coverage-accumulation scheme over a discretized target-surface mesh (loaded via `func_load_science_points_v2`, mapped to longitude/latitude, and tallied in `monostatic_observed_point`/`monostatic_num_point_observed`) is useful for assessing global imaging coverage over a mission timeline.
- **Health, cadence, and power/data bookkeeping.** Health/temperature state, an executive-gated measurement cadence (`measurement_wait_time`), and power/data bookkeeping consistent with the rest of the True_SC hardware framework are all present.
- **Visualization support.** Visualization support (3D attitude/coverage plots, 2D longitude-latitude coverage maps, percentage-coverage time history, and optional video capture) is implemented and useful for verifying pointing and coverage behavior during formulation-phase analysis.

- **Fidelity appropriate for CML 2–5.** This level of fidelity — a single boresight cone check plus pixel-count data volume — is consistent with CML 2–5 expectations, where medium/first-order fidelity is acceptable and flight-software-representative detector/optics models are not required.

5.2.3.2 What is not included / gaps

None of the following omissions are unusual or disqualifying for CML 2–5 trade-space work, where a coarse cone-visibility and data-volume model is sufficient and flight-representative imaging-chain fidelity is explicitly out of scope.

- **Single scalar FOV, no footprint or occlusion check.** The field-of-view test only checks a single boresight-to-target angle against one scalar `field_of_view` value; there is no rectangular/2D FOV (separate horizontal/vertical half-angles), no check that the target's full angular extent or footprint fits within the frame, and no occlusion, terrain shadowing, or off-nadir/limb geometry test — the target is treated as a point relative to the boresight cone.
- **No ground-sample-distance calculation.** There is no ground-sample-distance (GSD), pixel-scale, or spatial-resolution calculation relating range, focal length, and pixel pitch to surface resolution, despite `resolution` and `field_of_view` being available; footprint size on the target surface is never computed.
- **No illumination or phase-angle gating.** No illumination or phase-angle calculation is performed — the light source position is set only for MATLAB rendering, not evaluated as a phase angle, incidence angle, or used to gate visibility/exposure.
- **No sensor/optics model; scalar data-rate inputs.** There is no sensor noise, exposure time, saturation, blur, distortion, or optical model, and `instantaneous_data_generated_per_pixel`/`instantaneous_data_compression` are simple externally supplied scalars rather than derived quantities.
- **Undefined referenced routines.** The referenced mission-specific camera mode and `func_coverage_Generic` coverage-accumulation routine are called but not defined in this file, so their fidelity and completeness cannot be assessed here.

5.3 True_SC_Communication_Link

True_SC_Communication_Link models a logical communication *link* between a transmitter and a receiver — either a ground station (denoted by the sentinel index 0 for TX_spacecraft/RX_spacecraft) or a spacecraft radio antenna (indexed into mission.true_SC{...}.true_SC_radio_antenna). It is distinct from True_SC_Radio_Antenna, which models the physical antenna hardware state (TX/RX mode, instantaneous data rates in/out, power draw); this class instead owns the link-level bookkeeping: which antenna hardware is paired with which, whether the link currently has line-of-sight, what data rate the link should run at, and coordinating the TX antenna and RX antenna objects to actually move data between them during a given time step.

Each instance is constructed with a fixed pairing of TX/RX endpoints and hardware indices, a link-selection mode string (mode_true_SC_communication_link_selector, either 'Generic' or 'Multi-SC per GS'), and either a flag to compute the data rate from a link budget (flag_compute_data_rate) or a fixed, pre-specified data rate (given_data_rate). The main update method dispatches to one of two link-management strategies each time step, checks whether the link currently has visibility (obtained from the transmitting/receiving spacecraft's orbit-propagation visibility flag, not computed internally), determines the instantaneous data rate for the step, and — if the link is commanded active by the executive — flips on the paired TX and RX antenna objects and hands them the resulting data rate.

5.3.1 Inputs / Outputs

5.3.1.1 Inputs / Dependencies

- init_data **(constructor-time):** TX_spacecraft, TX_spacecraft_Radio_HW, RX_spacecraft, RX_spacecraft_Radio_HW, flag_compute_data_rate, given_data_rate, mode_true_SC_communication_link_selector, instantaneous_power_consumed (optional, defaults to 0.5 W).
- mission.true_GS_radio_antenna{...} **(if an endpoint is a ground station, i.e. spacecraft index 0):** name (for naming), and at runtime flag_executive, mode_true_GS_radio_antenna_selector, instantaneous_data_rate_transmitted/instantaneous_data_rate_received are written into the GS antenna object to start a TX/RX session.
- mission.true_SC{TX/RX_spacecraft}.true_SC_radio_antenna{...} **(if an endpoint is a spacecraft):** name (for naming), and at runtime flag_executive, mode_true_SC_radio_antenna_selector, instantaneous_data_rate_removed (TX side)/instantaneous_data_rate_generated (RX side) are written into the antenna object to start the link.
- True_SC_orbit_propagation **(of the non-ground-station endpoint spacecraft):** flag_visible_Earth, used as the line-of-sight gate for whether the link can carry data this step.
- mission.true_SC{TX_spacecraft}.software_SC_data_handling: total_data_storage, used to cap the effective data rate so a TX spacecraft cannot transmit more data than it currently has stored.
- mission.true_time: time_step, used in the data-rate-vs-storage cap calculation.

5.3.1.2 Outputs

- **Internal / stored outputs:** `flag_executive`, `flag_TX_RX_visible`, `this_data_rate` are computed in `func_communication_link_generic` and stored in `obj.store`.
- **Reported to `True_SC_Power` (if `i_SC > 0`):** `instantaneous_power_consumed` (link control overhead) is registered/updated via `func_initialize_list_HW_energy_consumed`/`func_update_instantaneous_power_consumed`.
- **Drives `True_SC_Radio_Antenna` / `True_GS_Radio_Antenna` objects:** rather than those classes reading from the link, the link class itself writes `flag_executive`, `mode_true_SC_radio_antenna_selector`/`mode_true_GS_radio_antenna_selector`, and the appropriate data-rate property directly into the TX and RX antenna objects each time it actuates a transfer — i.e. this class is the orchestrator that commands the antenna hardware, not a passive data source for it.
- **Read for visualization/reporting:** `store.flag_executive` and `store.this_data_rate` are read by plotting/reporting utilities (e.g. `func_plot_telecom.m`, `func_realtime_performance_plot.m`) and by mission-specific post-processing scripts, but these do not feed back into the simulation.
- **Set (commanded) by mission-specific software communication/executive scripts:** e.g. `func_update_software_SC_communication_DART.m`, `func_update_software_SC_communication_NISAR.m`, and `func_software_SC_executive_DART.m` all write `mission.true_SC{i_SC}.true_SC_communication_link{i_HW}.flag_executive` to turn a given comm link on/off for the current time step — the standard executive-flag command pattern, here used to decide which link(s) are actively transferring data.

5.3.2 Mathematical Equations

5.3.2.1 Data Rate Selection: Given Rate vs. Eclipse/No-Visibility

$$R_{\text{link}} = \begin{cases} R_{\text{given}} & \text{if } V_{\text{TX,RX}} = 1 \text{ and computed-rate flag} = 0 \\ 0 & \text{if } V_{\text{TX,RX}} = 0 \end{cases} \tag{5.6}$$

where R_{link} is the instantaneous link data rate for the current time step, R_{given} is a fixed, pre-specified data rate supplied at initialization, and $V_{\text{TX,RX}}$ is a Boolean visibility indicator for the link.

In the code, R_{link} is `obj.this_data_rate`, R_{given} is `obj.given_data_rate` (units of kbps), and $V_{\text{TX,RX}}$ is `obj.flag_TX_RX_visible`. Visibility itself is not computed by this class; it is read directly from `mission.true_SC{...}.true_SC_orbit_propagation.flag_visible_Earth` of whichever end of the link is the spacecraft (the ground-station end is assumed always visible from its own side). No range, elevation-angle, or other geometric quantity is computed inside `True_SC_Communication_Link` itself — the link-visibility geometry lives entirely in the orbit-propagation object it queries. The current implementation of `func_communication_link_generic` also explicitly only supports one ground-station-to-spacecraft leg at a time: if both `TX_spacecraft` and `RX_spacecraft` are nonzero (i.e., an inter-spacecraft crosslink), the code raises `error('Need to write this code where both are SC!')` and is not implemented.

5.3.2.2 Onboard Data Storage Rate Limiting

$$R'_{\text{link}} = \begin{cases} \dfrac{D_{\text{stored}}}{\Delta t} & \text{if } R_{\text{link}}\,\Delta t \geq D_{\text{stored}} \text{ and TX is a spacecraft and } V_{\text{TX,RX}} = 1 \\ R_{\text{link}} & \text{otherwise} \end{cases} \tag{5.7}$$

where D_{stored} is the total onboard data currently queued for downlink at the transmitting spacecraft and Δt is the simulation time step. This equation prevents the link from reporting a data rate that would transmit more data in one step than is actually stored onboard — i.e., it clamps the rate to exactly drain the buffer rather than overshoot it.

In the code this is the line `obj.this_data_rate = mission.true_SC{obj.TX_spacecraft}.software_SC_data_handling.total_data_storage / mission.true_time.time_step;`, guarded by the condition `(obj.TX_spacecraft > 0) && (obj.flag_TX_RX_visible == 1) && ((obj.this_data_rate * mission.true_time.time_step) >= mission.true_SC{obj.TX_spacecraft}.software_SC_data_handling.total_data_storage)`, evaluated only when the link is active (i.e., when `obj.flag_executive == 1`). D_{stored} maps to `total_data_storage` on the transmitting spacecraft's `software_SC_data_handling` object and Δt maps to `mission.true_time.time_step`.

5.3.2.3 Link Margin Computation (Intended but Not Implemented)

No link-budget or link-margin equation (e.g., received power, E_b/N_0, or margin versus a required threshold as a function of range, transmit power, antenna gains, and losses) is implemented anywhere in this file. The branch that is clearly intended to hold this calculation is gated by `obj.flag_compute_data_rate == 1` inside `func_communication_link_generic`: it sets `obj.this_data_rate = 0;` and then immediately calls `error('Write code to compute Link Margin!')`. As confirmed by reading the file, this branch is a stub that always raises an error if reached; every current mission configuration sets `flag_compute_data_rate = 0` and instead relies exclusively on the fixed `given_data_rate` described above, so this branch is never exercised in practice. This mirrors the same "intended but not implemented" link-margin gap noted for `True_GS_Radio_Antenna`.

5.3.3 CML 2–5 Suitability

5.3.3.1 What is included and correctly implemented

The class correctly implements the bookkeeping needed for a first-order, point-design-level communication link model.

- **Fixed TX/RX pairing and executive/visibility gating.** It establishes a fixed TX/RX pairing between either a ground station and a spacecraft antenna or between two spacecraft antennas (by index into the respective antenna hardware lists), resolves human-readable link names at construction time, and gates data flow on an executive-commanded flag together with a line-of-sight visibility flag obtained from orbit propagation.
- **Fixed-rate data-rate model appropriate for pre-negotiated link budgets.** The core data-rate model — a fixed, user-specified data rate applied whenever the link has visibility and zero rate otherwise (e.g., during eclipse/occultation) — is exactly the kind of medium-fidelity, pre-negotiated-link-budget approach appropriate for CML 2–5 trade studies, where analysts typically supply an assumed downlink rate from an external link-budget spreadsheet rather than recomputing RF physics inside the simulation.

- **Onboard-storage rate-limiting safeguard.** The onboard-storage rate-limiting logic (clamping the reported data rate so it never drains more data than is actually queued in a single time step) is a sound, physically meaningful safeguard for mass/data-budget studies.
- **Coordinated state across TX/RX objects and power system.** The class also correctly coordinates state across objects: it sets the paired TX and RX antenna objects' executive flags, mode selectors, and instantaneous data rates, and it registers link-control overhead power draw with the spacecraft power system, all of which are appropriate levels of fidelity for early trade-space analysis.

5.3.3.2 What is not included / gaps

Several gaps are present, most of which are acceptable simplifications for CML 2–5 but limit applicability to higher-fidelity work.

- **Link-margin data-rate path unimplemented.** The intended data-rate-from-link-margin path is entirely unimplemented: setting `flag_compute_data_rate = 1` leads directly to a hard `error('Write code to compute Link Margin!')`, so no RF link budget, range dependence, or margin threshold is ever evaluated by this class; all current missions bypass it via a hard-coded `given_data_rate`. This is acceptable for CML 2–5 (a fixed assumed data rate is a standard simplification at this maturity level) but means the tool cannot currently be used to self-consistently derive data rate from geometry, transmit power, and antenna performance — any such trade must be done offline and fed in as `given_data_rate`.
- **No spacecraft-to-spacecraft crosslink support in generic mode.** The generic link-management mode (`func_communication_link_generic`) explicitly does not support spacecraft-to-spacecraft crosslinks: if both link endpoints are spacecraft, the code raises `error('Need to write this code where both are SC!')`, so only ground-station-to-spacecraft links are functional in this mode.
- **Visibility is a pass-through Boolean with no independent geometry.** Link visibility is treated as a simple pass-through Boolean from the spacecraft's Earth-visibility flag; no independent range, elevation angle, or slant-distance calculation is performed within this class, so any geometry-dependent effects (e.g., rate falloff with range, minimum elevation mask, multi-hop relay geometry) are entirely delegated to (and limited by) whatever the orbit-propagation visibility flag itself captures.
- **Undefined alternate dispatch mode.** The alternate dispatch mode `'Multi-SC per GS'` calls `func_communication_link_MultiSC_per_GS`, which is not defined in this file, so its fidelity and completeness cannot be assessed from this source alone.

5.4 True_SC_Chemical_Thruster

The True_SC_Chemical_Thruster class models a single chemical (monopropellant-style, specific-impulse-driven) thruster mounted on the spacecraft body. It manages a simple operational state machine (idle → warming_up → ready → firing) that gates when thrust can actually be produced, tracks power consumption in each state, and computes the resulting thrust force vector, its transformation into the inertial frame, the disturbance torque it exerts about the spacecraft center of mass, and the propellant mass it consumes per firing. When commanded to fire by the software orbit-control layer (flag_executive and commanded_thrust), the class adds noise to the commanded thrust, clamps it to minimum_thrust/maximum_thrust, rotates the resulting body-frame force into the inertial frame using the attitude quaternion from True_SC_ADC, and injects that force into True_SC_Orbit_Propagation.control_force for orbit propagation while also injecting the corresponding torque into True_SC_ADC.disturbance_torque.

The class also interfaces with mass/propellant bookkeeping and other spacecraft subsystems: it computes an impulsive ΔV contribution that is accumulated into the software orbit-control object's executed ΔV tally, computes propellant mass consumed via the rocket-equation mass-flow relation and withdraws that mass from the assigned tank through True_SC_Fuel_Tank.func_consume_fuel, and reports its instantaneous power draw and data generation to True_SC_Power and True_SC_Data_Handling respectively. It uses True_SC_Body.location_COM as the moment-arm reference for torque calculation, so the thruster acts as the actuator-level bridge between software-commanded thrust and the truth-level rigid-body dynamics (orbit and attitude) and mass depletion of the spacecraft.

5.4.1 Inputs / Outputs

5.4.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumption, thruster_warm_up_power_consumed (optional), command_actuation_power_consumed (optional), instantaneous_data_generated_per_sample, chemical_thruster_noise, location, orientation, chemical_thruster_ISP, minimum_thrust, maximum_thrust, desired_fuel_tank (optional, defaults to tank index 1; 0 means "no tank consumption").
- True_SC_ADC **(same spacecraft):** attitude (quaternion), converted via quaternionToRotationMatrix to the body-to-inertial rotation used to transform the commanded thrust vector from body frame to inertial frame.
- True_SC_Body **(same spacecraft):** location_COM (center of mass), used with the thruster's location to compute the moment-arm vector for disturbance-torque calculation; also total_mass, used in the DeltaV-per-firing calculation.
- commanded_thrust**:** a magnitude *written into this object* by orbit-control software — see the write-back bullet below; the thruster reads this each cycle to decide what thrust to actually apply.
- flag_executive**:** also written into this object externally (same orbit-control software), gating whether the current commanded_thrust is actually honored this step.

5.4.1.2 Outputs

- **Internal / stored outputs:** thruster_state (idle/warming_up/ready/firing state machine), true_commanded_thrust (thrust with noise applied and clamped to [minimum_thrust, maximum

_thrust]), force_inertial, control_torque_CT, fuel_consumed_per_firing, total_fuel_consumed, pending_fire/pending_thrust, command_executed, warm_up_progress are computed in func_main_true_chemical_thruster and stored in obj.store.

- **Reported to True_SC_Power:** instantaneous_power_consumed (idle/warm-up/firing power states) registered/updated via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed.
- **Reported to True_SC_Data_Handling:** instantaneous_data_generated_per_sample registered/updated via func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated.
- **Writes into True_SC_orbit_propagation:** when firing, the thruster adds its force_inertial directly into mission.true_SC{i_SC}.true_SC_orbit_propagation.control_force (accumulated, not overwritten), i.e. this class is itself a writer into another class's state to apply thrust to the orbital dynamics.
- **Writes into True_SC_ADC:** when firing, control_torque_CT (the moment of the thrust force about the SC's center of mass) is added into mission.true_SC{i_SC}.true_SC_adc.disturbance_torque.
- **Writes into True_SC_Fuel_Tank:** when firing and desired_fuel_tank $\neq 0$, the thruster calls mission.true_SC{i_SC}.true_SC_fuel_tank{i_tank}.func_consume_fuel(obj.fuel_consumed_per_firing), depleting that tank's instantaneous_fuel_mass by the Tsiolkovsky-derived mass consumed this firing (see True_SC_Fuel_Tank below for the resulting mass-update chain into True_SC_Body).
- **Writes into Software_SC_Control_Orbit (software layer):** when firing, the thruster sets mission.true_SC{i_SC}.software_SC_control_orbit.flag_executive = 1 and directly accumulates the executed DeltaV into software_SC_control_orbit.data.total_DeltaV_executed, closing the loop so the orbit-control software can tell when its commanded maneuver has actually been achieved.
- **External class writes back into this class's properties: yes, explicitly.** Supporting_Functions/mission_specific/DART/func_command_DeltaV.m and func_update_sw_SC_ctrl_orbit_DART.m (software orbit-control code, invoked as part of software_SC_control_orbit) write mission.true_SC{i_SC}.true_SC_chemical_thruster{i_thruster}.commanded_thrust (the desired thrust magnitude) and .flag_executive (fire/don't-fire command) directly into each thruster object — this is the actuator-command path from software control down into the True hardware actuator.

5.4.2 Mathematical Equations

5.4.2.1 Thrust Force Vector (Body to Inertial Frame)

$$\vec{F}_{body} = T\,\hat{u}_{orientation}, \qquad \vec{F}_{inertial} = R^{T}_{BF\to IF}\,\vec{F}_{body} \tag{5.8}$$

Here T is the (noise-perturbed, saturation-limited) commanded thrust magnitude [N]; $\hat{u}_{orientation}$ is the unit thrust-direction vector of the nozzle in the spacecraft body frame; $R_{BF\to IF}$ is the rotation matrix built from the spacecraft attitude quaternion (inertial-to-body), whose transpose maps body-frame vectors into the inertial frame; $\vec{F}_{inertial}$ is the resulting thrust force vector in the inertial frame [N].

In the code, T corresponds to `obj.true_commanded_thrust`, computed as `thrust_to_use + obj.thruster_noise*(2*rand()-1)` and then clamped with `max(min(...,obj.maximum_thrust),obj.minimum_thrust)`. $\hat{u}_{orientation}$ is `obj.orientation`, and `force_body = obj.true_commanded_thrust * obj.orientation`. $R_{BF\to IF}$ is obtained from `quaternionToRotationMatrix(mission.true_SC{i_SC}.true_SC_adc.attitude)`, and `obj.force_inertial = (R_BF_IF.')*force_body'` implements the frame transform. The result is added into `mission.true_SC{i_SC}.true_SC_orbit_propagation.control_force`.

5.4.2.2 Disturbance Torque About Center of Mass

$$\vec{\tau}_{CT} = \vec{r} \times \vec{F}_{body}, \qquad \vec{r} = \vec{\ell}_{thruster} - \vec{r}_{COM} \tag{5.9}$$

Here $\vec{\ell}_{thruster}$ is the thruster mounting location in the body frame [m]; $\vec{r}_{COM}$ is the spacecraft center-of-mass location in the body frame [m]; $\vec{r}$ is the moment-arm vector from the center of mass to the thruster; $\vec{F}_{body}$ is the thrust force vector expressed in the body frame [N]; $\vec{\tau}_{CT}$ is the resulting disturbance torque [N·m].

In the code, $\vec{\ell}_{thruster}$ is `obj.location`, $\vec{r}_{COM}$ is `mission.true_SC{i_SC}.true_SC_body.location_COM`, and the moment arm is computed as `r = obj.location - ...location_COM`. The torque is `obj.control_torque_CT = cross(r, force_body)`, which is then added into `mission.true_SC{i_SC}.true_SC_adc.disturbance_torque`.

5.4.2.3 Propellant Mass Flow Rate (Rocket Equation Relation)

$$m_{consumed} = \frac{T\,\Delta t}{I_{sp}\,g_0} \tag{5.10}$$

Here T is the true (noise-perturbed, bounded) thrust magnitude [N]; Δt is the simulation time step over which the thruster fires [s]; I_{sp} is the thruster's specific impulse [s]; $g_0 = 9.80665$ m/s^2 is standard gravity; $m_{consumed}$ is the propellant mass consumed during that firing interval [kg]. This is the standard rocket-equation relation $\dot{m} = T/(I_{sp}g_0)$ integrated over one time step under the (first-order) assumption that thrust is constant across Δt.

In the code this is implemented directly as `obj.fuel_consumed_per_firing = (obj.true_commanded_thrust * dt) / (obj.isp * g0)`, with `dt = mission.true_time.time_step` and `g0 = 9.80665`. The running total is kept in `obj.total_fuel_consumed`, and the consumed mass is withdrawn from the tank via `mission.true_SC{i_SC}.true_SC_fuel_tank{i_tank}.func_consume_fuel(obj.fuel_consumed_per_firing)` when `obj.desired_fuel_tank` is nonzero.

5.4.2.4 Impulsive Delta-V Applied

$$\Delta\vec{V} = \frac{\|\vec{F}_{inertial}\|\,\Delta t}{m_{sc}}\,\frac{\vec{F}_{inertial}}{\|\vec{F}_{inertial}\|} = \frac{\vec{F}_{inertial}\,\Delta t}{m_{sc}} \tag{5.11}$$

Here $\vec{F}_{inertial}$ is the inertial-frame thrust force vector [N]; Δt is the time step [s]; m_{sc} is the current total spacecraft mass [kg]; $\Delta\vec{V}$ is the incremental velocity change delivered during this firing, expressed as impulse (force × time) divided by mass.

In the code, m_{sc} is `mission.true_SC{i_SC}.true_SC_body.total_mass`. The code computes `DeltaV_applied = (norm(obj.force_inertial)*dt/sc_mass) * (obj.force_inertial/norm(obj.force_inertial))`, which algebraically reduces to $\vec{F}_{inertial}\Delta t/m_{sc}$. This value is accumulated into `mission.true_SC{i_SC}.software_SC_control_orbit.total_DeltaV_executed`, which the software orbit-control logic uses to track progress toward `desired_control_DeltaV`.

5.4.3 CML 2–5 Suitability

5.4.3.1 What is included and correctly implemented

The class implements a physically sensible, first-order chemical thruster model appropriate for pre-project trade studies.

- **Separated thrust magnitude and direction with correct frame propagation.** It correctly separates commanded thrust magnitude (bounded by `minimum_thrust`/`maximum_thrust`, perturbed by a simple additive `thruster_noise` term) from thrust direction (a fixed body-frame unit vector, `orientation`), and correctly propagates the resulting force from the body frame to the inertial frame using the spacecraft's actual attitude quaternion via `quaternionToRotationMatrix`, rather than assuming a fixed inertial thrust direction.
- **Correct torque about the true center of mass.** The torque calculation uses a proper moment-arm cross product about the spacecraft's true (possibly time-varying) center of mass (`true_SC_body.location_COM`) rather than a hardcoded location.
- **Propellant consumption closes the loop with the fuel tank.** Propellant consumption is computed with the standard $\dot{m} = T/(I_{sp}g_0)$ relation and is actually withdrawn from a designated `True_SC_Fuel_Tank` object, closing the loop between thrust production and mass depletion; the class also supports assigning "no fuel tank" (`desired_fuel_tank == 0`) for idealized cases.
- **ΔV bookkeeping feeds software maneuver planning.** The impulsive ΔV bookkeeping feeds back into the software orbit-control object's executed-vs-desired ΔV tracking, which is a reasonable simplified way to couple truth-level thruster firing to software-level maneuver planning.
- **State machine, health flag, and time-history storage.** A basic warm-up/ready/firing state machine with distinct power draws per state, a health flag that disables firing, and a "pending fire" mechanism to avoid dropping a commanded burn across a state transition boundary are all present, along with storage of the key time histories (force, fuel consumed, torque, thruster state, warm-up progress) for post-run analysis. These are all appropriate levels of fidelity for CML 2–5 trade-space and point-design work.

5.4.3.2 What is not included / gaps

Several simplifications are present that would need revisiting for higher-fidelity or flight-representative work, though they are acceptable at CML 2–5.

- **Fixed I_{sp} with no propellant-state dependence.** Specific impulse (`isp`) is treated as a fixed constant with no dependence on propellant tank pressure, temperature, mixture ratio, or blowdown behavior as propellant depletes, and there is no bipropellant/mixture-ratio modeling despite the "chemical thruster" naming.
- **No sub-stepping of mass depletion within a time step.** The ΔV and mass-flow calculations both assume thrust and spacecraft mass are constant over the full time step `dt`, i.e., no sub-stepping or integration of mass depletion during a single firing interval.
- **No thrust-direction noise or alignment uncertainty.** Thrust direction noise is not modeled — only the scalar magnitude receives noise via `thruster_noise`, so mounting/alignment uncertainty and thrust-vector wander are not represented.

- **No thermal feedback.** The `temperature` property is initialized to 20 (Celsius) but is never updated by any method, so there is no thermal feedback on performance or health.
- **No minimum-impulse-bit, duty-cycle, or wear modeling; fixed timing constants.** There is no minimum-impulse-bit/pulsing model, duty-cycle limit, or thruster wear/degradation model beyond the static boolean `health` flag; warm-up time and the 300-second ready-state timeout are fixed constants rather than derived from any thermal or propellant-feed physics.
- **Torque zeroed outside firing state.** Torque is only computed and applied during the "firing" state and is explicitly zeroed at the end of each call, so there is no continuous or residual torque contribution outside an active firing pulse.
- **No plume impingement or multi-thruster coupling.** Finally, the class does not model plume impingement, nozzle exit-plane effects, or multi-thruster coupling (e.g., shared feed-line pressure drop across simultaneously firing thrusters on the same tank) — all reasonable omissions for this maturity level but worth flagging as gaps relative to flight-software-representative fidelity.

5.5 True_SC_Fuel_Tank

True_SC_Fuel_Tank models a single propellant tank as a simple depleting mass reservoir on board the spacecraft. Each instance is initialized with a maximum capacity, an initial fuel load, a physical location in the body frame, and nominal heater/electronics power and data-generation parameters; at run time its only state variable of consequence is the instantaneous remaining fuel mass, which is decremented whenever a thruster class (True_SC_Chemical_Thruster, True_SC_EP_Thruster, or True_SC_RCS_Thruster) calls its fuel-consumption method after a firing.

The class's principal role in the overall architecture is to serve as the authoritative propellant mass ledger that True_SC_Body reads when it recomputes total spacecraft mass, center of mass, and moment of inertia. Rather than pushing mass updates itself, True_SC_Fuel_Tank raises a dirty flag that tells the body class to pull the latest fuel mass on its next update cycle, decoupling the tank's bookkeeping from the body's inertial recomputation. It also reports its own housekeeping loads (heater power draw, telemetry data rate) into the spacecraft power and data-handling subsystems and logs its fuel-mass and power history to the mission store for later plotting and analysis.

5.5.1 Inputs / Outputs

5.5.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed, instantaneous_data_rate_generated, maximum_capacity, initial_fuel_mass, location.
- **No other True/Software class is read by** True_SC_Fuel_Tank **at runtime** apart from mission.storage/mission.true_SC{i_SC}.true_SC_power for bookkeeping (see Outputs). The tank's own state (instantaneous_fuel_mass) is depleted only via calls made *into* it by thruster classes, not by anything it reads.
- True_SC_Chemical_Thruster / True_SC_EP_Thruster / True_SC_RCS_Thruster **(same spacecraft, cross-class caller):** these actuator classes call obj.func_consume_fuel(fuel_mass_consumed) on the tank object they are configured to draw from (desired_fuel_tank index), directly decrementing instantaneous_fuel_mass and setting flag_update_SC_body = 1. This is effectively an external-class method call that mutates the tank's state, i.e. the thruster classes are consumers that command fuel depletion into this object.

5.5.1.2 Outputs

- **Internal / stored outputs:** instantaneous_fuel_mass, instantaneous_power_consumed are stored in obj.store each step via func_update_true_SC_fuel_tank_store.
- **Reported to** True_SC_Power**:** instantaneous_power_consumed (heaters/valves) registered/updated via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed.
- **Reported to** True_SC_Data_Handling**:** instantaneous_data_rate_generated registered via func_initialize_list_HW_data_generated.
- **Reported to** True_SC_Body**:** when fuel is consumed, the tank sets mission.true_SC{i_SC}.true_SC_body.flag_update_SC_body_total_mass_COM_MI = 1 (in func_main_true_SC_fuel_tank) rather than writing mass directly; True_SC_Body subsequently reads mission.true_SC{i_SC}.true_SC_fuel_tank{i_FT}.instantaneous_fuel_mass and .location itself to re-

compute aggregate mass, center of mass, and inertia whenever that flag is set — this is the mechanism by which thruster-driven fuel consumption ultimately changes the spacecraft's rigid-body mass properties.

- **External class writes back into this class's properties: yes.** Thruster classes (`True_SC_Chemical_Thruster`, `True_SC_EP_Thruster`, `True_SC_RCS_Thruster`) call the tank's own `func_consume_fuel` method, which decrements `instantaneous_fuel_mass` and sets `flag_update_SC_body`.

5.5.2 Mathematical Equations

5.5.2.1 Propellant Mass Depletion

$$m_{\text{fuel}}(t_{k+1}) = m_{\text{fuel}}(t_k) - \min\big(\Delta m_{\text{consumed}},\ m_{\text{fuel}}(t_k)\big) \tag{5.12}$$

Here $m_{\text{fuel}}(t_k)$ is the fuel mass remaining in the tank at the current time step; $\Delta m_{\text{consumed}}$ is the propellant mass a thruster requests to burn during that step (e.g., mass per firing, or mass flow rate integrated over the step, as computed by the calling thruster class); $m_{\text{fuel}}(t_{k+1})$ is the updated remaining fuel mass. The min(·) term enforces a floor so the tank cannot be driven to a negative mass.

This update is implemented in `func_consume_fuel(obj, fuel_mass_consumed)`. The method first checks `if fuel_mass_consumed > obj.instantaneous_fuel_mass` and, if so, clamps the request down to `obj.instantaneous_fuel_mass` (a commented-out `warning(...)` line shows that this saturation was originally meant to be flagged, but the warning is currently disabled). It then applies `obj.instantaneous_fuel_mass = obj.instantaneous_fuel_mass - fuel_mass_consumed` and sets `obj.flag_update_SC_body = 1` so that `True_SC_Body` will pull the new mass into its total-mass/center-of-mass/moment-of-inertia recomputation (`flag_update_SC_body_total_mass_COM_MI`) on the next main-loop pass. The tank itself performs no other physics: it does not compute a fuel fraction/percentage analogous to a battery's state of charge, does not compute its own center of mass or inertia contribution (those are handled downstream in `True_SC_Body.func_update_SC_body_mass`, where the propellant point mass is currently assigned a zero moment-of-inertia-over-mass tensor), and does not model tank pressure, temperature-dependent behavior, or ullage/slosh effects despite carrying a `temperature` property.

5.5.3 CML 2–5 Suitability

5.5.3.1 What is included and correctly implemented

The class correctly tracks a bulk propellant mass state and integrates it cleanly with the rest of the spacecraft model, which is an appropriate first-order pattern for point-design trade studies.

- **Non-negative mass floor.** It enforces a non-negative floor so the tank cannot report unphysical negative fuel.
- **Decoupled mass-properties hand-off.** It cleanly threads the tracked mass into the spacecraft's total mass/center-of-mass/inertia recomputation via a dirty-flag hand-off (`flag_update_SC_body` → `flag_update_SC_body_total_mass_COM_MI`) rather than tight coupling.
- **Body-frame location exposure.** It exposes the location of the tank in the body frame so downstream mass-properties calculations can use it.

- **Unified thruster interface.** It integrates with the multiple thruster types (`True_SC_Chemical_Thruster`, `True_SC_EP_Thruster`, `True_SC_RCS_Thruster`) through a single, simple `func_consume_fuel` interface.
- **Power/data budget integration.** It reports its heater power draw and telemetry data rate into the spacecraft power and data-handling bookkeeping so overall power/data budgets stay self-consistent.
- **Sufficient logging for trending.** Historical fuel mass and power consumption are logged to `obj.store` for trending, which is sufficient for the kind of top-level propellant-budget and delta-v margin assessments typical of CML 2–5 formulation work.

5.5.3.2 What is not included / gaps

Several omissions are present, though they are consistent with the medium-fidelity, non-flight-representative modeling expected at CML 2–5.

- **No first-class remaining-fuel fraction.** The tank has no notion of remaining-fuel fraction or percentage (unlike the analogous `state_of_charge`/`state_of_data_storage` metrics in `True_SC_Battery` and `True_SC_Onboard_Memory`), so any such margin metric must currently be recomputed ad hoc by calling code (e.g., dividing `instantaneous_fuel_mass` by `maximum_capacity` in plotting scripts) rather than being a first-class property.
- **No shape, pressure, or COM-shift modeling.** There is no shape model, tank pressure/temperature coupling, or center-of-mass shift as propellant depletes; the tank is treated as a fixed point location, and the moment-of-inertia contribution passed to `True_SC_Body` is hard-set to a zero tensor (`obj.mass.propellant{i_FT}.MI_over_m = zeros(3,3)`), with a commented-out block showing that a shape-model-based or point-mass-approximation inertia calculation was considered but never enabled.
- **No thermal or health modeling.** The `temperature` and `health` properties are initialized but never updated by any method in this file, so there is no thermal model, fault/leak model, or health-degradation logic despite the properties existing.
- **No rate limiting or reserve/shutoff logic.** There is also no depletion-rate limiting beyond the simple floor (no minimum-fuel/reserve threshold, low-fuel warning, or shutoff logic), and no handling of multiple propellant types or tank pressurization physics.
- **Adequate for CML 2–5, but limiting beyond it.** A depleting bulk-mass ledger with a non-negative floor is adequate for early trade-space and point-design propellant budgeting, but these omissions would need to be addressed before the class could support higher-fidelity CML work (e.g., detailed COM/MI as propellant sloshes or depletes, thermal/pressure effects, or flight-software-representative low-fuel fault management).

5.6 True_SC_Generic_Sensor

The True_SC_Generic_Sensor class is a generic, catch-all sensor model intended to represent any spacecraft sensor whose only relevant behavior, for the purposes of the simulation, is its power draw and the rate at which it generates data. Rather than modeling any particular sensing physics (e.g., optical, magnetic, or inertial measurement), the class simply reads an instantaneous power consumption and instantaneous data rate (either supplied directly by the user or looked up from a per-spacecraft-mode table in obj.data.instantaneous_power_consumed_per_SC_mode) and forwards these values to the spacecraft's power and data-handling bookkeeping classes (true_SC_power and true_SC_data_handling) each time the executive flag commands it to act.

Because it contains no sensor-specific physics, True_SC_Generic_Sensor functions as a lightweight placeholder or template: it can stand in for an unmodeled sensor in a spacecraft bus definition, or serve as a skeleton/starting point that a developer copies and extends when building a new, physically detailed sensor class. Its constructor and main-update method establish the standard pattern used throughout the True_Sensors_Actuators library (health/temperature initialization, executive-flag gating, storage array pre-allocation, and per-time-step store updates), while leaving the actual measurement-generation logic essentially empty.

5.6.1 Inputs / Outputs

5.6.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed, instantaneous_data_rate_generated, data (optional, may include instantaneous_power_consumed_per_SC_mode, a per-mode power lookup table).
- mission.true_SC{i_SC}.software_SC_executive **(same spacecraft):** this_sc_mode_value, flag_slew_sc_mode_exists — used, if obj.data.instantaneous_power_consumed_per_SC_mode is defined, to index the sensor's power draw by the spacecraft's current operating mode.
- mission.true_SC{i_SC}.software_SC_control_attitude **(same spacecraft):** flag_slew — combined with the executive's slew-mode-exists flag to override the mode-indexed power value specifically during a slew.
- **No other True-stack truth state (attitude, orbit, target, solar system) is read.** This class is intentionally generic/placeholder: it has no measurement model of its own and only models power/data bookkeeping as a function of SC mode.

5.6.1.2 Outputs

- **Internal / stored outputs:** flag_executive, instantaneous_data_rate_generated, instantaneous_power_consumed are stored in obj.store each step via func_update_true_SC_generic_sensor_store.
- **Reported to** True_SC_Power**:** instantaneous_power_consumed registered/updated via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed.
- **Reported to** True_SC_Data_Handling**:** instantaneous_data_rate_generated registered/updated via func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated.

- **No other class in the codebase was found referencing** `true_SC_generic_sensor` outside of its own class file and the `main_v3.m` update-loop call to `func_main_true_SC_generic_sensor`. It has no mission-specific consumers found in `Mission/*.m` or `Supporting_Functions/mission_specific/*`.
- No external class writes back into this class's properties.

5.6.2 Mathematical Equations

This class contains no genuine sensor physics or measurement equations. Its only computed quantity is a direct lookup/passthrough of instantaneous power consumption as a function of spacecraft mode (with a special-case override during slew), i.e. $P_{\text{inst}} =$ `data.instantaneous_power_consumed_per_SC_mode`(mode), which is a table lookup rather than a physical model, so no equation is presented here; the data-rate and power values are otherwise simply carried through from initialization or from the mode-indexed table into the spacecraft's power and data-handling accounting.

5.6.3 CML 2–5 Suitability

5.6.3.1 What is included and correctly implemented

The class correctly implements the standard MuSCAT hardware-object lifecycle expected at CML 2–5.

- **Standard constructor-time initialization.** Name, health, temperature, executive flag, and storage arrays are initialized at construction, sized to `mission.storage.num_storage_steps`.
- **Correct power/data registration.** The sensor's power draw and data generation are properly registered with the spacecraft's `true_SC_power` and `true_SC_data_handling` classes both at construction and at each executive-commanded time step.
- **Correct flag gating.** Behavior is correctly gated on `flag_executive` and `health`.
- **Mode-dependent power lookup.** A reasonable mode-dependent power lookup (including a slew-specific override) is provided for cases where power draw varies with spacecraft operating mode.
- **Suitable as a template/extension point.** As a template/extension point, it is well-suited to CML 2–3 trade studies where a sensor's presence needs to be represented in power and data budgets without committing to detailed sensor physics, and it gives a correctly wired skeleton for a developer to extend into a higher-fidelity custom sensor.

5.6.3.2 What is not included / gaps

The class provides no actual sensing physics.

- **No measurement or noise model.** There is no measurement model, no noise or error model, no field-of-view, geometry, or line-of-sight computation, and no relationship between the spacecraft state (attitude, position, environment) and any output "measurement" — `obj.data` is an arbitrary user-supplied struct with no defined sensing content.

- **No thermal or health-transition logic.** There is also no temperature dynamics (temperature is set once at construction and never updated) and no health-transition logic beyond a static on/off flag.
- **Placeholder only, not a usable sensor model.** Consequently, while it is appropriate as a bus-level power/data placeholder or as a starting template, it cannot by itself represent any sensor's actual measurement behavior at any CML level; a specific sensor class must be built on top of it (or modeled after it) before it can support trade studies that depend on sensor output quality, accuracy, or environmental sensitivity.

5.7 True_SC_IMU

True_SC_IMU models the true (ground-truth) behavior of an Inertial Measurement Unit — specifically a rate gyro — onboard the spacecraft. Its role in the simulation is to take the spacecraft's true angular velocity, as propagated by the truth attitude dynamics and exposed through True_SC_ADC (accessed here as mission.true_SC{i_SC}.true_SC_adc.angular_velocity), and corrupt it with sensor-realistic error sources to produce a simulated raw measurement vector. Depending on the configured mode, the class can pass the true rate through unmodified (Truth mode), add zero-mean Gaussian white noise (Simple mode), or add a more complete stochastic error model consisting of angle random walk (rate white noise), an initial constant bias, and a rate (bias) random walk with long-term stability bounding (Rate bias mode).

The class follows the same architectural pattern as other True_Sensors_Actuators classes: it is constructed once per hardware unit from an init_data struct and a mission object, precomputes discrete-time noise parameters (ARW/RRW standard deviations and a bias decay pole) from continuous-time power spectral density (PSD) inputs, and is stepped every attitude cycle by func_main_true_SC_imu, which enforces a measurement wait time, dispatches to the appropriate noise-mode subfunction, updates power/data-handling bookkeeping, and logs results to the store structure for post-processing.

5.7.1 Inputs / Outputs

5.7.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed, instantaneous_data_generated_per_sample, mode_true_SC_imu_selector (Truth/Simple/Rate bias), measurement_wait_time, measurement_noise, location, orientation, data (optional), rng_seed (optional), initial_bias (optional), arw_psd/rrw_psd (optional), bias_long_term_stab (optional).
- mission.true_time**:** time_step_attitude (used with measurement_wait_time to set sim_dt for ARW/RRW sigma computation), time_attitude (used to gate measurement cadence).
- True_SC_ADC **(same spacecraft):** angular_velocity — the true body angular rate, read directly in all three measurement modes (func_true_SC_imu_Truth, func_true_SC_imu_Simple, func_true_SC_imu_Bias) as the basis for the simulated gyro measurement.

5.7.1.2 Outputs

- **Internal / stored outputs:** measurement_vector (simulated angular-velocity measurement, with mode-dependent noise/bias added to the true rate) and rate_bias are computed each step and stored in obj.store, keyed to the attitude-rate storage index (k_storage_attitude).
- **Reported to** True_SC_Power**:** instantaneous_power_consumed registered/updated via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed_attitude.
- **Reported to** True_SC_Data_Handling**:** instantaneous_data_generated_per_sample registered/updated via func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated_attitude.

- **Read by `Software_SC_Estimate_Attitude` (same spacecraft):** this attitude-estimation software class, when using a Kalman-filter-type estimator, sets `mission.true_SC{i_SC}.true_SC_imu{i_HW}.flag_executive = 1` to request an IMU measurement each estimation cycle, and (through the sensor's public `measurement_vector`) consumes the IMU's simulated rate measurement as input to the attitude filter.
- **External class writes back into this class's properties:** only the generic executive-flag command pattern — `Software_SC_Estimate_Attitude.m` (KF branch) writes `flag_executive = 1` to trigger a measurement; no commanded value is written into the IMU object by any external class.

5.7.2 Mathematical Equations

5.7.2.1 Simple Mode: Rate Measurement with White Noise

$$\boldsymbol{\omega}_{meas} = \boldsymbol{\omega}_{true} + \sigma_{noise}\,\mathbf{n} \tag{5.13}$$

Here $\boldsymbol{\omega}_{meas}$ is the 1×3 simulated measured angular velocity vector [rad/s], $\boldsymbol{\omega}_{true}$ is the true spacecraft angular velocity [rad/s], σ_{noise} is the 1-sigma measurement noise standard deviation [rad/s], and $\mathbf{n}$ is a 1×3 vector of independent standard normal random samples.

This is implemented in `func_true_SC_imu_Simple` as `obj.measurement_vector = mission.true_SC{i_SC}.true_SC_adc.angular_velocity + obj.measurement_noise*randn(1,3)`, where `obj.measurement_noise` is σ_{noise} (set from `init_data.measurement_noise`) and `randn(1,3)` draws $\mathbf{n}$ from MATLAB's default (non-seeded) random stream.

5.7.2.2 Rate Bias Mode: Bias Random-Walk Propagation

$$b_k = (1-\alpha)\,b_{k-1} + \sigma_{rrw}\,\mathbf{n}_k \tag{5.14}$$

Here b_k is the current-step gyro rate bias vector [rad/s], b_{k-1} is the previous-step bias, α is a dimensionless discrete decay coefficient (the "angular frequency of the bias pole"), σ_{rrw} is the rate random walk (RRW) standard deviation per step [rad/s], and $\mathbf{n}_k$ is a 1×3 vector of standard normal samples drawn at step k.

This is implemented in `func_true_SC_imu_Bias` as `obj.rate_bias = (1 - obj.ang_freq_bias_pole)*obj.rate_bias_prev + obj.rrw_sigma*randn(obj.rand_stream,1,3)`, followed by `obj.rate_bias_prev = obj.rate_bias`. The decay term α (`obj.ang_freq_bias_pole`) is computed once in the constructor as $\alpha = \dfrac{S_{rrw}}{2\,\sigma_{stab}^2}\,\Delta t$ (`obj.rrw_psd/(2*(obj.bias_long_term_stab)^2)*obj.sim_dt`), where S_{rrw} is the RRW PSD height (`rrw_psd`), σ_{stab} is the bias long-term stability 1-sigma value (`bias_long_term_stab`), and Δt is the simulation attitude time step (`sim_dt`); this term provides a first-order approximation that bounds long-term bias growth, per the in-code comment. Random draws use the class's dedicated `rand_stream` (Mersenne Twister), which can be seeded via `obj.rng_seed` for repeatability.

5.7.2.3 Rate Bias Mode: Measured Rate with Noise and Bias

$$\boldsymbol{\omega}_{meas} = \boldsymbol{\omega}_{true} + \sigma_{arw}\,\mathbf{m} + b_k \tag{5.15}$$

Here $\boldsymbol{\omega}_{meas}$ is the measured angular velocity [rad/s], $\boldsymbol{\omega}_{true}$ is the true angular velocity from `True_SC_ADC`, σ_{arw} is the angle random walk (rate white noise) standard deviation [rad/s], $\mathbf{m}$ is an independent 1×3 standard normal draw, and b_k is the current bias from the random-walk propagation above.

This is implemented as the second statement of `func_true_SC_imu_Bias`: `obj.measurement_vector = mission.true_SC{i_SC}.true_SC_adc.angular_velocity + obj.arw_sigma*randn(obj.rand_stream,1,3) + obj.rate_bias`.

5.7.2.4 Discretization of ARW and RRW Standard Deviations from PSD

$$\sigma_{arw} = \sqrt{\frac{S_{arw}}{\Delta t}}, \qquad \sigma_{rrw} = \sqrt{S_{rrw}\,\Delta t} \tag{5.16}$$

Here σ_{arw} and σ_{rrw} are the discrete-time standard deviations used in the noise equations above [rad/s], S_{arw} is the angle random walk PSD height [rad^2/s], S_{rrw} is the rate random walk PSD height [(rad/s)^2/s], and Δt is the effective simulation time step used for the IMU update [s].

This is computed once in the constructor: `obj.arw_sigma = sqrt(obj.arw_psd/obj.sim_dt)` and `obj.rrw_sigma = sqrt(obj.rrw_psd*obj.sim_dt)`, where `obj.sim_dt = max([mission.true_time.time_step_attitude, obj.measurement_wait_time])` and `obj.arw_psd`/`obj.rrw_psd` default to representative gyro values (e.g. `(0.01*pi/180/sqrt(3600))^2`) but can be overridden via `init_data`.

5.7.3 CML 2–5 Suitability

5.7.3.1 What is included and correctly implemented

The class correctly implements a first-order, medium-fidelity gyro error model appropriate for point-design/trade-space work.

- **Graded fidelity options.** It provides a graded set of fidelity options (`Truth`, `Simple`, `Rate bias`) selectable per unit via `mode_true_SC_imu_selector`, allowing analysts to trade computational simplicity against realism.
- **Standard two-state gyro error decomposition.** The `Rate bias` mode implements the standard two-state gyro error decomposition used in early-mission analyses: angle random walk (white noise on the rate) plus a rate (bias) random walk with a decay term bounding long-term drift, both parameterized directly from PSD "height" values (a common gyro datasheet quantity) and converted to discrete-time sigmas using the actual simulation time step.
- **Seedable reproducibility.** Reproducibility is supported through a dedicated, seedable random stream (`rand_stream`, `rng_seed`).
- **Framework integration.** The class also integrates with the broader framework's power consumption and data-generation bookkeeping (`true_SC_power`, `true_SC_data_handling`) and with the storage/logging system, and it respects an executive-enable flag, health state, and a measurement wait time to throttle the sampling rate independent of the attitude integration step.
- **Fidelity matched to CML 2–5.** This level of fidelity is well matched to CML 2–5 needs, where the goal is representative sensor error statistics rather than flight-software-accurate implementation.

5.7.3.2 What is not included / gaps

Several gaps and simplifications are apparent, consistent with early-formulation fidelity but worth noting.

- **Simple mode not reproducible.** The `Simple` mode uses the global `randn` function (unseeded, not tied to `obj.rand_stream`), so it is not reproducible/seed-controlled the way `Rate bias` mode is, and it ignores any `initial_bias` entirely.
- **No misalignment, scale-factor, or temperature dependence.** There is no cross-axis misalignment, scale-factor error, non-orthogonality, or temperature-dependent error modeling despite a `temperature` property existing on the object (it is set once to 10 degrees C and never used in any measurement equation).
- **No quantization, saturation, or latency modeling.** There is no quantization/ADC resolution model, no sensor saturation or dynamic-range limiting, and no measurement latency/time-tagging jitter beyond the coarse `measurement_wait_time` gate.
- **Mounting geometry stored but unused.** The `location` and `orientation` properties (sensor placement and boresight in the body frame) are stored but never referenced in any of the three measurement functions, so single-axis or multi-IMU mounting geometry is not actually applied to the computed measurement vector.
- **Self-documented as incomplete.** The `Rate bias` case is explicitly flagged in a code comment as "TBD: include rate bias model," and the RRW decay/pole formulation is described in the code's own comments as only a first-order approximation.
- **No fault injection or degraded-health behavior.** No fault injection, health-dependent degraded-measurement behavior, or sensor failure modes are implemented beyond the binary on/off `health` gate. These omissions are acceptable for CML 2–5 trade studies but would need to be addressed before the model could be considered flight-software-representative.

5.8 True_SC_Magnetorquer

This class models a single magnetorquer actuator used for spacecraft attitude control. A magnetorquer produces a control torque by driving a commanded current through a coil to generate a magnetic dipole moment along a fixed body-frame axis (`orientation`); this dipole interacts with the local ambient magnetic field to produce a torque via $\vec{\tau} = \vec{m} \times \vec{B}$. Because the tool is written for Earth-orbiting spacecraft, `True_SC_Magnetorquer` also computes the ambient field itself, using a tilted geomagnetic dipole model evaluated at the spacecraft's position relative to Earth. The class enforces that it is only usable when the spacecraft's navigation target is Earth (`assert(strcmp(mission.true_target{obj.index_Earth_target}.name, 'Earth'), ...)`), since the dipole field model and its ECEF tilt/longitude parameters are Earth-specific.

Each simulation step, `func_main_true_SC_magnetorquer` first refreshes the local ambient field via `func_update_true_SC_magnetorquer_magnetic_field`, which transforms the spacecraft's position from the inertial (J2000) frame into the Earth-fixed (ECEF) frame, evaluates the tilted-dipole field there, and rotates the result back through J2000 into the spacecraft body frame using the target and ADC rotation matrices. If the magnetorquer is commanded (`flag_executive` and nonzero `commanded_dipole_moment`), the class saturates the command at `maximum_dipole_moment`, adds a uniformly distributed noise term (`moment_noise`), forms the dipole moment vector along `orientation`, and crosses it with the body-frame field to get `actual_torque`, which is added directly into `true_SC_adc.control_torque`. Power consumption is obtained by interpolating a lookup table of dipole moment vs. power (`power_consumed_dipole_moment_array`) and is reported to `True_SC_Power` via `func_update_instantaneous_power_consumed_attitude`; a fixed per-sample data volume is likewise reported to `True_SC_Data_Handling`. Health, storage, and command-flag reset logic mirror the pattern used by other actuator classes in `True_Sensors_Actuators`.

5.8.1 Inputs / Outputs

5.8.1.1 Inputs / Dependencies

- `init_data` **(constructor-time):** `location`, `orientation` (normalized dipole axis), `maximum_dipole_moment`, `moment_noise`, optional `mass`, `power_consumed_dipole_moment_array` (power-vs-dipole-moment lookup table), `instantaneous_data_generated_per_sample`, and optional tilted-dipole Earth field parameters `dipole_field_constant`, `dipole_tilt_angle`, `dipole_tilt_longitude`.

- `True_SC_Orbit_Propagation` **(same spacecraft):** reads `index_relative_target` at construction to identify the Earth target index, and reads `position_relative_target` every step to compute the SC's position relative to Earth for the magnetic field model. The constructor asserts the navigation target is named `'Earth'`.

- `mission.true_target{obj.index_Earth_target}` **(mission-level):** reads `rotation_matrix` (Earth body-fixed ECEF $\leftrightarrow$ J2000) each step to rotate the tilted-dipole field between frames.

- `True_SC_ADC` **(same spacecraft):** reads `rotation_matrix` every step to rotate the inertial-frame magnetic field into the SC body frame.

- `Software_SC_Control_Attitude` **(same spacecraft):** the hooks `commanded_dipole_moment` and `flag_executive` exist for this class to be commanded, analogous to reaction wheels/thrusters, but no mission-specific attitude-control function in the codebase was found actually writing to them — the one mission using this class (`Mission_NISAR_Magnetorquer.m`)

runs attitude control in "NISAR Oracle" mode, which moves `true_SC_adc.attitude` directly and never populates the magnetorquer's commanded moment.

5.8.1.2 Outputs

- **Internal / stored outputs:** `obj.store.commanded_dipole_moment`, `obj.store.actual_torque`, `obj.store.magnetic_field_body`, `obj.store.saturated`.
- **Reported to `True_SC_ADC` (same spacecraft):** `actual_torque` (dipole moment × magnetic field) is added directly into `mission.true_SC{i_SC}.true_SC_adc.control_torque` every attitude step.
- **Reported to `True_SC_Power` (same spacecraft):** `instantaneous_power_consumption/instantaneous_power_consumed` registered/updated via `func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed_attitude`.
- **Reported to `True_SC_Data_Handling` (same spacecraft):** `instantaneous_data_generated_per_sample` registered/updated via `func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated_attitude`.
- External write-back: the write-back interface (`commanded_dipole_moment`, `flag_executive`) exists on the class but is currently unused/unwired in the shipped mission configurations, unlike the analogous (and actually wired) write-back path for `True_SC_Reaction_Wheel` and `True_SC_RCS_Thruster`.

5.8.2 Mathematical Equations

5.8.2.1 Tilted-Dipole Ambient Magnetic Field

$$\vec{B}_{ECEF} = \frac{k_{dip}}{r^3}\left(3\,(\hat{d}\cdot\hat{r})\,\hat{r} - \hat{d}\right) \tag{5.17}$$

where k_{dip} is the dipole field constant $\mu_0 M_{Earth}/(4\pi)$ [T m^3], r is the spacecraft distance from Earth's center [m], $\hat{r}$ is the unit position vector in the Earth-fixed (ECEF) frame, and $\hat{d}$ is the unit vector along the tilted magnetic dipole axis, itself defined by a tilt angle from Earth's rotation axis and an east longitude.

This is computed in `func_update_true_SC_magnetorquer_magnetic_field`, where `obj.dipole_field_constant` is k_{dip} (default 7.94×10^{15} T m^3), `r_norm` and `r_hat` are r and $\hat{r}$, and `obj.dipole_axis_ECEF` (built from `obj.dipole_tilt_angle` and `obj.dipole_tilt_longitude`, defaults 169.2° and −72.68°) is $\hat{d}$. The result `B_ECEF` is then rotated through the inertial frame into the body frame via `target.rotation_matrix` and `true_SC_adc.rotation_matrix` to produce `obj.magnetic_field_body`.

5.8.2.2 Commanded Dipole Moment with Noise and Saturation

$$m_{true} = \mathrm{sat}(m_{cmd},\, m_{max}) + n, \qquad n \sim \mathcal{U}(-\delta_m,\, \delta_m) \tag{5.18}$$

where m_{cmd} is the commanded scalar dipole moment magnitude, m_{max} is the maximum achievable dipole moment, sat$(\cdot)$ clamps to $\pm m_{max}$, δ_m is the moment noise amplitude, and n is a uniformly distributed random noise draw.

In the code, `obj.commanded_dipole_moment` is clamped against `obj.maximum_dipole_moment` (setting `obj.saturated`), `current_moment_noise = obj.moment_noise * (2*rand() - 1)` implements n, and the sum is stored in `obj.true_commanded_dipole_moment`, i.e. m_{true}.

5.8.2.3 Dipole Moment Vector and Control Torque

$$\vec{m} = \hat{u}\, m_{true}, \qquad \vec{\tau} = \vec{m} \times \vec{B}_{body} \tag{5.19}$$

where $\hat{u}$ is the unit orientation axis of the torquer in the body frame, m_{true} is the noisy, saturated commanded moment magnitude, $\vec{B}_{body}$ is the ambient field in the body frame, and $\vec{\tau}$ is the resulting control torque.

This maps directly to `obj.actual_dipole_moment_vector = obj.orientation * obj.true_commanded_dipole_moment` for $\vec{m}$, and `obj.actual_torque = cross(obj.actual_dipole_moment_vector, obj.magnetic_field_body)` for $\vec{\tau}$, which is then added into `true_SC_adc.control_torque`.

5.8.2.4 Power Consumption from Lookup Table Interpolation

$$P = f_{interp}\big(|m_{true}|;\, \{m_i\},\, \{P_i\}\big) \tag{5.20}$$

where $\{m_i\}$ and $\{P_i\}$ are the tabulated dipole-moment/power pairs, $|m_{true}|$ is the magnitude of the true commanded dipole moment, and f_{interp} is piecewise-linear interpolation, with clamping to the endpoint power values outside the tabulated moment range.

This corresponds to `obj.power_consumed_dipole_moment_array` providing $\{P_i\}$ (row 1) and $\{m_i\}$ (row 2, sorted before use), `abs_moment = abs(obj.true_commanded_dipole_moment)`, and `interp1(dipole_moments, power_consumptions, abs_moment, 'linear')` computing P, stored in `obj.instantaneous_power_consumption` / `obj.instantaneous_power_consumed`. There is no physics-based I^2R or coil-current model; power is purely table-driven.

5.8.3 CML 2–5 Suitability

5.8.3.1 What is included and correctly implemented

The class implements the core torque physics needed for trade-space attitude-control studies.

- **Correct torque computation.** Torque is correctly computed as the cross product of the commanded dipole moment vector and the local magnetic field, $\vec{\tau} = \vec{m} \times \vec{B}$, and this torque is fed into `true_SC_adc.control_torque` for use by the attitude dynamics.
- **Saturation and noise modeling.** Saturation at a configurable `maximum_dipole_moment` and randomized moment noise are both present, giving first-order actuator imperfection modeling appropriate to this maturity level.
- **Self-consistent ambient field model.** Critically, and unlike many trade-study-level magnetorquer models, the ambient magnetic field is not left as an external stub — the class self-consistently computes Earth's field using a tilted-dipole model (with configurable tilt angle, tilt longitude, and dipole strength defaulting to standard first-order geomagnetic values) and correctly threads the frame transformations from ECEF to J2000 to the spacecraft body frame using the existing target and ADC rotation matrices.
- **Power/data integration.** Power consumption is handled via a configurable lookup table interpolated against commanded moment, integrating cleanly with `True_SC_Power`, and a fixed data-volume-per-sample value integrates with `True_SC_Data_Handling`.
- **Standard hardware-object bookkeeping.** Health-off behavior, command-flag reset each cycle, and storage of key time histories (`commanded_dipole_moment`, `actual_torque`, `magnetic_field_body`, `saturated`) are all implemented, matching the pattern of sibling actuator/sensor classes.

5.8.3.2 What is not included / gaps

None of the following gaps are expected at CML 2–5, where a self-consistent first-order dipole field plus a simple $\vec{m} \times \vec{B}$ torque and table-driven power (as implemented here) is standard and sufficient fidelity for trade studies.

- **First-order dipole field only, Earth-hard-coded.** The ambient field model is a first-order tilted dipole only — it omits higher-order (quadrupole and beyond) terms, secular variation, and any real-world magnetic anomaly maps (e.g. IGRF), and it is hard-coded to Earth via the `assert` on the navigation target, so the actuator cannot be used as-is for non-Earth-orbiting missions or bodies with different (or negligible) intrinsic fields.
- **No electrical/thermal dynamics.** There is no model of coil inductance/electrical dynamics, temperature-dependent resistance, or hysteresis in the dipole response; the commanded moment is applied instantaneously with no time constant, and power is a static lookup rather than derived from I^2R or a coil model.
- **Simplistic noise model, no cross-coupling or degradation.** The noise model is a single uniform random draw per step with no first-order Markov/correlation structure, and there is no model of torquer degradation, or interaction/cross-coupling between multiple magnetorquers' fields.
- **Main caveat is the Earth-only restriction.** The main caveat for future users is the Earth-only restriction if the tool is ever extended to other planetary environments.

5.9 True_SC_Onboard_Computer

The True_SC_Onboard_Computer class models a single onboard computer (OBC) unit as one of possibly several hardware elements on a spacecraft. It tracks basic operational state — whether the unit is switched on (health), whether the spacecraft executive has commanded it to operate (flag_executive), its temperature, and its processor utilization — and reports two externally visible quantities to the rest of the spacecraft bus: an instantaneous power draw (in Watts) and an instantaneous generated data rate (in kbps, or alternatively data volume per sample). These values are registered with the spacecraft's True_SC_Power and True_SC_Data_Handling classes at construction time and updated every simulation step so that the OBC's contribution to the spacecraft-level power and data budgets is properly accounted for.

Functionally, the class is a bookkeeping/passthrough model rather than a physics model: when the executive flag is set and the unit is healthy, the OBC reports a fixed, pre-specified power consumption and data generation rate to the power and data-handling subsystems; otherwise it reports nothing. The processor_utilization property is tracked and logged but, per the code's own comment, is not used to modulate the power or data outputs — the computer's contribution is constant regardless of CPU load. Temperature is initialized once (15 °C) and never subsequently updated within this class.

5.9.1 Inputs / Outputs

5.9.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed [W], instantaneous_data_rate_generated [kbps], optional processor_utilization [%] (defaults to 10), optional name, optional data struct.
- mission.storage **(mission-level):** reads num_storage_steps, k_storage, k_storage_attitude, flag_store_this_time_step to size and index the store.
- No other True/Software class truth or state is read by this class — it has no cross-class input dependency beyond its own init_data and the generic storage bookkeeping. flag_executive is set once at the end of mission setup scripts (e.g. Mission_DART.m, Mission_NISAR.m) and the class then runs continuously.

5.9.1.2 Outputs

- **Internal / stored outputs:** obj.store.flag_executive, obj.store.instantaneous_data_rate_generated, obj.store.instantaneous_power_consumed, obj.store.processor_utilization.
- **Reported to** True_SC_Power **(same spacecraft):** func_initialize_list_HW_energy_consumed (constructor) and func_update_instantaneous_power_consumed (main loop) read instantaneous_power_consumed to include the computer's constant power draw in the spacecraft power budget.
- **Reported to** True_SC_Data_Handling **(same spacecraft):** func_initialize_list_HW_data_generated (constructor) and func_update_instantaneous_data_generated (main loop) read instantaneous_data_rate_generated to add the computer's housekeeping data rate to the spacecraft data budget.

- No external class writes back into this class's own properties, aside from the one-time `flag_executive = 1` activation performed directly by Mission setup scripts (not by another True/Software class) immediately after construction.

5.9.2 Mathematical Equations

This class contains no genuinely computed physics or dynamics. It is a constant-value passthrough: at each simulation step, if the onboard computer is commanded on (`flag_executive` = 1) and healthy (`health` = 1), it simply forwards its pre-specified, unchanging `instantaneous_power_consumed` and `instantaneous_data_rate_generated` values to the spacecraft power and data-handling classes; no equation relates these outputs to processor utilization, temperature, or any other state variable, so there is no meaningful equation to document here.

5.9.3 CML 2–5 Suitability

5.9.3.1 What is included and correctly implemented

The class provides a reasonable first-order representation of an onboard computer for trade-space studies at CML 2–5.

- **Configurable power/data budget integration.** It correctly exposes configurable, user-specified instantaneous power consumption and data generation rate (or data-per-sample, via the `instantaneous_data_type` flag) that feed directly into the spacecraft-level power (`True_SC_Power`) and data-handling (`True_SC_Data_Handling`) budget classes through dedicated initialization and update calls, which is appropriate for a point-design-level resource-budgeting tool.

- **Health and executive gating.** It supports a simple on/off `health` state and a separate `flag_executive` command gate, so that the computer's contribution to power and data budgets can be turned on or off during a simulation (e.g., to represent commanded shutdown or a failed unit) without requiring detailed fault modeling.

- **Processor utilization property with default.** A `processor_utilization` property with a sensible default (10%) is included and logged in the `store` structure, giving downstream analysis access to a utilization time history even though it is not dynamically driven.

- **Clean storage/logging integration.** The class also integrates cleanly with the mission's storage/logging framework, recording `flag_executive`, `instantaneous_data_rate_generated`, `instantaneous_power_consumed`, and `processor_utilization` at each storage step, which is sufficient fidelity for budget trending and margin analysis at this maturity level.

5.9.3.2 What is not included / gaps

Several aspects fall short of representative fidelity, though this is expected and acceptable at CML 2–5.

- **No utilization-to-output coupling.** Processor utilization has no effect on power draw or data generation — the code comment explicitly states operation occurs "regardless of processor utilization," so there is no coupling between workload and power/thermal/data outputs, which would be needed for any duty-cycle- or workload-dependent analysis.

- **No thermal model.** Temperature is initialized to a fixed 15 °C in the constructor but is never updated anywhere in the class, so there is effectively no thermal model despite the property's presence.
- **Binary health model only.** The health model is binary (on/off) only, with no intermediate degraded states, radiation-induced upsets, restart/reboot behavior, or processing-throughput limits (e.g., no representation of instruction throughput, memory, or task queuing).
- **No fault-propagation modeling.** There is also no model of how the computer's own faults could propagate (e.g., corrupting data generation or triggering safe mode) beyond the simple on/off gating already described. These omissions are consistent with a medium-fidelity, budget-bookkeeping representation appropriate for pre-project formulation trade studies, not a flight-software-representative OBC model.

5.10 True_SC_Onboard_Clock

The True_SC_Onboard_Clock class provides a truth-model representation of a spacecraft's onboard clock within the MuSCAT simulation. Rather than propagating any independent clock dynamics, the class periodically samples the mission's true simulation time (mission.true_time.time and mission.true_time.date) and perturbs that sample with a bounded random offset, producing a "measured" time and date that other onboard software or subsystem classes can read via the object's measurement_vector and measurement_time properties. A wait-time gate (measurement_wait_time) controls how frequently a new clock reading is taken, mimicking the discrete update cadence of a real onboard timing peripheral rather than continuous time-tagging.

Beyond its role as a time source, the class follows the same boilerplate pattern used by other True_Sensors_Actuators hardware models in MuSCAT: it tracks a health flag, a nominal temperature, and an executive flag that gates whether the sensor is commanded to act in a given time step, and it forwards its instantaneous power consumption and data-generation rate to the spacecraft's power and data-handling classes. All computed quantities are logged into a store structure once per storage step for later post-processing and plotting, consistent with the rest of the truth-side sensor/actuator framework.

5.10.1 Inputs / Outputs

5.10.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed [W], instantaneous_data_rate_generated [kbps], mode_true_SC_onboard_clock_selector (only 'Simple' implemented), measurement_wait_time [sec], optional measurement_noise [sec] (defaults to 0), optional name.
- mission.true_time **(mission-level):** reads time and date at construction and every main-loop update (when measurement_wait_time has elapsed) to form measurement_vector = [time, date] with injected clock noise.

5.10.1.2 Outputs

- **Internal / stored outputs:** obj.store.measurement_vector, obj.store.measurement_time, obj.store.measurement_noise.
- **Reported to** Software_SC_Executive **(same spacecraft):** the executive sets mission.true_SC{i_SC}.true_SC_onboard_clock{i_HW}.flag_executive = 1 to force a measurement, then reads measurement_vector(1) and measurement_vector(2) to populate its own obj.time and obj.date — the spacecraft's onboard notion of time, as opposed to mission.true_time, which is ground truth.
- **Reported to** True_SC_Power **(same spacecraft):** instantaneous_power_consumed read via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed.
- **Reported to** True_SC_Data_Handling **(same spacecraft):** instantaneous_data_rate_generated read via func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated.

- No external class writes measurement data into this class; the only external write is `Software_SC_Executive` setting `flag_executive = 1` to trigger a clock reading, which the clock itself resets to 0 at the end of its own main function.

5.10.2 Mathematical Equations

5.10.2.1 Measurement Noise Draw

$$n_k = \sigma_{\text{noise}} \cdot 2 \cdot (u_k - 0.5)\,, \qquad u_k \sim \mathcal{U}(0,1) \tag{5.21}$$

Here σ_{noise} is the configured clock measurement noise amplitude (in seconds), u_k is a uniformly distributed random draw on $[0,1]$ taken at update step k, and n_k is the resulting noise sample applied to that step's clock reading. Note that although the code comment labels σ_{noise} as a "1-sigma standard deviation," the actual construction produces a value uniformly distributed on $[-\sigma_{\text{noise}}, +\sigma_{\text{noise}}]$, not a Gaussian-distributed error.

This corresponds directly to the line `this_measurement_noise = obj.measurement_noise*2*(rand-0.5)`, computed both in the constructor and in `func_main_true_SC_onboard_clock`, using the property `obj.measurement_noise` set from `init_data.measurement_noise` (default 0 if not supplied).

5.10.2.2 Clock Measurement Vector

$$\begin{bmatrix} t_{\text{meas},k} \\ d_{\text{meas},k} \end{bmatrix} = \begin{bmatrix} t_{\text{true},k} + n_k \\ d_{\text{true},k} + n_k \end{bmatrix} \tag{5.22}$$

$t_{\text{true},k}$ and $d_{\text{true},k}$ are the true mission time and date at step k, n_k is the same noise sample defined above (identically applied to both time and date), and $t_{\text{meas},k}$, $d_{\text{meas},k}$ are the resulting clock-reported time and date.

This maps to `obj.measurement_vector = [(mission.true_time.time + this_measurement_noise) (mission.true_time.date + this_measurement_noise)]`, with the first element also copied into `obj.measurement_time = mission.true_time.time` (stored without the noise term).

5.10.2.3 Measurement Update Trigger

$$\text{take new measurement if} \quad t_{\text{true}} - t_{\text{meas,prev}} \geq \Delta t_{\text{wait}} \tag{5.23}$$

t_{true} is the current true mission time, $t_{\text{meas,prev}}$ is the time stored from the previous clock reading (`obj.measurement_time`), and Δt_{wait} is the configured minimum inter-measurement interval.

This corresponds to the conditional `if (mission.true_time.time - obj.measurement_time) >= obj.measurement_wait_time` in `func_main_true_SC_onboard_clock`, using `obj.measurement_wait_time` set from `init_data.measurement_wait_time`. Only when this condition holds does the `'Simple'` case of `obj.mode_true_SC_onboard_clock_selector` redraw the noise and update `measurement_vector`/`measurement_time`; any other mode string causes `error('Clock mode not defined!')`.

There is no equation governing clock drift rate, oscillator frequency offset, or accumulated timing error over mission duration in this file — each triggered update independently re-samples true mission time plus a fresh, memoryless noise draw, rather than integrating a bias/drift state forward in time.

5.10.3 CML 2–5 Suitability

5.10.3.1 What is included and correctly implemented

The class implements a functioning, minimal clock-measurement abstraction that is adequate for CML 2–5 trade-space work.

- **Bounded, gated clock-error measurement.** It produces a bounded, randomly perturbed time/date reading (`measurement_vector`, `measurement_time`) derived from true mission time, gated by a configurable sampling cadence (`measurement_wait_time`).
- **Reasonable stand-in for point-design studies.** For point-design studies where the analyst mainly needs to inject a representative clock knowledge error of a specified magnitude into downstream navigation/timing calculations, this bounded-noise approach is a reasonable first-order stand-in.
- **Standard MuSCAT hardware-object pattern.** The class also correctly follows the standard MuSCAT hardware-object pattern: health/executive flags, constructor initialization from an `init_data` struct with sensible defaults (e.g., `measurement_noise` defaults to 0 if omitted), registration of instantaneous power consumption and data generation with the spacecraft's power (`true_SC_power`) and data-handling (`true_SC_data_handling`) classes, and consistent per-step storage logging via `func_update_true_SC_onboard_clock_store` gated on `mission.storage.flag_store_this_time_step`.
- **Properly wired resource budgeting.** This resource-budget integration (power/data) is properly wired and self-consistent within the file.

5.10.3.2 What is not included / gaps

There is no actual clock drift, bias-growth, or oscillator frequency-offset model.

- **No persistent/accumulating error.** Each update independently redraws an i.i.d. noise sample and re-anchors to true mission time, so the clock error does not accumulate or persist across time steps the way a real onboard oscillator's timing error would.
- **Cannot represent time-correlated drift.** This means the class cannot represent time-correlated clock behavior (e.g., growing divergence between onboard and true time over long baselines without ground contact), which may be a relevant gap even at CML 2–5 if clock drift over a comm outage is part of the trade.
- **Only one mode implemented.** Only a single mode, `'Simple'`, is implemented; the `mode_true_SC_onboard_clock_selector` switch has no other cases, so there is no higher- or lower-fidelity alternative clock model available, and selecting any unimplemented mode string throws a hard error rather than degrading gracefully.
- **No temperature coupling.** Finally, the `temperature` property is initialized (10 °C) but is never read, updated, or coupled to the clock's timing error in this file, so any implication of temperature-dependent clock performance is not actually modeled here.

5.11 True_SC_Onboard_Memory

True_SC_Onboard_Memory models a single onboard memory or mass-storage unit (e.g., a solid-state recorder) as a handle object with a fixed maximum capacity, an instantaneous stored-data level, and a derived percent-full metric. Each instance is constructed from an init_data struct supplying its rated maximum_capacity (in kb), its instantaneous power draw, and its instantaneous data-generation rate (or, alternatively, a per-sample data-generation quantity, selected via the instantaneous_data_type flag to avoid repeated isprop() calls at every timestep). At construction the object registers itself with the spacecraft's True_SC_Power object (via func_initialize_list_HW_energy_consumed) and with True_SC_Data_Handling (via func_initialize_list_HW_data_generated), so that those higher-level classes are aware this memory exists as a power load and as a data sink/source.

The class itself does not implement the fill/drain bookkeeping of the memory contents; that sequential fill/drain algorithm across multiple memories and downlink paths is owned by True_SC_Data_Handling, which writes back into this object's instantaneous_capacity property each timestep. What True_SC_Onboard_Memory itself computes, in its func_main_true_SC_onboard_memory method, is (1) the resulting state-of-data-storage percentage from whatever instantaneous_capacity value True_SC_Data_Handling has left it with, (2) an optional mode-dependent instantaneous power draw, and (3) a sanity check that the stored data never goes negative. It also maintains simple bookkeeping state (name, health flag, temperature) and a store struct for time-history logging of capacity and percent-full, in the same pattern used by the other True_Sensors_Actuators classes.

5.11.1 Inputs / Outputs

5.11.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumed [W], instantaneous_data_rate_generated [kbps], maximum_capacity [kb], optional name, optional data struct (may contain instantaneous_power_consumed_per_SC_mode).
- mission.true_SC{i_SC}.software_SC_executive **(same spacecraft):** reads this_sc_mode_value to index into obj.data.instantaneous_power_consumed_per_SC_mode and determine mode-dependent power draw.
- mission.true_SC{i_SC}.software_SC_control_attitude **(same spacecraft):** reads flag_slew to override the power lookup with a dedicated "Slew" mode's power value when a slew is in progress.
- True_SC_Data_Handling **(same spacecraft):** the single largest cross-class dependency — True_SC_Data_Handling directly writes into obj.instantaneous_capacity (see Outputs below) rather than this class computing its own fill level from internal logic.

5.11.1.2 Outputs

- **Internal / stored outputs:** obj.store.instantaneous_capacity, obj.store.state_of_data_storage, obj.store.maximum_capacity. state_of_data_storage is recomputed every step as 100× instantaneous_capacity / maximum_capacity.
- **Reported to** True_SC_Power **(same spacecraft):** instantaneous_power_consumed read via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed.

- **Reported to `True_SC_Data_Handling` (same spacecraft):** the memory's own housekeeping data rate is read via `func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated`; separately, `maximum_capacity` and `state_of_data_storage/instantaneous_capacity` are read by plotting and mission-specific aggregation code (e.g. `func_update_mean_state_of_data_storage.m`).
- **External write-back confirmed:** `True_SC_Data_Handling` directly reads and overwrites `mission.true_SC{i_SC}.true_SC_onboard_memory{i_memory}.instantaneous_capacity` in multiple places — when distributing newly generated instrument/sensor data into memory, when removing data during downlink/TX, and when enforcing capacity limits (clamping at `maximum_capacity` or at 0). This class's own main function only recomputes `state_of_data_storage` from whatever `instantaneous_capacity` was last set to by `True_SC_Data_Handling`.

5.11.2 Mathematical Equations

5.11.2.1 State of Data Storage (Percent Full)

$$\text{SoDS} = 100 \times \frac{C_{\text{inst}}}{C_{\max}} \tag{5.24}$$

SoDS is the state of data storage, expressed as a percentage of capacity used; C_{inst} is the instantaneous amount of data currently stored in the memory, in kilobits (kb); $C_{\max}$ is the fixed maximum data storage capacity of the memory unit, in kb.

This is computed as `obj.state_of_data_storage = 100*obj.instantaneous_capacity/obj.maximum_capacity;`, both at initialization in the constructor (with `instantaneous_capacity` set to 0) and every timestep inside `func_main_true_SC_onboard_memory`. Note that `instantaneous_capacity` itself is not solved for here: it is written into this object externally by `True_SC_Data_Handling`'s fill/drain logic before this line executes, so this equation is purely a derived percentage, not a state propagation. The class also guards against an invalid state by throwing an error (`error('Data in memory is negative!')`) if `obj.instantaneous_capacity < 0`, but this is a runtime check rather than a computed quantity.

5.11.2.2 Mode-Dependent Instantaneous Power Consumption

$$P_{\text{inst}} = \begin{cases} P_{\text{mode}}(m_{\text{slew}}), & \text{if a Slew mode exists and the spacecraft is slewing} \\ P_{\text{mode}}(m), & \text{if a per-mode power table is defined} \\ P_{\text{inst}}, & \text{otherwise (unchanged, set at construction)} \end{cases} \tag{5.25}$$

P_{inst} is the instantaneous power consumed by the memory unit, in Watts; $P_{\text{mode}}(\cdot)$ is a lookup table of power draw indexed by spacecraft operating mode, one entry per mode; m is the current spacecraft mode value; m_{slew} is the mode index corresponding specifically to a "Slew" mode, when one is defined and active.

If the optional field `obj.data.instantaneous_power_consumed_per_SC_mode` exists, `obj.instantaneous_power_consumed` is set by indexing that array with `mission.true_SC{i_SC}.software_SC_executive.this_sc_mode_value`. If the executive additionally reports `flag_slew_sc_mode_exists == 1` and the attitude control software reports `flag_slew == 1`, the power value is overridden using the mode index returned by `func_find_this_sc_mode_value(...,'Slew')` instead. If the optional per-mode table is absent, `instantaneous_power_consumed` simply retains the constant value supplied in `init_data` at construction. The resulting value is then reported outward to `True_SC_Power` via `func_update_instantaneous_power_consumed`, and the

memory's data-generation value is likewise reported to `True_SC_Data_Handling` via `func_update_instantaneous_data_generated`; neither of those downstream computations is performed inside this file.

5.11.3 CML 2–5 Suitability

5.11.3.1 What is included and correctly implemented

The class provides exactly the information a point-design trade study needs about an onboard memory.

- **Core capacity/state bookkeeping.** It provides a rated maximum capacity, a normalized percent-full state (SoDS) that is trivial to plot and threshold against margin requirements, a health flag, and a nominal power draw that can be either a constant or made mode-dependent (including a special case for slewing) without requiring any change to how the memory interacts with the rest of the spacecraft model.
- **Negative-capacity sanity check.** The negative-capacity guard (`error('Data in memory is negative!')`) is a useful first-order sanity check that catches bookkeeping errors elsewhere in the simulation (e.g., in `True_SC_Data_Handling`'s fill/drain logic) before they silently propagate into a nonphysical state.
- **Correct power/data endpoint registration.** The constructor's registration calls into `True_SC_Power` and `True_SC_Data_Handling` correctly establish this memory as both a power load and a data-handling endpoint, consistent with how other hardware objects in `True_Sensors_Actuators` integrate with the spacecraft-level bus/power/data managers.
- **Standard time-history logging.** The time-history logging pattern (`store.instantaneous_capacity`, `store.state_of_data_storage`), gated on `mission.storage.flag_store_this_time_step`, matches the standard storage convention used throughout the tool and is sufficient for post-run analysis of memory margin over a mission timeline.
- **Appropriate fidelity for CML 2–5.** For CML 2–5 purposes, where only first-order, mode-level fidelity is expected, this level of detail (capacity, percent full, health, and a mode-switchable power draw) is appropriate and adequate.

5.11.3.2 What is not included / gaps

None of the following omissions are problems at CML 2–5, where flight-software-representative fidelity (e.g., real file-system semantics, actual bit-error-rate models, thermal derating of storage media) is explicitly not required.

- **No degradation, radiation, or thermal modeling.** The class contains no model of memory degradation, radiation-induced bit errors, wear/endurance limits, or temperature-dependent behavior; `temperature` is initialized to a fixed 10 °C and is never updated anywhere in this file, so it is effectively a placeholder rather than a computed state.
- **No health transitions.** Likewise `health` is initialized to 1 (nominal) and never transitioned to any other state within this class, so no fault or failure model exists here (any such behavior would have to be injected externally).

- **No zero-capacity validation.** The class performs no validation that `maximum_capacity` is positive before dividing by it in the SoDS calculation, so a misconfigured `init_data.maximum_capacity` of zero would produce `NaN/Inf` rather than a caught error, though this is a minor robustness gap rather than a fidelity gap.
- **Passive state container only.** Most importantly, all of the interesting dynamics — how `instantaneous_capacity` actually rises and falls over time as data is generated, downlinked, or moved between memories — live entirely in `True_SC_Data_Handling`; this class is a passive state container plus a percentage/power calculator, and any correctness issues in the fill/drain sequencing itself are out of scope for this file.

5.12 True_SC_Radio_Antenna

The True_SC_Radio_Antenna class models the physical radio antenna hardware carried onboard the spacecraft, playing the same hardware-truth role for the space segment that True_GS_Radio_Antenna plays for the ground segment. Each instance represents one antenna (dipole or high-gain) with a body-frame location and boresight orientation, and it operates in one of two mutually exclusive modes, TX or RX, selected by mode_true_SC_radio_antenna_selector. The class tracks the instantaneous power drawn while transmitting or receiving, a baseline (housekeeping) data rate that is always generated, and placeholders for the RF link-budget quantities (antenna gain, noise temperature, beamwidth, required energy per bit, line loss, and coding gain) that would in principle be needed to compute a downlink/uplink margin.

Structurally, the class does not perform any RF propagation or data-transport physics itself; instead it acts as a bookkeeping and pass-through node. On every simulation step it selects the correct power draw for the current TX/RX mode (optionally overridden by a per-spacecraft-mode power lookup table), pushes that instantaneous power to True_SC_Power for energy accounting, and pushes its instantaneous generated/removed data rates to True_SC_Data_Handling so that the SC-side onboard-storage ledger can be updated. The actual over-the-link data transfer and any RF signal computations are the responsibility of True_SC_Communication_Link (and its ground-side counterpart), with this class simply exposing the antenna's hardware state and health to that link model and to the SC executive/data-handling subsystems.

5.12.1 Inputs / Outputs

5.12.1.1 Inputs / Dependencies

- init_data **(constructor-time):** antenna_type ('Dipole' / 'High Gain'), location, orientation, TX_power_consumed, RX_power_consumed, base_data_rate_generated, optional link-budget parameters antenna_gain, noise_temperature, beamwidth, energy_bit_required, line_loss, coding_gain, optional data (may contain instantaneous_TX/RX_power_consumed_per_SC_mode).

- mission.true_SC{i_SC}.software_SC_executive **(same spacecraft):** reads this_sc_mode_value to index into the optional per-mode TX/RX power lookup tables.

- True_SC_Communication_Link / Software_SC_Communication**-type classes (same and other spacecraft):** drive this class externally by setting flag_executive, mode_true_SC_radio_antenna_selector, and the instantaneous data rate properties (see write-back below); location and orientation are read by True_SC_Communication_Link and mission-specific pointing functions to compute link geometry.

5.12.1.2 Outputs

- **Internal / stored outputs:** obj.store.flag_executive, obj.store.mode_TX_RX, obj.store.instantaneous_data_rate_generated, obj.store.instantaneous_data_rate_removed, obj.store.instantaneous_power_consumed.

- **Reported to** True_SC_Power **(same spacecraft):** instantaneous_power_consumed (TX or RX value depending on mode) read via func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed.

- **Reported to `True_SC_Data_Handling` (same spacecraft):** `instantaneous_data_rate_generated` read via `func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated`; `instantaneous_data_rate_removed` read via `func_initialize_list_HW_data_removed/func_update_instantaneous_data_removed` — this antenna is one of the few hardware classes that both adds and removes data from the spacecraft's memory budget (RX ingests data, TX downlinks/removes it).
- **Reported to `True_SC_Communication_Link` (link-budget calculations, same or paired spacecraft):** reads `antenna_gain`, `noise_temperature`, `beamwidth`, `energy_bit_required`, `coding_gain`, `name`, `location`, `orientation` when computing achievable TX/RX data rate between a TX and RX antenna pair, potentially on different spacecraft.
- **External write-back confirmed:** `True_SC_Communication_Link` writes `flag_executive = 1`, `mode_true_SC_radio_antenna_selector` (`'TX'`/`'RX'`), and `instantaneous_data_rate_removed/instantaneous_data_rate_generated` directly into this class's properties each time a link is active; mission-specific communication functions similarly set `flag_executive` and `mode_true_SC_radio_antenna_selector` to schedule TX/RX windows. This class resets `flag_executive` and the instantaneous rates to baseline at the end of its own main function.

5.12.2 Mathematical Equations

5.12.2.1 Instantaneous Power Consumption by TX/RX Mode

$$P_{\text{inst}} = \begin{cases} P_{\text{TX}}, & \text{if mode = TX and no per-mode table is defined} \\ P_{\text{TX}}^{(m)}, & \text{if mode = TX and a per-mode table is defined} \\ P_{\text{RX}}, & \text{if mode = RX and no per-mode table is defined} \\ P_{\text{RX}}^{(m)}, & \text{if mode = RX and a per-mode table is defined} \end{cases} \tag{5.26}$$

P_{inst} is the instantaneous electrical power [W] drawn by the antenna at the current time step; P_{TX} and P_{RX} are the fixed, constructor-supplied power draws [W] for transmit and receive operation respectively; $P_{\text{TX}}^{(m)}$ and $P_{\text{RX}}^{(m)}$ are optional, spacecraft-mode-dependent power draws [W] indexed by the current spacecraft operating mode m.

This logic is implemented in `func_main_true_SC_radio_antenna`. The `switch obj.mode_true_SC_radio_antenna_selector` block sets `obj.instantaneous_power_consumed = obj.TX_power_consumed` (or `obj.RX_power_consumed`) by default; if `obj.data` contains the field `instantaneous_TX_power_consumed_per_SC_mode` (or the RX analogue), the value is instead looked up as `obj.data.instantaneous_TX_power_consumed_per_SC_mode(mission.true_SC{i_SC}.software_SC_executive.this_sc_mode_value)`. The resulting `obj.instantaneous_power_consumed` is then forwarded to `True_SC_Power` via `func_update_instantaneous_power_consumed` and `func_initialize_list_HW_energy_consumed`.

5.12.2.2 Data-Rate Bookkeeping (Generated/Removed)

$$R_{\text{gen}} = R_{\text{base}}, \qquad R_{\text{rem}} = 0 \tag{5.27}$$

R_{gen} is the instantaneous data rate [kbps] reported to the onboard data-handling ledger as newly generated data (e.g. housekeeping telemetry produced by the antenna); R_{base} is the constant, constructor-supplied baseline data-generation rate [kbps]; R_{rem} is the instantaneous data rate [kbps] reported as removed (i.e. downlinked) from onboard storage by this antenna.

R_{gen} corresponds to `obj.instantaneous_data_rate_generated`, which is set equal to `obj.base_data_rate_generated` at construction and is reset to that same value at the end of every call to `func_main_true_SC_radio_antenna`. R_{rem} corresponds to `obj.instantaneous_data_rate_removed`, which is initialized to 0 and likewise reset to 0 at the end of every step; the file provides no internal logic that increases `instantaneous_data_rate_removed` above zero, so any nonzero downlink removal rate must be written into this property from outside the class (e.g. by `True_SC_Communication_Link`) between the reset and the next call to `func_update_instantaneous_data_generated`/`func_update_instantaneous_data_removed` on `True_SC_Data_Handling`. Both quantities are passed by reference into the data-handling object each step, making this class purely a source/sink bookkeeping node rather than a computer of data-rate physics.

5.12.2.3 Link Margin / E_b/N_0 Parameters (Declared but Not Computed)

$$\left[G_{\text{ant}},\ T_{\text{noise}},\ B,\ \left(\frac{E_b}{N_0}\right)_{\text{req}},\ L_{\text{line}},\ G_{\text{code}}\right] \longrightarrow \text{(no link-margin equation evaluated in this file)} \tag{5.28}$$

G_{ant} is the antenna gain [dB] (`antenna_gain`); T_{noise} is the system noise temperature [K] (`noise_temperature`); B is the receiver beamwidth [MHz] (`beamwidth`); $(E_b/N_0)_{\text{req}}$ is the minimum required energy-per-bit-to-noise ratio [dB] (`energy_bit_required`); L_{line} is the pointing/line loss [dB] (`line_loss`); G_{code} is the coding gain [dB] (`coding_gain`). These are exactly the same category of link-budget parameters previously documented for `True_GS_Radio_Antenna`.

In the constructor, the block guarded by `if isfield(init_data, 'antenna_gain')` assigns `obj.antenna_gain`, `obj.noise_temperature`, `obj.beamwidth`, `obj.energy_bit_required`, and `obj.coding_gain` from `init_data`, and all six quantities are declared as properties under the comment "Optional (only for Link Margin Calculations)." Notably, `line_loss` is declared as a property but is never assigned anywhere in this file, not even inside the conditional block that populates the other five fields. As with `True_GS_Radio_Antenna`, no method in `True_SC_Radio_Antenna` (nor, per prior review, in `True_SC_Communication_Link`) ever combines these six quantities into an actual E_b/N_0 or link-margin calculation; they are stored purely as static, optionally-populated metadata. This confirms the same gap identified on the ground-station side is mirrored, and in fact slightly worse (one unassigned field), on the spacecraft side.

5.12.3 CML 2–5 Suitability

5.12.3.1 What is included and correctly implemented

The class correctly captures the essential hardware bookkeeping needed for trade-space-level spacecraft power and data-volume analysis at CML 2–5.

- **Mode-dependent power and data bookkeeping.** It distinguishes TX and RX operating modes, tracks distinct fixed power draws for each mode with an optional spacecraft-mode-dependent power lookup table (allowing coarse per-mode power differentiation without requiring detailed RF electronics modeling), and consistently feeds instantaneous power draw into `True_SC_Power` and instantaneous generated/removed data rate into `True_SC_Data_Handling` every simulation step.
- **Health/executive gating and deterministic reset.** Health/executive gating (`flag_executive`, `health`) and the reset-to-baseline pattern for data rates at the end of each step are implemented consistently and give a clean, deterministic per-step behavior appropriate for point-design or first-order trade studies.

- **Storage/logging.** The storage/logging machinery (`func_update_true_SC_radio_antenna_store`) faithfully records mode, power, and data-rate history for post-run analysis, which is sufficient fidelity for CML 2–5 architecture trades where flight-software-representative behavior is not required.

5.12.3.2 What is not included / gaps

The main gap concerns the unused link-budget fields, with a minor secondary gap in data-rate removal.

- **Unused link-budget fields, no link margin computed.** The six link-budget fields (`antenna_gain`, `noise_temperature`, `beamwidth`, `energy_bit_required`, `line_loss`, `coding_gain`) are declared and, except for `line_loss`, conditionally populated from initialization data, but no method in this file (or, per established prior review, in `True_SC_Communication_Link`) ever uses them to compute a link margin, received E_b/N_0, or any other RF performance metric — the intended link-budget capability is a stub.

- **Externally driven data-rate removal.** In addition, `instantaneous_data_rate_removed` has no internal mechanism in this class to become nonzero: it is only ever set to 0, so the actual downlink/removal data rate must be injected from outside the class, and this file offers no visibility into how or whether that occurs.

- **Relative severity for CML 2–5 purposes.** For CML 2–5 purposes, the absent link-margin computation is the more consequential gap, since even a first-order comm-link closure check (a common early trade-space discriminator) cannot currently be produced from these stored fields; the missing internal computation of removed data rate is comparatively minor bookkeeping that is likely intentional given the described role of `True_SC_Communication_Link`.

5.13 True_SC_RCS_Thruster

True_SC_RCS_Thruster models a single reaction-control-system (RCS) micro-thruster mounted on the spacecraft bus. Structurally it is very close to True_SC_Chemical_Thruster: both are handle classes with a fixed orientation and location in the body frame, a noisy commanded thrust, an Isp-based fuel-consumption model, warm-up power/timing bookkeeping, health and temperature properties, and hooks into the spacecraft power, data-handling, and fuel-tank classes. The key functional difference is in how the resulting force/torque pair is used: an RCS thruster is treated as an intentional attitude-control actuator, whereas the chemical thruster is treated as the primary orbit-control (translational) actuator. Concretely, True_SC_RCS_Thruster adds its body-frame torque directly to true_SC_adc.control_torque (an intended control input to the attitude dynamics), while its reaction force is rotated into the inertial frame and added only as a small disturbance_force to orbit propagation (scaled down by the ratio of the attitude to orbit integration time steps). This is the opposite convention from True_SC_Chemical_Thruster, which adds its force to orbit_propagation.control_force (the intended orbit-control input) and its torque to true_SC_adc.disturbance_torque (an unwanted torque to be rejected by attitude control). This role reversal is consistent with the physical roles of the two hardware types: RCS thrusters provide fine attitude torques (and incidental, usually negligible, translational impulse), while chemical thrusters provide large translational burns (with an incidental, usually undesired, torque due to thrust-vector/CG misalignment).

Unlike True_SC_Chemical_Thruster, this class has no explicit thruster-state machine (idle/warming_up/ready/firing), no pending-fire logic across cycles, and no explicit accumulation or reporting of delta-V; it is driven each attitude time step by a single boolean command_actuation flag (set upstream, per the code comment, by a thruster-selection/allocation routine such as "optimize_thruster_dart" in the attitude control software), fires for that step once any configured warm-up time has elapsed, and otherwise reports zero force, torque, power, and data.

5.13.1 Inputs / Outputs

5.13.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_power_consumption, instantaneous_data_generated_per_sample, mode_true_SC_RCS_thruster_selector, location, orientation (normalized), thruster_noise, minimum_thrust, maximum_thrust, RCS_thruster_ISP, optional desired_fuel_tank index (defaults to 1), optional name.
- True_SC_ADC **(same spacecraft):** reads attitude (quaternion) every firing to rotate control_force_MT from body frame to inertial frame.
- True_SC_Body **(same spacecraft):** reads location_COM every firing to compute the moment arm for control_torque_MT = cross(location - COM, control_force_MT).
- True_SC_Fuel_Tank{desired_fuel_tank} **(same spacecraft):** calls func_consume_fuel(fuel_consumed_per_firing) on the designated tank every firing, driving fuel depletion.
- mission.true_time **(mission-level):** reads time_step_attitude to scale fuel consumption and the inertial disturbance-force contribution.
- Software_SC_Control_Attitude / **mission-specific attitude-optimization functions (same spacecraft):** write command_actuation and commanded_thrust into this object (see write-back below), read every main-loop step to decide whether/how hard to fire.

5.13.1.2 Outputs

- **Internal / stored outputs:** `obj.store.true_commanded_thrust`, `obj.store.control_torque_MT`, `obj.store.control_force_MT`, `obj.store.fuel_consumed_per_firing`, `obj.store.total_fuel_consumed`.
- **Reported to `True_SC_ADC` (same spacecraft):** `control_torque_MT` is added directly into `mission.true_SC{i_SC}.true_SC_adc.control_torque` every firing.
- **Reported to `True_SC_Orbit_Propagation` (same spacecraft):** the inertial-frame control force is added into `mission.true_SC{i_SC}.true_SC_orbit_propagation.disturbance_force`, scaled by the ratio of attitude to orbit time steps.
- **Reported to `True_SC_Fuel_Tank` (same spacecraft):** drives fuel depletion via `func_consume_fuel`.
- **Reported to `True_SC_Power`/`True_SC_Data_Handling` (same spacecraft):** `instantaneous_power_consumed`/`instantaneous_data_generated_per_sample` read via the standard attitude-rate update functions.
- **External write-back confirmed:** `command_actuation` and `commanded_thrust` are written into `mission.true_SC{i_SC}.true_SC_RCS_thruster{i}` by `Software_SC_Control_Attitude`'s mission-specific actuator-selection chain (e.g. `func_decompose_control_torque_into_thrusters_optimization_kkt.m`, `func_optimize_thruster_pinv.m`, invoked from `func_actuator_selection_DART.m`) when attitude error is large, wheel torque is insufficient, or wheel saturation is predicted. This class resets its own `command_actuation`, `commanded_thrust`, `control_torque_MT`, `control_force_MT` to zero at the end of each cycle.

5.13.2 Mathematical Equations

5.13.2.1 True Commanded Thrust with Random Noise

$$F_{\text{true}} = F_{\text{cmd}} + n_T\,(2u - 1)\,, \qquad u \sim \mathcal{U}(0,1) \tag{5.29}$$

F_{true} is the actual (noisy) scalar thrust magnitude produced by the thruster during the current attitude step; F_{cmd} is the thrust commanded by the spacecraft flight software; n_T is the configured thruster noise amplitude; u is a uniform random draw.

In the code this is `obj.true_commanded_thrust = obj.commanded_thrust + current_thruster_noise`, where `current_thruster_noise = obj.thruster_noise * (2*rand() - 1)`. Unlike `True_SC_Chemical_Thruster`, the result is not subsequently clamped to [`minimum_thrust`, `maximum_thrust`] — those bounds are stored as properties but are not enforced in this file's main function.

5.13.2.2 Thrust Force Vector: Body Frame and Rotation to Inertial Frame

$$\vec{F}_B = \hat{u}\,F_{\text{true}}, \qquad \vec{F}_I = \mathcal{R}(q^*)\;\vec{F}_B \tag{5.30}$$

$\vec{F}_B$ is the thrust force vector in the spacecraft body frame; $\hat{u}$ is the unit-norm thruster orientation vector (body frame); $\vec{F}_I$ is the same force vector rotated into the inertial frame; q^* is the conjugate of the spacecraft's attitude quaternion; $\mathcal{R}(\cdot)$ denotes the quaternion vector-rotation operator.

This is computed as `obj.control_force_MT = obj.orientation * obj.true_commanded_thrust` and `control_force_MT_IF = qvtran(qcon(mission.true_SC{i_SC}.true_SC_adc.attitude), obj.control_force_MT)`. The physical operation (rotate a body-frame force

into the inertial frame using the current attitude) is the same idea used in `True_SC_Chemical_Thruster` (there via a rotation matrix from `quaternionToRotationMatrix`), just implemented with a quaternion vector-transform utility instead.

5.13.2.3 Control Torque About the Center of Mass

$$\vec{\tau}_{\mathrm{MT}} = (\vec{r}_{\mathrm{loc}} - \vec{r}_{\mathrm{COM}}) \times \vec{F}_B \tag{5.31}$$

$\vec{\tau}_{\mathrm{MT}}$ is the torque generated by this thruster about the spacecraft center of mass, in the body frame; $\vec{r}_{\mathrm{loc}}$ is the thruster mounting location; $\vec{r}_{\mathrm{COM}}$ is the spacecraft's center-of-mass location.

This is `obj.control_torque_MT = cross(obj.location - mission.true_SC{i_SC}.true_SC_body.location_COM, obj.control_force_MT)`. This torque is then added directly to `true_SC_adc.control_torque` (an intentional actuator input to the attitude dynamics) — the same cross-product form used in `True_SC_Chemical_Thruster`, but there the resulting torque instead feeds `disturbance_torque`, reflecting the RCS thruster's role as the primary attitude-control actuator rather than an unwanted torque source.

5.13.2.4 Propellant Mass Flow Rate (Rocket Equation)

$$\dot{m}_{\mathrm{firing}} = \frac{F_{\mathrm{true}}\,\Delta t_{\mathrm{att}}}{I_{sp}\,g_0} \tag{5.32}$$

$\dot{m}_{\mathrm{firing}}$ is the propellant mass consumed during the current firing step; Δt_{att} is the attitude-loop integration time step; I_{sp} is the thruster specific impulse; $g_0 = 9.80665$ m/s^2 is standard gravity.

This is `obj.fuel_consumed_per_firing = (obj.true_commanded_thrust * mission.true_time.time_step_attitude) / (obj.isp * g0)`, accumulated into `obj.total_fuel_consumed` and drawn from `true_SC_fuel_tank{i_tank}` via `func_consume_fuel`. This is the identical rocket-equation form used in `True_SC_Chemical_Thruster`, with the notable difference that it uses the *attitude* time step rather than the (typically coarser or differently-paced) orbit time step, consistent with RCS thrusters firing on the attitude control cadence.

5.13.2.5 Reaction Force Contribution to Orbital Disturbance

$$\vec{F}_{\mathrm{dist}} \;+=\; \vec{F}_I\,\frac{\Delta t_{\mathrm{att}}}{\Delta t_{\mathrm{orb}}} \tag{5.33}$$

$\vec{F}_{\mathrm{dist}}$ is the cumulative disturbance force acting on the orbit propagator; Δt_{att} and Δt_{orb} are the attitude and orbit integration time steps, respectively.

This is `mission.true_SC{i_SC}.true_SC_orbit_propagation.disturbance_force = ... + (control_force_MT_IF * mission.true_time.time_step_attitude / mission.true_time.time_step)`. This equation has no counterpart in `True_SC_Chemical_Thruster`: it is a multi-rate bookkeeping step that converts an impulse computed on the (typically faster) attitude cadence into an average force compatible with the (typically slower) orbit-propagation cadence, and it deliberately routes the RCS reaction force into the orbit disturbance accounting rather than into the orbit's intentional control force, reflecting that RCS thrust is treated as a minor, non-primary contributor to translational motion.

5.13.3 CML 2–5 Suitability

5.13.3.1 What is included and correctly implemented

The class implements the correct first-order rigid-body actuator physics needed at CML 2–5.

- **Core thrust/torque/propellant physics.** It provides a body-frame thrust vector along a configurable orientation, a torque computed as the cross product of the moment arm (mounting location minus center of mass) with that force, and a standard rocket-equation propellant consumption model tied into a fuel-tank object.
- **Correct separation of torque and translational-disturbance roles.** It correctly and deliberately distinguishes an RCS thruster's role from a main-propulsion thruster's role within the simulation's bookkeeping: torque is delivered as an intentional input to attitude control (`control_torque`), while the reaction force is delivered as a (rate-scaled) disturbance to orbit propagation, which is an appropriate medium-fidelity representation of "RCS torques the vehicle; any accompanying translational impulse is secondary."
- **Noise, health, and warm-up modeling.** A simple uniform random noise term on commanded thrust, a health flag that can zero out the thruster's output, a temperature property, and warm-up timing/power-state logic are all present, giving trade-space studies a way to represent actuator uncertainty, faults, and finite response time without needing flight-software-representative logic.
- **System-level wiring.** The class is also properly wired into the spacecraft power, data-handling, and fuel-tank subsystems, which matters for system-level resource-budget trades at this maturity level.

5.13.3.2 What is not included / gaps

Several limitations exist, ranging from an unenforced thrust bound to modeling simplifications appropriate for this maturity level.

- **Unenforced thrust limits.** `minimum_thrust` and `maximum_thrust` are declared and initialized but are never enforced (no clamping) on `true_commanded_thrust` in `func_main_true_SC_RCS_thruster`, unlike the chemical-thruster class, which explicitly clamps its noisy thrust to [`minimum_thrust`, `maximum_thrust`]; as written, noise could in principle push the effective thrust outside the physically achievable range or even negative.
- **No minimum-impulse-bit or duty-cycle modeling.** There is no minimum-impulse-bit (MIB) or sub-step pulse-width modeling: the thruster is simply "on" (full computed thrust) or "off" for an entire attitude time step according to `command_actuation`, so short-pulse / duty-cycle firing patterns characteristic of real RCS hardware are not represented — this is a reasonable simplification for CML 2–5 but is a real fidelity gap if fine impulse-bit effects matter to a trade.
- **No thruster pairing/clustering logic.** The class also models only a single, independent thruster; there is no logic here for thruster pairing/clustering to guarantee a pure torque couple (zero net translational force) — that behavior, if desired, must be enforced entirely by the calling attitude-control software's thruster selection logic, not by this class.
- **No delta-V tracking.** No delta-V state is tracked (in contrast to `True_SC_Chemical_Thruster`), consistent with an attitude-actuator role but a gap if RCS is also relied upon for translational trims/orbit maintenance in a given mission concept.
- **Simplified noise and warm-up models.** Finally, the noise model is a simple uncorrelated uniform draw and the warm-up model is a linear timer with no thrust ramp-up profile or thermal effects — both acceptable simplifications, not defects, at CML 2–5.

5.14 True_SC_Reaction_Wheel

True_SC_Reaction_Wheel models a single momentum-storage reaction wheel used for spacecraft attitude control. Each instance tracks its own moment of inertia, angular velocity, and angular momentum, and integrates a commanded angular acceleration (set externally by the attitude control logic) forward in time, subject to torque, acceleration, velocity, and rate-of-change limits, plus injected angular-velocity noise. From the realized wheel dynamics the class computes the actual torque the wheel applies to itself ($\tau = I\alpha$) and, critically, applies the equal-and-opposite reaction torque directly onto the spacecraft bus by decrementing mission.true_SC{i_SC}.true_SC_adc.control_torque. This confirms that True_SC_Reaction_Wheel is the actual source of both the per-wheel angular momentum (total_momentum) that True_SC_ADC later sums via func_update_total_wheel_momentum, and of the reaction torque fed back into the attitude dynamics — True_SC_ADC does not compute wheel momentum or torque itself, it only consumes the values this class produces.

In addition to the rotational dynamics, the class interfaces with True_SC_Power (registering itself for power accounting at construction via func_initialize_list_HW_energy_consumed and reporting instantaneous_power_consumed each step via func_update_instantaneous_power_consumed_attitude) and with True_SC_Data_Handling (registering and reporting instantaneous_data_generated_per_sample). A simple health flag allows the wheel to be disabled, in which case torque, velocity, and power/data outputs are all zeroed.

5.14.1 Inputs / Outputs

5.14.1.1 Inputs / Dependencies

- init_data **(constructor-time):** location, orientation, max_angular_velocity, instantaneous_data_generated_per_sample, power_consumed_angular_velocity_array, angular_velocity_noise, angular_velocity (initial), either moment_of_inertia directly or radius + mass (disk approximation), optional max_torque/maximum_acceleration.
- True_SC_Body **(same spacecraft):** reads num_hardware_exists.num_reaction_wheel at construction to validate wheel count (3, 4, 5, or 6) and set the geometric envelope_ratio for the torque/momentum envelope model.
- mission.true_time **(mission-level):** reads time_step_attitude every step to integrate commanded angular acceleration into angular velocity.
- Software_SC_Control_Attitude / **mission-specific actuator-selection functions (same spacecraft):** write flag_executive and commanded_angular_acceleration into this object (see write-back below), read to decide whether/how to spin up or down.

5.14.1.2 Outputs

- **Internal / stored outputs:** obj.store.angular_velocity, obj.store.torque, obj.store.saturated, obj.store.commanded_angular_acceleration, obj.store.actual_angular_acceleration, obj.store.actual_torque, obj.store.max_angular_velocity.
- **Reported to** True_SC_ADC **(same spacecraft):** actual_torque (reaction torque, equal and opposite to the torque applied to the wheel) is subtracted directly from mission.true_SC{i_SC}.true_SC_adc.control_torque every attitude step.

- **Reported to `Software_SC_Control_Attitude` (same spacecraft):** `maximum_torque`, `moment_of_inertia`, `orientation`, `total_momentum/angular_velocity`, and `health` are read by mission-specific wheel-command functions when solving for commanded wheel accelerations and checking for predicted saturation.
- **Reported to `True_SC_Power/True_SC_Data_Handling` (same spacecraft):** `instantaneous_power_consumed` (interpolated from `power_consumed_angular_velocity_array`) and `instantaneous_data_generated_per_sample` read via the standard attitude-rate update functions.
- **External write-back confirmed:** `flag_executive` and `commanded_angular_acceleration` are written into `mission.true_SC{i_SC}.true_SC_reaction_wheel{i}` by `Software_SC_Control_Attitude`'s mission-specific actuator-selection chain (`func_apply_reaction_wheel_commands.m` applies per-wheel accelerations from `func_compute_rw_command_kkt.m/_pinv.m`, invoked from `func_actuator_selection_DART.m`); `func_handle_desaturation.m` separately overrides these during desaturation handling. This class resets its own `flag_executive/commanded_angular_acceleration` to zero at the end of each cycle.

5.14.2 Mathematical Equations

5.14.2.1 Moment of Inertia of the Wheel

$$I = \frac{1}{2} m r^2 \tag{5.34}$$

I = moment of inertia of the wheel [kg·m^2]; m = wheel mass [kg]; r = wheel radius [m]. This is the standard solid-disk-about-its-central-axis formula.

Computed in the constructor as `obj.moment_of_inertia = 0.5 * obj.mass * obj.radius^2`, only when `init_data` does not already supply a `moment_of_inertia` field (in which case that value is used directly).

5.14.2.2 Maximum Wheel Acceleration

$$\alpha_{max} = \frac{\tau_{max}}{I} \tag{5.35}$$

α_{max} = maximum angular acceleration of the wheel [rad/s^2]; τ_{max} = maximum torque the wheel motor can produce [Nm]; I = moment of inertia [kg·m^2].

`obj.maximum_acceleration = obj.maximum_torque / obj.moment_of_inertia`, computed in the constructor when `init_data` provides `max_torque` but not `maximum_acceleration` directly. If no `max_torque` is supplied at all, `maximum_torque`, `momentum_capacity`, and `maximum_acceleration` are all set to `Inf` (i.e. unconstrained wheel).

5.14.2.3 Commanded Torque from Angular Acceleration

$$\vec{\tau}_{actual} = I\, \alpha_{cmd}\, \hat{u} \tag{5.36}$$

$\vec{\tau}_{actual}$ = actual torque vector applied by the wheel motor [Nm]; I = moment of inertia [kg·m^2]; α_{cmd} = commanded angular acceleration (after being clipped to $\pm\alpha_{max}$) [rad/s^2]; $\hat{u}$ = unit vector giving the wheel's spin-axis orientation in the body frame.

In `func_main_true_reaction_wheel`, `obj.commanded_angular_acceleration` is first clamped to $\pm$`obj.maximum_acceleration`, then `torque_magnitude = obj.moment_of_inertia * obj.commanded_angular_acceleration` and `obj.actual_torque = torque_magnitude * obj.orientation`.

5.14.2.4 Angular Velocity Update with Noise and Rate Limiting

$$\omega_{unc} = \omega_k + \alpha_{cmd}\,\Delta t + n \tag{5.37}$$

$$\omega_{k+1} = \mathrm{clip}\Big(\omega_k + \mathrm{sign}(\omega_{unc} - \omega_k)\,\min(|\omega_{unc} - \omega_k|,\ \Delta\omega_{max}),\ -\omega_{max},\ \omega_{max}\Big) \tag{5.38}$$

ω_k = wheel angular velocity at the current step [rad/s]; ω_{unc} = unconstrained (unclamped) next-step velocity; Δt = attitude integration time step [s]; n = injected angular-velocity noise (uniform on $[-\sigma, \sigma]$, applied only while a nonzero acceleration is commanded); $\Delta\omega_{max}$ = maximum allowed velocity change per step (normally $0.1\,\omega_{max}$, tightened to $0.02\,\omega_{max}$ if the commanded acceleration would reverse the wheel's spin direction while it is spinning above $0.3\,\omega_{max}$); ω_{max} = `max_angular_velocity`.

`current_vel_noise = obj.angular_velocity_noise * (2*rand()-1);` `unconstrained_velocity = obj.angular_velocity + obj.commanded_angular_acceleration * mission.true_time.time_step_attitude + current_vel_noise;` `max_velocity_change` (set to `0.1*obj.max_angular_velocity`, or `0.02*obj.max_angular_velocity` on a detected reversal); the clamp to `limited_velocity` and then to `new_velocity = max(min(limited_velocity, obj.max_angular_velocity), -obj.max_angular_velocity).` The realized acceleration is back-computed as `obj.actual_angular_acceleration = (new_velocity - obj.angular_velocity) / mission.true_time.time_step_attitude` before `obj.angular_velocity` is overwritten with `new_velocity`.

5.14.2.5 Wheel Angular Momentum

$$\vec{h} = I\,\omega\,\hat{u} \tag{5.39}$$

$\vec{h}$ = angular momentum vector stored in the wheel [kg·m^2/s]; I = moment of inertia [kg·m^2]; ω = current wheel angular velocity [rad/s]; $\hat{u}$ = spin-axis orientation unit vector.

`momentum_magnitude = obj.moment_of_inertia * obj.angular_velocity; obj.total_momentum = momentum_magnitude * obj.orientation;`, computed each call of `func_main_true_reaction_wheel` (and once at construction time). This is the exact per-wheel quantity that `True_SC_ADC.func_update_total_wheel_momentum` later reads and vector-sums across all wheels to obtain total system wheel momentum — the summation itself is not performed here, only the per-wheel value.

5.14.2.6 Reaction Torque Applied to the Spacecraft Bus

$$\vec{\tau}_{bus} \leftarrow \vec{\tau}_{bus} - \vec{\tau}_{actual} \tag{5.40}$$

$\vec{\tau}_{bus}$ = net control torque accumulator acting on the spacecraft body, held on the ADC object; $\vec{\tau}_{actual}$ = torque the wheel motor applies to the wheel. Newton's third law gives the wheel a reaction torque on the bus equal and opposite to the torque the wheel applies to itself.

`mission.true_SC{i_SC}.true_SC_adc.control_torque = mission.true_SC{i_SC}.true_SC_adc.control_torque - obj.actual_torque;`. This is the sole place in the reviewed code where a reaction wheel's reaction torque is injected into the spacecraft's attitude dynamics; `True_SC_ADC` itself only reads/accumulates this field, it does not compute the torque.

5.14.2.7 Saturation Flag

$$\text{saturated} = (|\omega| \geq 0.8\,\omega_{max}) \tag{5.41}$$

ω = current angular velocity; ω_{max} = `max_angular_velocity`. The wheel is flagged saturated once it reaches 80% of its rated maximum speed, ahead of the hard velocity clamp above.

`obj.saturated = (abs(obj.angular_velocity) >= obj.max_angular_velocity * 0.80);`.

5.14.2.8 Power Consumption from Angular Velocity

$$P(\omega) = \text{interp1}\big(\{\omega_i\}, \{P_i\}, \omega\big) \tag{5.42}$$

$P(\omega)$ = instantaneous power drawn by the wheel motor [W]; $\{\omega_i\}, \{P_i\}$ = the paired velocity/power lookup table supplied at construction; ω = current wheel angular velocity, clamped to the table's endpoints outside its range.

`power_consumptions/angular_velocities` are extracted from `obj.power_consumed_angular_velocity_array`, sorted by velocity, and linear-interpolated (`interp1(...,'linear')`) at `obj.angular_velocity` to set `obj.instantaneous_power_consumption` (and its alias `obj.instantaneous_power_consumed`). Note the code first computes a physics-based estimate `obj.instantaneous_power_consumption = abs(obj.actual_torque * obj.angular_velocity)` (i.e. $P = |\tau\,\omega|$) immediately after the torque/velocity update, but this value is unconditionally overwritten a few lines later by the table-interpolated result, so only the lookup-table value is actually used downstream.

5.14.3 CML 2–5 Suitability

5.14.3.1 What is included and correctly implemented

The class implements a physically sound, self-consistent first-order reaction-wheel model appropriate for trade-space and point-design work.

- **Core wheel dynamics and limits.** It includes moment of inertia from mass/radius geometry or direct override, torque-limited and acceleration-limited response to a commanded acceleration, first-order Euler integration of wheel speed with injected noise, hard velocity saturation limiting, and a saturation-warning threshold at 80% of rated speed.
- **Correct momentum and torque coupling to the bus.** It correctly computes and tracks the vector sign of the individual wheel's stored angular momentum ($\vec{h} = I\omega\hat{u}$), and correctly applies the reaction torque (equal and opposite to the wheel's own torque) onto the spacecraft's `control_torque` accumulator, which is the physically appropriate way to couple a wheel's internal dynamics to bus attitude dynamics.
- **Power modeling and system wiring.** Power draw is modeled via a velocity-dependent lookup table (with sorting and edge clamping), and the class correctly registers itself with `True_SC_Power` and `True_SC_Data_Handling` for system-level power and data-volume accounting.
- **Fault flag and telemetry.** A health flag provides a simple fault mode (wheel off, zero torque/momentum/power/data), and per-step telemetry (velocity, torque, commanded/actual acceleration, saturation flag) is stored for post-run analysis.
- **Overall fidelity match to CML 2–5.** This level of fidelity — geometric/physical parameterization, first-order dynamics, torque/speed/momentum limits, and coupling to power and dynamics — is consistent with CML 2–5 trade studies and does not require flight-software-representative implementation.

5.14.3.2 What is not included / gaps

None of the following gaps are problems for CML 2–5 use, where first-order/medium-fidelity actuator behavior is sufficient.

- **No motor/control-loop model.** There is no explicit motor/control-loop model (e.g. current draw, back-EMF, motor time constant, or bearing/viscous friction torque); torque is applied instantaneously and losses only appear implicitly through the empirical power-vs-speed lookup table.
- **Ad hoc rate-limiting for numerical stability.** The ad hoc rate-limiting logic (10% of max velocity per step generally, tightened to 2% on a detected direction reversal, with hard-coded thresholds and a debug print rather than a logged/telemetered event) is a simulation-stability safeguard rather than a physical wheel characteristic, and is not derived from any wheel spec.
- **Dead torque-based power path.** The physically motivated power estimate $P = |\tau\omega|$ is computed but then discarded in favor of the lookup table, so the torque-based power path is effectively dead code.
- **Momentum capacity not enforced.** Momentum capacity (`momentum_capacity`) is defined as a property and set to `Inf` in the unconstrained-torque branch, but is never computed from the wheel's actual max torque/speed in the torque-limited branch nor used anywhere to limit `total_momentum`, so true momentum saturation (as opposed to velocity saturation) is not enforced.
- **Unused thermal and min-acceleration properties.** Thermal behavior (`temperature`) and the `min_acceleration` property are declared but never updated or used in `func_main_true_reaction_wheel` — flight-software-representative fidelity (e.g. detailed motor electronics, thermal derating, or verified momentum-envelope enforcement) is not required at this maturity level.

5.15 True_SC_Remote_Sensing

This class models a generic, mode-selectable remote sensing instrument (e.g. a scanner, radiometer, or coverage-mapping payload) and tracks its cumulative footprint coverage of a target body over the simulation. Rather than computing a single instantaneous image as `True_SC_Camera` does, `True_SC_Remote_Sensing` maintains a fixed mesh of surface science points (loaded via `func_load_science_points_v2`) and, at each commanded measurement, dispatches via `mode_true_SC_remote_sensing_selector` to either a `'Generic'` coverage test implemented directly in this file (`func_update_true_SC_remote_sensing_Generic`) or a `'mission specific'` function resolved by name via `str2func`. The `'Generic'` test combines a limb-angle visibility check (is the mesh point on the near side of the target, as seen from the spacecraft) with a boresight field-of-view check (is the point within the instrument's cone, measured from the spacecraft to the point), and falls back to the single angularly-closest point if no mesh point satisfies both. The instrument's boresight is defined by a location and orientation unit vector in the spacecraft body frame, and its field of view is a single scalar in degrees, consistent with the camera class's boresight/FOV convention.

Interfaces mirror the other True_Sensors_Actuators hardware classes: the executive commands the instrument on/off via `flag_executive`; when active and past its `measurement_wait_time` cadence, it consumes power (`instantaneous_power_consumed`, optionally a lookup table indexed by spacecraft mode) and generates data at a continuous rate (`instantaneous_data_rate_generated`, in kbps) that is registered with `True_SC_Power` and `True_SC_Data_Handling` at construction. Coverage progress is visualized in body/target-fixed 3-D views, a static longitude/latitude map (using `cspice_reclat`, exactly as in `True_SC_Camera`), and a time history of percent-of-mesh-observed, optionally recorded to video.

5.15.1 Inputs / Outputs

5.15.1.1 Inputs / Dependencies

- `init_data` **(constructor-time):** `name`, `instantaneous_power_consumed`, `mode_true_SC_remote_sensing_selector` (`'Generic'` or `'mission specific'`), `instantaneous_data_rate_generated`, `measurement_wait_time`, `location`, `orientation`, `field_of_view`, `flag_show_coverage_plot`, `num_points`, and optionally `data`, `mode_selector_name`.
- `True_SC_ADC` **(same spacecraft):** `rotation_matrix`, used in `func_update_true_SC_remote_sensing_Generic` (the `'Generic'` mode handler) to rotate `obj.orientation` from the body frame into the target-centered frame to form the boresight pointing vector.
- `True_SC_Orbit_Propagation` **(same spacecraft):** `position_relative_target` and `index_relative_target`, used to compute spacecraft-to-target geometry (limb angle, line-of-sight to mesh points) for the field-of-view/coverage test and for plotting.
- `mission.true_target{i_target}` **(target body, mission-level):** `radius` and `rotation_matrix`, used to rotate the unit-sphere mesh points into the target-centered inertial frame for the coverage geometry test and plots.
- `Software_SC_Executive` **(same spacecraft):** `flag_executive` is set externally by mission-specific executive logic (e.g. for NISAR) to command a measurement; `this_sc_mode_value` is read to select a mode-dependent power/data rate if defined; `time` gates the visualization refresh rate.

- `mission.true_time: time`, used to gate repeat measurements against `measurement_wait_time`.

5.15.1.2 Outputs

- **Internal / stored outputs:** `monostatic_observed_point`, `monostatic_num_point_observed`, `flag_executive`, `instantaneous_data_rate_generated`, `instantaneous_power_consumed` are stored in `obj.store` each step; coverage percentage and plots/video/Excel report rows are generated by `func_visualize_SC_remote_sensing_coverage_during_sim`.
- **Reported to `True_SC_Power`:** `instantaneous_power_consumed` registered/updated via `func_initialize_list_HW_energy_consumed`/`func_update_instantaneous_power_consumed`.
- **Reported to `True_SC_Data_Handling`:** `instantaneous_data_rate_generated` registered/updated via `func_initialize_list_HW_data_generated`/`func_update_instantaneous_data_generated`.
- **Reported to visualization/post-processing (same file, plus mission-specific extensions, e.g. for NISAR):** the local subplot helper functions (`func_true_SC_remote_sensing_coverage_subplot_3d/_2d/_attitude/_time`) and the report/Excel-writing logic in `func_visualize_SC_remote_sensing_coverage_during_sim` read `pos_points`, `spherical_points`, `monostatic_observed_point`, `name`, `orientation` directly for plotting and reporting, without modifying them; additional mission-specific functions may read these same fields.
- External mission-specific executive functions WRITE `flag_executive` on this class's instances to command measurements — the one documented case of an external class writing into this class's own properties, limited to the standard executive-command flag, not to any measurement/physics output.

5.15.2 Mathematical Equations

5.15.2.1 Measurement Cadence (Temporal Sampling Gate)

$$\text{take measurement if} \quad (t_{\text{sim}} - t_{\text{meas,prev}}) \geq \Delta t_{\text{wait}} \tag{5.43}$$

where t_{sim} is the current mission simulation time, $t_{\text{meas,prev}}$ is the time of the previous measurement, and Δt_{wait} is the instrument's minimum re-measurement interval.

This gate is implemented in `func_main_true_SC_remote_sensing` as `(mission.true_time.time - obj.measurement_time) >= obj.measurement_wait_time`, with `obj.measurement_time` updated to `mission.true_time.time` whenever a new measurement is taken. It is the only explicit temporal/duty-cycle law in the class; it governs how often the coverage function (and therefore power draw and data generation) is invoked, analogous to the camera class's imaging cadence.

5.15.2.2 Limb-Angle Visibility Test

$$\theta_{\text{limb}} = \arccos\left(\frac{R_{\text{target}}}{d_{\text{SC-target}}}\right), \qquad \text{point } i \text{ visible if } \beta_i \leq \theta_{\text{limb}} \tag{5.44}$$

where R_{target} is the target body's radius, $d_{\text{SC-target}}$ is the instantaneous spacecraft-to-target-center distance, θ_{limb} is the resulting limb (horizon) half-angle as seen from the spacecraft, and β_i is the angle between the target-center-to-spacecraft direction and the target-center-to-mesh-point-i direction. A

mesh point passes this test only if it lies on the near side of the target body relative to the spacecraft, i.e. within the visible disk rather than over the horizon.

In `func_update_true_SC_remote_sensing_Generic`, this is `limb_angle = acosd(mission.true_target{i_target}.radius/distance_SC_Target)`, and β_i is `dot_prod_angle_array = real(acosd(this_pos_points_normalized * this_SC_relative_position_normalize'))`, with the pass/fail mask `idx_array = logical(dot_prod_angle_array <= limb_angle)`. This is a purely geometric horizon test; it does not account for topographic self-shadowing on an irregular shape model, only the limb of the (spherical-radius) target.

5.15.2.3 Boresight Field-of-View Test

$$\gamma_i = \arccos\Big(\hat{u}_i \cdot \hat{b}\Big), \qquad \text{point } i \text{ in view if } \gamma_i \le \text{FOV} \tag{5.45}$$

where $\hat{u}_i$ is the unit vector from the spacecraft to mesh point i, $\hat{b} = \mathbf{R}_B\,\hat{o}$ is the instrument boresight unit vector (body-frame `orientation` rotated by the ADC's body-to-inertial `rotation_matrix`), γ_i is the resulting boresight-to-point angle, and FOV is the instrument half-angle field of view (property `field_of_view`, in degrees).

In the code, `this_orientation = (mission.true_SC{i_SC}.true_SC_adc.rotation_matrix * obj.orientation')'` forms $\hat{b}$; the spacecraft-to-mesh-point unit vectors are `this_pos_points_relative_SC_normalize`; and γ_i is `dot_prod_angle_array = real(acosd(this_pos_points_relative_SC_normalize * this_orientation'))`. The combined visible-and-in-FOV mesh-point set is `idx_array = idx_array & logical(dot_prod_angle_array <= obj.field_of_view)`, and `index_point_science_mesh = find(idx_array)`. If this set is empty (no mesh point satisfies both the limb and FOV tests simultaneously), the code falls back to the single angularly-closest point via `[min_angle, index_point_science_mesh] = min(dot_prod_angle_array)`, so a measurement always registers at least one observed point rather than silently observing nothing.

5.15.2.4 Surface Point Geodetic Mapping

$$(r_i, \lambda_i, \phi_i) = \texttt{cspice_reclat}(\vec{p}_i), \qquad \lambda_i^\circ = \lambda_i \cdot \frac{180}{\pi}, \quad \phi_i^\circ = \phi_i \cdot \frac{180}{\pi} \tag{5.46}$$

where $\vec{p}_i$ is the i-th unit-sphere science point position, and (r_i, λ_i, ϕ_i) are the SPICE-returned radius, longitude, and latitude (radians) of that point, converted to degrees $(\lambda_i^\circ, \phi_i^\circ)$.

This is computed once per point at construction time in the loop over `obj.pos_points`, storing results in `obj.spherical_points`, via `[radius, lon, lat] = cspice_reclat(obj.pos_points(i,:)')` followed by `rad2deg(lon)`, `rad2deg(lat)`. This is the identical technique used in `True_SC_Camera` to map surface science/target points into longitude/latitude for coverage plotting, confirming the two classes share a common geodetic-mapping convention.

5.15.2.5 Monostatic Coverage Fraction

$$C(t) = \frac{100}{N_{\text{points}}}\, N_{\text{obs}}(t) \tag{5.47}$$

where N_{points} is the total number of mesh points on the target (`obj.num_points`) and $N_{\text{obs}}(t)$ is the cumulative count of distinct points observed at least once by simulation time t (`obj.monostatic_num_point_observed`).

This fraction is computed directly in `func_visualize_SC_remote_sensing_coverage_during_sim` as `(100/obj.num_points)*obj.monostatic_num_point_observed` (and its time series), giving a scalar coverage-percentage metric that is the class's principal science-return figure of merit. The

per-point observation tally feeding $N_{\text{obs}}(t)$, `obj.monostatic_observed_point`, is incremented inside `func_update_true_SC_remote_sensing_Generic` (or a mission-specific equivalent) by looping over `index_point_science_mesh` — for each index, `obj.monostatic_num_point_observed` is incremented only the first time that point transitions from an observed-count of zero, and `obj.monostatic_observed_point` is incremented unconditionally every time the point is seen again.

5.15.3 CML 2–5 Suitability

5.15.3.1 What is included and correctly implemented

The class provides a clean, reusable scaffold for trading remote-sensing instrument concepts at a point-design level, and its `'Generic'` geometry test is implemented directly in this file rather than delegated externally.

- **Self-contained, verifiable coverage geometry.** The `'Generic'` mode's limb-angle visibility test and boresight field-of-view test are both implemented directly in `func_update_true_SC_remote_sensing_Generic` within this file, so the geometric fidelity of the default coverage determination can be fully verified from this class alone, rather than depending on an unreviewed external function.
- **Robust fallback behavior.** When no mesh point satisfies both the limb and FOV tests, the code falls back to recording the single angularly-closest point rather than silently registering zero observations for that step, which avoids spuriously flat coverage curves when the instrument is pointed only approximately at the target.
- **Configurable geometry and pluggable coverage mode.** It offers configurable body-frame location/orientation and a single-scalar field of view, a pluggable mode selector (`'Generic'` vs. `'mission specific'`) so different coverage algorithms can be swapped without touching the bookkeeping code, and a time-gated measurement cadence (`measurement_wait_time`) that gives a first-order instrument duty cycle.
- **Power/data interfaces and mode-dependent power lookup.** Power and data interfaces are correctly wired into `True_SC_Power` and `True_SC_Data_Handling` at construction and each timestep, and a spacecraft-mode-dependent power lookup (`instantaneous_power_consumed_per_SC_mode`, and separately `instantaneous_data_rate_generated_per_SC_mode`) allows quick power/data-rate-profile trades across ConOps modes.
- **Coverage metric and visualization.** The cumulative monostatic coverage metric (percent of mesh points observed) and its longitude/latitude and 3-D visualizations reuse the same SPICE-based geodetic conversion as the camera class, giving a consistent, sufficiently accurate coverage-fraction figure of merit for trade-space studies.
- **Consistent integration with system conventions.** Executive gating, health/temperature placeholders, and storage/plotting/video infrastructure follow the same conventions as the other True_Sensors_Actuators classes, so it integrates consistently into the broader system-level power, data, and coverage budgets expected at CML 2–5.

5.15.3.2 What is not included / gaps

Several second-order geometry effects, data-path, and noise-modeling limitations exist.

- **No range, resolution, swath, or self-occlusion modeling.** The limb and FOV tests are purely angular (no slant-range, incidence-angle, ground-sample-distance, swath-width, or push-broom scan-line computation), and the limb test uses the target's spherical radius rather than the actual shape model, so it does not capture self-shadowing from an irregular body or terrain occlusion of a nominally visible point.

- **Constant-rate data model; unused per-sample path.** Data generation is modeled only as a constant rate in kbps rather than a per-measurement data-volume equation (contrast with the camera's per-image size calculation); the alternate "per-sample" data path implied by `instantaneous_data_type`/`instantaneous_data_generated_per_sample` is declared in the property list and comments but never assigned or exercised anywhere in this file, indicating an incomplete/unused code path.

- **No sensor noise or knowledge-error propagation.** There is no modeling of sensor noise, integration time, spectral/radiometric response, or attitude-knowledge-error propagation into the coverage test.

- **Fallback point may still be out of true view.** The nearest-point fallback (when no mesh point passes both tests) always registers an observation even when the instrument's actual footprint does not intersect the target at all, which could overstate coverage in edge cases where the spacecraft is pointed well off-target.

- **Acceptable for CML 2–5, not CML 6+.** None of these are problems for CML 2–5 use, where first-order geometric coverage and rate-based resource accounting are acceptable fidelity, but they would need to be added before the model could support CML 6+ analyses requiring real link-budget-driven data rates, resolution, or incidence-angle-dependent performance.

5.16 True_SC_RPS

True_SC_RPS models a Radioisotope Power System (RPS) — an RTG/MMRTG-style radioisotope thermoelectric generator — as one of the electrical power sources aboard the spacecraft, alongside solar arrays. Unlike a solar panel, its output does not depend on Sun visibility, eclipse state, incidence angle, or spacecraft attitude; instead, the electrical power it can produce decays monotonically and deterministically with elapsed time since a specified Beginning-Of-Life (BOL) date, driven by the radioisotope's (effective) half-life. Each instance tracks a beginning-of-life power level (power_BOL), a half-life (half_life, defaulting to 87.7 years, the physical half-life of Pu-238), and a BOL date (BOL_date, defaulting to mission start if unspecified), from which it computes the fraction of BOL power still available and hence the present maximum and instantaneous power output.

The class follows the same constructor/store/main pattern as other True_Sensors_Actuators hardware classes. On construction and at every simulation step, func_update_RPS_instantaneous_power_generated recomputes the decay fraction and resulting power, and the object registers itself with True_SC_Power (via func_initialize_list_HW_energy_consumed / func_initialize_list_HW_energy_generated and their corresponding func_update_... calls) so the RPS's generated power and its own fixed housekeeping power draw (instantaneous_power_consumed) feed into the spacecraft's overall power/energy balance. It also registers a data-generation rate with True_SC_Data_Handling, consistent with other hardware units that produce housekeeping telemetry.

5.16.1 Inputs / Outputs

5.16.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name, instantaneous_power_consumed, instantaneous_data_rate_generated, mass, power_BOL, and optionally half_life (defaults to 87.7 years, Pu-238) and BOL_date_string (defaults to mission.true_time.t_initial_date).
- mission.true_time**:** date, read every step to compute elapsed_time since BOL_date and hence the radioisotope decay fraction; t_initial_date read once at construction if no explicit BOL_date_string is given.
- **Internal state only:** unlike True_SC_Solar_Panel, the RPS reads no attitude (True_SC_ADC), orbit/eclipse (True_SC_Orbit_Propagation), or solar-system truth — its power output decays monotonically with elapsed time since Beginning Of Life and does not depend on Sun visibility, incidence angle, or spacecraft attitude; the only other dependency is obj.health, which zeroes instantaneous_power_generated when unhealthy.

5.16.1.2 Outputs

- **Internal / stored outputs:** instantaneous_power_generated, maximum_power, decay_fraction are computed each step and stored in obj.store.
- **Reported to** True_SC_Power**:** instantaneous_power_generated and instantaneous_power_consumed registered/updated via the same generated/consumed power interface used by True_SC_Solar_Panel.
- **Reported to** True_SC_Data_Handling**:** instantaneous_data_rate_generated registered/updated via func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated.

- **Reported to `True_SC_Body` (mass properties):** `obj.mass` is set from `init_data.mass` at construction; unlike `True_SC_Solar_Panel`, no corresponding wiring was found in `True_SC_Body.func_update_SC_body_mass` that folds an RPS mass contribution into the spacecraft's aggregate mass/COM/inertia (only `true_SC_solar_panel` and `true_SC_fuel_tank` are consumed there) — the RPS's mass is a stored property but, as far as could be verified, is not currently read by any other class.
- No external class writes back into this class's own properties.

5.16.2 Mathematical Equations

5.16.2.1 Radioisotope Decay-Based Power Output

The RPS models its available power as decaying with elapsed time since Beginning-Of-Life according to a half-life law:

$$P_{max}(t) = P_{BOL} \cdot 2^{-\frac{t-t_{BOL}}{T_{1/2}}} \tag{5.48}$$

where P_{BOL} is the electrical power output at Beginning-Of-Life, t is the current mission true time, t_{BOL} is the Beginning-Of-Life epoch, $T_{1/2}$ is the (effective) half-life expressed in the same time units as $t - t_{BOL}$, and $P_{max}(t)$ is the maximum power the unit could produce at time t, irrespective of its health state. This is mathematically equivalent to the standard exponential radioactive decay law $P(t) = P_{BOL}\, e^{-\lambda(t-t_{BOL})}$ with decay constant $\lambda = \ln(2)/T_{1/2}$, simply expressed in base-2 half-life form.

In the code, elapsed time is computed as `elapsed_time = max(mission.true_time.date - obj.BOL_date, 0)` (seconds, clamped to be non-negative), and the half-life is converted from years to seconds via `half_life_sec = obj.half_life * 365.25 * 86400`. The decay fraction is computed directly as `decay_fraction = 2^( -elapsed_time / half_life_sec )`, and the maximum power as `maximum_power = power_BOL * decay_fraction`. The comments in the code note that `half_life` may be set to the radioisotope's physical half-life (e.g., 87.7 years for Pu-238) or to an empirical "effective" half-life that also lumps in thermocouple degradation, as is common practice for low-fidelity RPS power budgeting (e.g., for GPHS-RTG or MMRTG designs).

5.16.2.2 Instantaneous Power Generated (Health Gating)

The power actually delivered at a given time step is the decayed maximum power, gated by a binary health flag:

$$P_{gen}(t) = \begin{cases} P_{max}(t), & \text{health} = 1 \\ 0, & \text{health} \neq 1 \end{cases} \tag{5.49}$$

where $P_{gen}(t)$ is the instantaneous electrical power actually produced by the RPS at time t, and health is an integer state (1 = switched on/nominal; 0 = switched off) that is a property of the object but is not itself computed or degraded anywhere within this file. In the code this corresponds to the `if obj.health == 1` branch in `func_update_RPS_instantaneous_power_generated`, which sets `obj.instantaneous_power_generated = obj.maximum_power` when healthy, or `0` otherwise.

No thermal-to-electric conversion efficiency, thermal model, or output-current/voltage model is computed in this file; `power_BOL` is treated as a direct, user-specified electrical output number, and `temperature` is only initialized to a fixed 10 °C constant and never updated.

5.16.3 CML 2–5 Suitability

5.16.3.1 What is included and correctly implemented

The class correctly captures the single first-order physical effect that dominates RPS power budgeting at the concept/point-design stage.

- **Half-life decay physics, decoupled from geometry.** It models monotonic power decline with time due to radioisotope decay (and, if desired, lumped thermocouple degradation) via a half-life law, independent of orbital/attitude/eclipse geometry, which correctly distinguishes RPS behavior from solar array behavior.
- **Configurable BOL parameters.** It supports a configurable BOL power level, a configurable (defaulting to physically correct Pu-238) half-life, and a configurable BOL date, defaulting sensibly to mission start if not specified, which is adequate fidelity for trade-space and point-design power budget studies across mission phases.
- **Internally consistent decay calculations.** The decay-fraction and maximum-power/instantaneous-power-generated calculations are internally consistent (elapsed time clamped at zero, decay fraction bounded in $[0, 1]$ at $t \geq t_{BOL}$).
- **System wiring and telemetry.** The class is properly wired into the spacecraft power ledger (`True_SC_Power`) and data-handling ledger (`True_SC_Data_Handling`) so its output participates in overall spacecraft resource accounting, and it stores time-history data (`instantaneous_power_generated`, `maximum_power`, `decay_fraction`) for later analysis, consistent with the pattern used elsewhere in `True_Sensors_Actuators`.
- **Overall fidelity match to CML 2–5.** This level of fidelity is appropriate for and sufficient to support CML 2–5 trade studies (e.g., comparing RPS sizing/BOL power/number of units against mission duration and expected end-of-life power margins).

5.16.3.2 What is not included / gaps

None of the following omissions are unusual or problematic for CML 2–5 scope, but they bound the class's applicability to higher-fidelity analyses.

- **No health-degradation or fault model.** There is no model of how `health` itself changes — it is fixed at 1 on construction and never updated within this file, so any RPS failure, degraded-mode, or fault-injection behavior would have to be driven externally; there is no logic here for random or scheduled failures.
- **No thermal model.** There is no thermal model: `temperature` is initialized to a constant 10 °C and never recomputed, so there is no coupling between RPS waste-heat output and spacecraft thermal balance, even though real RPS units are significant heat sources.
- **No conversion-efficiency or electrical-output model.** There is no explicit thermoelectric conversion efficiency, output voltage/current, or degradation-vs-temperature model; `power_BOL` is an externally supplied constant rather than derived from thermal input power and a conversion efficiency, so cross-checking against a physical Watts-of-heat-in/Watts-of-electricity-out budget is not possible from this class alone.

- **Deterministic decay, no anomalies.** The half-life-based decay is deterministic and smooth, with no stochastic or lot-to-lot variability, and no provision for step-change anomalies (e.g., a suddenly failed thermocouple string).
- **Applicability boundary.** These omissions mean the class should not be relied upon for flight-fidelity power-system verification, fault-management analysis, or thermal-balance closure.

5.17 True_SC_Science_Processor

True_SC_Science_Processor is a generic, mode-dispatching hardware container in the same architectural family as True_SC_Generic_Sensor: on its own, the base class file implements only the standard MuSCAT actuator/sensor lifecycle — name, health, temperature, and executive-flag initialization; pre-allocated storage arrays for instantaneous power consumed, instantaneous data rate generated, and instantaneous data removed per sample; registration of those quantities with the spacecraft's true_SC_power and true_SC_data_handling classes at construction; and a mode_true_SC_science_processor_selector switch (currently supporting only the value 'mission specific') that looks up and calls an externally supplied constructor function (func_true_SC_science_processor_constructor_<mission>) and per-step update function (func_true_SC_science_processor_<mission>) by name via str2func. No EP-thruster, beam, or target-interaction physics is written into this file itself.

The class's role as an electric-propulsion science/kinetic-impactor processor comes entirely from what a mission plugs into this dispatch mechanism. True_SC_Science_Processor itself lives in MuSCAT_Matlab_v2, but its one implemented mission plugin lives in a separate, firewalled mission repository, which is instantiated by that mission's own mission script and reaches this class via a path-added copy of MuSCAT_Matlab_v2. In that plugin (invoked from func_main_true_SC_science_processor through the 'mission specific' case), the per-step function models an ion beam emitted by a co-indexed True_SC_EP_Thruster striking the target body's shape model (via a helper beam/target-intersection function), computes obj.data.momentum_transfer_efficiency, obj.data.EP_thruster_force_on_target, and obj.data.EP_thruster_torque_on_target into the free-form obj.data struct that this base class merely allocates and passes through, and then — after sanity-checking momentum_transfer_efficiency for bounds/NaN — directly accumulates EP_thruster_force_on_target/EP_thruster_torque_on_target into mission.true_target{1}.control_force/control_torque itself (there is no separate True_Target_Computed method performing this accumulation; an older codebase snapshot, MuSCAT_Matlab_v2_OLD, did have a dedicated True_Target_Computed.func_update_SB_control_force method that looped over science-processor hardware units, but the current True_Target_Computed.m has no such method or reference to this class — the mission plugin performs the accumulation inline instead). Of the two target-side properties, only control_force is actually integrated into the target's translational dynamics (inside func_orbit_inertial_static_SB_v6_Sun_centered.m); control_torque is accumulated and stored but never integrated, since the target's attitude is currently kinematically prescribed rather than dynamically propagated. True_SC_Science_Processor is therefore the SC-side handle/interface object through which the mission's EP-beam-on-target model exposes its force/torque outputs to the target dynamics; the momentum-transfer physics itself lives in the mission plugin, not in this base file.

5.17.1 Inputs / Outputs

5.17.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name, instantaneous_power_consumed, instantaneous_data_rate_generated, instantaneous_data_removed_per_sample, mode_true_SC_science_processor_selector (currently only 'mission specific' is implemented), and optionally data, mode_selector_name.
- **Mission-specific plugin functions (resolved by name via str2func, not a MuSCAT class):** the constructor looks up func_true_SC_science_processor_constructor_

<mode_selector_name> for additional mission-specific initialization, and func_main_true_SC_science_processor dispatches each step to func_true_SC_science_processor_<mode_selector_name>. No fixed truth dependency (ADC/orbit/power) is hard-coded in the base class itself — all cross-class reads happen inside these externally supplied plugin functions.

- Software_SC_Executive **(same spacecraft):** this_sc_mode_value, read if obj.data.instantaneous_power_consumed_per_SC_mode is defined.
- **Executive gating:** flag_executive is forced to 1 unconditionally every step directly by main_v3.m (not by a mission executive class), i.e. the Science Processor always runs when hardware exists for it, unlike most other instruments which require an executive command.

5.17.1.2 Outputs

- **Internal / stored outputs:** flag_executive, instantaneous_data_rate_generated, instantaneous_data_removed_per_sample, instantaneous_power_consumed are stored in obj.store each step, plus any mission-specific storage fields added via an optional func_update_true_SC_science_processor_store_<mission> plugin.
- **Reported to** True_SC_Power**:** instantaneous_power_consumed registered via func_initialize_list_HW_energy_consumed at construction; main-loop updates happen inside the mission-specific plugin.
- **Reported to** True_SC_Data_Handling**:** instantaneous_data_rate_generated and instantaneous_data_removed_per_sample registered via func_initialize_list_HW_data_generated/func_initialize_list_HW_data_removed at construction.
- **Reported to** True_Target_Computed **(mission-specific, in a separate firewalled mission repository):** the mission's science-processor plugin computes obj.data.momentum_transfer_efficiency/EP_thruster_force_on_target/torque and then itself accumulates EP_thruster_force_on_target/torque directly into mission.true_target{1}.control_force/control_torque — a live, confirmed cross-class write (see True_Target_Computed for the full accumulation/reset/consumption details). The current True_Target_Computed.m within MuSCAT_Matlab_v2 has no method that itself reaches into this class; the accumulation happens from the mission plugin's side instead, so True_SC_Science_Processor within MuSCAT_Matlab_v2 alone (i.e. without that mission plugin) has no consumer for these fields.
- flag_executive is force-set to 1 every step by main_v3.m itself (the top-level simulation loop, not another instrument/software class) immediately before calling func_main_true_SC_science_processor. No other external class writes back into this class's own properties; mission-specific physics outputs are written into obj.data by the class's own dispatched plugin functions, which (for at least one mission) also reach out to write into True_Target_Computed as described above.

5.17.2 Mathematical Equations

This base class file (True_SC_Science_Processor.m) contains no equations of its own — it only allocates obj.data, dispatches to a mission-specific function, and manages power/data/storage bookkeeping. The momentum-transfer, force, and torque physics referenced by True_Target_Computed are computed entirely inside a mission-specific plugin, living in a separate firewalled

mission repository, that is reached through this class's dispatch mechanism. Since the interface this class exposes to `True_Target_Computed` is the object of interest, the equations actually populating `obj.data.momentum_transfer_efficiency`, `obj.data.EP_thruster_force_on_target`, and `obj.data.EP_thruster_torque_on_target` in that plugin are presented below, with an explicit note that they live outside `True_SC_Science_Processor.m` itself and outside the `MuSCAT_Matlab_v2` core codebase.

5.17.2.1 Beam Footprint Radius and Gaussian Divergence Width

$$R_{\phi_0} = \tan(\phi_0)\, d, \qquad \sigma = \frac{R_{\phi_0}}{\sqrt{-2\ln(1-p)}} \tag{5.50}$$

R_{ϕ_0} is the beam footprint radius at the target silhouette plane; ϕ_0 is the beam divergence half-angle; d is the distance from the thruster to the silhouette plane measured along the beam axis; σ is the Gaussian standard deviation (assumed equal in both transverse directions) such that a circle of radius R_{ϕ_0} encloses a specified fraction p of the beam's total momentum.

In `func_compute_beam_on_target.m` this is `phi0_radius = tan(divergence_angle) * distance_to_plane` (with `divergence_angle = beam_state.Phi0`), and, for the Gaussian beam fidelity, `sigma_x = phi0_radius / sqrt(-2 * log(1 - pct))` with `pct = beam_state.beam_radius_pct`. For the uniform-beam fidelity, `mesh_radius = phi0_radius` directly (no Gaussian tail). These parameters originate from the co-indexed `True_SC_EP_Thruster` object, not from `True_SC_Science_Processor` itself.

5.17.2.2 Momentum Transfer Efficiency

$$\eta = \sum_{i \in \text{lit}} w_i, \qquad w_i = \frac{I_i\, r_i}{\sum_j I_j\, r_j} \tag{5.51}$$

η is the momentum transfer efficiency (the fraction of the beam's total momentum that lands on the illuminated, target-facing part of the silhouette); I_i is the raw beam intensity (uniform or Gaussian) sampled at mesh point i on a polar grid centered on the beam/silhouette-plane intersection; r_i is the radial coordinate of point i (the polar-area weighting factor, since polar-grid cell area scales with radius); the sum in the denominator runs over the entire beam mesh, while the numerator sum runs only over mesh points i that fall inside a lit (target-facing) triangular face of the target's shape model.

This is `intensity_weighted = intensity.*r_grid; intensity_norm = intensity_weighted / sum(sum(intensity_weighted))`, followed by `intensity_norm_in = intensity_norm(in)` (where `in` flags mesh points located, via `pointLocation` on a `triangulation` of the lit faces, inside the silhouette) and `obj.data.momentum_transfer_efficiency = sum(intensity_norm_in)`. For the point-beam fidelity, this reduces to the unconditional constant `obj.data.momentum_transfer_efficiency = 1` (full intercept assumed, with no pointing check); for a beam pointed away from the target, `func_zero_beam_output.m` sets it to 0. `True_Target_Computed.func_update_SB_control_force` treats values outside $[0, 1]$ or `NaN` as warning conditions that should never occur.

5.17.2.3 EP Thruster Force on Target

$$\vec{F}_{\text{target}} = F_{\text{mag}} \sum_{i \in \text{lit}} w_i\, \hat{u}_i \tag{5.52}$$

$\vec{F}_{\text{target}}$ is the net force delivered to the target body by the intercepted ion beam; $F_{\text{mag}} = \|\vec{F}_{\text{thruster,inertial}}\|$ is the thrust magnitude of the emitting EP thruster; $\hat{u}_i$ is the unit vector, in the inertial frame, from the thruster to intercepted impact point i; w_i is the same intensity/area weight as above.

This is `thrust_magnitude = beam_state.thrust_magnitude` (from `norm(ep_thruster.force_inertial)`) and `obj.data.EP_thruster_force_on_target = thrust_magnitude * sum(intensity_norm_in.*thrust_vectors, 1)`, where `thrust_vectors` are the per-impact-point unit vectors from the thruster location to each impact point. For the point-beam fidelity, this reduces to `obj.data.EP_thruster_force_on_target = -mission.true_SC{i_SC}.true_SC_ep_thruster{i_HW}.force_inertial` (the full reaction of the force applied to the spacecraft, by Newton's third law, with no torque).

5.17.2.4 EP Thruster Torque on Target About the Target Centroid

$$\vec{\tau}_{\text{target}} = F_{\text{mag}} \sum_{i \in \text{lit}} w_i \, (\vec{r}_i - \vec{r}_{\text{COM}}) \times \hat{u}_i \tag{5.53}$$

$\vec{\tau}_{\text{target}}$ is the net torque delivered to the target body about its center of mass, expressed in the inertial frame; $\vec{r}_i$ is the inertial-frame position of impact point i; $\vec{r}_{\text{COM}}$ is the target's constant-density centroid (precomputed once in the target body frame and rotated into the inertial frame each call).

This is `COM2BI = beam_impact_points_J2000 - COM_SB_IF`; `obj.data.normalized_torque = sum(cross(COM2BI, intensity_norm_in.*thrust_vectors, 2), 1)`; and `obj.data.EP_thruster_torque_on_target = obj.data.normalized_torque * thrust_magnitude`. Per the in-code assumption comment, ions are modeled as fully absorbed at the silhouette plane (through the centroid) rather than at the true near-surface impact point, so there is no incidence-angle/sputtering/reflection dependence. The torque is explicitly computed and stored in the inertial frame; per an accompanying comment it is not currently integrated into any target attitude dynamics, since `True_Target_Computed` currently prescribes the target's rotation kinematically rather than dynamically. For the point-beam fidelity, torque is identically zero.

5.17.3 CML 2–5 Suitability

5.17.3.1 What is included and correctly implemented

The base `True_SC_Science_Processor` class correctly implements the standard MuSCAT hardware lifecycle expected at CML 2–5.

- **Standard lifecycle and system-wiring.** It implements health/temperature/executive-flag initialization, storage-array pre-allocation, and registration of instantaneous power consumption, data-rate generation, and data removal with `true_SC_power` and `true_SC_data_handling` both at construction and each step through the mission-specific function.
- **Executive/health gating.** The `flag_executive/health` gating in `func_main_true_SC_science_processor` correctly disables the mission-specific update (and thus any EP-beam-on-target computation) when the unit is switched off or not commanded, which is an appropriate first-order fault/idle representation for trade studies.
- **Clean, mission-agnostic extension point.** Architecturally, routing all mission-specific physics through a single `obj.data` struct and a name-based dispatch (`'mission specific'`) is a clean, appropriately lightweight extension point for CML 2–5: it lets a mission implement a genuinely useful, medium-fidelity EP-beam-on-target model — silhouette-based beam/target

intersection, uniform or Gaussian intensity weighting, momentum transfer efficiency, and a resulting force and torque on the target computed by exact polygon/triangulation intersection — and expose exactly the three fields that `True_Target_Computed.func_update_SB_control_force` needs, without requiring the base class or the target class to know anything about beam geometry.

- **Sound systems-engineering practice.** This decoupling is sound systems-engineering practice for a point-design trade tool and is sufficient fidelity for scoping deflection/kinetic-impact or EP-beam momentum-transfer concepts at CML 2–5.

5.17.3.2 What is not included / gaps

The physics itself, and its validation, live entirely outside this base class.

- **No EP-thruster physics in the base class itself.** `True_SC_Science_Processor.m` itself contains no EP-thruster, beam, or momentum-transfer physics; the class name and its generic properties (`instantaneous_power_consumed`, `instantaneous_data_rate_generated`, `instantaneous_data_removed_per_sample`) describe a generic instrument/data-processing bookkeeping object, and the actual physics referenced by `True_Target_Computed` exists only in an externally supplied, by-convention-named plugin function that this class locates at runtime via `exist()`/`str2func()`.

- **Silent failure if the mission plugin is missing.** If that mission-specific constructor or update function is absent, the constructor only prints a warning and silently leaves `obj.data` unpopulated rather than raising an error, so a mission that forgets to implement the plugin (or misnames it) would have an uninitialized `obj.data.momentum_transfer_efficiency`/`EP_thruster_force_on_target`/`EP_thruster_torque_on_target` the first time `True_Target_Computed` tries to read them (in the per-step update path, by contrast, an undefined mission mode does raise a hard error).

- **No internal validation of physics values.** The base class also performs no validation of the force/torque/efficiency values it is passed through `obj.data` — the range and NaN sanity checks live entirely outside this class, in `True_Target_Computed`.

- **No multi-thruster support within a single instance.** There is no built-in support for multiple EP thrusters sharing one science-processor instance; the data contract carries exactly one force/torque/efficiency triplet per hardware unit, so a spacecraft with several EP thrusters acting on the target needs one `True_SC_Science_Processor` instance per thruster, which is a workable but implicit convention rather than an enforced one.

5.18 True_SC_Science_Radar

The True_SC_Science_Radar class models an onboard science radar instrument used to build up global (or regional) monostatic surface coverage of a target body over the course of a simulation. Rather than propagating detailed electromagnetic pulse physics, the class represents the target surface as a discretized mesh of unit-sphere sample points (loaded via func_load_science_points_v2, with mesh density selectable from a menu of preset point counts) and, at each measurement epoch, determines which mesh points fall within the radar's field of view of the spacecraft boresight. Each observed point is tallied to build a running record of how many times every surface location has been "seen," from which a global percentage coverage metric is derived. The class interfaces with the spacecraft attitude/orbit propagation to obtain the current geometry, with a mode_true_SC_science_radar_selector switch that dispatches either to a 'Generic' coverage test implemented directly in this file (func_update_true_SC_science_radar_generic) or to a mission-specific handler resolved by name, and with the True_SC_Power and True_SC_Data_Handling classes to which it reports instantaneous power draw and data volume generated by each radar measurement.

Operationally, the instrument is enabled by an executive flag (flag_executive) and health flag, and only takes a new measurement once a minimum revisit/measurement wait time has elapsed since the previous one. When a measurement is triggered, the mode-specific function computes the angle between the spacecraft-to-target line of sight and each mesh point, selects either the single closest point (if field_of_view is set to zero) or all points within the specified angular field of view, updates the monostatic observation counters, and pushes an instantaneous data volume into the spacecraft's data handling ledger. The class also carries a visualization routine that plots the 3D spacecraft/target geometry, the boresight orientation vector, the mesh points colored by number of times observed, a 2D longitude/latitude coverage map, and a time history of percentage coverage, and can optionally record this to an MPEG-4 video for post-processing review.

5.18.1 Inputs / Outputs

5.18.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name, instantaneous_power_consumed, mode_true_SC_science_radar_selector ('Generic' or 'mission specific'), instantaneous_data_rate_generated, measurement_wait_time, location, orientation, field_of_view, flag_show_radar_plot, num_points, and optionally data.
- True_SC_ADC **(same spacecraft):** rotation_matrix, used only in the visualization routine (func_visualize_SC_radar_coverage_during_sim) to draw the boresight orientation; the 'Generic' coverage test itself does not read attitude (see Mathematical Equations below).
- True_SC_Orbit_Propagation **(same spacecraft):** position_relative_target and index_relative_target, used for the field-of-view/nadir-point geometric test and for plotting.
- mission.true_target{i_target} **(target body, mission-level):** radius and rotation_matrix, used to rotate the mesh unit vectors for the angular-separation test and plots.
- Software_SC_Executive **(same spacecraft):** this_sc_mode_value/this_sc_mode read for mode-indexed power and plot titling; time gates the visualization refresh.
- mission.true_time**:** time, gates repeat measurements against measurement_wait_time.

5.18.1.2 Outputs

- **Internal / stored outputs:** `monostatic_observed_point`, `monostatic_num_point_observed`, `flag_executive`, `instantaneous_data_rate_generated`, `instantaneous_power_consumed` are stored in `obj.store`.
- **Reported to `True_SC_Power`:** `instantaneous_power_consumed` registered/updated via `func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed`.
- **Reported to `True_SC_Data_Handling`:** `instantaneous_data_rate_generated` registered/updated via `func_initialize_list_HW_data_generated/func_update_instantaneous_data_generated`.
- **Read by shared SC visualization/plotting utilities:** draw the radar's boresight orientation and coverage mesh on spacecraft 3D attitude plots, read-only.
- **External executive write (DART mission):** `func_update_software_SC_communication_DART.m` directly sets `mission.true_SC{i_SC}.true_SC_science_radar{i_HW}.flag_executive = 0` to block radar operation during a conflicting DTE-communication mode — the one documented case of an external class writing into this class's own property, limited to the executive gating flag. No external class modifies this radar's computed measurement/coverage outputs.

5.18.2 Mathematical Equations

5.18.2.1 Boresight-to-Surface-Point Angular Separation

$$\theta_i = \arccos(\hat{p}_i \cdot \hat{s}), \qquad i = 1, \ldots, N_{\text{points}} \tag{5.54}$$

where $\hat{p}_i$ is the unit vector (in the target-centered rotating frame) to the i-th surface mesh point, $\hat{s}$ is the unit vector from the target center to the spacecraft, and θ_i is the angular separation between the point's local vertical and the spacecraft line of sight, expressed in degrees. This angle is used as a proxy for whether a given surface point lies within the radar's field of view, given the assumption that the radar boresight is aligned with (or its footprint is centered on) the sub-spacecraft point.

In the code, this is computed in `func_update_true_SC_science_radar_generic` (the `'Generic'` mode handler, implemented directly in `True_SC_Science_Radar.m`) as `dot_prod_angle_array = real(acosd(this_pos_points * SC_pos_normalize'))`, where `this_pos_points` are the mesh unit vectors rotated into the target frame via `mission.true_target{i_target}.rotation_matrix`, and `SC_pos_normalize` is computed with `func_normalize_vec` from the spacecraft-minus-target position vector. The corresponding property `obj.field_of_view` (in degrees) sets the threshold applied to θ_i.

5.18.2.2 Field-of-View / Nadir-Point Selection Criterion

$$\mathcal{I} = \begin{cases} \{\, \arg\min_i \theta_i \,\}, & \text{FOV} = 0 \\ \{\, i : \theta_i \le \text{FOV} \,\}, & \text{FOV} > 0 \end{cases} \tag{5.55}$$

where $\mathcal{I}$ is the index set of surface mesh points considered "observed" during the current measurement, and FOV is the radar half-angle field of view in degrees (property `field_of_view`). When FOV $= 0$ the model degenerates to a single-point (nadir/closest-point) observation, representative

of a narrow-beam altimeter; when FOV > 0 it represents a conical footprint on the surface, representative of a wider-beam imaging radar swath.

This is implemented directly with MATLAB's `min` and `find` functions: `[min_angle, index_point_science_mesh] = min(dot_prod_angle_array)` for the zero-FOV case, and `index_point_science_mesh = find(dot_prod_angle_array <= obj.field_of_view)` otherwise, both inside `func_update_true_SC_science_radar_generic`.

5.18.2.3 Monostatic Coverage Accumulation

$$c_i(t_{k+1}) = c_i(t_k) + \mathbb{1}[i \in \mathcal{I}(t_k)]\,, \qquad N_{\text{obs}}(t_{k+1}) = N_{\text{obs}}(t_k) + \sum_i \mathbb{1}[c_i(t_k) = 0 \ \wedge \ i \in \mathcal{I}(t_k)] \quad (5.56)$$

where c_i is the cumulative number of times mesh point i has been observed, and N_{obs} is the running count of distinct mesh points observed at least once. The fractional surface coverage reported by the visualization and stored time history is $100\, N_{\text{obs}}/N_{\text{points}}$.

These quantities correspond to the properties `monostatic_observed_point` (vector c_i) and `monostatic_num_point_observed` (scalar N_{obs}), both updated in the loop over `index_point_science_mesh` in `func_update_true_SC_science_radar_generic`, and the percentage coverage is computed inline in `func_visualize_SC_radar_coverage_during_sim` as `(100/obj.num_points)*obj.monostatic_num_point_observed`.

5.18.2.4 Instantaneous Data Volume Generated per Measurement

$$\Delta D = \dot{D}\,\Delta t \quad (5.57)$$

where ΔD is the data volume (kb) generated by the radar during one simulation time step in which a measurement is taken, $\dot{D}$ is the instantaneous data rate (kbps), and Δt is the simulation time step. This models the radar as a constant-rate data source that only accumulates data when an actual measurement is executed (i.e., only when the elapsed time since the previous measurement exceeds `measurement_wait_time`).

This corresponds to the property `instantaneous_data_rate_generated` (kbps) and is realized in the shared data-handling routine `func_update_instantaneous_data_generated` (in `True_SC_Data_Handling.m`), called from `func_update_true_SC_science_radar_generic`, as `this_instantaneous_data_generated = equipment.instantaneous_data_rate_generated * mission.true_time.time_step`. Instantaneous power draw is handled analogously through `obj.instantaneous_power_consumed`, either as a fixed constant or, if `obj.data.instantaneous_power_consumed_per_SC_mode` is defined, indexed by the current spacecraft mode.

5.18.3 CML 2–5 Suitability

5.18.3.1 What is included and correctly implemented

The class provides a reasonable first-order representation of radar surface coverage suitable for early trade-space and point-design studies.

- **Mesh-based geometric coverage test.** It discretizes the target surface with a selectable-density point mesh and computes, at each measurement epoch, a purely geometric angular test (spacecraft-to-target line of sight versus each mesh point) to decide which points are "observed," supporting both a narrow nadir-point (altimeter-like) mode and a wider conical field-of-view (imaging-radar-like) mode via a single threshold parameter.

- **Coverage accumulation metric.** It correctly accumulates monostatic hit counts per surface point and a running total-coverage metric, which is exactly the kind of coverage-percentage figure of merit needed at CML 2–5 to compare instrument/orbit/pointing design options.
- **System-level integration.** The class is properly wired into the broader spacecraft simulation: it gates measurements on an executive/health flag and a measurement revisit time, and reports instantaneous power draw to `True_SC_Power` and instantaneous data volume to `True_SC_Data_Handling` using the same rate-times-time-step convention used by other instruments in the codebase.
- **Mission-specific extension point.** Beyond the `'Generic'` coverage test implemented directly in this file, a `'mission specific'` case dispatches by name (via `str2func`) to an externally supplied, mission-named handler, so the interface needed to add mission-specific radar behavior already exists without requiring any change to this base class.
- **Analyst visualization.** The built-in 3D/2D visualization and optional video capture of coverage evolution provide useful analyst insight without being required for the core numerics.

5.18.3.2 What is not included / gaps

The model contains no actual radar physics.

- **No range, resolution, or link-budget physics.** There is no slant-range or range equation, no computation of incidence angle relative to local surface normal (the angular test only uses the spacecraft-target line of sight versus mesh point position, not a true grazing/incidence angle), no swath width or ground resolution computation, no link budget (transmit power, antenna gain, SNR, backscatter coefficient), no pulse timing, bandwidth, or along-/cross-track resolution, and no distinction between different radar techniques (SAR, altimeter, scatterometer) beyond the ad hoc field-of-view parameter.
- **Data/power decoupled from geometry.** Data volume and power are both modeled as simple constant (or mode-indexed constant) rates multiplied by time step, with no dependence on range, swath size, or number of points actually observed in a given measurement, so the data/power model is not tied to the geometric coverage computation at all.
- **Coverage-test simplifications.** The mesh-based coverage test assumes the point of interest is defined purely by the spacecraft-to-target-center vector rather than an actual boresight/pointing vector derived from spacecraft attitude (the `orientation` property is stored and used for plotting only, not in the field-of-view test itself), and it makes no self-occlusion, terrain-shadowing, or line-of-sight-blockage check.
- **No bistatic/multistatic capability.** Only monostatic accumulation is implemented.
- **Acceptable for CML 2–5, not CML 6+.** None of these are problems for CML 2–5 use, where first-order geometric coverage and rate-based resource accounting are acceptable fidelity, but they would need to be added before the model could support CML 6+ analyses requiring real link-budget-driven data rates, resolution, or incidence-angle-dependent backscatter performance.

5.19 True_SC_Solar_Panel

True_SC_Solar_Panel models a single solar panel/array as a first-order power source for the spacecraft. Each instance carries its own triangulated shape_model (vertices, faces, solar-cell-side normal, reflectance factors for SRP), a mass, a packing fraction, and a solar cell efficiency, from which the constructor derives face areas, total active area, center of mass, and (for cuboid-shaped panels) a body-frame inertia tensor about the panel's own center of mass. These mass and inertia contributions feed into the aggregate rigid-body mass properties assembled by True_SC_Body. At runtime, func_update_SP_instantaneous_power_generated queries True_SC_orbit_propagation for the Sun-visibility flag and spacecraft position (to build the Sun vector and range), and queries True_SC_ADC for the current body-to-inertial rotation_matrix so that the panel's fixed body-frame normal can be expressed in the same frame as the Sun vector. This combination determines the Sun incidence angle on the panel and, together with the solar flux at the spacecraft's heliocentric range, the instantaneous power the panel can generate.

The panel's generated power is reported back to True_SC_Power (via func_initialize_list_HW_energy_generated / func_update_instantaneous_power_generated) alongside its own housekeeping power draw (instantaneous_power_consumed), and any data it generates is reported to True_SC_Data_Handling. The class thus acts as a leaf hardware component in the True stack: it consumes attitude and orbit/eclipse truth from True_SC_ADC and True_SC_orbit_propagation, and produces power/mass/data truth consumed by True_SC_Power, True_SC_Data_Handling, and True_SC_Body.

5.19.1 Inputs / Outputs

5.19.1.1 Inputs / Dependencies

- init_data **(constructor-time, from the mission setup script):** name, instantaneous_power_consumed, instantaneous_data_rate_generated, shape_model (Vertices, Faces, Face_orientation_solar_cell_side, reflectance factors, type), mass, packing_fraction, and type (panel articulation type).
- True_SC_orbit_propagation **(same spacecraft):** flag_visible_Sun (eclipse gate) and position (spacecraft position, used with the Sun's position to build the Sun-to-spacecraft vector Sun_vector and its range).
- True_SC_ADC **(same spacecraft):** rotation_matrix (body-to-inertial attitude), used to rotate the panel's fixed body-frame Face_orientation_solar_cell_side normal into the inertial frame for the Sun-incidence-angle calculation.
- mission.true_solar_system**:** SS_body{index_Sun}.position (Sun position, to form the Sun vector), solar_constant_AU, and AU_distance (both used in the inverse-square solar flux attenuation for maximum_power).
- **Internal state:** obj.health (0/1 flag) gates whether power/geometry are computed at all, independent of the external classes above.

5.19.1.2 Outputs

- instantaneous_power_generated, maximum_power, Sun_incidence_angle**:** computed each call to func_update_SP_instantaneous_power_generated and stored in obj.store for later analysis; these are not written to by any other class.

- **Reported to `True_SC_Power`:** the panel's `instantaneous_power_generated` and `instantaneous_power_consumed` are read (not modified) by `True_SC_Power` via `func_initialize_list_HW_energy_generated/func_update_instantaneous_power_generated` and `func_initialize_list_HW_energy_consumed/func_update_instantaneous_power_consumed`, which accumulate them into `True_SC_Power`'s own `instantaneous_total_power_generated/instantaneous_total_power_consumed` and per-HW energy arrays.
- **Reported to `True_SC_Data_Handling`:** the panel's `instantaneous_data_rate_generated` is read by `func_update_instantaneous_data_generated`, which accumulates it into `True_SC_Data_Handling`'s `instantaneous_data_generated` and per-HW data array.
- **Reported to `True_SC_Body`:** the panel's `mass`, `shape_model.r_CM`, and `shape_model.I_through_r_CM` (computed once at construction) are read by `True_SC_Body` and folded into the spacecraft's aggregate mass, center-of-mass, and inertia-tensor properties.
- **No external class writes back to this panel's properties** — all cross-class interaction is the panel reading truth from `True_SC_ADC/True_SC_orbit_propagation` and other classes reading (not modifying) the panel's computed outputs.

5.19.2 Mathematical Equations

5.19.2.1 Sun Incidence Angle

$$\theta_{\text{Sun}} = \cos^{-1}(\hat{\mathbf{s}} \cdot (\mathbf{R}_B\, \hat{\mathbf{n}}_{\text{cell}})) \tag{5.58}$$

where $\hat{\mathbf{s}}$ is the unit vector from the spacecraft to the Sun expressed in the inertial frame, $\mathbf{R}_B$ is the body-to-inertial rotation matrix, and $\hat{\mathbf{n}}_{\text{cell}}$ is the solar-cell-side face normal expressed in the body frame.

In the code, the Sun vector is computed as `Sun_vector = mission.true_solar_system.SS_body{...index_Sun}.position - mission.true_SC{i_SC}.true_SC_orbit_propagation.position`, normalized via `func_normalize_vec` into `Sun_vector_normalized`. The incidence angle is then `obj.Sun_incidence_angle = real(acosd(dot(Sun_vector_normalized', mission.true_SC{i_SC}.true_SC_adc.rotation_matrix * obj.shape_model.Face_orientation_solar_cell_side')))`, i.e. $\hat{\mathbf{n}}_{\text{cell}}$ is `shape_model.Face_orientation_solar_cell_side` rotated by `true_SC_adc.rotation_matrix`. This is the only place attitude truth enters the panel model.

5.19.2.2 Maximum Power (Inverse-Square Solar Flux Attenuation)

$$P_{\max} = S_{1\text{AU}} \left(\frac{d_{1\text{AU}}}{r_{\text{Sun}}}\right)^2 A_{\text{total}}\, \eta_{\text{pack}}\, \eta_{\text{cell}} \tag{5.59}$$

where $S_{1\text{AU}}$ is the solar constant at 1 AU, $d_{1\text{AU}}$ is the length of one astronomical unit, r_{Sun} is the instantaneous Sun-spacecraft range, A_{total} is the panel's total active face area, η_{pack} is the solar-cell packing fraction, and η_{cell} is the solar cell efficiency.

This is computed as `obj.maximum_power = mission.true_solar_system.solar_constant_AU * (mission.true_solar_system.AU_distance/norm(Sun_vector))^2 * obj.shape_model.Face_total_area * obj.packing_fraction * obj.solar_cell_efficiency`, i.e. the maximum power the panel could generate if the Sun were exactly along the panel normal, before the cosine (pointing) loss is applied. It uses the standard inverse-square falloff of insolation with distance from the Sun.

5.19.2.3 Instantaneous Power Generated (Cosine Law with Eclipse/Health Gating)

$$P_{\text{gen}} = \begin{cases} 0 & \text{health} = 0 \text{ or eclipsed} \\ P_{\max} \cos\theta_{\text{Sun}} & \theta_{\text{Sun}} \le 90^\circ \\ 0 & \theta_{\text{Sun}} > 90^\circ \end{cases} \tag{5.60}$$

where $P_{\max}$ and θ_{Sun} are as defined above, health is the panel's `health` flag, and "eclipsed" refers to `true_SC_orbit_propagation.flag_visible_Sun` being zero.

In `func_update_SP_instantaneous_power_generated`, the outer condition `(obj.health == 1) && (mission.true_SC{i_SC}.true_SC_orbit_propagation.flag_visible_Sun == 1)` gates whether any power/geometry calculation is done at all; in the eclipsed/unhealthy branch, `maximum_power` and `instantaneous_power_generated` are forced to zero and `Sun_incidence_angle` is set to `inf`. When healthy and sunlit, `obj.instantaneous_power_generated = obj.maximum_power * cosd(obj.Sun_incidence_angle)` if `obj.Sun_incidence_angle <= 90`, otherwise it is zero (panel facing away from the Sun). This is the standard cosine (Lambertian) law for flat-plate solar array power.

5.19.2.4 Panel Face Area (Heron's Formula) and Total Active Area

$$A_f = \sqrt{s(s-a)(s-b)(s-c)}, \qquad s = \frac{a+b+c}{2}, \qquad A_{\text{total}} = \sum_f A_f \tag{5.61}$$

where a, b, c are the three edge lengths of triangular face f (computed from the Euclidean norms between its three `Vertices`), s is the semi-perimeter, A_f is the face area, and A_{total} is the sum over all faces of `shape_model.Faces`.

This is implemented in the constructor's loop over `f=1:size(obj.shape_model.Faces,1)`, computing `a`, `b`, `c` from `obj.shape_model.Vertices(vertex_index(...),:)`, then `this_face_area = sqrt(s*(s-a)*(s-b)*(s-c))`, accumulated into `Face_area(f)` and `Face_total_area`, stored as `obj.shape_model.Face_area` and `obj.shape_model.Face_total_area`. This total area is exactly the A_{total} used in the maximum-power equation above.

5.19.2.5 Cuboid Mass Properties (Inertia Tensor and Volume)

$$\mathbf{I}_{CM} = \text{diag}\left(\frac{W^2+H^2}{12},\ \frac{L^2+H^2}{12},\ \frac{L^2+W^2}{12}\right), \qquad V = L\,W\,H \tag{5.62}$$

where L, W, H are the panel's bounding-box extents along the body-frame x, y, z axes, $\mathbf{I}_{CM}$ is the moment-of-inertia tensor about the panel's center of mass assuming a uniform-density rectangular cuboid, and V is the panel volume.

This is implemented only for `obj.shape_model.type = 'cuboid'` (the `switch` statement's `otherwise` branch raises `error('Havent written yet!')` for any other shape), producing `obj.shape_model.I_through_r_CM = diag([1/12*(W^2+H^2), 1/12*(L^2+H^2), 1/12*(L^2+W^2)])` and `obj.shape_model.volume = L*W*H`, alongside `obj.shape_model.r_CM = mean(obj.shape_model.Vertices, 1)`. Together with `obj.mass` (from `init_data.mass`), these values are the panel's contribution to the spacecraft's aggregate mass properties in `True_SC_Body`.

5.19.3 CML 2–5 Suitability

5.19.3.1 What is included and correctly implemented

The panel implements the physically dominant first-order effects needed for formulation-level power budgeting.

- **Core power-scaling physics.** It implements flat-plate cosine (Lambertian) power scaling with Sun incidence angle, inverse-square attenuation of solar flux with heliocentric distance, and simple multiplicative derates for packing fraction and cell efficiency.
- **Self-consistent incidence angle.** Sun incidence angle is computed self-consistently from actual attitude truth (the ADC's `rotation_matrix`) and actual Sun-relative geometry (from orbit propagation), rather than being an input parameter, so day/night and off-pointing losses fall out naturally as the spacecraft attitude and orbit evolve.
- **Eclipse and health handling.** Eclipse handling is present and binary, which is appropriate fidelity for CML 2–5 trade studies, and a health/on-off state is modeled, zeroing power output when unhealthy.
- **Geometry-derived properties.** Geometric area is derived rigorously from the actual triangulated shape model via Heron's formula rather than an assumed single number, and mass/inertia properties (for cuboid panels) are derived from actual bounding-box dimensions rather than hardcoded, so panel sizing changes propagate correctly into the mass properties used elsewhere.
- **Modular delegation.** The class cleanly delegates power and data bookkeeping to `True_SC_Power` and `True_SC_Data_Handling`, which is a reasonable modular decomposition for a point-design tool.

5.19.3.2 What is not included / gaps

Only one panel geometry has an implemented mass-properties calculation, and several second-order effects are absent.

- **Single supported panel geometry.** Only one panel geometry, `'cuboid'`, has an implemented inertia/volume calculation; any other `shape_model.type` throws `error('Havent written yet!')`, so deployable/articulated panels with non-cuboid shapes cannot currently get mass properties from this class.
- **No penumbra modeling.** There is no explicit modeling of partial eclipse (penumbra) — eclipse is a hard binary flag from orbit propagation, not a fractional attenuation.
- **No temperature/aging/self-shadowing effects.** There is no cell-temperature dependence of efficiency, no degradation/aging model over mission life, no radiation- or micrometeoroid-induced performance loss, and no shadowing of one face/panel by another spacecraft structure (self-shadowing) — the incidence-angle calculation only considers the single face normal against the Sun vector, not occlusion.
- **No gimbal/tracking logic.** The `'active_deployed_gimballed'` panel type is declared in comments as changing orientation in the body frame, but no gimbal/tracking logic (e.g. Sun-tracking control law) is implemented in this file; the rotation applied to the face normal is only the spacecraft body attitude, not any panel-local articulation.

- **Unused reflectance factors.** Bidirectional/reflective effects noted in the shape model comments (`Face_reflectance_factor_solar_cell_side`, `Face_reflectance_factor_opposite_side`) are stored but not used anywhere in this file for power computation (they are flagged as being for SRP elsewhere).
- **Acceptable for CML 2–5.** None of these are blockers for CML 2–5 medium-fidelity trade studies, but they would need attention before flight-software-representative or CML 6+ fidelity.

5.20 True_SC_Star_Tracker

The True_SC_Star_Tracker class models a spacecraft star tracker as a quaternion-output attitude sensor. Rather than performing an explicit star-catalog lookup and pattern-matching solution from True_Stars (i.e., it does not reference all_stars_unit_vector or any star-identification/QUEST-type algorithm), the class instead derives its reported attitude directly from the truth attitude quaternion held by True_SC_ADC (mission.true_SC{i_SC}.true_SC_adc.attitude) and layers one of several selectable statistical error models on top of it via mode_true_SC_star_tracker_selector. The four modes are: "Truth" (pass-through of the true attitude), "Simple" (small-angle Gaussian cross-boresight/about-boresight noise applied as an error quaternion), "Simple with Sun outside FOV" (the Simple noise model gated by a boresight-to-Sun exclusion-angle test, with no measurement produced if the Sun falls inside the sensor field of view), and "Bias + First Order Gauss Markov" (a combined noise-equivalent-angle, constant unit-to-unit bias, and first-order Gauss-Markov drift error budget intended to emulate realistic star tracker error sources such as low-frequency spatial error, LFSE).

The class interfaces with True_SC_ADC by reading its computed truth attitude (and, for the Sun-exclusion mode, its rotation_matrix) and by exposing measurement_vector as the sensor's reported attitude quaternion for downstream attitude determination/estimation consumers. It also interfaces with True_SC_Power and True_SC_Data_Handling to register instantaneous power draw and generated data volume, and uses a dedicated Mersenne-Twister random stream (rand_stream) with an optional seed for Monte Carlo repeatability of the injected measurement noise.

5.20.1 Inputs / Outputs

5.20.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name, instantaneous_power_consumed, instantaneous_data_generated_per_sample, mode_true_SC_star_tracker_selector ('Truth', 'Simple', 'Simple with Sun outside FOV', or 'Bias + First Order Gauss Markov'), measurement_wait_time, location, field_of_view, and mode-dependent noise/bias/FOGM parameters.
- mission.true_time: time_attitude (gates repeat measurement) and time_step_attitude (sets the FOGM discretization step).
- True_SC_ADC **(same spacecraft):** attitude (true body-to-inertial quaternion) is the fundamental truth input corrupted by each mode's noise model to produce measurement_vector; rotation_matrix is read in 'Simple with Sun outside FOV' mode to rotate the boresight into the inertial frame.
- mission.true_solar_system **and** True_SC_Orbit_Propagation **(same spacecraft):** Sun position and spacecraft position, used in 'Simple with Sun outside FOV' mode to check whether the Sun falls inside the sensor's exclusion field of view.
- Software_SC_Estimate_Attitude **(same spacecraft):** sets obj.flag_executive = 1 (in 'KF' mode) once per star-tracker unit to command execution.

5.20.1.2 Outputs

- **Internal / stored outputs:** measurement_vector (quaternion) and fogm_SF are stored in obj.store at every attitude-loop storage step.

- **Reported to** `True_SC_Power/True_SC_Data_Handling`: `instantaneous_power_consumed/instantaneous_data_generated_per_sample` registered/updated via the standard attitude-rate update functions.
- **Reported to** `Software_SC_Estimate_Attitude` **/ attitude Kalman filter:** the stored `measurement_vector` is the intended sensor input to the GNC attitude-estimation filter in `'KF'` mode; in the current base-class `'Truth'` path, the estimator bypasses the star tracker and reads `True_SC_ADC.attitude` directly.
- **Read by shared SC visualization/plotting and validation harnesses:** read-only consumption of `store.measurement_vector`, `alignment_BF_SF`, etc.
- **External executive write:** `Software_SC_Estimate_Attitude` (in `'KF'` mode) sets `flag_executive = 1` for every star-tracker unit each attitude step to command a measurement; this is limited to the executive command flag — no code path was found reading `measurement_vector` directly from `Software_SC_Estimate_Attitude.m` in the current codebase (the Truth-branch star-tracker-enable lines are commented out).

5.20.2 Mathematical Equations

5.20.2.1 Sun-Exclusion Field-of-View Test

$$\theta_{\text{sun}} = \angle\left(\mathbf{C}_{BI}\,\hat{\mathbf{b}}_{SF},\ \hat{\mathbf{s}}_I\right), \qquad \text{measurement valid if } \theta_{\text{sun}} \geq \text{FOV} \tag{5.63}$$

Here $\hat{\mathbf{b}}_{SF}$ is the sensor boresight unit vector expressed in the body frame after rotation by the current attitude, $\mathbf{C}_{BI}$ is the inertial-to-body rotation matrix, $\hat{\mathbf{s}}_I$ is the unit vector from the spacecraft to the Sun, and FOV is the star tracker's field-of-view half-angle in radians.

In the code, this is implemented in the "Simple with Sun outside FOV" case of `func_main_true_SC_star_tracker`: `this_orientation` is computed as `mission.true_SC{i_SC}.true_SC_adc.rotation_matrix * obj.orientation'`, the Sun direction is `Sun_vector_normalized` from `func_normalize_vec`, and the angle is computed with `func_angle_between_vectors`, compared against `deg2rad(obj.field_of_view)`. If the Sun is inside the FOV cone, `obj.measurement_vector` is set to `nan(1,4)` (no measurement); otherwise `func_true_SC_star_tracker_Simple` is invoked. This is the only place in the file where visibility/exclusion geometry (as opposed to a star catalog) is evaluated; no per-star FOV test against `True_Stars`' catalog occurs anywhere in the class.

5.20.2.2 Simple Small-Angle Attitude Error Model

$$\boldsymbol{\theta}_{SF} = \begin{bmatrix}\sigma_\perp n_1\\ \sigma_\perp n_2\\ \sigma_\parallel n_3\end{bmatrix}, \qquad \boldsymbol{\theta}_{BF} = \mathbf{C}_{BF\leftarrow SF}\,\boldsymbol{\theta}_{SF}, \qquad q_{\text{err}} = \left[\sin\!\left(\tfrac{\|\boldsymbol{\theta}_{BF}\|}{2}\right)\frac{\boldsymbol{\theta}_{BF}}{\|\boldsymbol{\theta}_{BF}\|},\ \cos\!\left(\tfrac{\|\boldsymbol{\theta}_{BF}\|}{2}\right)\right], \qquad q_{\text{meas}} = q_{\text{err}}\otimes q \tag{5.64}$$

$\sigma_\perp$ and $\sigma_\parallel$ are the 1-sigma cross-boresight and about-boresight angular noise standard deviations, n_1, n_2, n_3 are independent standard normal draws, $\mathbf{C}_{BF\leftarrow SF}$ is the sensor-frame-to-body-frame alignment DCM, q_{true} is the spacecraft's true attitude quaternion, and q_{meas} is the reported (noisy) attitude, obtained via Hamilton quaternion multiplication ($\otimes$) followed by normalization.

This corresponds to `func_true_SC_star_tracker_Simple`: `angle_error_SF = obj.angle_sigma_SF(:).*randn(...)`, `angle_error_BF = (obj.alignment_BF_SF*angle_error_SF).'`, `error_quaternion_B = vec2q(angle_error_BF/angle_error_norm, angle_error_norm)`, and

`obj.measurement_vector = qnorm(qmult(error_quaternion_B, mission.true_SC{i_SC}.true_SC_adc.attitude))`. The code explicitly retains a commented-out "OLD Incorrect Method" that additively perturbed the quaternion components, documenting that the multiplicative small-angle-error construction was adopted as the corrected approach.

5.20.2.3 Bias + First-Order Gauss-Markov (FOGM) Error Budget

$$x_k = \phi\, x_{k-1} + w_k, \qquad \phi = e^{-\Delta t/\tau}, \qquad w_k \sim \mathcal{N}\big(0,\ \sigma_{\text{fogm}}^2\,(1-\phi^2)\big) \tag{5.65}$$

$$\boldsymbol{\theta}_{SF} = \boldsymbol{\theta}_{\text{NEA}} + \mathbf{b}_{SF} + \mathbf{x}_{SF} \tag{5.66}$$

x_k is the FOGM error state (per axis, cross/cross/boresight), ϕ is the discrete one-step decay coefficient, Δt is the propagation time step (`obj.sim_dt`), τ is the FOGM correlation time constant, σ_{fogm} is the FOGM steady-state 1-sigma, w_k is the per-step driving noise, $\boldsymbol{\theta}_{\text{NEA}}$ is white noise-equivalent angle noise resampled every measurement, and $\mathbf{b}_{SF}$ is a constant per-unit bias sampled once at construction. The resulting $\boldsymbol{\theta}_{SF}$ is transformed into the body frame and converted into an error quaternion exactly as in the Simple model.

This is implemented in `func_true_SC_star_tracker_bias_fogm`: `nea_SF`, the recursive update `obj.fogm_SF = (obj.fogm_phi*obj.fogm_SF(:) + obj.fogm_driving_sigma_SF.*randn(...))'`, and `total_angle_SF = nea_SF + obj.bias_SF(:) + obj.fogm_SF(:)`, followed by the same `vec2q`/`qmult`/`qnorm` sequence used in the Simple model. The coefficients `fogm_phi` and `fogm_driving_sigma_SF` are precomputed in the constructor from `fogm_tau`, `fogm_sigma_cross`, `fogm_sigma_boresight`, and `obj.sim_dt = max([mission.true_time.time_step_attitude, obj.measurement_wait_time])`, and `obj.bias_SF` is drawn once from `bias_sigma_cross`/`bias_sigma_boresight` at construction (or taken from `init_data.bias_SF` if supplied).

5.20.3 CML 2–5 Suitability

5.20.3.1 What is included and correctly implemented

The class provides a graduated set of fidelity options appropriate for pre-project trade studies.

- **Graduated noise-model hierarchy.** It offers a noiseless truth pass-through, a first-order Gaussian small-angle noise model with physically distinct cross-boresight and about-boresight sigmas (matching how real star trackers report anisotropic accuracy about vs. across boresight), and a more detailed bias-plus-FOGM error budget that separately represents white noise-equivalent angle, constant per-unit bias, and slowly correlated drift (LFSE) — a reasonable medium-fidelity representation of star tracker error character without requiring a flight-software-representative star-identification pipeline.

- **Correct attitude-error mechanics.** Attitude error is applied correctly as a multiplicative quaternion perturbation (axis-angle → quaternion via `vec2q`, applied with `qmult`, and renormalized with `qnorm`), and the code documents that an earlier additive-noise implementation was identified as incorrect and replaced.

- **Mounting misalignment.** Sensor mounting misalignment is captured through `alignment_BF_SF`, used consistently to rotate the sensor-frame noise into the body frame in both the Simple and Bias+FOGM models.

- **Sun-exclusion FOV check.** A Sun-exclusion FOV check (boresight-to-Sun angle vs. `field_of_view`) models the realistic operational constraint that a star tracker cannot produce a valid solution when blinded by the Sun.
- **Monte Carlo and system integration.** Monte Carlo repeatability is supported via a dedicated, seedable random stream, and the sensor integrates with the spacecraft power and data-handling accounting classes and gates measurement generation by a configurable `measurement_wait_time`, consistent with system-level resource-budget trade studies at CML 2–5.

5.20.3.2 What is not included / gaps

Despite the class name, there is no actual star-catalog consumption or star-identification processing.

- **No star-catalog/attitude-determination pipeline.** There is no star visibility test against `True_Stars`' `all_stars_unit_vector`, and no star-pattern-matching/attitude-determination algorithm (e.g., QUEST) anywhere in this file; the reported attitude is generated directly from the ADC's known truth attitude plus injected statistical noise, not from a simulated star image or star correspondence solution.
- **No star-count/geometry-dependent effects.** Effects that depend on the number and geometry of visible stars in the FOV (e.g., degraded accuracy or loss of track when insufficient stars are present, false star identification, stray light from bodies other than the Sun such as the Earth or Moon, or magnitude/catalog-density limitations) are not modeled.
- **Limited bright-body exclusion.** The Sun-exclusion logic checks only the Sun and not other bright-body interference.
- **Unused properties.** The `temperature` and `health` properties exist but are not used to modulate noise or measurement availability beyond a simple on/off gate (`flag_executive`, `health==1`).
- **Acceptable for CML 2–5.** These omissions are consistent with the first-order, non-flight-representative fidelity expected at CML 2–5, where a statistically parameterized attitude-noise sensor model is an accepted substitute for a full star-identification simulation.

5.21 True_SC_Sun_Sensor

The True_SC_Sun_Sensor class models a single sun sensor unit mounted on the spacecraft, whose role is to provide a measurement related to the Sun direction (or, in the implemented logic, the spacecraft attitude conditioned on Sun visibility) for use by the attitude determination and control chain. Each sensor instance is defined by a mounting location and boresight orientation in the body frame, a field_of_view half-angle, a measurement update period (measurement_wait_time), and a 1-sigma angular measurement_noise. The class interfaces with True_SC_ADC to obtain the true spacecraft attitude quaternion and rotation matrix, and with True_solar_system/orbit propagation to obtain the true relative Sun position needed to evaluate Sun visibility.

The sensor supports three fidelity modes selected via mode_true_SC_sun_sensor_selector: *Truth* (perfect knowledge of the true attitude quaternion), *Simple* (true attitude quaternion corrupted by a small-angle random-axis quaternion perturbation), and *Simple with Sun in FOV* (the Simple noise model, but gated by an explicit check of whether the Sun direction falls inside the sensor's field of view, otherwise the measurement is invalidated to NaN). Like the other true sensor classes, it also participates in the spacecraft power and data-handling bookkeeping and in the mission-wide time-history storage of its measurement.

5.21.1 Inputs / Outputs

5.21.1.1 Inputs / Dependencies

- init_data **(constructor-time):** name, instantaneous_power_consumed, instantaneous_data_generated_per_sample, mode_true_SC_sun_sensor_selector ('Truth', 'Simple', or 'Simple with Sun in FOV'), measurement_wait_time, measurement_noise, location, orientation, field_of_view.
- mission.true_time**:** time_attitude, gates repeat measurements against measurement_wait_time.
- True_SC_ADC **(same spacecraft):** attitude is the fundamental truth corrupted by each mode's noise model to produce measurement_vector; rotation_matrix is read in 'Simple with Sun in FOV' mode.
- mission.true_solar_system **and** True_SC_Orbit_Propagation **(same spacecraft):** Sun position and spacecraft position, used in 'Simple with Sun in FOV' mode to test whether the Sun is within this sensor's field of view (unlike a star tracker, a sun sensor requires the Sun in view to produce a valid measurement).

5.21.1.2 Outputs

- **Internal / stored outputs:** measurement_vector (quaternion) is stored in obj.store at every attitude-loop storage step.
- **Reported to** True_SC_Power/True_SC_Data_Handling**:** instantaneous_power_consumed/instantaneous_data_generated_per_sample registered/updated via the standard attitude-rate update functions.
- **Read by shared SC visualization/plotting utilities:** read-only consumption of orientation/stored measurement data for drawing the sensor boresight on attitude plots.

- **No consumption by `Software_SC_Estimate_Attitude` was found:** unlike the star tracker, no code path in `Software_SC_Estimate_Attitude.m` (Truth or KF) reads `true_SC_sun_sensor`; a KF plugin comment explicitly states the sun sensor is ignored for now.
- No external class writes back into this class's own properties.

5.21.2 Mathematical Equations

5.21.2.1 Sensor Boresight Direction in the Reference Frame

$$\mathbf{b} = \mathbf{R}_{B\to I}\,\mathbf{o} \tag{5.67}$$

where $\mathbf{o}$ is the unit boresight/normal vector of the sensor expressed in the spacecraft body frame, $\mathbf{R}_{B\to I}$ is the attitude rotation matrix of the spacecraft, and $\mathbf{b}$ is the resulting boresight direction used for the field-of-view test.

In the code, this corresponds to `this_orientation = (mission.true_SC{i_SC}.true_SC_adc.rotation_matrix * obj.orientation')'`, where `obj.orientation` is the sensor's body-frame normal vector and `rotation_matrix` comes from `True_SC_ADC`.

5.21.2.2 True Sun-Direction Vector

$$\hat{\mathbf{s}} = \frac{\mathbf{r}_\odot - \mathbf{r}_{SC}}{\|\mathbf{r}_\odot - \mathbf{r}_{SC}\|} \tag{5.68}$$

where $\mathbf{r}_\odot$ is the position of the Sun body and $\mathbf{r}_{SC}$ is the spacecraft position, and $\hat{\mathbf{s}}$ is the resulting unit Sun-direction vector.

This is computed as `Sun_vector = mission.true_solar_system.SS_body{mission.true_solar_system.index_Sun}.position - mission.true_SC{i_SC}.true_SC_orbit_propagation.position`, normalized via `Sun_vector_normalized = func_normalize_vec(Sun_vector)`.

5.21.2.3 Field-of-View Visibility Test

$$\cos^{-1}(\mathbf{b}\cdot\hat{\mathbf{s}}) \;\le\; \mathrm{FOV} \tag{5.69}$$

where $\mathbf{b}$ and $\hat{\mathbf{s}}$ are the boresight and Sun-direction unit vectors defined above, and FOV is the sensor half-angle field of view (converted from degrees to radians). If this condition holds, the Sun is considered within the sensor's field of view and a measurement is produced; otherwise the measurement is invalidated.

This corresponds to the check `func_angle_between_vectors(this_orientation, Sun_vector_normalized) <= deg2rad(obj.field_of_view)` in mode *Simple with Sun in FOV*; when false, `obj.measurement_vector = nan(1,4)`.

5.21.2.4 Measurement Noise Model (Small-Angle Quaternion Perturbation)

$$\delta\mathbf{q} = \left[\sin\!\left(\frac{\delta\theta}{2}\right)\hat{\mathbf{a}},\;\; \cos\!\left(\frac{\delta\theta}{2}\right)\right], \qquad \mathbf{q}_{meas} = \delta\mathbf{q}\otimes\mathbf{q}_{true} \tag{5.70}$$

where $\delta\theta$ is the 1-sigma angular measurement noise (`measurement_noise`, in radians) used directly as a fixed rotation-error magnitude, $\hat{\mathbf{a}}$ is a uniformly-sampled random unit rotation axis, $\delta\mathbf{q}$ is the resulting small-angle error quaternion, $\mathbf{q}_{true}$ is the true attitude quaternion from `True_SC_ADC`, $\otimes$ denotes quaternion multiplication, and $\mathbf{q}_{meas}$ is the resulting noisy measurement quaternion (re-normalized to be proper).

In the code this is `error_axis = rand(1,3) - 0.5` normalized to unit length, `error_quaternion = [sin(error_angle/2)*error_axis, cos(error_angle/2)]` with `error_angle = obj.measurement_noise`, both passed through `func_quaternion_properize`, and combined via `obj.measurement_vector = func_quaternion_multiply(error_quaternion, mission.true_SC{i_SC}.true_SC_adc.attitude)` in `func_true_SC_sun_sensor_Simple`. A comment in the source explicitly notes that an earlier implementation added noise directly to quaternion components (additive Gaussian noise on each element) and that this axis-angle multiplicative approach was adopted as the "New Correct Method" to properly preserve quaternion unit norm.

5.21.2.5 Truth-Mode Output

$$\mathbf{q}_{meas} = \mathbf{q}_{true} \tag{5.71}$$

where $\mathbf{q}_{true}$ is the true spacecraft attitude quaternion from `True_SC_ADC`. This is implemented as `obj.measurement_vector = func_quaternion_properize(mission.true_SC{i_SC}.true_SC_adc.attitude)` in `func_true_SC_sun_sensor_Truth`, i.e. the sensor returns the exact attitude with no noise or FOV gating.

5.21.3 CML 2–5 Suitability

5.21.3.1 What is included and correctly implemented

The class provides a clean, switchable fidelity hierarchy appropriate for trade-space and point-design studies.

- **Switchable fidelity hierarchy.** It offers Truth / Simple / Simple with Sun in FOV modes, letting an analyst run with perfect knowledge or with a first-order noise-and-visibility model without changing the surrounding architecture.
- **Correct quaternion noise mechanics.** The quaternion noise injection is implemented with correct rotational-error mechanics (axis-angle perturbation composed by quaternion multiplication and re-normalization), which the in-code comments show was a deliberate fix over a naive additive-noise approach — this is a genuinely correct, medium-fidelity representation of small-angle attitude/measurement error, well beyond what CML 2–5 requires.
- **Sun-visibility gating.** The Sun-visibility gating combines the sensor boresight (rotated into the reference frame via the true attitude rotation matrix) with the true relative Sun-direction vector and a single scalar field-of-view half-angle, which is a standard first-order way to decide whether a sun sensor "sees" the Sun.
- **System integration.** The class also correctly integrates with the mission's time-stepping (health/executive flags, measurement wait-time cadence), storage/logging of the measurement history, and the spacecraft power and data-handling subsystems, consistent with the rest of the True_Sensors_Actuators framework.

5.21.3.2 What is not included / gaps

The measurement actually produced is not a true body-frame Sun-direction output, and several second-order effects are absent.

- **Not a true two-axis sun-angle output.** The measurement actually produced is the (possibly noisy) spacecraft attitude quaternion, not a body-frame Sun-direction angle/vector; the sensor's own `orientation` vector is used only to test FOV visibility in the "Simple with Sun in FOV" mode and is never used to compute the reported measurement itself, so despite its name the class does not model the characteristic two-axis sun-angle output of a real sun sensor.
- **No intensity, falloff, or occlusion modeling.** There is no cosine (Lambertian) intensity response, no sensor field-of-view falloff shape (only a hard angular cutoff), and no modeling of eclipse, Sun blockage by spacecraft structures/solar panels, or albedo/stray-light effects.
- **Noise independent of geometry.** The noise model uses a single fixed 1-sigma angle regardless of Sun incidence angle, so there is no dependency of measurement accuracy on where in the FOV the Sun falls (real sensors typically degrade near the FOV edge).
- **No bias/misalignment/temperature/fault modeling.** There is no bias, misalignment, quantization, or temperature-dependent behavior modeled (the `temperature` property exists but is never used in the measurement logic), and no correlated/time-varying noise or fault modes beyond the simple on/off `health` flag.
- **Acceptable for CML 2–5.** For CML 2–5 formulation-level work these omissions are acceptable, since the intent at this maturity level is first-order/medium-fidelity representation rather than flight-software-representative sensor fidelity, and the existing Truth/Simple/FOV-gated hierarchy already supports the kind of trade-space sensitivity studies expected at this stage.

Chapter 6

SC System-Level and Functional-Level Autonomy Software Layer Classes

6.1 Software_SC_Communication

Software_SC_Communication is the flight-software-side model of a spacecraft's communication subsystem. It does not itself simulate radio-frequency links or antenna pointing (that physical behavior lives in True_SC_Communication_Link and True_SC_Radio_Antenna); instead it represents the onboard logic that decides *when* and *how* the vehicle should attempt to communicate, and then hands control off to mission-specific implementations. Each timestep, Software_SC_Executive sets the object's flag_executive to indicate that the communication subsystem has been tasked to run. If that flag is set, func_main_software_SC_communication dispatches, via a switch on mode_software_SC_communication_selector, to one of several mission-specific update functions (_Dart, _NISAR, a Multi-GS Generic handler, additional mission-specific named functions, or a generically-named mission specific function resolved through str2func). These external functions (not contained in this file) presumably use the tracked state — last_communication_time, wait_time_comm_dte, attitude_error_threshold, this_attitude_error — to decide whether a downlink/uplink contact should occur, and would in turn command True_SC_Communication_Link / True_SC_Radio_Antenna objects.

After dispatch, the class calls func_update_instantaneous_data_generated on the spacecraft's true_SC_data_handling object, tying the communication mode's outcome back into onboard data-volume bookkeeping (the amount of data generated per sample, instantaneous_data_generated_per_sample, or an instantaneous data rate, selected via instantaneous_data_type). At the end of each cycle the class records flag_executive and this_attitude_error into the store history buffers and resets flag_executive to 0 and this_attitude_error to inf, awaiting the next executive tasking cycle. In effect, this class is a thin state container and mode-dispatch layer for spacecraft communication scheduling logic, with the substantive contact-window and link-activation computations delegated to mission-specific subroutines not present in this file.

6.1.1 Inputs / Outputs

6.1.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_data_generated_per_sample; mode_software_SC_communication_selector (required: 'mission specific', 'DART', 'NISAR', 'Multi-GS Generic', or an additional mission-specific mode name); attitude_error

_threshold_deg (converted to radians); optional data, mode_selector_name, last_communication_time (default 0), wait_time_comm_dte (default 3600 s).

- True_SC_Body **(same spacecraft):** reads name for identification/logging.
- Software_SC_Executive **(same spacecraft):** reads this_sc_mode to gate whether a downlink/telecom opportunity is attempted this cycle.
- True_SC_ADC **(same spacecraft):** reads rotation_matrix to evaluate antenna pointing error against the ground station/relay.
- True_SC_Radio_Antenna **(same spacecraft):** reads orientation to compute link geometry and the attitude-error threshold check.
- True_Solar_System **(mission-level):** reads SS_body{...}.position to compute line-of-sight/geometry to Earth or a relay body.
- Software_SC_Estimate_Orbit **(same spacecraft):** reads position to determine spacecraft location relative to ground stations.
- True_SC_Communication_Link **(same spacecraft):** reads this_data_rate to estimate how much data can be downlinked.
- Software_SC_Data_Handling **(same spacecraft):** reads total_data_storage and mean_state_of_data_storage to decide whether a downlink is warranted and how much backlog exists.

6.1.1.2 Outputs

- **Internal / stored outputs:** flag_executive, this_attitude_error, data.transmission_complete, updated last_communication_time.
- **Reported to / written into** True_SC_Radio_Antenna**:** mission-specific communication-strategy functions set mission.true_SC{i_SC}.true_SC_radio_antenna{1}.flag_executive and mode_true_SC_radio_antenna_selector = "TX", commanding the onboard antenna's TX/RX mode for the current cycle.
- **Reported to / written into** True_GS_Radio_Antenna **(ground segment):** sets mode_true_GS_radio_antenna_selector = "RX" on the paired ground-station antenna object, coordinating the two ends of the link.
- **Reported to / written into** True_SC_Communication_Link**:** sets flag_executive = 1 to activate the link model for data-rate/throughput computation this cycle.
- **Reported to / written into** Software_SC_Data_Handling **(peer software object):** upon a completed downlink, zeros mean_state_of_data_storage and total_data_storage, simulating the onboard data buffer being emptied.
- This class both reads geometry/state from True_SC_ADC, True_SC_Radio_Antenna, Software_SC_Estimate_Orbit, and Software_SC_Data_Handling, and actively overwrites commanded mode/flag properties on True_SC_Radio_Antenna, True_GS_Radio_Antenna, and True_SC_Communication_Link each cycle, plus resets the data-storage state on its Software_SC_Data_Handling peer — a software-driven write-back pattern, unlike passive hardware classes.

6.1.2 Mathematical Equations

This file contains no genuinely computed equations of its own. The class is purely dispatch/bookkeeping logic: it stores configuration/state variables (attitude error threshold, last communication time, DTE wait time, instantaneous data-generation parameters) and, when tasked by the executive, forwards execution to one of several mission-specific functions (`func_update_software_SC_communication_DART`, `_NISAR`, `_MultiGS_Generic`, or a mission-named handle resolved via `str2func`) that are defined elsewhere and not visible in `Software_SC_Communication.m`. Any contact-window prediction, data-volume threshold, or scheduling formula would live in those external, mission-specific implementations rather than in this class itself, so no equation is reproduced here to avoid misattributing logic that this file does not actually contain.

6.1.3 CML 2–5 Suitability

6.1.3.1 What is included and correctly implemented

The class correctly implements the executive-gated update pattern common to the `Software_SC_*` family, providing a reasonable first-order representation of communication-scheduling bookkeeping.

- **Executive-gated duty cycle.** It only acts when `flag_executive` is set by `Software_SC_Executive`, and resets that flag and the attitude-error state (`this_attitude_error` back to `inf`) after each cycle, which is an appropriate first-order representation of a duty-cycled onboard task.
- **Contact-timing state.** It maintains the state needed for simple contact-timing logic — `last_communication_time` and `wait_time_comm_dte` (defaulting to a 3600 s / 1-hour cadence) — and an `attitude_error_threshold` (converted from degrees to radians via `deg2rad`), both reasonable, medium-fidelity proxies for "can/should I talk to the ground now" decisions appropriate at CML 2–5.
- **Clean data-volume delegation.** It cleanly separates data-volume bookkeeping by delegating to `true_SC_data_handling` (via `func_initialize_list_HW_data_generated` and `func_update_instantaneous_data_generated`).
- **Extensible mission-specific dispatch.** It supports multiple mission configurations (DART, NISAR, Multi-GS Generic, or an arbitrary mission-specific function looked up by name) through a single, extensible mode-selector pattern, plus standard `store` history logging of `flag_executive` and `this_attitude_error` for post-run analysis.

6.1.3.2 What is not included / gaps

None of the substantive communication-scheduling computation lives in this file itself.

- **No visible scheduling logic.** Contact-window prediction, link/antenna selection, data-volume-based activation thresholds, or ground-station visibility checks are not present in this file; they all live in mission-specific functions (e.g. `func_update_software_SC_communication_DART`) that are called but not shown, so this class alone cannot be assessed for algorithmic fidelity.
- **No visible hardware interface.** There is no visible interface back to `True_SC_Communication_Link` or `True_SC_Radio_Antenna` within this file (no antenna selection, link

budget, or slew/pointing command); those interactions, if they exist, occur inside the unlisted mission-specific handlers.

- **Silent fallback on undefined mode.** The `otherwise` branch of the mode `switch` only calls `disp('Communication mode not defined!')` and silently continues rather than raising an error or halting the timestep, which could mask a misconfiguration.
- **Unverifiable data-type flag.** There is no explicit handling of the `instantaneous_data_type` flag within this file itself (it is referenced only in a comment as governing which data-handling call path is used), so its effect cannot be verified from this class alone.

These gaps are consistent with, and acceptable for, a CML 2–5 point-design/trade-space tool, since flight-software-representative fidelity (e.g., real contact-window computation, link-margin checks, or robust error handling) is not required at this maturity level.

6.2 Software_SC_Control_Attitude

Software_SC_Control_Attitude is the flight-software attitude control law of the MuSCAT simulation. Each attitude timestep, its main method func_main_software_SC_control_attitude dispatches, via obj.mode_software_SC_control_attitude_selector, into one of several mission-specific control pipelines (DART, NISAR, additional mission-specific pipelines, or a generic "mission specific" handle). Each pipeline first calls a guidance step that sets body/inertial pointing vectors and computes a desired target quaternion (desired_attitude) and rate (desired_angular_velocity), then a control step that turns the attitude/rate error against Software_SC_Estimate_Attitude's current attitude/rate estimate into a commanded body torque (desired_control_torque), and finally an allocation step that maps that torque command onto actuator commands sent to True_SC_Reaction_Wheel (angular wheel accelerations) and/or True_SC_RCS_Thruster (individual thruster on/off and thrust levels), or, in "Oracle"/direct modes, writes the desired attitude straight into True_SC_ADC bypassing actuators entirely. The class is only executed when Software_SC_Executive sets flag_executive = 1, and it reports back an instantaneous attitude pointing error (error_angle_desired_attitude, computed against the estimator's attitude via func_error_angle_between_quaternions) that can be used by the executive or other software modules for mode logic.

Beyond direct control, the class also implements actuator-management logic that is representative of, but simpler than, real flight software: threshold-based selection between reaction wheels and thrusters, a least-squares/KKT-based thruster mixing solver, a reaction-wheel pseudo-inverse allocator with momentum-management heuristics to avoid abrupt torque-direction reversals, simplified wheel-saturation prediction, and a simplified desaturation procedure that dumps wheel momentum through thrusters. A separate slew-generation path (SLERP-based) produces intermediate attitude/rate commands for large re-orientations when a nonzero max_slew_rate is configured, rather than commanding the full attitude step discontinuously.

6.2.1 Inputs / Outputs

6.2.1.1 Inputs / Dependencies

- init_data **(constructor-time):** mode_software_SC_control_attitude_selector (required); optional max_slew_rate (default 0), data, mode_selector_name, control_gain, reaction_wheel_attitude_control_threshold (default 0.1 rad).
- True_SC_RCS_Thruster **(same spacecraft, constructor-time):** reads location, orientation, maximum_thrust, minimum_thrust, health of every installed thruster to build the control-allocation matrix.
- True_SC_Reaction_Wheel **(same spacecraft):** constructor-time reads orientation, moment_of_inertia, health, maximum_torque to build the wheel-contribution matrix; runtime reads angular_velocity, max_angular_velocity, health, saturated to drive the desaturation logic.
- True_SC_Body **(same spacecraft):** reads num_hardware_exists.* (RCS thruster/reaction wheel counts) and location_COM for allocation geometry.
- Software_SC_Estimate_Attitude **(same spacecraft):** reads attitude to compute error_angle_desired_attitude between estimated and desired attitude.

6.2.1.2 Outputs

- **Internal / stored outputs:** `desired_attitude`, `desired_angular_velocity`, `desired_control_torque`, `error_angle_desired_attitude`, `thruster_contribution_matrix`, `pinv_reaction_wheel_contribution_matrix`, `max_rw_torque`, `max_mt_torque`, `desaturation_procedure`, `flag_slew`.
- **Reported to / written into `True_SC_ADC`:** in Oracle/truth-passthrough control modes, directly overwrites `mission.true_SC{i_SC}.true_SC_adc.attitude = obj.desired_attitude` and `.angular_velocity = obj.desired_angular_velocity` (and, in a DART direct-torque submode, `.control_torque`), bypassing physical actuation entirely.
- **Reported to / written into `True_SC_Reaction_Wheel`:** sets `flag_executive = 1` and `commanded_angular_acceleration = wheel_accelerations(i)` on each wheel — the exact property that `True_SC_Reaction_Wheel`'s own main function reads to spin up the wheel on the next physics step.
- **Reported to / written into `True_SC_RCS_Thruster`:** sets `command_actuation = 1` and `commanded_thrust = x(i)` on selected thrusters, consumed directly by the thruster's own actuation logic the following step.
- `True_SC_Magnetorquer`: no write-back exists — `commanded_dipole_moment` is declared on `True_SC_Magnetorquer` but is never assigned by any current mission-specific attitude-control logic.
- This class reads estimated attitude from `Software_SC_Estimate_Attitude` and sensor/health state from `True_SC_Reaction_Wheel/True_SC_RCS_Thruster`, then overwrites `True_SC_ADC.attitude/angular_velocity/control_torque` (Oracle modes), `True_SC_Reaction_Wheel.commanded_angular_acceleration`, and `True_SC_RCS_Thruster.commanded_thrust/command_actuation` every cycle it executes — a direct software-to-actuator command write-back, not present in passive hardware classes.

6.2.2 Mathematical Equations

6.2.2.1 Quaternion Attitude Error

$$q_{err} = q_{desired} \otimes q_{actual}^{-1}, \qquad \theta_{err} = 2\arccos\left(|q_{desired} \cdot q_{actual}|\right) \tag{6.1}$$

$q_{desired}$ and q_{actual} are unit attitude quaternions (body frame relative to inertial frame), $\otimes$ is quaternion multiplication, and θ_{err} is the shortest-path rotation angle between the two attitudes (accounting for the quaternion double-cover by taking the absolute value of the dot product).

This is implemented in `func_error_angle_between_quaternions.m` (used in the class's main loop to set `obj.error_angle_desired_attitude`, taking the minimum over $q_{desired}$ and $-q_{desired}$) and in `func_get_attitude_error.m` (used by `func_actuator_selection_DART` to decide between wheels and thrusters) and in `q2error.m` (used inside the PD control law). The vector-form error used for feedback, $\delta\theta$, is obtained from the vector part of q_{err} scaled by the rotation angle (exact form in `q2error.m` with `mflag=1`, small-angle linear form $2\,q_{err}(1:3)$ with `mflag=0`).

6.2.2.2 PD Attitude Control Law with Feed-Forward and Gyroscopic Compensation

$$\tau_{cmd} = J\left(K_d\,\delta\omega + K_p\,\delta\theta\right) + J\dot{\omega}_{desired} - \tau_{dist,ff} + \omega_{est} \times \left(J\,\omega_{est}\right) \tag{6.2}$$

J is the spacecraft body-frame inertia matrix (`software_SC_executive.data.inertia_BF`); $K_p = k_p I_3$ and $K_d = k_d I_3$ are diagonal proportional and derivative gains built from `obj.data.control_gain` $= [k_p,\ k_d]$; $\delta\omega = \omega_{desired} - \omega_{est}$ is the angular-rate error and $\delta\theta$ is the quaternion attitude error from `q2error`; $\dot{\omega}_{desired}$ is `obj.desired_angular_accel` (feed-forward commanded angular acceleration); $\tau_{dist,ff}$ is a feed-forward disturbance-torque estimate from a mission-specific electric-propulsion disturbance-compensation function (subtracted from the command); and the last term is the gyroscopic (Euler) cross-coupling torque $\omega_{est} \times (J\omega_{est})$.

This is implemented in `function_update_desired_control_torque_PD.m` (variables `ufb`, `uff`, `uff_dist`, `vc`, summed into `obj.desired_control_torque`), invoked from the 'DART Control PD' mode in `func_update_software_SC_control_attitude_DART.m`.

6.2.2.3 Asymptotically-Stable Quaternion-Feedback Control Law

$$\omega_r = \omega_{desired} + (0.5\, Z(\beta_{est}))^{+}\, \lambda_r\, (\beta_{desired} - \beta_{est})\,, \qquad \tau_{cmd} = -K_r\, (\omega_{est} - \omega_r) \tag{6.3}$$

β_{est} and $\beta_{desired}$ are the (properized) estimated and desired attitude quaternions; $Z(\beta)$ is the 4×3 quaternion-kinematics matrix built from the quaternion vector part via a skew-symmetric operator ($Z(1{:}3, 1{:}3) = \beta_4 I_3 + [\beta_{1:3}]_\times$, $Z(4, 1{:}3) = -\beta_{1:3}^T$), with $(\cdot)^+$ its Moore-Penrose pseudo-inverse; K_r and λ_r are the two scalar gains stored in `obj.data.control_gain = [K_r, Lambda_r]`; ω_r is a reference angular velocity that drives the quaternion error to zero, and the commanded torque is proportional feedback on the rate error relative to that reference.

Implemented in `function_update_desired_control_torque_asymptotically_stable.m` (variables `Zbeta`, `omega_r`, `obj.desired_control_torque`), invoked by several mission-specific 'Control Asymptotically Stable' modes (including DART and NISAR).

6.2.2.4 Reaction-Wheel Command Allocation (Pseudo-Inverse)

$$\dot{h}_{wheel} = B_{rw}^{+}\, (-\tau_{cmd})\,, \qquad B_{rw}(:, i) = I_{w,i}\, \hat{a}_i \tag{6.4}$$

B_{rw} is the $3 \times N_{rw}$ reaction-wheel momentum-contribution matrix, with each column equal to the wheel's moment of inertia $I_{w,i}$ times its spin-axis unit vector $\hat{a}_i$ (zeroed for unhealthy wheels); B_{rw}^{+} is its pseudo-inverse; $\dot{h}_{wheel}$ is the vector of commanded wheel angular accelerations; the sign is negated because the torque commanded to the wheels must be equal and opposite to the torque desired on the spacecraft body.

B_{rw} is built in `func_initialize_reaction_wheel_contribution.m` and inverted once into `obj.pinv_reaction_wheel_contribution_matrix`; the allocation itself is `func_compute_rw_command_pinv.m` (variable `wheel_accelerations`), followed by heuristic momentum-management limiting in `func_manage_wheel_momentum.m` and safety clamping/dispatch in `func_apply_reaction_wheel_commands.m`.

6.2.2.5 Thruster Torque Allocation via Constrained Least Squares (KKT)

$$\min_x \|Ax - b\|_2^2 \quad \text{s.t.} \quad Cx = \tau_{cmd}, \qquad \begin{bmatrix} A^T A & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} = \begin{bmatrix} A^T b \\ \tau_{cmd} \end{bmatrix} \tag{6.5}$$

x is the vector of thrust magnitudes for reduced thruster pairs producing opposite torque directions; C is the constraint matrix whose columns are each pair's net torque-per-unit-thrust direction (from the position$\times$orientation cross product, `matrix_opposite_torque`); $A = I$ and $b = 0$ in the current implementation, so the cost term is a minimum-thrust-effort regularizer; z is the vector of Lagrange multipliers enforcing the torque-matching constraint $Cx = \tau_{cmd}$. The system is solved

once per allocation call via a pre-computed LU decomposition ($E = LUP^{-1}$, i.e. `P,L,U`) of the KKT matrix E.

Set up in `func_initialize_optimization_data.m` (variables `A`, `b`, `C`, `E`, `L`, `U`, `P`, stored in `obj.optim_data`); solved each call in `func_decompose_control_torque_into_thrusters_optimization_kkt.m` (variables `f`, `xz`, `x_opposite`, expanded to per-thruster commands `x` and written to `true_SC_RCS_thruster{i}.commanded_thrust`), with a final proportional scale-down if any commanded thrust exceeds `obj.max_thrust`.

6.2.2.6 SLERP-Based Slew Trajectory Generation

$$q(\tau) = \frac{\sin\left[(1-\tau)\,\theta_{tot}\right] q_{start} + \sin(\tau\,\theta_{tot})\, q_{end}}{\sin(\theta_{tot})}, \qquad \theta_{tot} = 2\arccos\left(q_{err,4}\right) \tag{6.6}$$

q_{start} and q_{end} are the current and target attitude quaternions (sign-aligned for shortest path); θ_{tot} is the total rotation angle between them; $\tau \in [0, 1]$ is the normalized interpolation parameter across $N = \lceil(\theta_{tot}/w_{max})/dt\rceil$ discrete steps sized so that the average angular rate does not exceed the configured maximum slew rate w_{max} (`obj.max_slew_rate`).

Implemented in `func_update_software_SC_control_attitude_Slew.m` (helper functions `func_generate_slerp_path`, `func_slerp`), which is only exercised when `obj.desired_slew_angle_change` (from `func_error_angle_between_quaternions`) exceeds the per-step slew-angle threshold $w_{max} \cdot dt$; the corresponding instantaneous body angular velocity is then reconstructed from consecutive quaternions in the path via the error-quaternion/axis-angle method in `func_quat_to_angular_velocity`.

6.2.3 CML 2–5 Suitability

6.2.3.1 What is included and correctly implemented

The class implements a genuine closed-loop attitude control architecture rather than a stub.

- **Two distinct control laws.** A PD-plus-feedforward law with gyroscopic compensation and electric-propulsion disturbance feedforward, and a Lyapunov-style "asymptotically stable" quaternion-feedback law, are both driven by quaternion attitude error against the `Software_SC_Estimate_Attitude` state rather than the true attitude, which is appropriate closed-loop fidelity for CML 2–5 trade studies.

- **Non-trivial actuator allocation.** A reaction-wheel pseudo-inverse mixing law with cached maximum torque capability, and a constrained least-squares/KKT thruster-mixing solver that correctly accounts for opposite-thrust pairs and per-thruster force limits, are both recognizable, non-trivial allocation methods.

- **Rule-based actuator selection and desaturation.** Simple, reasonable rule-based logic exists for choosing between wheels and thrusters based on attitude-error magnitude and torque-capability thresholds, plus a first-order wheel-saturation predictor and desaturation-via-thrusters procedure.

- **Tunable-fidelity guidance modes.** An "Oracle" direct-attitude-set mode and a SLERP-based slew-rate-limited trajectory generator are both available for lower-fidelity guidance-only studies, giving the tool flexibility to trade fidelity vs. runtime — exactly the kind of tunable fidelity CML 2–5 point designs need.

- **Multi-mission mode dispatch.** The mode-selector switch structure cleanly supports multiple missions (e.g. DART and NISAR, plus additional mission-specific pipelines) and multiple control fidelities per mission from one class, with storage/logging of key control quantities for post-run analysis.

6.2.3.2 What is not included / gaps

Several elements fall short of true flight-software fidelity, consistent with CML 2–5 expectations of first-order/medium-fidelity representations.

- **No gain-scheduling or integral term.** The gains (`obj.data.control_gain`) are simple scalar Kp/Kd or Kr/lambda pairs with no gain-scheduling, anti-windup, or rate limiting despite an unused `integral_error` property being initialized (no PID integral term is ever applied in the control laws).

- **Ad hoc torque-capability estimate.** The reaction-wheel maximum-torque capability calculation applies an ad hoc "configuration factor" ($1/\sqrt{N_{active}}$) explicitly flagged in-code as "a conservative estimate," not a geometrically derived bound.

- **Heuristic saturation/desaturation logic.** The wheel-saturation predictor and momentum-management logic are explicitly labeled "simplified"/heuristic (fixed thresholds like 0.8, 0.5, 0.3 of max velocity, fixed minimum time-steps) rather than derived from a rigorous saturation/momentum-budget model, and desaturation dumps momentum directly through thrusters with a fixed 10%-per-step wheel deceleration rather than a torque-balanced thruster/wheel co-control scheme.

- **Unresolvable mission-specific modes.** Several of the `mode_software_SC_control_attitude_selector` cases (e.g. certain mission-specific update/constructor functions) reference external functions that were not found in the repository search, meaning those modes are not resolvable in the current codebase state and cannot be verified for fidelity.

- **No flight-representative command/fault modeling.** There is no explicit actuator command quantization, telemetry/fault-detection logic, or flight-software-representative timing/latency modeling — all of which is appropriately out of scope for a CML 2–5 trade-space tool but would need to be added for higher-fidelity or flight-representative software verification.

6.3 Software_SC_Control_Orbit

Software_SC_Control_Orbit is the orbit/maneuver-planning flight-software layer of the MuSCAT spacecraft model. It owns the state that describes a desired orbital correction — a target Delta-V vector (desired_control_DeltaV), the time horizon and burn timing over which it must be executed (time_horizon_DeltaV, time_DeltaV, burn_duration), the resulting commanded thruster forces (F1_cmd_body, F2_cmd_body, desired_control_thrust), and a running tally of Delta-V actually delivered by the propulsion hardware (total_DeltaV_executed). It is the natural counterpart to True_SC_Chemical_Thruster and True_SC_RCS_Thruster: those classes physically fire thrusters and increment total_DeltaV_executed toward the target set here, while this class is responsible for deciding *when* a maneuver is needed, *how large* it should be, and for tracking flags such as desired_DeltaV_needs_to_be_executed, desired_DeltaV_achieved, and desired_attitude_for_DeltaV_achieved that gate execution (the last of these implies coordination with an attitude-control class Software_SC_Control_Attitude so the spacecraft is pointed correctly before a burn is commanded). It also carries relative-navigation/intercept quantities (rel_pos_desired, rel_vel_desired, position_error, velocity_error, error_Integral, intercept_SB_position/velocity, intercept_distance, desired_intercept_distance) that are consistent with closed-loop guidance relative to a target body's orbit as propagated by True_SC_Orbit_Propagation, and fuel-feasibility bookkeeping (estimated_fuel_required, min_fuel_threshold, flag_insufficient_fuel) that would be checked against the propellant state tracked by the thruster classes.

Structurally, this base class is a thin constructor/storage/dispatcher shell: func_main_software_SC_control_orbit is the entry point called once per control cycle, and it simply switches on mode_software_SC_control_orbit_selector (‘mission specific’ or ‘Inactive’) to call a mission-specific implementation function (e.g., func_update_software_SC_control_orbit_DART) resolved by name via str2func, which is not contained in this file. All of the actual orbit-determination, Lambert-solver targeting (e.g., the mission-referenced func_compute_TCM_Lambert_Battin), and thrust-allocation logic therefore lives downstream in those mode-specific functions; this class supplies the shared property set, initializes default values, records per-timestep history into obj.store, and updates the SC data-handling bookkeeping (func_update_instantaneous_data_generated) that models the data volume the controller itself generates.

6.3.1 Inputs / Outputs

6.3.1.1 Inputs / Dependencies

- init_data **(constructor-time):** mode_software_SC_control_orbit_selector (required: 'mission specific' or 'Inactive'); optional max_time_before_control (default 0), data, mode_selector_name.
- True_SC_Fuel_Tank **(same spacecraft):** reads instantaneous_fuel_mass to determine remaining propellant before commanding a maneuver.
- True_SC_Chemical_Thruster **(same spacecraft):** reads health, thruster_warm_up_time, flag_warming_up, maximum_thrust, minimum_thrust, orientation to select healthy thrusters and compute thrust vectors.
- Software_SC_Control_Attitude **(same spacecraft):** consulted for attitude error to decide whether the spacecraft is sufficiently pointed to execute a burn.

6.3.1.2 Outputs

- **Internal / stored outputs:** `flag_executive`, `max_time_before_control`, `data`.
- **Reported to / written into `True_SC_Chemical_Thruster` (warm-up):** sets `flag_warming_up = true` and `flag_executive = false`, commanding the thruster to begin its warm-up cycle before firing is permitted.
- **Reported to / written into `True_SC_Chemical_Thruster` (DeltaV commanding):** sets `commanded_thrust = thrust_vector(i)` and `flag_executive = 1` on healthy thrusters — `commanded_thrust` is documented in `True_SC_Chemical_Thruster.m` as "comes from the SC's orbit control software" and is consumed by that class's main function to produce actual thrust/DeltaV.
- **`True_SC_EP_Thruster`:** no write-back exists in the current mission-specific orbit-control logic.
- Note the coupling is bidirectional for the executive flag: `True_SC_Chemical_Thruster` itself sets `mission.true_SC{i_SC}.software_SC_control_orbit.flag_executive = 1` when it fires, feeding back into this software object's own scheduling state.
- This class reads propellant and thruster-health state from `True_SC_Fuel_Tank`/`True_SC_Chemical_Thruster`, then overwrites `True_SC_Chemical_Thruster.commanded_thrust`, `.flag_executive`, and `.flag_warming_up` each cycle it executes — a direct commanded-burn write-back into the hardware actuator object, distinct from the read-only pattern of hardware-tier classes.

6.3.2 Mathematical Equations

6.3.2.1 Delta-V Magnitude Tracking and Execution Threshold

The only quantitative relationships actually evaluated in this base file are the Delta-V magnitude bookkeeping and the minimum-Delta-V execution criterion used to decide whether a computed correction is worth commanding to the thrusters:

$$\|\Delta v_{\text{desired}}\| = \left\|\vec{\Delta v}_{\text{desired}}\right\|, \qquad \|\Delta v_{\text{executed}}\| = \left\|\vec{\Delta v}_{\text{executed}}\right\| \tag{6.7}$$

$$\text{execute burn} \iff \left\|\vec{\Delta v}_{\text{desired}}\right\| \geq \Delta v_{\min} \tag{6.8}$$

Here $\vec{\Delta v}_{\text{desired}}$ is the commanded three-axis Delta-V vector to be achieved, $\vec{\Delta v}_{\text{executed}}$ is the cumulative Delta-V actually delivered by the thrusters, and $\Delta v_{\min}$ is a fixed threshold below which a maneuver is not worth executing (e.g., to avoid commanding negligible burns that only waste propellant on valve cycling or attitude re-pointing).

In the MATLAB code, $\vec{\Delta v}_{\text{desired}}$ is the property `desired_control_DeltaV` and $\vec{\Delta v}_{\text{executed}}$ is `total_DeltaV_executed` (the same property that `True_SC_Chemical_Thruster.m` and `True_SC_RCS_Thruster.m` increment as they fire). Their norms are computed and logged every storage step in `func_update_software_SC_Control_Orbit_store` as `obj.store.deltaV_magnitude_desired(...) = norm(obj.desired_control_DeltaV)` and `obj.store.deltaV_magnitude_executed(...) = norm(obj.total_DeltaV_executed)`. The threshold $\Delta v_{\min}$ is the property `threshold_minimum_deltaV`, initialized to 0.01 (units of m/s per the property comment) in the constructor; the comparison itself is not evaluated inside this file but the property exists specifically to be consulted by the mode-specific update functions before flagging `desired_DeltaV_needs_to_be_executed`.

Beyond this bookkeeping, `Software_SC_Control_Orbit.m` contains no Lambert-problem solution, no targeting/porkchop search, and no burn-duration or thrust-direction derivation — it only declares the properties (`time_horizon`, `time_intercept`, `time_horizon_DeltaV`, `time_DeltaV`, `burn_duration`, `intercept_SB_position`, `intercept_SB_velocity`, `intercept_distance`, `desired_intercept_distance`, `flag_position_velocity_burn`, `desired_control_DeltaV_position_burn`/`_velocity_burn`) that such a solver would populate. Given the naming conventions and the mission-specific dispatch, the actual trajectory-correction-maneuver math (almost certainly including any Lambert–Battin-based targeting such as `func_compute_TCM_Lambert_Battin`) is implemented in external, mission-specific functions (e.g. `func_update_software_SC_control_orbit_DART`) that are not defined in this file.

6.3.3 CML 2–5 Suitability

6.3.3.1 What is included and correctly implemented

The class provides a coherent, self-consistent bookkeeping framework for orbit-control state that is well matched to point-design/trade-space fidelity.

- **Desired vs. executed Delta-V tracking.** It distinguishes desired versus executed Delta-V (`desired_control_DeltaV` vs. `total_DeltaV_executed`) as three-component vectors, cleanly separates position-burn and velocity-burn Delta-V components with associated timing, and exposes a minimum-Delta-V execution threshold to avoid modeling non-physical infinitesimal burns.
- **Fuel-feasibility bookkeeping.** It carries fuel-feasibility flags (`flag_insufficient_fuel`, `estimated_fuel_required`, `min_fuel_threshold`) alongside the Delta-V state, which is the right level of abstraction for CML 2–5 studies that need to ask "can this maneuver be afforded" without needing flight-software-representative fuel-slosh or tank-pressure models.
- **Reusable mode-dispatch architecture.** The mode-selector dispatch pattern is an appropriate architecture for reusing one control-orbit interface across different mission concepts, letting each concept plug in its own maneuver-planning fidelity while keeping a common Delta-V/thruster-commanding interface with the true-hardware thruster classes.
- **Adequate post-run visibility.** Storage of Delta-V magnitudes, commanded body-frame forces, and control errors every step gives adequate visibility into maneuver performance for trade studies without requiring high-rate flight-telemetry-like logging.

6.3.3.2 What is not included / gaps

As delivered in this file, the class is essentially an empty shell for maneuver planning.

- **No targeting/burn-optimization logic.** No targeting algorithm, Lambert solver, or burn-optimization logic is present here, so the fidelity of any actual trajectory-correction maneuver depends entirely on the unseen mode-specific functions it dispatches to, and this file alone cannot be assessed for whether that downstream logic meets even first-order fidelity.
- **No slew-time or Delta-V derivation.** There is no explicit attitude-slew/pointing-time model beyond a boolean achieved-flag (`desired_attitude_for_DeltaV_achieved`), no closed-form or iterative solution shown for converting a desired position/velocity change into a Delta-V vector, and no thrust-direction or burn-duration derivation (e.g., no $\Delta v = \int (F/m)\, dt$

or Tsiolkovsky-type relation appears in this file, even though `burn_duration` and `desired_control_thrust` exist as properties).

- **Hard failure on unsupported modes.** The `otherwise` branches in the mode switch simply `error(...)`, so unsupported mode strings fail hard rather than degrading gracefully, and the `'Inactive'` mode performs no control at all (acceptable for a placeholder but worth noting as a gap if invoked unintentionally).

For CML 2–5 purposes these omissions are generally acceptable since first-order physics suffices and the true maneuver math is expected to live in the mode-specific functions rather than this dispatcher/state class; however, anyone assessing overall maneuver-planning fidelity must inspect those external functions (particularly whatever implements the Lambert–Battin-based TCM referenced elsewhere in the codebase) rather than this class file alone.

6.4 Software_SC_Data_Handling

The Software_SC_Data_Handling class represents the flight-software-level data-management function onboard the spacecraft: rather than modeling the physical fill and drain of individual memory banks, it aggregates the truth-layer memory state into flight-software-relevant summary quantities (total_data_storage and mean_state_of_data_storage) that a downlink-planning or executive process would use. Each simulation step, its func_main_software_SC_data_handling method dispatches on mode_software_SC_data_handling_selector (currently 'Generic' or an additional mission-specific mode) to recompute these aggregate metrics, then calls func_update_instantaneous_data_generated on the corresponding True_SC_Data_Handling object to register any data this "equipment" instance has generated (using either an instantaneous rate or a per-sample amount, selected by instantaneous_data_type).

This class is best understood as a lightweight consumer/aggregator layered on top of True_SC_Data_Handling, which is the truth-layer class that actually owns and mutates the physical per-memory-bank state (filling and draining each True_SC_Onboard_Memory bank's instantaneous_capacity, bounded by maximum_capacity, and tracking the list/array of hardware data sources). Software_SC_Data_Handling does not itself change memory bank contents; instead, its mode-specific update functions loop over the true onboard memory banks and sum their instantaneous_capacity and maximum_capacity values to produce a single scalar total_data_storage (total queued/onboard data awaiting downlink, in kb) and a percentage-based mean_state_of_data_storage. This aggregate total_data_storage value is exactly what True_SC_Communication_Link.m reads to rate-limit the downlink: if the nominal link rate would transmit more data in one timestep than is actually stored onboard, the transmit rate is clamped so the buffer drains to exactly zero rather than going negative. Thus the true/software split mirrors the physical-state-vs-flight-software-view pattern used elsewhere in MuSCAT: True_SC_Data_Handling is the ground truth of what is stored where, while Software_SC_Data_Handling is the flight-software-representative summary of "how much data is onboard" that other software-layer classes consume for downlink planning and rate limiting.

6.4.1 Inputs / Outputs

6.4.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_data_generated_per_sample; optional mode_software_SC_data_handling_selector (default 'Generic'; additional mission-specific mode names are also supported); optional data.
- True_SC_Body **(same spacecraft):** reads name and num_hardware_exists.num_onboard_memory for identification and to size the memory-aggregation loop.
- True_SC_Onboard_Memory **(same spacecraft):** via func_update_mean_state_of_data_storage.m, reads instantaneous_capacity and maximum_capacity of every onboard memory unit to compute mean_state_of_data_storage.
- Software_SC_Communication **(same spacecraft):** this class's own mean_state_of_data_storage and total_data_storage are overwritten (zeroed) by Software_SC_Communication on completed downlink — a write-back received rather than issued.

6.4.1.2 Outputs

- **Internal / stored outputs:** `mean_state_of_data_storage` (%), `total_data_storage` (kb), `flag_executive`.
- **No write-back to `True_SC_Onboard_Memory` or any hardware actuator exists.** This class is a pure read/aggregate consumer of `True_SC_Onboard_Memory` capacity data; the actual filling/draining of memory capacity is performed by the separate hardware-tier class `True_SC_Data_Handling` (distinct from this software class despite the similar name), not by `Software_SC_Data_Handling` itself.
- **Written into by `Software_SC_Communication`:** `mean_state_of_data_storage` and `total_data_storage` are reset to 0 after a successful downlink, simulating the onboard buffer being emptied.
- Unlike the control classes, `Software_SC_Data_Handling` issues no commands to hardware; it only reads `True_SC_Onboard_Memory` capacities to report an aggregate storage-state metric, and is itself a write-back *target* of `Software_SC_Communication` rather than a writer of other objects.

6.4.2 Mathematical Equations

6.4.2.1 Total Onboard Data Storage

$$\text{total_data_storage} = \sum_{i=1}^{N_{\text{mem}}} C_{\text{inst},i} \tag{6.9}$$

where N_{mem} is the number of onboard memory banks on the spacecraft and $C_{\text{inst},i}$ is the instantaneous stored data (in kb) in memory bank i, as tracked by the corresponding `True_SC_Onboard_Memory` object.

In the `'Generic'` mode (function `func_update_mean_state_of_data_storage`, called from `func_main_software_SC_data_handling`), the loop runs over `1:mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_onboard_memory` and accumulates each bank's `mission.true_SC{i_SC}.true_SC_onboard_memory{i_memory}.instantaneous_capacity` into a local variable `total_instantaneous_capacity`, which is then assigned to `obj.total_data_storage`. In at least one mission-specific mode, the identical sum is computed but the loop is restricted to `1:(num_onboard_memory - 1)`, deliberately excluding the last onboard memory bank from the aggregate.

6.4.2.2 Mean State of Data Storage

$$\text{mean_state_of_data_storage} = 100 \times \frac{\displaystyle\sum_{i=1}^{N_{\text{mem}}} C_{\text{inst},i}}{\displaystyle\sum_{i=1}^{N_{\text{mem}}} C_{\text{max},i}} \tag{6.10}$$

where $C_{\text{max},i}$ is the maximum storage capacity (in kb) of memory bank i (property `maximum_capacity` of the corresponding `True_SC_Onboard_Memory` object), and the numerator is the same instantaneous-capacity sum used above.

This is computed in the same loop as above: a second running total, `total_maximum_capacity`, accumulates each bank's `maximum_capacity`, and the ratio `100 * total_instantaneous_capacity`

`/ total_maximum_capacity` is assigned to `obj.mean_state_of_data_storage`. Note that the per-step data-generation bookkeeping call, `func_update_instantaneous_data_generated`, does not itself modify `total_data_storage` or `mean_state_of_data_storage` when invoked without the optional memory-selection arguments (as it is here); it only accumulates into the truth object's own `instantaneous_data_generated` tally. The aggregate storage metrics in `Software_SC_Data_Handling` are therefore fully recomputed from the truth-layer memory banks each step, not incrementally updated alongside data generation.

6.4.3 CML 2–5 Suitability

6.4.3.1 What is included and correctly implemented

The class correctly implements a simple, self-consistent flight-software-style abstraction of onboard data storage suitable for point-design and trade-space studies.

- **Aggregate storage metrics.** A scalar `total_data_storage` and percentage `mean_state_of_data_storage` are derived each timestep directly from the sum of instantaneous and maximum capacities across all true onboard memory banks, which is exactly the level of fidelity needed to drive downlink-rate-limiting logic in `True_SC_Communication_Link.m`.
- **Mission-specific extension mechanism.** The class supports multiple selectable modes (`'Generic'` summing over all memory banks, and at least one mission-specific mode that excludes the last bank), demonstrating a workable mission-specific extension mechanism, and it properly initializes and updates its own storage-history arrays consistent with the rest of the framework's time-history logging pattern.
- **Consistent data-source registration.** The registration of this software object as a data-generating "equipment" item with the truth-layer `True_SC_Data_Handling` class is functionally wired up, allowing the software layer to represent itself as a data source alongside other instruments/subsystems.

6.4.3.2 What is not included / gaps

The class contains no onboard data-management logic beyond the aggregate capacity summary.

- **No prioritization or memory-selection policy.** There is no prioritization of data types, no compression, no onboard data deletion/overwrite policy, and no logic for deciding which memory bank to write new data into.
- **No packet/queue-level bookkeeping.** There is no explicit downlink-queue or packet-level bookkeeping — `total_data_storage` is a single lumped-sum figure across all banks, not a per-data-product or per-priority breakdown, so it cannot represent flight-software features such as store-and-forward prioritization, data quotas, or ground-commanded playback selection.
- **Unused executive flag.** The `flag_executive` property is set and reset but never meaningfully used within this file to gate behavior (it is always reset to 0 at the end of the main function without being read elsewhere in the class).
- **External mission-specific dependency.** At least one mission-specific mode's function is not present in the `MuSCAT_Matlab_v2` repository itself, so that dispatch branch is only resolvable when a separate, firewalled mission repository is on the MATLAB path.

Overall, these omissions are consistent with and acceptable at CML 2–5: the class provides medium-fidelity, first-order representative behavior (aggregate storage state feeding a downlink rate-limit) without requiring true flight-software-representative fidelity such as packet-level queue management or onboard autonomy logic.

6.5 Software_SC_Estimate_Attitude

The Software_SC_Estimate_Attitude class is the spacecraft's onboard attitude determination (AD) software: it produces the flight-software *estimate* of attitude, angular velocity, angular acceleration, gyro bias, and their associated uncertainties that downstream flight software (in particular Software_SC_Control_Attitude) consumes to compute control torques/wheel commands. This is distinct from the noise-free truth attitude state propagated by True_SC_ADC: Software_SC_Estimate_Attitude is meant to represent what the spacecraft's own algorithms *believe* the attitude to be, based only on sensor data that would be available onboard. The class is driven by a mode selector, mode_software_SC_estimate_attitude_selector, with values 'Truth' (bypass estimation and copy the true ADC state directly), 'Truth with Noise' (documented as a planned mode but not implemented in the executed switch statement), and 'KF' (a Kalman-filter-based estimator).

In 'KF' mode the class owns an AttitudeKalmanFilter object (obj.attitude_filter) that fuses gyro angular-rate measurements from True_SC_IMU (used continuously, every filter execution, for quaternion propagation and bias estimation) with attitude quaternion measurements from True_SC_Star_Tracker (used opportunistically, gated by a meas_update_occurred flag, as a discrete measurement update). Sun-sensor data (True_SC_Sun_Sensor) is not referenced anywhere in this class or in the estimation functions it calls — only star tracker and IMU measurements are actually fused. After each estimation step the class also re-derives the estimated body-frame angular momentum from the estimated angular velocity, the total spacecraft inertia, and the current reaction-wheel momenta, and it toggles flag_executive on the star tracker(s)/IMU(s) so that hardware is exercised on the appropriate cadence. The resulting attitude, angular_velocity, dot_angular_velocity, gyro_bias, and their uncertainty fields are the values that Software_SC_Control_Attitude reads to close the attitude control loop, so this class is the sole boundary between "truth" sensing and "estimated" state used for control.

6.5.1 Inputs / Outputs

6.5.1.1 Inputs / Dependencies

- init_data **(constructor-time):** mode_software_SC_estimate_attitude_selector (required: 'Truth', 'Truth with Noise', or 'KF'); optional data; for 'KF' mode: initial_est_q_BF_IF, initial_est_gyro_bias_BF, initial_sigma_theta_BF, initial_sigma_gyro_bias_BF, gyro_sigma, bias_sigma, startracker_sigma_SF, initial_est_omega_BF, startracker_C_BF_SF, fed into an internal AttitudeKalmanFilter object.
- True_SC_Body **(same spacecraft):** reads name, num_hardware_exists.num_star_tracker/.num_imu/.num_reaction_wheel, and total_MI for sensor-fusion loop sizing and momentum/rate bookkeeping.
- True_SC_Reaction_Wheel **(same spacecraft):** reads total_momentum of every wheel (summed into angular_momentum_BF).
- True_SC_ADC **(same spacecraft, 'Truth' mode):** reads attitude, angular_velocity, dot_angular_velocity as a direct truth passthrough.
- True_SC_Star_Tracker, True_SC_IMU **(same spacecraft, 'KF' mode):** reads their measurement vectors as filter inputs.

6.5.1.2 Outputs

- **Internal / stored outputs:** `attitude`, `angular_velocity`, `gyro_bias`, `dot_angular_velocity`, `angular_momentum_BF`, `attitude_filter`, plus corresponding `*_uncertainty` fields.
- **Reported to / written into `True_SC_Star_Tracker` / `True_SC_IMU` (`'KF'` mode only):** sets `flag_executive = 1` on each sensor to re-arm it for its next measurement update — a scheduling flag only, not a write of sensor data or physical state.
- **No write-back to `True_SC_ADC` or any actuator exists.** All `true_SC_adc.*` accesses are read-only (truth passthrough in `'Truth'` mode).
- **Consumed by `Software_SC_Control_Attitude`:** its `attitude` property is read to compute pointing error for the control loop.
- This class is predominantly a reader of `True_SC_Star_Tracker`, `True_SC_IMU`, `True_SC_Reaction_Wheel`, and `True_SC_Body` truth/measurement data; its only cross-object write is the sensor `flag_executive` re-arm flag, with no commanded-value write-back into any actuator or truth-state property — unlike `Software_SC_Control_Attitude`, this estimator does not drive hardware.

6.5.2 Mathematical Equations

6.5.2.1 Truth-Mode Attitude Estimate (Trivial Pass-Through)

In `'Truth'` mode, the estimator performs no filtering at all: it simply copies the true ADC state and forces the reported uncertainty to zero.

$$\hat{q}_{BF}^{IF} = q_{BF}^{IF,true}, \qquad \hat{\boldsymbol{\omega}}_{BF} = \boldsymbol{\omega}_{BF}^{true}, \qquad \dot{\hat{\boldsymbol{\omega}}}_{BF} = \dot{\boldsymbol{\omega}}_{BF}^{true}, \qquad \boldsymbol{\sigma}_{\theta} = \boldsymbol{\sigma}_{\dot{\omega}} = \mathbf{0} \tag{6.11}$$

Here $q_{BF}^{IF,true}$, $\boldsymbol{\omega}_{BF}^{true}$, and $\dot{\boldsymbol{\omega}}_{BF}^{true}$ are the truth quaternion, angular velocity, and angular acceleration held by `True_SC_ADC`, and $\boldsymbol{\sigma}_{\theta}, \boldsymbol{\sigma}_{\dot{\omega}}$ are the reported (here, zero) uncertainties. This is implemented in `func_update_software_SC_estimate_attitude_Truth`, which sets `obj.attitude`, `obj.angular_velocity`, `obj.dot_angular_velocity` equal to the corresponding `true_SC_adc` fields, with `obj.attitude_uncertainty` and `obj.angular_velocity_uncertainty` hard-set to zero vectors. This mode is honestly trivial: there is no sensor fusion, no filtering, and no uncertainty quantification — it is a pure truth pass-through used when attitude-estimation error is not of interest.

6.5.2.2 KF-Mode Quaternion Propagation (Gyro Kinematics)

When `mode_software_SC_estimate_attitude_selector = 'KF'`, `obj.attitude_filter.run(...)` is invoked every filter execution, and it always performs a rate-driven quaternion propagation step using the measured body rate:

$$\hat{\boldsymbol{\omega}}_{BF} = \boldsymbol{\omega}_{BF}^{meas} - \hat{\mathbf{b}}_{BF}, \qquad \dot{\hat{q}}_{BF}^{IF} = \tfrac{1}{2}\, \hat{q}_{BF}^{IF} \otimes \left[C(\hat{q}_{BF}^{IF})^{-1} \hat{\boldsymbol{\omega}}_{BF};\, 0 \right], \qquad \hat{q}_{BF}^{IF} \leftarrow \mathrm{norm}\Big(\hat{q}_{BF}^{IF} + \dot{\hat{q}}_{BF}^{IF}\, \Delta t \Big) \tag{6.12}$$

Here $\boldsymbol{\omega}_{BF}^{meas}$ is the raw gyro measurement (body frame), $\hat{\mathbf{b}}_{BF}$ is the current gyro bias estimate, $\hat{q}_{BF}^{IF}$ is the body-to-inertial attitude quaternion estimate (scalar-last), $\otimes$ is quaternion multiplication, and Δt is the fixed filter time step. In the code, these correspond to

`obj.est_omega_BF = meas_omega_B - obj.est_gyro_bias_BF`, `omega_IF = qvtran(qcon(obj.est_q_BF_IF), obj.est_omega_BF)`, `qdot_J = 0.5*qmult(obj.est_q_BF_IF, omega_q)`, and `obj.est_q_BF_IF = qnorm(obj.est_q_BF_IF + qdot_J*obj.filter_dt)`, all inside the private `propagate` method of the `AttitudeKalmanFilter` object. This is a simple first-order (Euler) quaternion integration, not a higher-order or exact exponential-map propagation.

6.5.2.3 KF-Mode Error-State Covariance Propagation

Alongside quaternion propagation, the filter propagates a 6-state (3 attitude-error + 3 gyro-bias) linearized error covariance using a discretized state-transition matrix and process noise:

$$\mathbf{\Phi} = \begin{bmatrix} \mathbf{I}_3 - [\hat{\boldsymbol{\omega}}_{BF}\times]\Delta t & -\mathbf{I}_3\,\Delta t \\ \mathbf{0}_3 & \mathbf{I}_3 \end{bmatrix}, \qquad \mathbf{Q}_k = \begin{bmatrix} \left(\sigma_v^2\Delta t + \frac{1}{3}\sigma_u^2\Delta t^3\right)\mathbf{I}_3 & -\frac{1}{2}\sigma_u^2\Delta t^2\,\mathbf{I}_3 \\ -\frac{1}{2}\sigma_u^2\Delta t^2\,\mathbf{I}_3 & \sigma_u^2\Delta t\,\mathbf{I}_3 \end{bmatrix}, \qquad \mathbf{P}^- = \mathbf{\Phi}\,\mathbf{P}\,\mathbf{\Phi}^T + \mathbf{Q}_k \tag{6.13}$$

$[\hat{\boldsymbol{\omega}}_{BF}\times]$ is the skew-symmetric matrix of the current bias-corrected rate estimate, σ_v (gyro noise, `gyro_sigma`) and σ_u (gyro bias random-walk noise, `bias_sigma`) are scalar, isotropic noise parameters, and $\mathbf{P}$ is the 6×6 error covariance. In the code this is the `Phi11/Phi12/Phi21/Phi22` and `Qk11/Qk12/Qk21/Qk22` block assembly and `obj.P = Phi*obj.P*(Phi.') + Qk` inside `AttitudeKalmanFilter.propagate`. This is a standard first-order linearized (EKF-style) propagation of a multiplicative attitude-error state, propagated in the body frame (the class's own header comment notes this is an approximation valid for low-agility vehicles rather than the more rigorous inertial-frame propagation).

6.5.2.4 KF-Mode Star-Tracker Measurement Update

When a fresh star-tracker measurement is available (`startracker_update_occurred = 1`), the filter performs a linear Kalman measurement update using the quaternion residual between the star-tracker measurement and the current attitude estimate:

$$\mathbf{S} = \mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R}, \qquad \mathbf{K} = \mathbf{P}\mathbf{H}^T\mathbf{S}^{-1}, \qquad \mathbf{P}^+ = (\mathbf{I}_6 - \mathbf{K}\mathbf{H})\,\mathbf{P}, \qquad \delta q = q_{BF}^{meas} \otimes \left(\hat{q}_{BF}^{IF}\right)^{-1}, \qquad \begin{bmatrix} \delta\hat{\boldsymbol{\theta}} \\ \delta\hat{\mathbf{b}}_{BF} \end{bmatrix} = \mathbf{K}\,\boldsymbol{\theta}(\delta q) \tag{6.14}$$

$\mathbf{H} = [\mathbf{I}_3\ \mathbf{0}_3]$ maps the 6-state error vector to the observed attitude-error subspace, $\mathbf{R}$ is the star-tracker measurement noise covariance rotated from sensor frame to body frame via the star-tracker alignment DCM, $\boldsymbol{\theta}(\delta q)$ is the axis-angle (small-angle) vector extracted from the residual quaternion δq, and $\delta\hat{\boldsymbol{\theta}}$, $\delta\hat{\mathbf{b}}_{BF}$ are the resulting attitude and gyro-bias corrections. In code this is `S = obj.H*obj.P*obj.H_T + obj.startracker_R_BF`, `K = obj.P*(obj.H_T)/S`, `obj.P = (obj.I6 - K*obj.H)*obj.P`, `dq_residual = qmult(meas_q_B_J, qcon(obj.est_q_BF_IF))`, `state_vector_correction = K*dtheta_residual_B(:)`, and the subsequent quaternion-correction and `obj.est_gyro_bias_BF = obj.est_gyro_bias_BF + bias_correction_BF` lines inside `AttitudeKalmanFilter.update`. After propagation (and update, if it ran), `Software_SC_Estimate_Attitude` pulls the filter's internal state into its own `attitude`, `angular_velocity`, `gyro_bias`, `attitude_uncertainty`, and `gyro_bias_uncertainty` properties. Note that `dot_angular_velocity` in this mode is simply hard-set to zero (not filtered or differentiated).

6.5.2.5 Estimated Body-Frame Angular Momentum

Independent of which estimation mode is active, `func_update_angular_momentum` recomputes the estimated total body-frame angular momentum each cycle from the estimated angular velocity and

the current reaction-wheel momenta:

$$\hat{\mathbf{h}}_{BF} = \sum_{i=1}^{n_{RWA}} \mathbf{h}_{RWA,i,BF} + \mathbf{I}_{BF}\,\hat{\boldsymbol{\omega}}_{BF} \tag{6.15}$$

$\mathbf{h}_{RWA,i,BF}$ is the total momentum of the i-th reaction wheel, $\mathbf{I}_{BF}$ is the total spacecraft inertia tensor, and $\hat{\boldsymbol{\omega}}_{BF}$ is the estimated body angular velocity just produced by the Truth or KF branch above. This corresponds exactly to `rwa_mom_BF = rwa_mom_BF + true_SC_reaction_wheel{ii}.total_momentum(:).'` accumulated in a loop, followed by `obj.angular_momentum_BF = (rwa_mom_BF + (true_SC_body.total_MI*obj.angular_velocity(:))')`. A code comment notes solar-array momentum is not yet included when arrays are actively slewing.

6.5.3 CML 2–5 Suitability

6.5.3.1 What is included and correctly implemented

The class provides a mode-selectable estimator that lets a trade study run either a zero-cost truth-following AD model or a physically meaningful filtered estimator.

- **Tunable-fidelity mode selector.** A `'Truth'` vs. `'KF'` mode selector is exactly the kind of tunable-fidelity structure appropriate for CML 2–5 point-design and trade-space work; the `'Truth'` mode is a correctly and honestly implemented trivial pass-through (no hidden filtering, uncertainty explicitly zeroed).
- **Structurally correct Kalman filter.** The `'KF'` mode implements a standard, structurally correct multiplicative/error-state Kalman filter for spacecraft attitude determination: gyro-driven quaternion propagation with bias removal, a properly discretized linear error-state transition matrix and process-noise covariance for a 6-state (attitude-error + gyro-bias) filter, and a star-tracker-triggered measurement update using a Kalman gain, quaternion residual, and multiplicative attitude correction with bias correction — this is first-order/medium-fidelity attitude filtering consistent with what CML 2–5 studies need.
- **Exposed uncertainty for trade analysis.** The class correctly propagates and exposes covariance-derived uncertainties, which supports Monte Carlo and covariance-based trade analyses.
- **Correct hardware/software interfacing.** It correctly interfaces with hardware truth objects, turns on the star tracker and IMU hardware models it depends on, and feeds a coherent estimated state forward for `Software_SC_Control_Attitude` to consume, plus a self-consistent estimated angular-momentum computation.

6.5.3.2 What is not included / gaps

Several gaps limit the estimator's fidelity relative to a real onboard attitude-determination system.

- **Sun sensor not fused.** Sun-sensor data (`True_SC_Sun_Sensor`) is never read or fused anywhere in this class or its called estimation functions, despite being a commonly expected AD input; only star tracker and IMU measurements are used.
- **Unimplemented and stub modes.** The `'Truth with Noise'` mode is documented in a property comment as a valid selector value but has no corresponding case in the switch

statement — selecting it would fall through to `otherwise` and raise an error, so it is an unimplemented placeholder. The generic `func_update_software_SC_estimate_attitude_KF` function (as opposed to the mission-specific variant actually called) is an empty stub that just copies truth-ADC values with no filtering at all — only the mission-specific KF path is a real filter, and both that function and the `AttitudeKalmanFilter` class it depends on were found to reside outside the main repository tree, indicating the general-purpose KF integration is not yet fully merged/self-contained.

- **Single-sensor hardcoding.** The KF call hardcodes the first star tracker and first IMU, so multiple star trackers/IMUs are not actually fused even though the class enables `flag_executive` on all of them.
- **No angular-acceleration estimate.** `dot_angular_velocity` is not estimated in KF mode (hard-set to zero).
- **Simplified noise model and frame approximation.** The gyro noise and bias random-walk parameters are scalar/isotropic rather than per-axis, and the covariance is explicitly propagated in the body frame as a stated first-order approximation rather than the more rigorous inertial-frame formulation, which is an acceptable simplification at this maturity level but is a known limitation.
- **No outlier rejection or fault handling.** There is no measurement outlier rejection, innovation gating, or sensor fault/failure handling in the update step, and solar-array momentum contributions are explicitly noted as not yet included in the angular-momentum estimate — gaps that are unremarkable for CML 2–5 fidelity but would need to be closed for flight-representative software.

6.6 Software_SC_Estimate_Orbit

Software_SC_Estimate_Orbit is the onboard orbit-determination/navigation flight-software model for each spacecraft: it produces the spacecraft's estimate of its own position and velocity (in the inertial frame and relative to a designated target/small body), the target's own position and velocity, and associated uncertainties, which downstream flight software (notably Software_SC_Control_Orbit, e.g. Lambert/Battin TCM targeting and target-intercept prediction in the DART implementation) consumes instead of ever touching the truth propagator directly. The class is deliberately mode-switched via mode_software_SC_estimate_orbit_selector: a 'Truth' mode that copies the state directly from True_SC_Orbit_Propagation (and the true target ephemeris) with zero uncertainty, a 'TruthWithErrorGrowth' mode that still copies the truth position/velocity but attaches a time-since-last-visibility-driven uncertainty that resets low whenever a camera reports the target in view and grows linearly while it is not, and placeholders for additional mission-specific modes that dispatch to mission-specific update/constructor functions not present in this repository snapshot. In every mode implemented here, the estimate is a direct pass-through/deterioration of the truth state rather than a measurement-fused filter (no camera, radiometric, or IMU measurements are actually ingested into a correction step); the class's job is to gate what "known" state and its associated confidence bounds are handed to guidance and control, and to report its per-sample data volume to True_SC_Data_Handling for telemetry/downlink budgeting.

6.6.1 Inputs / Outputs

6.6.1.1 Inputs / Dependencies

- init_data **(constructor-time):** mode_software_SC_estimate_orbit_selector (required: 'Truth', 'Truth with Noise', 'TruthWithErrorGrowth', or 'mission specific'); optional data, mode_selector_name, compute_wait_time (default 0).
- True_SC_Orbit_Propagation **(same spacecraft):** reads name_relative_target, index_relative_target, position, velocity, position_relative_target, velocity_relative_target as the orbit-truth source.
- True_Target **(mission-level, indexed by** index_relative_target**):** reads position, velocity for relative-navigation truth.
- Software_SC_Executive **(same spacecraft):** reads time to set obj.compute_time, gating how often the estimate is refreshed.
- True_SC_Body, True_SC_Camera **(same spacecraft, 'TruthWithErrorGrowth' mode):** reads num_hardware_exists.num_camera and flag_target_visible to model navigation-error growth when a target is out of view.

6.6.1.2 Outputs

- **Internal / stored outputs:** position, velocity, position_relative_target, velocity_relative_target, position_target, velocity_target, compute_time, plus corresponding *_uncertainty fields.
- **No write-back whatsoever.** Every true_SC_orbit_propagation.* and true_target{...}.* reference in this class and its mode-handler functions is strictly read-only;

no hardware or truth-state property is ever assigned — the strictest pure-reader among the software classes, not even touching a hardware `flag_executive` scheduling flag.

- **Consumed by downstream logic (indirect):** its `position`/`velocity` outputs are read by mission executives, pointing-vector functions, communication-strategy functions, and (for DART) maneuver/Lambert-targeting functions, mediated through mission-specific helper functions rather than a direct reference from `Software_SC_Control_Orbit`.

- This class only reads truth state from `True_SC_Orbit_Propagation` and `True_Target` and never overwrites any property on them or on any actuator — orbit estimation, unlike orbit/attitude control, performs no write-back into the hardware or truth layer.

6.6.2 Mathematical Equations

6.6.2.1 Truth-Mode State Pass-Through (No Estimation Error)

$$\hat{\mathbf{r}} = \mathbf{r}_{\text{true}}, \quad \hat{\mathbf{v}} = \mathbf{v}_{\text{true}}, \quad \sigma_{\mathbf{r}} = \mathbf{0}, \quad \sigma_{\mathbf{v}} = \mathbf{0} \tag{6.16}$$

Here $\hat{\mathbf{r}}$ and $\hat{\mathbf{v}}$ are the estimated spacecraft position [km] and velocity [km/sec] (inertial, and separately relative to the target), $\mathbf{r}_{\text{true}}$ and $\mathbf{v}_{\text{true}}$ are the corresponding truth states from the orbit propagator, and $\sigma_{\mathbf{r}}$, $\sigma_{\mathbf{v}}$ are the reported 1σ (or bound) uncertainties, which are set identically to zero. This is not truly an "estimator" in the navigation-filter sense; it is a fidelity mode that assumes perfect knowledge, useful for isolating guidance/control performance from navigation error.

In the code, this is implemented in `func_update_software_SC_estimate_orbit_Truth.m`, which sets `obj.position`, `obj.velocity`, `obj.position_relative_target`, `obj.velocity_relative_target`, `obj.position_target`, `obj.velocity_target` directly equal to `mission.true_SC{i_SC}.true_SC_orbit_propagation.position/velocity` and `mission.true_target{obj.index_relative_target}.position/velocity`, while every `_uncertainty` field is set to a zero vector. This mode is selected via `mode_software_SC_estimate_orbit_selector = 'Truth'` in `func_main_software_SC_estimate_orbit`, and is also always run once at construction time regardless of the configured mode to initialize state before mission-specific constructor logic runs.

6.6.2.2 Visibility-Gated Linear Error-Growth Model

$$\sigma_p(t) = \begin{cases} \sigma_{p,0} & \text{if target visible at } t \\ \sigma_{p,0} + \dot{\sigma}_p \left(t - t_{\text{vis}}\right) & \text{if target not visible at } t \end{cases} \tag{6.17}$$

$$\sigma_v(t) = \begin{cases} \sigma_{v,0} & \text{if target visible at } t \\ \sigma_{v,0} + \dot{\sigma}_v \left(t - t_{\text{vis}}\right) & \text{if target not visible at } t \end{cases} \tag{6.18}$$

where $\sigma_p(t)$ and $\sigma_v(t)$ are the isotropic (equal in all three axes) position [km] and velocity [km/sec] uncertainties at the current compute time t; $\sigma_{p,0} = 0.01$ km and $\sigma_{v,0} = 0.001$ km/sec are the base (target-visible) uncertainties; $\dot{\sigma}_p$ and $\dot{\sigma}_v$ are constant position/velocity error-growth rates; and t_{vis} is the last time any onboard camera reported the target in view. This is a simple linear (random-walk-like, but deterministic rather than stochastic) uncertainty inflation with time-since-last-fix — it grows the reported error bound the longer the target has been out of camera view, and snaps back to the base value the instant visibility is regained. It is not a covariance propagation and does not affect the estimated position/velocity values themselves, which remain equal to truth in this mode as well — only the reported uncertainty bound changes.

In the code (`func_update_software_SC_estimate_orbit_TruthWithErrorGrowth.m`), t_{vis} is stored in `obj.data.last_target_visible_time` (initialized to `-inf` in the constructor), and target visibility is determined by looping over `mission.true_SC{i_SC}.true_SC_camera{i_HW}.flag_target_visible`. The growth rates $\dot{\sigma}_p$ and $\dot{\sigma}_v$ are `obj.data.position_error_growth_rate` (0.01 km/sec) and `obj.data.velocity_error_growth_rate` (0.001 km/sec^2), set at construction. The elapsed time $t - t_{\text{vis}}$ is `time_since_target_visible`, clamped to a default of 3600 sec if negative. When visible, `obj.position_uncertainty`, `obj.velocity_uncertainty`, `obj.position_relative_target_uncertainty`, and `obj.velocity_relative_target_uncertainty` are all set to the base values; when not visible they are set to `base + growth_rate * time_since_target_visible` in each axis. The target's own position/velocity uncertainty uses the same base but only half the growth increment, reflecting an assumption that target ephemeris knowledge degrades more slowly than relative navigation knowledge. The spacecraft's own inertial state is still copied directly from `true_SC_orbit_propagation`, i.e. truth, unmodified by this uncertainty growth.

6.6.3 CML 2–5 Suitability

6.6.3.1 What is included and correctly implemented

The class correctly separates the "truth" state from the "estimated" state used by guidance and control, which is the right architectural pattern for CML 2–5 trade studies.

- **Clean truth/estimate separation.** `Software_SC_Control_Orbit` and DART's Lambert/Battin TCM and target-intercept logic explicitly read the estimator's position/velocity/target fields, never the truth object (`True_SC_Orbit_Propagation`), letting a study test guidance algorithms against imperfect (or perfect) knowledge without building a real filter.
- **Cheap, tunable fidelity mode.** The mode selector gives a clean, cheap way to run a point design first with perfect knowledge and then add a first-order knowledge-degradation model that ties uncertainty growth to a physically meaningful trigger (loss of camera lock on the target), which is exactly the "first-order/medium fidelity, not flight-representative" level of realism appropriate to this maturity band.
- **Auditable, tunable error-growth model.** The error-growth model is simple, auditable, and cheaply tunable for sensitivity studies on how navigation gaps affect downstream maneuver planning, and both modes correctly report per-sample data volume to the data-handling/telemetry system and store full time histories of both state and uncertainty for post-run analysis.

6.6.3.2 What is not included / gaps

There is no actual estimation filter anywhere in this file or its mission-specific update functions.

- **No measurement model or covariance propagation.** No measurement model, Kalman gain, covariance propagation/update, or cross-correlation between position and velocity uncertainty is present (all uncertainties are isotropic, diagonal, and hand-set rather than derived from sensor noise statistics or a dynamics-consistent state-transition matrix).
- **Hard-coded, cosmetic error growth.** The `'TruthWithErrorGrowth'` uncertainty growth is linear in time with hard-coded constants baked directly into the source rather than derived from any camera/OD error budget, and the reported uncertainty never actually perturbs the

returned position/velocity values — the "estimate" is truth-valued in all cases, only its claimed error bar changes, so any control logic that is sensitive to actual estimation error (rather than just its reported bound) would not see a difference between modes.

- **Unresolvable mission-specific modes.** The dispatch code references several additional mission-specific modes and calls out to the corresponding functions, none of which exist in the current codebase snapshot, so those modes cannot be exercised or verified here.
- **No consistency checks or latency model.** There is no target-relative-state consistency check (e.g. ensuring `position_relative_target` equals `position - position_target` under uncertainty) and no explicit latency/compute-time delay model beyond stamping `obj.compute_time` — the estimate is always current-time, with no representation of onboard OD latency or update cadence limitations that a flight-representative navigation filter would have.

6.7 Software_SC_Executive

Software_SC_Executive is the central spacecraft mode manager (i.e., the ConOps state machine) for each simulated spacecraft. It maintains the authoritative current-mode state (this_sc_mode, a string, and its paired integer encoding this_sc_mode_value) that every other hardware and software class in the simulation queries to determine mode-dependent behavior — for example, the mode-dependent power tables referenced elsewhere in this report key off software_SC_executive.this_sc_mode/this_sc_mode_value, and other classes check flag_slew_sc_mode_exists to know whether a dedicated Slew mode is modeled for a given mission configuration. The class itself does not hard-code the mode list or the transition logic: the set of valid modes is supplied externally via init_data.sc_modes (a cell array of strings), and the actual rules governing when and how the spacecraft transitions between modes (e.g., Sun-Safe → Slew → Science Observation) are delegated to mission-specific plug-in functions.

Mode transitions are determined/commanded through a dynamic dispatch pattern keyed on mode_software_SC_executive_selector (currently only the 'mission specific' case is implemented). At construction, in the per-step store update, and in the main step function, the class builds a function name string such as func_software_SC_executive_<mode_selector_name> and, using str2func, calls that mission-specific handler, passing itself and the mission and i_SC context. That external function is responsible for evaluating whatever conditions (sensor readings, timers, commands, fault flags, etc.) drive a mode change and for writing the new value into obj.this_sc_mode. After the mission-specific handler returns, Software_SC_Executive immediately re-derives this_sc_mode_value from this_sc_mode via func_find_this_sc_mode_value, updates the spacecraft's onboard time/date from the onboard clock hardware object, notifies the SC data-handling class of the new instantaneous data generation, and records this_sc_mode_value, time, and date into the store structure for the full mission timeline so other classes and post-processing can look up "what mode was the spacecraft in at time t."

6.7.1 Inputs / Outputs

6.7.1.1 Inputs / Dependencies

- init_data **(constructor-time):** sc_modes (required, cell array of mode-name strings, e.g. DART: {'Point camera to Target','Maximize SP Power','Point Thruster along DeltaV direction','DTE Comm'}); mode_software_SC_executive_selector (required, only 'mission specific' implemented); optional data, mode_selector_name (default mission.name), compute_wait_time (default 0), flag_slew_sc_mode_exists (default 0).
- True_SC_Body **(same spacecraft):** reads name for identification.
- True_SC_Onboard_Clock **(same spacecraft):** reads measurement_vector(1)/(2) (time/date) after setting that clock's flag_executive = 1 to schedule the read.
- **Broad mission-state inputs (via mission-specific** func_software_SC_executive_<Mission>.m**):** ground-station visibility, Software_SC_Power.mean_state_of_charge, orbit position, and other subsystem state used to select the active mode.

6.7.1.2 Outputs

- **Internal / stored outputs:** this_sc_mode (string, set by mission-specific logic), this_sc_mode_value (integer index via func_find_this_sc_mode_value), name, time, date, data, mode

_selector_name.

- **Reported to / written into** `True_SC_Onboard_Clock`**:** sets `flag_executive = 1` to schedule a time/date read each cycle.
- **Reported to / written into every other** `Software_SC_*` **subsystem object (and some** `True_SC_*` **sensor objects):** e.g. `software_SC_control_attitude.flag_executive = 1`, `software_SC_communication.flag_executive = 1` (only in `'DTE Comm'/'Telecom'` modes), `true_SC_camera{1}.flag_executive = 1` — this is a scheduling/enable switch set each cycle (not a commanded physical value), and each subsystem self-clears it to 0 at the end of its own main method.
- `this_sc_mode/this_sc_mode_value` **read widely across the codebase:** pointing-vector logic switches primary/secondary pointing target by mode string; communication-strategy functions gate ground-link computation; power-consumption lookup tables in `True_SC_Generic_Sensor`, `True_SC_Onboard_Memory`, `True_SC_Radio_Antenna`, `True_SC_Remote_Sensing`, `True_SC_Science_Processor`, `True_SC_Science_Radar` index by `this_sc_mode_value`; `flag_slew_sc_mode_exists` combined with `Software_SC_Control_Attitude.flag_slew` overrides the lookup with a dedicated "Slew" index; `Storage` logs mode per spacecraft for dashboards/plotting.
- The Executive does not command actuators directly — its write-back is the `flag_executive` enable flag broadcast to essentially every other `Software_SC_*` and several `True_SC_*` objects each cycle (gating whether they run), while its `this_sc_mode/this_sc_mode_value` outputs are read (not written) by numerous other classes to drive pointing, communication, and power-consumption decisions.

6.7.2 Mathematical Equations

This class contains no genuine physical or dynamical equations. Its role is purely that of a dispatcher/bookkeeping shell: it holds the current mode state, forwards control to externally-supplied mission-specific mode-logic functions (not present in this file), and stores the resulting mode history. The only quantitative operation performed directly in this file is a discrete lookup/indexing operation that converts the current mode name into its integer code, which is documented below for completeness.

6.7.2.1 Mode-Value Lookup (String-to-Index Encoding)

$$\text{this_sc_mode_value} = j \quad \text{such that} \quad \text{sc_modes}\{j\} = \text{this_sc_mode}, \qquad j \in \{1, 2, \ldots, N_{\text{modes}}\} \tag{6.19}$$

sc_modes is the ordered cell array of all valid spacecraft mode names for the mission (e.g., Sun-Safe, Slew, Science Observation); this_sc_mode is the current mode as a string; $N_{\text{modes}} = \text{length(sc_modes)}$ is the total number of defined modes; j is the resulting integer index, i.e. this_sc_mode_value.

The mode list is set once from `init_data.sc_modes` in the constructor and stored unchanged in `obj.sc_modes` and `obj.store.sc_modes`. The integer encoding is (re)computed by the helper `func_find_this_sc_mode_value(obj, obj.this_sc_mode)`, called once at construction (initializing from `obj.this_sc_mode = obj.sc_modes{1}`) and once every step inside `func_main_software_SC_executive`, immediately after the mission-specific mode-transition function has potentially updated `obj.this_sc_mode`. The resulting `this_sc_mode_value` is what other classes' mode-dependent

power/data tables key on, and it is what gets archived into `obj.store.this_sc_mode_value` every storage step.

6.7.3 CML 2–5 Suitability

6.7.3.1 What is included and correctly implemented

The class correctly implements the core bookkeeping a trade-space-level ConOps state machine needs.

- **Queryable mode timeline.** A single authoritative current-mode string and its integer encoding are both refreshed every step and stored at every storage timestep, giving a queryable mode timeline for post-processing and for other classes' mode-dependent power/data-rate lookups.
- **Mission-configurable mode list.** The mode list itself is not hard-coded but is supplied through `init_data.sc_modes`, so different missions/point designs can define arbitrary mode sets without modifying this class.
- **Reasonable dispatch pattern.** The dispatch-to-mission-specific-function pattern (using `exist(...)` checks and `str2func`) is a reasonable, lightweight way to let each mission plug in its own mode-transition rules at the constructor, per-step store, and main-step stages, appropriate for a flexible trade-space tool that must support many different spacecraft/ConOps concepts without rewriting the executive itself.
- **Concrete downstream coupling.** The class correctly keeps its time synchronized to the SC's onboard clock hardware object each main step and pushes mode-driven data-generation updates to the SC data-handling class, tying the abstract "mode" concept to concrete downstream effects (power/data), exactly the level of fidelity expected at CML 2–5.
- **Lightweight slew-mode flag.** The `flag_slew_sc_mode_exists` flag is a simple, appropriately lightweight mechanism for letting other classes branch on whether a mission's mode set includes a distinct Slew mode.

6.7.3.2 What is not included / gaps

All actual mode-transition logic lives entirely outside this file.

- **No transition rules or validation.** The conditions, thresholds, or timers that decide when the spacecraft moves from one mode to another are all in mission-specific functions that are dynamically invoked but not shown here; the executive itself contains no transition rules, no allowed/disallowed mode-transition matrix, and no validation that a mission-specific handler only sets `this_sc_mode` to a value actually present in `sc_modes`.
- **No dwell-time enforcement or transition log.** There is no minimum-dwell-time or time-in-mode enforcement, and no transition history/log beyond the raw per-step mode-value array in `store` (no record of, e.g., number of mode transitions, or fault-triggered vs. commanded transitions).
- **Unused property.** The `compute_wait_time` property is initialized in the constructor but is never read or used anywhere in this file's methods, indicating an unused/placeholder field rather than a modeled computational-delay effect.

- **No generic fallback implementation.** The dispatch switch-statements in the constructor, store-update, and main-step methods only implement the `'mission specific'` case; any other value of `mode_software_SC_executive_selector` produces only a warning (constructor/store) or a hard error (main step) rather than a generic fallback behavior, so the class provides no default/generic state-machine implementation of its own.
- **No fault-protection hierarchy.** There is no fault-protection or safing hierarchy encoded here (e.g., automatic forced transition to a safe mode on anomaly detection) — any such behavior, if it exists, must be implemented entirely inside the external mission-specific functions and is therefore outside the scope of what can be verified from this file.

These omissions are consistent with, and acceptable for, a CML 2–5 point-design/trade-space tool, since flight-software-representative mode-transition logic, fault protection, and transition validation are not required at this maturity level.

6.8 Software_SC_Power

Software_SC_Power is the flight-software layer that monitors and (nominally) manages spacecraft power state onboard the simulated flight computer, as distinct from True_SC_Power, which is the truth-layer bookkeeping class that actually tallies power generation, consumption, and balance for the bus. Where True_SC_Power represents "what physically happens" to the power system, Software_SC_Power represents "what the onboard software perceives and would act on." In this implementation, the software's view of power state is a single aggregate quantity, mean_state_of_charge, computed each time it runs by pulling the instantaneous and maximum capacities directly from the truth-layer battery objects and averaging them across all batteries on the spacecraft. Execution is gated by flag_executive, which is set externally (by mission-specific executive functions) and consumed once per call to func_main_software_SC_power, mirroring the duty-cycled task pattern used by the other Software_SC_* flight-software classes.

The class provides a mode_software_SC_power_selector switch ('Generic' and an additional mission-specific mode) intended as an extension point for mission-specific power-management behavior (e.g., mode transitions, load shedding, or safe-mode entry triggered by low state of charge), but as written, both branches perform the same action: they simply call func_update_mean_state_of_charge to refresh the aggregate SoC estimate. The mission-specific branch contains a commented-out call to a presumably more elaborate, mission-specific update function, indicating that richer power-management decision logic was planned but is not currently active for any mission using this file. After updating SoC, the class also invokes func_update_instantaneous_data_generated on the spacecraft's data-handling object, logging the housekeeping/telemetry data volume this software task itself generates.

6.8.1 Inputs / Outputs

6.8.1.1 Inputs / Dependencies

- init_data **(constructor-time):** instantaneous_data_generated_per_sample; mode_software_SC_power_selector (required: 'Generic' or an additional mission-specific mode name); optional data.
- True_SC_Body **(same spacecraft):** reads name and num_hardware_exists.num_battery to size the battery-aggregation loop.
- True_SC_Battery **(same spacecraft):** via func_update_mean_state_of_charge.m, reads instantaneous_capacity and maximum_capacity of every battery to compute mean_state_of_charge.
- Software_SC_Executive **(same spacecraft):** sets this class's flag_executive = 1 to trigger its update each cycle — a write received, not issued.

6.8.1.2 Outputs

- **Internal / stored outputs:** mean_state_of_charge (%), flag_executive.
- **No write-back to** True_SC_Battery**,** True_SC_Solar_Panel**, or any hardware on/off flag exists.** This class is a purely passive read-only aggregator of battery capacity data.

- **Read by `Software_SC_Executive` / mission-specific mode logic:** `mean_state_of_charge` is read by mission executives to select power-related modes (e.g. low charge selects a "Maximize SP Power" mode).
- Unlike the control classes, `Software_SC_Power` issues no commands to hardware; it only reads `True_SC_Battery` capacities to report an aggregate charge-state metric, with its own `flag_executive` written by `Software_SC_Executive` as the sole external write it receives.

6.8.2 Mathematical Equations

The only quantity actually computed by this class (via the mission-specific helper function `func_update_mean_state_of_charge.m` that it calls) is the fleet-averaged battery state of charge. There is no threshold-comparison, load-shedding, or mode-transition equation implemented anywhere in `Software_SC_Power.m` itself — the mode-selector switch/case structure is decision scaffolding without an underlying computed criterion, and the data-generation call simply reuses the shared data-handling bookkeeping equations documented elsewhere. Accordingly, only one equation is presented below.

6.8.2.1 Mean State of Charge

$$\mathrm{SoC}_{\mathrm{mean}} = 100 \times \frac{\sum_{i=1}^{N_{\mathrm{batt}}} C_{\mathrm{inst},i}}{\sum_{i=1}^{N_{\mathrm{batt}}} C_{\mathrm{max},i}} \tag{6.20}$$

Here $C_{\mathrm{inst},i}$ is the instantaneous energy capacity (in Watt-hours) of the i-th battery, $C_{\mathrm{max},i}$ is its maximum rated capacity (Watt-hours), N_{batt} is the number of batteries on the spacecraft, and $\mathrm{SoC}_{\mathrm{mean}}$ is the resulting fleet-averaged state of charge expressed as a percentage.

In the code, this is computed in `func_update_mean_state_of_charge.m`, which is called from the `Software_SC_Power` constructor and from every branch of the `mode_software_SC_power_selector` switch inside `func_main_software_SC_power`. The summation loops over `mission.true_SC{i_SC}.true_SC_body.num_hardware_exists.num_battery` batteries, accumulating `true_SC_battery{i_batt}.instantaneous_capacity` and `true_SC_battery{i_batt}.maximum_capacity` from the truth layer, and the result is stored in `obj.mean_state_of_charge`, which is subsequently logged each storage step into `obj.store.mean_state_of_charge`.

6.8.3 CML 2–5 Suitability

6.8.3.1 What is included and correctly implemented

The class correctly implements the executive-gated execution pattern common to the `Software_SC_*` suite.

- **Executive-gated duty cycle.** It only updates state when `flag_executive` is set by the mission's executive function, and resets the flag after each call, an appropriate first-order representation of a duty-cycled onboard software task for trade-space studies.
- **Fleet-averaged SoC metric.** It correctly aggregates state of charge across an arbitrary number of batteries into a single mean SoC metric, a reasonable, low-fidelity abstraction of what a real power-management task would monitor.

- **Extensible mode scaffolding.** The mode-selector scaffolding (with an explicit error on an undefined mode) gives a clean, extensible hook for adding mission-specific behavior later without needing to fork the class.
- **Consistent storage and data-source registration.** Time-history storage of `flag_executive` and `mean_state_of_charge` is correctly wired up, consistent with the storage conventions used elsewhere in the codebase, enabling post-run plotting/analysis of SoC trends; it also correctly participates in the shared data-handling bookkeeping by registering itself and reporting its own data generation.

6.8.3.2 What is not included / gaps

No actual power-management decision logic is implemented in this file.

- **No decision logic.** There is no threshold comparison against `mean_state_of_charge`, no load-shedding, and no triggering of safe-mode or other mode transitions anywhere in this file, despite the property and mode-selector structure suggesting that this is the intended purpose of the class.
- **Dormant mission-specific case.** The additional mission-specific case's real logic is present only as a commented-out function call, so no mission currently exercises anything beyond the generic SoC-averaging behavior.
- **No downstream effect.** The computed `mean_state_of_charge` is never fed forward to command anything — it is stored for logging but does not influence any actuator, load, or executive state within this class.
- **No representation of onboard knowledge limits.** There is no representation of estimation error, sensor limitations, or restricted onboard visibility into power state: the value is read directly from the truth-layer battery objects rather than being derived from a flight-software-only telemetry/estimation chain, so it does not model what the real flight software would or would not be able to observe.
- **No hysteresis or safe-mode criteria.** There is also no hysteresis, priority-based load-shedding table, or safe-mode entry/exit criteria of any kind.

For CML 2–5 purposes this is acceptable — flight-software-representative fidelity is not required at this maturity level — but the class should be understood as a placeholder for future power-management logic and a SoC-trending/data-bookkeeping utility, not as an implemented power-management algorithm.

Chapter 7

Main File

7.1 main_v3

main_v3 is the master time loop that drives the entire MuSCAT simulation once a mission has been configured. Each pass of the outer loop (indexed by k, over mission.true_time.num_time_steps) advances the true time/date, updates the solar system and every target body, and then updates every spacecraft's translational dynamics (SRP, gravity gradient, body mass properties, orbit propagation) and onboard clock/computer. Inside each coarse step, an inner Attitude Dynamics Loop (indexed by k_attitude, over mission.true_time.num_time_steps_attitude) advances attitude at the finer sub-step: it updates each spacecraft's ADC, sun sensors, star trackers, and IMUs, runs the software attitude estimator and controller, and then fires RCS thrusters, reaction wheels, and magnetorquers based on the resulting commands. After the attitude loop completes, the outer loop resumes with a final per-spacecraft block that updates cameras, telescopes, science radar, remote sensing, communication software and hardware (radio antennas, communication links), generic sensors, solar panels, RPS, orbit-control software, chemical/EP thrusters, science processors, fuel tanks, batteries, power software, data handling (both truth and software layers), and onboard memory — followed by updates to the ground station's radio antennas, software enterprise, and the ground station object itself.

This ordering encodes the dependency structure of the whole simulation: environment and target state must be current before spacecraft dynamics are propagated; translational and attitude states must be current before sensors/estimators run; estimated state must be current before control laws run; and control commands must be current before actuators (which draw power and consume propellant) are updated, which in turn must be current before the power and data-handling ledgers are closed out for the step. At the end of every outer-loop iteration, the script clears the last MATLAB warning (to avoid the same warning repeating every step), updates the real-time visualization, and checks mission.storage.flag_stop_sim to allow an early, graceful termination of the run. After the loop exits (either by reaching num_time_steps or via an early stop), it clears all loaded SPICE kernels (cspice_kclear) and sets mission.flag_stop_sim = 1.

Chapter 8

Mission Classes

8.1 Mission_DART

Mission_DART is the top-level initialization script that configures a single-spacecraft simulation of the Double Asteroid Redirection Test concept within MuSCAT. It sets the mission time span (an initial epoch of 02-NOV-2018 propagated for two days at a 5-second dynamics time step), loads the spacecraft's SPICE trajectory kernel (traj_daresim_simple.bsp) to initialize its Sun-centered position and velocity, and defines Bennu as the target asteroid via True_Target_SPICE. The script builds out the spacecraft hardware manifest passed into the various True_SC_* classes, including a camera, six sun sensors, three star trackers, one IMU, a 12-thruster RCS cluster plus a single chemical thruster, three reaction wheels, dual onboard computers and batteries, a fuel tank, and a dipole radio antenna for Earth downlink/uplink.

On the software side, the init_data structures passed to the Software_SC_* classes select mission-specific algorithms rather than generic defaults: the executive cycles through modes such as pointing the camera at the target, maximizing solar power, and aligning the thruster along the delta-V direction; orbit control and attitude control both use dedicated 'DART' control laws; orbit estimation uses a truth-with-error-growth model; and the communication software schedules direct-to-Earth contacts roughly every 42 minutes. Solar radiation pressure is enabled while Earth gravity-gradient torque is explicitly disabled, reflecting DART's role as an interceptor rather than an Earth-orbiting satellite.

8.2 Mission_NISAR

Mission_NISAR is the top-level configuration script that instantiates and initializes the mission object for the NISAR (NASA-ISRO Synthetic Aperture Radar) mission concept. It sets up a single-spacecraft, single-target (Earth) scenario, using SPICE-based target ephemerides and a Solar System model including the Sun and Moon, over a mission-specific window tied to a selectable case study (e.g., "Hurricane Helene" from 24 September 2024 or "Texas heavy rain" from 3 July 2025) with a 60-second simulation time step. The spacecraft is placed in a nearly circular, Sun-synchronous-like low Earth orbit (altitude 747 km, inclination 98.4°) defined via classical orbital elements and propagated with relative dynamics, while the multi-body shape model constructs a detailed bus, deployable boom, and tilted 12 m radar reflector geometry.

The script also builds out the spacecraft's hardware manifest (solar panels, battery, onboard memory/clock/computer, fuel tank, and two remote-sensing radar payloads modeled as L-band and S-band SAR instruments with multi-kilowatt power draws and downlink rates up to hundreds of Mbps) and links it to 17 ground station radio antennas spanning NASA Near Earth Network and ISRO sites for uplink/downlink communication. Finally, it configures the onboard software stack (Software_SC_* objects) with mission-specific mode selectors such as "NISAR Oracle" attitude control and "NISAR" communication logic, alongside truth-based attitude and orbit estimation, before invoking main_v3.m to run the simulation and post-process ground-track and Excel outputs.

8.3 Mission_NISAR_2SC

Mission_NISAR_2SC extends the single-spacecraft NISAR configuration to a two-spacecraft formation flying in the same 747 km, 98.4°-inclination orbit about Earth, propagated in a relative dynamics frame (mission.frame = 'Relative'). The two spacecraft are distinguished only by their right ascension of ascending node (RAAN), which is offset by 90° between SC1 and SC2 for each scenario (e.g. 12° vs. 102° for the Texas heavy rain case, 91° vs. 181° for Hurricane Helene), placing the pair in a phased along-track/cross-track configuration rather than a true tight formation. Each spacecraft carries an identical hardware manifest replicated from the single-SC NISAR bus and payload (deployable boom-mounted 12 m reflector, L-band and S-band SAR instruments, dual solar panels, battery, fuel tank, onboard computer, and chemical thruster), and each independently links to the same 17-antenna ground station network, so the "2SC" aspect here is expressed through duplicated per-spacecraft hardware and communication links (using the 'Multi-SC per GS' communication-link mode) rather than an explicit inter-satellite radio link between the two spacecraft.

The script otherwise follows the same initialization flow as the single-spacecraft Mission_NISAR script — time, storage, star catalog, solar system, target body, and ground station setup occur once, and the spacecraft-specific blocks (body shape, orbit, attitude, power, data handling, sensors, software executive, and communication) are then executed inside a loop over mission.num_SC = 2, so all per-spacecraft objects are stored as separate cell entries under mission.true_SC{i_SC}. Visualization and post-processing are likewise adapted for the two-spacecraft case, calling func_visualize_SC_NISAR_2SC for the 3D dashboard and looping over both spacecraft when producing ground-track plots and Excel output via func_post_process_NISAR_output_to_Excel.

Bibliography